Fullstack Java and Angular Enterprise Platform: Architecture

Order Management and Fulfillment platform. Design, trade-offs, failure handling and honest verification status.

Synthetic demonstration project

Independently designed for engineering-interview preparation. All data is synthetic: no real customer data, credentials, proprietary APIs or confidential processes are used.

Document: PDF 1 of 2, Architecture (21 sections, a traceability appendix and an integration-defects appendix)

Stack: Java 17, Spring Boot 3.5.16, Spring Cloud Gateway, Resilience4j, Kafka, PostgreSQL 16, Flyway, Redis (optional), Angular 20, React 19 with Redux Toolkit, Docker, Kubernetes, Terraform (AWS reference), GitHub Actions, OpenTelemetry, Prometheus, Grafana

Companion: Fullstack_Java_Angular_Deep_Dive_Interview_Guide.pdf

Built: 2026-10-19. Test statistics and the validation report are read from disk at build time; rebuild after the integration run.

Evidence labels used throughout: Executed = run in this environment with recorded output; Documented / not executed = files or design present but not run here; Conceptual = design discussion only.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Table of contents

1. Business problem and requirements . . . **7**

Purpose of this document . . . 7
The business problem . . . 7
Actors . . . 7
Functional requirements . . . 8
Non-functional requirements . . . 8
Explicit scope boundaries and assumptions . . . 8
Technology choices at a glance . . . 9

2. System context and component diagrams . . . **10**

System context . . . 10
Context diagram . . . 11
Component and service map . . . 12
Where the code lives . . . 12

3. Service boundaries . . . **14**

Principle . . . 14
The four services and the gateway . . . 14
Why four services and not one . . . 14
Synchronous versus asynchronous communication . . . 14
Contracts between services . . . 15
Service discovery and configuration: an honest note . . . 15
Shared library trade-off . . . 15

4. Frontend architecture: Angular and React . . . **16**

Why there are two front ends . . . 16
Dual UI, one gateway . . . 16
Angular architecture . . . 16
React architecture . . . 21
React state-management design . . . 25
Angular and React side by side . . . 26
Dual-UI testing matrix . . . 26

5. REST API design . . . **28**

Conventions (all services) . . . 28
Endpoint groups . . . 28
Design decisions worth defending . . . 29
Example: create order and replay . . . 29
What is not done (honest list) . . . 30
Captured API examples . . . 32

6. Database ER diagram . . . **55**

One database per service . . . 55

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Order lifecycle and state machine . . . **60**
Rules . . . 60
8. Kafka event flows . . . **64**
Topology . . . 64
Contract: topics, envelope, payloads and delivery semantics . . . 66
Delivery semantics in depth . . . 68
1. The three vocabularies . . . 68
2. The chain . . . 68
3. Failure windows . . . 68
4. Honest limits . . . 69
9. Saga orchestration and compensation . . . **70**
1. What a saga is . . . 70
2. Happy path (orchestrated by order-service) . . . 70
3. Compensation paths . . . 70
4. Eventual consistency . . . 71
5. Trade-offs . . . 71
6. What users see . . . 72
Sequence diagrams . . . 73
Contract: saga rules . . . 76
10. Inventory concurrency handling . . . **77**
The problem . . . 77
The model . . . 77
Strategy 1 (default): atomic conditional update . . . 77
Strategy 2 and 3 . . . 78
Defence in depth . . . 78
Evidence . . . 78
Sequence: check-then-act versus atomic update . . . 79
11. Security architecture . . . **80**
Authentication and authorization flow . . . 80
Front-end security: token storage and Content Security Policy (Angular and React) . . . 81
0. Verification legend and honest status . . . 82
1. Assets, actors, trust boundaries . . . 82
2. STRIDE by element . . . 84
3. The ten threats . . . 84
4. Summary matrix . . . 87
5. Test matrix (existing vs still open) . . . 88
6. OWASP Top 10 (2021) mapping . . . 89
7. OWASP API Security Top 10 (2023) mapping . . . 89

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Demo-only vs production controls . . . 90
9. Secrets management . . . 90
10. Integration-pass security findings and caveats (live stack, 2026-10-18) . . . 91
11. React front end notes (frontend/oms-ui-react, added 2026-10-19) . . . 92
12. Docker and Kubernetes deployment . . . 93
Honest status first . . . 93
Container images . . . 93
Docker Compose . . . 93
Kubernetes . . . 94
Rollout and database changes . . . 95
AWS reference . . . 95
Trade-offs . . . 95
Deployment diagram . . . 96
Deployment and rollback procedure . . . 97
0. Honest status . . . 97
1. Environments and overlays . . . 97
2. Pipeline (design) . . . 98
3. Rolling update mechanics (from base/*.yaml) . . . 98
4. Release procedure (per environment) . . . 99
5. Database migrations: expand / contract . . . 99
6. Kafka consumer concerns during deploy and rollback . . . 100
7. Rollback decision tree . . . 100
8. Dev-to-prod differences and demo-only controls to remove . . . 101
9. Backout checklist (printable) . . . 101
13. CI/CD pipeline . . . 102
Pipeline . . . 102
14. Observability architecture . . . 105
Three signals, one correlation id . . . 105
15. Failure and recovery flows . . . 108
Failure catalogue . . . 108
Why these mechanisms are sufficient and where they stop . . . 109
Key code . . . 109
Order of operations for an on-call engineer . . . 110
16. Testing strategy . . . 112
Strategy . . . 112
Techniques used . . . 114
Quality gates . . . 114
Evidence for this build . . . 114

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Front-end testing: React numbers and the dual-UI end-to-end matrix . . . 115
1. Result at a glance . . . 116
2. Commands to reproduce (from the repo root) . . . 116
3. Checks that were run . . . 117
4. NOT EXECUTED (stated plainly) . . . 119
5. Other honest observations . . . 120
6. Finalization pass (after the integration run) . . . 120
7. React front end verification (frontend/oms-ui-react, 2026-10-19) . . . 121
17. AI-assisted development workflow . . . 123
Principle . . . 123
The workflow . . . 123
The flawed-AI catalogue . . . 123
Reviewing AI-generated code: a five-minute method . . . 124
Risks and controls . . . 124
Honesty statement . . . 124
18. Architecture Decision Records . . . 126
ADR-0001: Four services plus a gateway instead of a modular monolith . . . 126
ADR-0002: Kafka event design (envelope, topics, keys, versioning) . . . 128
ADR-0003: Orchestrated saga for order processing . . . 129
ADR-0004: Database per service . . . 131
ADR-0005: Idempotency at the API, the consumer and the state machine . . . 132
ADR-0006: Concurrency control for stock reservation . . . 133
ADR-0007: URL-path API versioning with additive-only evolution . . . 134
ADR-0008: CI/CD pipeline and deployment model . . . 135
ADR-0009: Transactional outbox for event publication . . . 137
ADR-0010: JWT bearer tokens, OIDC-compatible resource servers, backend-authoritative authorizati... . . . 138
ADR-0011: Observability stack (Prometheus, Grafana, Loki, Tempo, OpenTelemetry) . . . 139
ADR-0012: Angular state management with signals and small stores, not NgRx . . . 140
ADR-0013: Two front ends (Angular and React + Redux) on one backend contract . . . 142
19. Architecture mind map . . . 144
20. Troubleshooting scenarios . . . 145
How to use this section . . . 145
Symptom to first action . . . 145
A repeatable investigation method . . . 145
Debugging exercises included in the repository . . . 145
Runbook index . . . 146
Index . . . 146
Severity guide . . . 146

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

On-call basics . . . 147
Command cheat sheet (all illustrative: not executed against a live cluster) . . . 147
Metrics referenced (contract section 7 plus Micrometer defaults) . . . 148
Related . . . 148
Runbook 01 - Orders stuck in processing . . . 148
Runbook 02 - Kafka consumer lag . . . 150
Runbook 03 - Inventory reservation failures under concurrency . . . 152
Runbook 04 - Database latency spikes . . . 155
Runbook 05 - Duplicate order events . . . 157
Runbook 06 - Service unavailable (503 / readiness DOWN) . . . 159
Runbook 07 - Failed deployment . . . 161
Runbook 08 - JVM memory problems . . . 163
21. Limitations and future enhancements . . . 166
Known limitations (by design or by environment) . . . 166
Documentation freshness . . . 167
Future enhancements (prioritised) . . . 168
Appendix A. Job-description traceability matrix . . . 169
Appendix B. Integration defects found and fixed . . . 180

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Business problem and requirements

Purpose of this document

This document explains the architecture of an Order Management and Fulfillment platform (OMS) that was built as a runnable, interview-preparation reference for a Senior Full Stack Java and Angular engineer role. The original brief mandated Angular; the user later explicitly asked for a React frontend as well, so the platform has two feature-equivalent browser UIs (Angular 20 and React 19 with Redux Toolkit) on one unchanged gateway contract (ADR-0013). It is written so that a reader can explain every design choice, point at the code that implements it, and state honestly which parts were executed and which are documented only. All data in the system is synthetic. There are no real customers, no real payment or email providers and no real credentials.

> Reading guide: each section ends with the evidence you would show in an interview. Evidence labels used everywhere are **Executed** (run in this environment, output recorded), **Documented / not executed** (files exist, were statically checked at most) and **Conceptual** (discussed, not built).

The business problem

A mid-sized B2B supplier sells physical goods to business customers. The operations team has five recurring problems:

Problem in the business	Consequence	What the platform does about it
Two buyers order the last units at the same moment	Overselling, cancelled orders, angry customers	Stock is reserved with one atomic conditional database update, so the database decides who wins
A customer double-clicks "Place order" or the network retries	Duplicate orders	`Idempotency-Key` header, fingerprint check, stored replay response
A step in the order flow fails halfway (stock reserved, shipping fails)	Stock locked forever, money or goods in limbo	Orchestrated saga with compensation, recovery job and reconciliation
Staff cannot see why an order is stuck	Long investigations, guesswork	Status history, saga step log, audit events, correlation ids, metrics, runbooks
Notifications go out twice, or never	Customer confusion	Idempotent consumers, unique (event, template) notification key, retry with backoff

Actors

- **Customer** (role `CUSTOMER`): browses the catalog, places and cancels their own orders, reads their own notifications.
- **Operator** (role `OPERATOR`): sees every order, adjusts stock, retries failed orders, fulfillments and notifications, reads audit and messaging health.
- **Administrator** (role `ADMIN`): everything an operator can do plus product and customer management and reconciliation.
- **Service** (role `SERVICE`): machine identity used for service-to-service REST calls on `/api/v1/internal/**`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Functional requirements

1. Authenticate with a JWT; authorise by role and by resource ownership (a customer only sees their own orders).
2. Browse products and availability; staff manage products and adjust stock with a movement ledger.
3. Place an order with several lines. The API answers `202 Accepted` immediately and processing continues asynchronously.
4. Orders move through a strict lifecycle: `CREATED`, `VALIDATED`, `INVENTORY_RESERVED`, `CONFIRMED`, `FULFILLED`, or the terminal outcomes `REJECTED`, `CANCELLED`, `FAILED`, with `COMPENSATION_PENDING` as the transitional compensation state.
5. Reserve stock without ever overselling; reservations expire after a TTL if the order is not confirmed.
6. Simulated fulfillment (picking, packing, shipping, completion) with deterministic failure injection (`FAILTOWN`).
7. Simulated email notifications with retry, backoff and duplicate protection.
8. Cancel an order from any non-terminal pre-fulfillment state and release stock through compensation.
9. Operational tooling: failed-order list, saga inspection, retry, reconciliation run, messaging and audit endpoints, aggregate health.
10. A browser UI for customers and operators, delivered twice on the same contract: Angular 20 (`frontend/oms-ui`) and React 19 with Redux Toolkit (`frontend/oms-ui-react`).

Non-functional requirements

Quality	Target in this design	How it is addressed
Correctness under concurrency	No negative or oversold stock, ever	Atomic conditional UPDATE, database CHECK constraints, tests under concurrency
Reliability	No lost events, no duplicated effects	Transactional outbox, idempotent consumers, DLT, recovery job
Availability	One service failing must not take down the others	Database per service, async messaging, circuit breaker at the edge and on REST calls
Security	OWASP API Top 10 aware	JWT at edge and service, ownership checks, rate limits, body limits, masked logs, no secrets in code
Observability	Any order traceable end to end	Correlation id, W3C traceparent, structured logs, Prometheus metrics, Grafana dashboards, runbooks
Evolvability	Add fields without breaking consumers	URL versioning, additive-only changes, tolerant readers, schemaVersion in the envelope
Operability	Repeatable build and deploy	Maven reactor, Docker images, Kustomize overlays, GitHub Actions, rollback procedure
Testability	Fast feedback on domain rules	Pure `OrderStateMachine`, H2 in PostgreSQL mode, embedded Kafka, BDD scenarios

Explicit scope boundaries and assumptions

- Payment, tax, pricing rules, real shipping carriers and real email or SMS delivery are out of scope; they are simulated or absent.
- The identity provider is simulated by a dev-only token endpoint in the gateway. The resource-server configuration is OIDC compatible, so a real issuer can be plugged in by configuration.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Capacity targets are illustrative design targets, not measured results. Load scripts exist (`performance/`) but a production-scale run was not performed.
- The reference platform runs locally without Docker (embedded Kafka and H2). Docker, Kubernetes and AWS artefacts exist and are validated statically; whether they were run live is stated in the evidence section.

Technology choices at a glance

Layer	Choice	Version	Why
Language and runtime	Java	17	LTS, records and sealed types, widely available in enterprises
Framework	Spring Boot	3.5.16	Mature ecosystem, Actuator, auto-configuration
Edge	Spring Cloud Gateway (reactive) with Resilience4j	Spring Cloud 2025.0.0, Resilience4j 2.2.0	Routing, filters, circuit breaker, rate limiting
Messaging	Apache Kafka	3.9 (KRaft) in Compose	Durable log, per-key ordering, consumer groups
Database	PostgreSQL	16 (H2 in PostgreSQL mode for local and tests)	ACID, row locks, rich indexing
Migrations	Flyway	via Spring Boot	Same SQL for H2 and PostgreSQL
Cache and limiter store	Redis	optional	Only for rate limiting and product cache; degrades to in-memory
Frontend	Angular	20	Standalone components, signals, functional guards and interceptors
Frontend (second UI)	React, Redux Toolkit (RTK Query), React Router, MUI, Vite	React 19, RTK 2, Router 7	Slices for client state, RTK Query for server state, same gateway contract and same 18 end-to-end specs
Packaging	Docker, Kustomize, Terraform (AWS reference)	n/a	Immutable images, environment overlays
CI	GitHub Actions	n/a	Build, test, scan, image, validate
Observability	Micrometer, OpenTelemetry, Prometheus, Grafana, Loki, Tempo	n/a	Metrics, logs, traces

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. System context and component diagrams

System context

The platform is used by three human roles through either of two browser applications (Angular and React, same contract), and by machines through the same REST API. Everything outside the dashed boundary in the diagram is either infrastructure (Kafka, PostgreSQL, Redis, the observability stack) or an external party that is simulated: the identity provider (a dev-only token endpoint stands in for it) and the email or SMS provider (a simulated sender that never leaves the process).

Reading the diagram:

- The browser (either UI) talks only to the gateway. The gateway is the single public entry point and the only component that mints demo tokens.
- Services never call each other through the gateway. The one synchronous service-to-service call (order to inventory validation) goes directly over the cluster network with a short-lived SERVICE token.
- Kafka carries commands and events between services. Each service reads and writes only its own database.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Context diagram

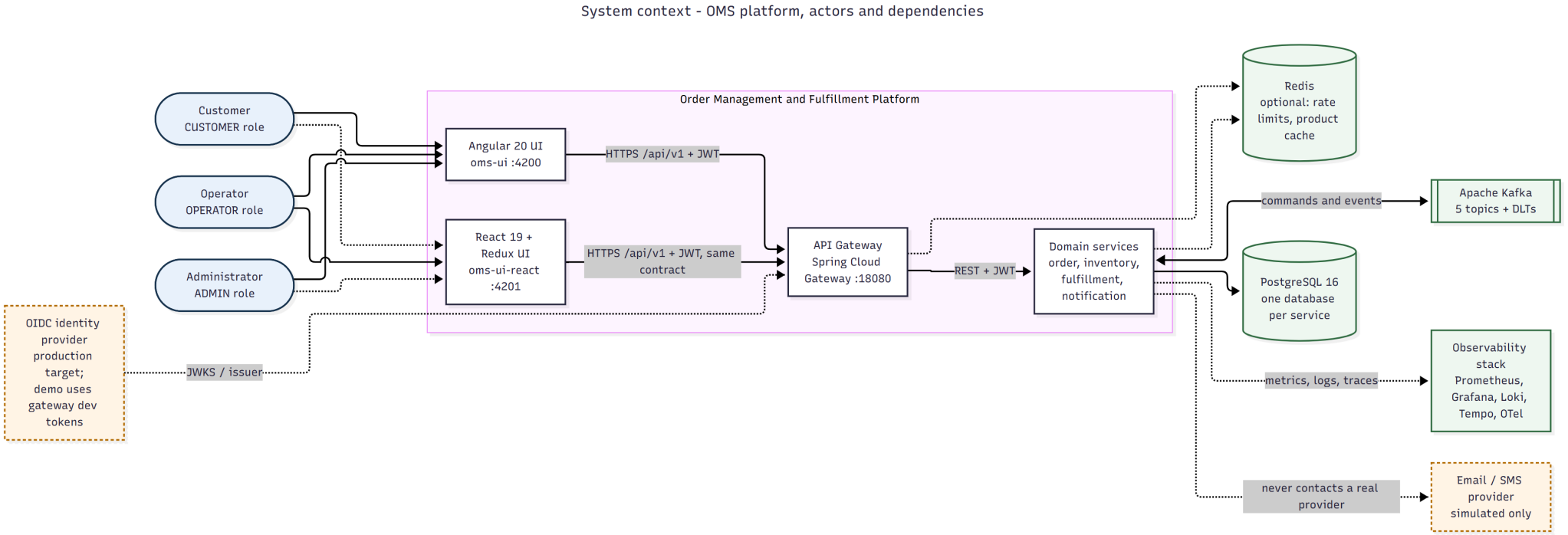


Figure 1. System context: actors, platform boundary, infrastructure and simulated externals

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Component and service map

The second diagram zooms into the four services. Inside each service the layering is the same: REST controllers, application services with transaction boundaries, domain classes without framework dependencies (`OrderStateMachine`, `FulfillmentStateMachine`), repositories, messaging listeners, scheduled jobs, and the outbox relay from `platform-common`.

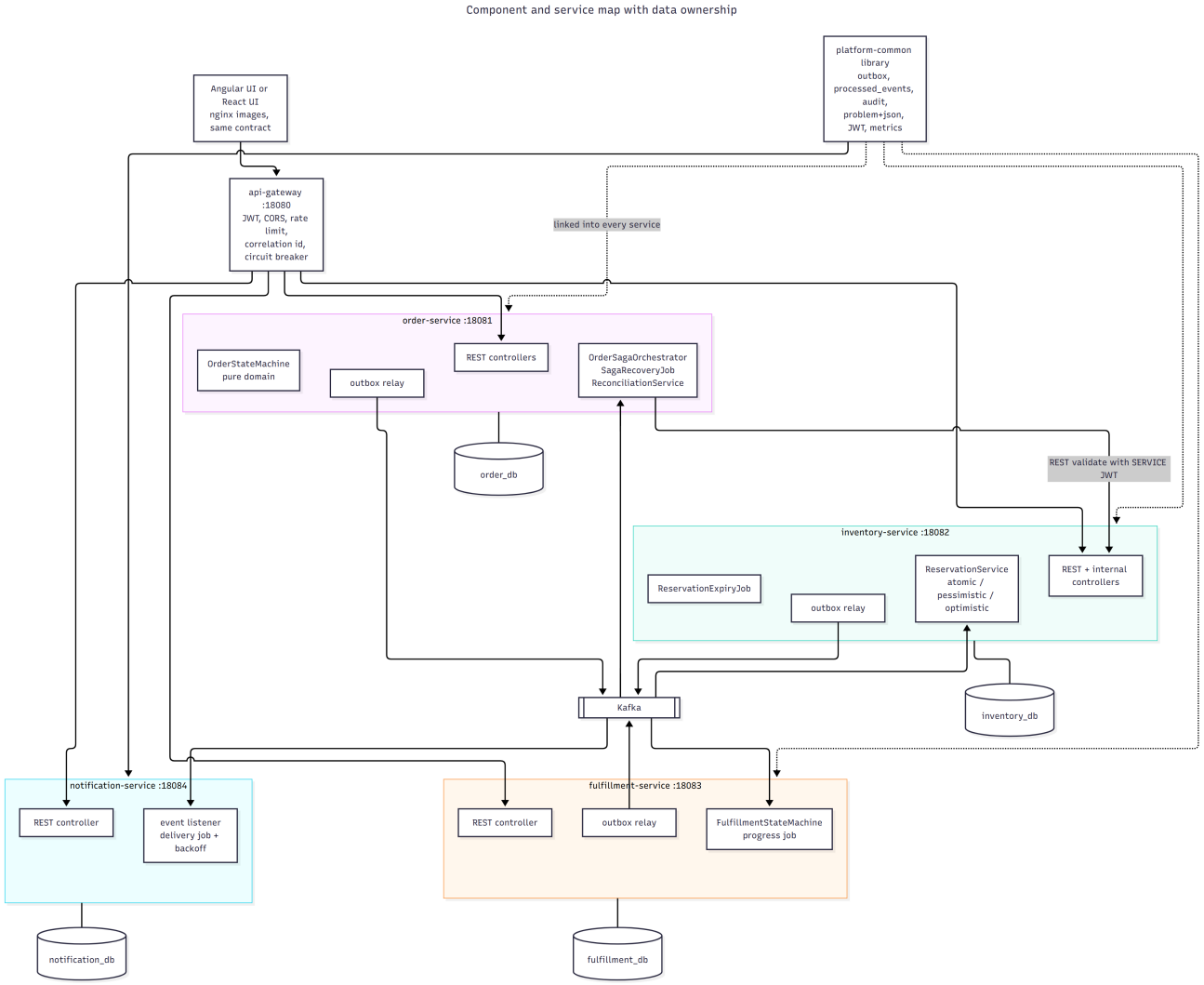


Figure 2. Component and service map with data ownership

Where the code lives

Area	Path
Reactor build	`services/pom.xml` (gateway module in `gateway/`)
Shared library	`services/platform-common/`
Test helpers	`services/platform-test-support/`
Local no-Docker stack (embedded Kafka)	`services/local-stack/`
Order, inventory, fulfillment, notification	`services/<name>-service/`
Edge	`gateway/`
Angular UI	`frontend/oms-ui/`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Path
React UI (second front end, same contract)	`frontend/oms-ui-react/`
Container, Compose, Kubernetes, AWS	`infrastructure/`
Dashboards, alerts, collectors	`observability/`
Load tests	`performance/`
AI workspace	`ai-engineering/`
Specs and OpenAPI exports	`specs/`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3. Service boundaries

Principle

Service boundaries follow data ownership and rate of change, not technical layers. Each service owns its tables exclusively and exposes data only through its REST API or its events. No service reads another service's database.

The four services and the gateway

Component	Port (local)	Owns	Responsibility
api-gateway	18080	Routes, edge policy	JWT validation, CORS allow-list, rate limiting, request size limit, correlation id, circuit breaker, dev token endpoint (profile `dev`), aggregate health
order-service	18081	customers, orders, order_items, order_status_history, saga_instances, saga_step_log, idempotency_keys	Order API, `OrderStateMachine`, saga orchestration, recovery job, reconciliation
inventory-service	18082	products, stock, inventory_reservations (+items), stock_movements	Catalog, stock, atomic reservation, expiry, internal validation endpoint
fulfillment-service	18083	fulfillments, fulfillment_events	Simulated picking, packing, shipping, retry, failure injection
notification-service	18084	notifications, notification_attempts, order_contacts	Consumes events only, simulated delivery with backoff
platform-common (library)	n/a	Shared tables `outbox_events`, `processed_events`, `audit_events` in every service database	Outbox relay, idempotent consumer, audit, problem+json, JWT, correlation id, Micrometer, ops endpoints

Every service database also contains the three shared tables from `services/platform-common/src/main/resources/db/common/V1__outbox_processed_audit.sql`, but each service has its own copy and its own rows. Flyway versions V1 to V8 are reserved for the shared library, service schemas start at V10, seed data at V1000 (dev profile only).

Why four services and not one

The decision is recorded in ADR-0001. In short: the four capabilities have different consistency needs and different failure behaviour. Inventory must serialise on hot rows; fulfillment is slow and flaky by nature; notification is allowed to lag. Splitting them lets each fail and scale independently and makes the saga, outbox and idempotency patterns real instead of decorative. The cost is distributed-systems complexity, which is why the documents are explicit about eventual consistency. A modular monolith would be the right default for a small team with no independent scaling needs; ADR-0001 discusses that trade-off honestly.

Synchronous versus asynchronous communication

Interaction	Style	Reason
Browser to gateway to services	REST over HTTP	Request/response user interactions

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Interaction	Style	Reason
order-service to inventory-service validation	REST (`POST /api/v1/internal/products/validate`) with SERVICE JWT, timeouts 2 s connect / 3 s read, Resilience4j retry and circuit breaker	The saga needs prices and availability to price the order before reserving
Reserve, release, confirm, ship, notify	Kafka commands and events	Decoupled, durable, replayable, naturally retried

Contracts between services

- REST contracts are published as OpenAPI (`specs/openapi/`, exported with `scripts/export-openapi.mjs`).
- Event contracts are JSON envelopes defined in `services/platform-common/src/main/java/com/enterprise/oms/common/events/EventEnvelope.java` with payload records in `EventPayloads.java` and topic names in `Topics.java`.
- Both are versioned additively; see the API versioning and event schema evolution documents.

Service discovery and configuration: an honest note

This project does **not** use Eureka or Spring Cloud Config. Routes are static URIs from properties (`oms.gateway.routes.<svc>.uri`), because in Docker Compose and Kubernetes the platform already provides DNS-based service discovery (Compose service names, Kubernetes `Service` objects) and configuration (environment variables, ConfigMaps, Secrets). Adding a discovery server or config server would add two more stateful components without adding capability in these runtimes. Spring Cloud is used where it earns its place: Gateway for the edge and Resilience4j for fault tolerance. In an interview, say exactly this and add that Eureka/Config would be reasonable on a VM-based estate without an orchestrator.

Shared library trade-off

`platform-common` removes boilerplate (outbox, idempotency, error format, security) but couples services to one library version. The mitigation is semantic versioning of the library, additive changes, and keeping domain logic out of it. A shared library is not a shared database, which is the coupling that matters most.

Source: `services/platform-common/src/main/java/com/enterprise/oms/common/events/EventEnvelope.java` *(lines 14 to 31)*

```
public record EventEnvelope<T>(
        UUID eventId,
        String eventType,
        int schemaVersion,
        Instant occurredAt,
        String correlationId,
        UUID causationId,
        String aggregateType,
        String aggregateId,
        String producer,
        T payload) {

    public static final int CURRENT_SCHEMA_VERSION = 1;

    /** Builds a new envelope with a random eventId and {@code occurredAt = now}. */
    public static <T> EventEnvelope<T> of(String eventType, String aggregateType, String aggregateId,
                                          String producer, String correlationId, UUID causationId, T
payload) {
        return new EventEnvelope<>(UUID.randomUUID(), eventType, CURRENT_SCHEMA_VERSION, Instant.now(),
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Frontend architecture: Angular and React

Why there are two front ends

The original brief mandated Angular, so `frontend/oms-ui` (Angular 20, standalone components, signal stores, ADR-0012) was built first. The user later explicitly asked for a React frontend as well, so `frontend/oms-ui-react` (React 19, Redux Toolkit with RTK Query, React Router, MUI) was added as a second, feature-equivalent UI. The decision is ADR-0013. Three rules kept the second UI cheap and honest:

- **One contract, no backend change.** Both UIs call the same `/api/v1` gateway paths and follow the same problem+json errors, role rules, idempotency and order state machine. Nothing under `services/` or `gateway/` was changed for React, and nothing in the backend knows which UI is calling.
- **Parity is proven by tests, not prose.** The same 18 Playwright specs run unchanged against both UIs; `E2E_UI` selects which one.
- **Idiomatic code in each framework**, not an Angular port written in JSX: RTK Query and slices on one side, signal stores and RxJS on the other.

Dual UI, one gateway

Both single-page apps are served same-origin with `/api` proxied to the gateway (the Vite or Angular dev server, `scripts/serve-ui.mjs`, or nginx in the container images), so no CORS is needed. The gateway is the only public entry point for either UI. In the local run the Angular UI is on port 4200 and the React UI on 4201 (`bash scripts/run-local.sh --ui both`); in Kubernetes the Angular UI owns the Ingress path `/` and the React UI is an optional Deployment that the Ingress does not route (reach it with `kubectl port-forward`). The system context diagram in section 2 shows both UIs in front of the same gateway.

Aspect	Angular UI	React UI
Directory	`frontend/oms-ui`	`frontend/oms-ui-react`
Dev and local port	4200	4201
Language and build	TypeScript strict, Angular CLI	TypeScript strict, Vite
Unit tests	Karma and Jasmine, 149 of 149	Vitest, Testing Library and MSW, 207 of 207
End-to-end	18 of 18 mock, 18 of 18 live	18 of 18 mock, 18 of 18 live (`E2E_UI=react`)
Image and Kubernetes manifest	nginx-unprivileged image, base manifest	same pattern; never built or applied (no Docker, no cluster)

Angular architecture

Decision records: ADR-0012 state management, ADR-0010 security. Verified against `frontend/oms-ui/src/app` on disk. Angular 20, standalone components only (no NgModules), Angular Material.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

1. Layers and folders

Layer	Folder	Responsibility
Bootstrap	`app.config.ts`, `main.ts`, `app.routes.ts`	providers, interceptor order, `provideAppInitializer` that loads `config.json`, lazy routes
Core (singletons)	`core/`	`config.service.ts`, `auth/` (`auth.store.ts`, `guards.ts`), `http/` (`interceptors.ts`, `api-error.ts`), `api/apis.ts` (typed API clients), `models.ts` (types mirroring the contract), `notify/toaster.ts`, `state/` (`list-store.ts`, `cart.store.ts`, `polling.ts`)
Features	`features/`	one folder per screen: catalog, dashboard, fulfillment, inventory, login, notifications, orders (list, create, detail, form, detail store), ops (ops dashboard, failed orders, audit), misc
Shared UI	`shared/components/`	presentational components: `data-table`, `page-header`, `status-chip`, `status-timeline`, `timeline`, `confirm-dialog`, `feedback`
Testing	`testing/test-utils.ts` and `*.spec.ts` beside code	Karma/Jasmine

Dependency direction: features depend on core and shared; core never imports features; shared components take inputs and emit outputs and do not call APIs.

2. Standalone components and lazy loading

Every feature route uses `loadComponent: () => import(...)`, so each screen is its own chunk and the initial bundle contains only bootstrap, login and shared code. `withComponentInputBinding()` lets route params bind to component `input()`s. Route `title`s are set per route. `app.config.ts` uses `provideZoneChangeDetection({ eventCoalescing: true })`, so the app is still zone-based; signals work with it and make a future zoneless switch feasible.

3. State with signals

- `AuthStore`: token and identity as signals, mirrored to `sessionStorage`; `isAuthenticated`, roles and customer id derived; `logout('expired')` called by the error interceptor on 401.
- `CartStore`: a private writable `signal` of lines with `computed` `count` and `total`.
- `list-store.ts`: generic paginated-list state (`status`, `data`, `error`) with pure update functions (`listLoading`, `listLoaded`, `listFailed`, `withFilters`, `withPage`, `withSort`); changing filters or sort resets to page 0.
- `order-detail.store.ts`: per-screen store combining order, history and polling.
- Server data is refetched rather than mirrored in a global cache. After an admin retry the UI ignores the 2xx body and re-fetches (contract section 5.5).

4. HTTP pipeline (interceptors)

Order in `app.config.ts`: `correlationInterceptor`, `authInterceptor`, `retryInterceptor`, `errorInterceptor`. Requests flow left to right, errors right to left.

- `correlationInterceptor`: adds `X-Correlation-Id` (UUID) to API calls so a click can be traced through gateway and services (ADR-0011).
- `authInterceptor`: adds `Authorization: Bearer` only to API calls, never to the token endpoint or foreign origins.
- `retryInterceptor`: retries only GET/HEAD, twice, with 200 ms then 400 ms backoff, only for status 0, 502, 503, 504. Mutations are never retried here; order creation relies on `Idempotency-Key` so the user (or caller) can safely resubmit.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- `errorInterceptor`: normalizes everything into `ApiError` (reads RFC 7807 `code`, `correlationId`, `errors[]`), logs out on 401 (except on login), toasts 429 with `Retry-After`. The UI shows a generic message for login failure and never echoes server `detail`.

5. Guards are UX, not security

`authGuard`, `roleGuard` (reads route `data.roles`, for example staff = OPERATOR and ADMIN) and `guestGuard` only hide screens a user cannot use and avoid dead-end navigation. Anyone can edit client code or call the API directly; every endpoint re-checks roles and ownership in the backend (ADR-0010, contract section 4). Never put a secret or an authorization decision in the bundle. The cart and role-dependent menus are convenience only.

6. Polling

Order status is eventually consistent (saga), so the detail screen polls: `pollUntil(fetchFn, isDone, {initialMs, maxMs, factor 1.5}, stop$)` in `core/state/polling.ts` emits immediately, then re-polls with growing delay capped at `maxMs`, completes on a terminal status or when the component is destroyed (`stop$`), and ends on error so the screen can offer a manual refresh. Defaults come from `config.json`. There is no WebSocket or SSE; that is a possible later upgrade if polling load matters.

7. Runtime configuration

`public/config.json` holds `{ "apiBaseUrl": "/api/v1", "pollIntervalMs": 1500, "pollMaxIntervalMs": 8000 }`. `ConfigService.load()` runs in `provideAppInitializer` before the app renders, falling back to `DEFAULT_CONFIG`. The same build runs anywhere: in the nginx image (`frontend/oms-ui/nginx.conf`, `Dockerfile`) the file is generated from `OMS_API_BASE_URL` and `/api/` is proxied to `API_GATEWAY_URL`; in dev `proxy.conf.json` forwards `/api` to the gateway on 18080. Serving UI and API same-origin means no CORS configuration is needed.

8. Testing

Pure functions (list reducers, `backoffDelay`, error normalization) are tested without TestBed; stores, guards, interceptors and screens have `*.spec.ts` files next to them. Tests run with Karma (`karma.conf.js`); ESLint config in `eslint.config.js`. Browser-level end-to-end checks live in `tests/e2e`. Not verified here: behaviour on browsers other than the headless one used by Karma.

9. React/Redux to Angular mapping

A working React + Redux Toolkit app now sits beside this one (`frontend/oms-ui-react`, react-architecture.md, ADR-0013); the concept table below is the quick reference, and the React app's README has the implementation-backed side-by-side table.

React / Redux idea	Angular in this project
Function component	Standalone `@Component` class with template
Props	`input()` / `@Input`; route params through `withComponentInputBinding`
Callback props, events	`output()`
`useState`	`signal()`
`useMemo` / selector	`computed()`
`useEffect`	`effect()` for side effects on signals; RxJS subscriptions with `takeUntilDestroyed` for IO
Redux store slice	injectable store class (`AuthStore`, `CartStore`, list store)
Reducer	exported pure functions (`listLoaded`, `withFilters`)
Action / dispatch	method call on the store

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

React / Redux idea	Angular in this project
Redux Thunk / Saga / RTK Query	RxJS operators (`switchMap`, `retry`, `expand` in `pollUntil`); no built-in query cache
Context provider	DI (`providedIn: 'root'` or component `providers`)
React Router loaders and guards	route `canActivate` guards, lazy `loadComponent`
Axios interceptors / middleware	`HttpInterceptorFn` chain
`React.lazy` + Suspense	`loadComponent` dynamic import
Error boundary	`provideBrowserGlobalErrorListeners` plus the error interceptor and toaster
`.env` build-time variables	runtime `config.json` loaded by `provideAppInitializer`
Redux DevTools	none by default (see ADR-0012 revisit triggers)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Architecture diagram

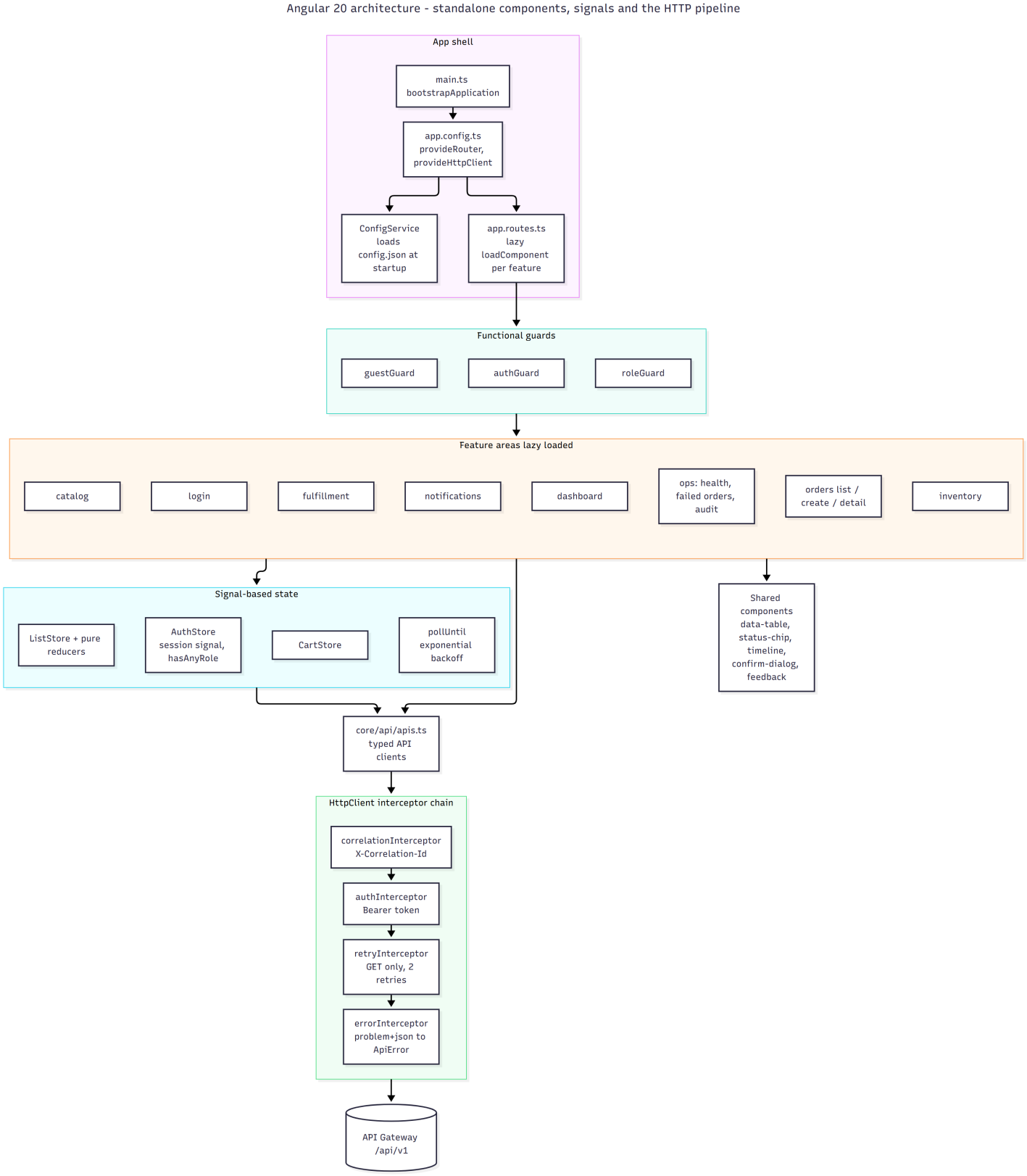


Figure 3. Angular architecture: routes, guards, stores and HTTP pipeline

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Source: `frontend/oms-ui/src/app/core/http/interceptors.ts` *(lines 50 to 61)*

```
export const retryInterceptor: HttpInterceptorFn = (req, next) => {
  if (req.method !== 'GET' && req.method !== 'HEAD') return next(req);
  return next(req).pipe(
    retry({
      count: MAX_GET_RETRIES,
      delay: (err, attempt) => (isTransient(err) ? timer(200 * 2 ** (attempt - 1)) : throwError(() =>
err)),
    }),
  );
};

/** Normalizes every failure to ApiError, logs out on 401 (except on login), and notifies on 429. */
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
```

React architecture

Second front end for the OMS, feature-equivalent to the Angular UI (see angular-architecture.md and ADR-0013). It uses only the gateway contract in 00-shared-contract.md; nothing in the backend knows which UI is calling.

Layers

```
main.tsx            loadConfig(config.json) -> createAppStore() -> <App/>
App.tsx             Provider (Redux) > ThemeProvider (MUI) > BrowserRouter > routes + ToastHost +
session-expiry timer
routes.tsx          React.lazy() per screen; guards as layout routes (RequireAuth, RequireRole,
GuestOnly)
layout/AppShell     skip link, app bar (cart, account menu), role-aware nav (<nav aria-label="Main
navigation">), ErrorBoundary + Suspense
store/              Redux Toolkit
  authSlice         session (memory), logoutReason, selectors; pure helpers (sessionFromToken,
hasAnyRole, parseStoredSession)
  cartSlice         cart lines + pure helpers shared in spirit with the Angular CartStore
  uiSlice           toast queue, mobile drawer
  baseQuery         the "interceptor chain": headers, GET-only retry, problem+json normalization, 401
logout, 429 toast
  api               RTK Query endpoints (all server state), tags and invalidation
  index             store factory, sessionStorage mirror, cache reset on logout
hooks/              useListQuery (page/size/sort/filters), useDebouncedValue, useOrderPolling,
useSessionExpiry
components/         PageHeader, DataTable (+pagination), StatusChip, StatusTimeline, EmptyState,
ErrorBanner, LoadingOverlay,
                    ConfirmDialog (+useConfirm), ToastHost, ErrorBoundary, guards
features/           login, dashboard, catalog, inventory (+StockAdjustDialog), orders (list/create/
detail), ops (dashboard, failed orders, audit),
                    notifications, fulfillment, misc (403/404)
```

Request flow

```
component --useXQuery/useXMutation--> RTK Query --> omsBaseQuery
                                                      1. fetchBaseQuery (URL = runtime apiBaseUrl +
path)
                                                         prepareHeaders: X-Correlation-Id (UUID),
Authorization: Bearer (never on /auth/token)
                                                      2. GET/HEAD + (network error | 502 | 503 | 504):
retry 2x with 200/400 ms backoff
                                                      3. any failure -> ApiError { status, code,
userMessage, correlationId, fieldErrors, retryAfterSeconds }
                                                      4. 401 (not the login call, signed in) ->
dispatch(loggedOut('expired')) -> RequireAuth redirects,
                                                         listener middleware resets the whole RTK Query
cache and the cart slice clears
                                                         429 -> dispatch(toast.warn("... Try again in
Ns."))
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`ApiError` is a plain serializable object (not an `Error` subclass) so it can live in the Redux store and the RTK Query cache. The user-facing message always comes from a code-to-text table; the server `detail` is kept for diagnostics only and the sign-in screen replaces 401/403 text with one generic sentence.

Server state vs client state

Kind	Where	Why
Orders, products, stock, reservations, fulfillments, notifications, audit, ops, health	RTK Query cache	fetched, cached 30 s after last use, refetched on mount/arg change, invalidated by mutations
Session (token, roles, customerId), cart, toasts	slices	cross-screen client state; session mirrored to `sessionStorage`
Filters, page, size, sort of one screen	`useListQuery` (reducer in the component)	local to the screen, serialised into the query args; changing a filter resets to page 0
Form values	React Hook Form	per form, validated by Zod schemas
Idempotency key	`IdempotencyKeyHolder` in component state	one logical submission; see below

Order creation and idempotency

`IdempotencyKeyHolder` (same rules as Angular): a new key for each new logical submission; the same key when the previous attempt had an unknown outcome (network error, status 0, or 5xx) and the request body fingerprint is unchanged; a new key after a definite answer (2xx/4xx) or when the body changed (same key plus a different body would be 422 `IDEMPOTENCY_KEY_REUSED`). The submit button is disabled while the mutation is in flight. 400 field errors (`items[0].quantity`, `shippingAddress.city`) are mapped to controls via `setError`; unknown fields stay in the banner. Because `POST /orders` is never retried by the base query, retry is always a user decision with the key preserved.

Order detail polling

`useOrderPolling` calls one RTK Query endpoint (`getOrderSnapshot`: order + history + saga for staff, fetched in parallel; a failing saga call only blanks the saga section). `pollingInterval` is recomputed after every completed fetch with exponential backoff (1.5 s x 1.5^n, capped at 8 s, from `config.json`) and becomes 0 at a terminal status, on error, after "Stop auto-refresh", and on unmount (RTK Query releases the subscription and its timer). The hook adjusts its small state during render (the documented pattern for derived state) instead of in effects.

Security

- Route guards and role-aware navigation are UX only; the backend authorizes every call (other customers' orders return 404).
- Token: Redux memory + `sessionStorage` (survives reload, per tab, readable by script on the origin: XSS exposure). Production should use authorization code + PKCE with a BFF and HttpOnly cookies. The login form has no prefilled password; the password is never stored.
- `returnUrl` accepts only same-app relative paths (no `//`, no scheme, no backslash).
- No CDN fonts or icons (system font stack; no icon package). nginx CSP: `default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'` (Emotion injects styles at runtime), `connect-src 'self'`, `frame-ancestors 'none'`. The React build emits no inline scripts.
- The UI renders server strings only as React text nodes (escaped); there is no `dangerouslySetInnerHTML`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- On logout every cached response and the cart are discarded, so the next user of the tab sees nothing of the previous session.

Accessibility

Skip link; the page `h1` is focusable and takes focus when a screen mounts (also sets `document.title`); labelled controls (MUI `TextField` labels) with error text; `role="alert"` for inline errors, `role="status"` for toasts (never competing with alerts), loading and status changes; dialogs trap focus, close on Escape and restore focus (`alertdialog` for confirmations, initial focus on the reason field or on the safe button); sortable headers are buttons with `aria-sort`; scroll regions are focusable; status is text plus glyph, never colour only; the add/remove item rows move focus to a stable control; `prefers-reduced-motion` respected. `eslint-plugin-jsx-a11y` runs in CI. No automated axe/Lighthouse pass has been run (see README limitations).

Testing

Layer	Tool	What
Pure logic	Vitest	slices, selectors, error normalization, config, timeline, Zod schemas, idempotency holder, list reducers, backoff
API layer	Vitest + MSW	headers, retries (GET only), 401/429 side effects, serializable errors, invalidation, persistence
Screens	Testing Library + MSW	the whole app is rendered (routes, guards, shell, toasts) at a URL with a seeded store: forms, dialogs, polling, null prices, role-aware navigation, pagination/filter/sort
End to end	Playwright (`tests/e2e`, `E2E_UI=react`)	the same 18 lifecycle specs as Angular, against the mock backend and the live stack

Angular to React mapping

See the table in `frontend/oms-ui-react/README.md` (signals/RxJS/Angular Material on one side, Redux Toolkit/RTK Query/MUI on the other).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Architecture diagram

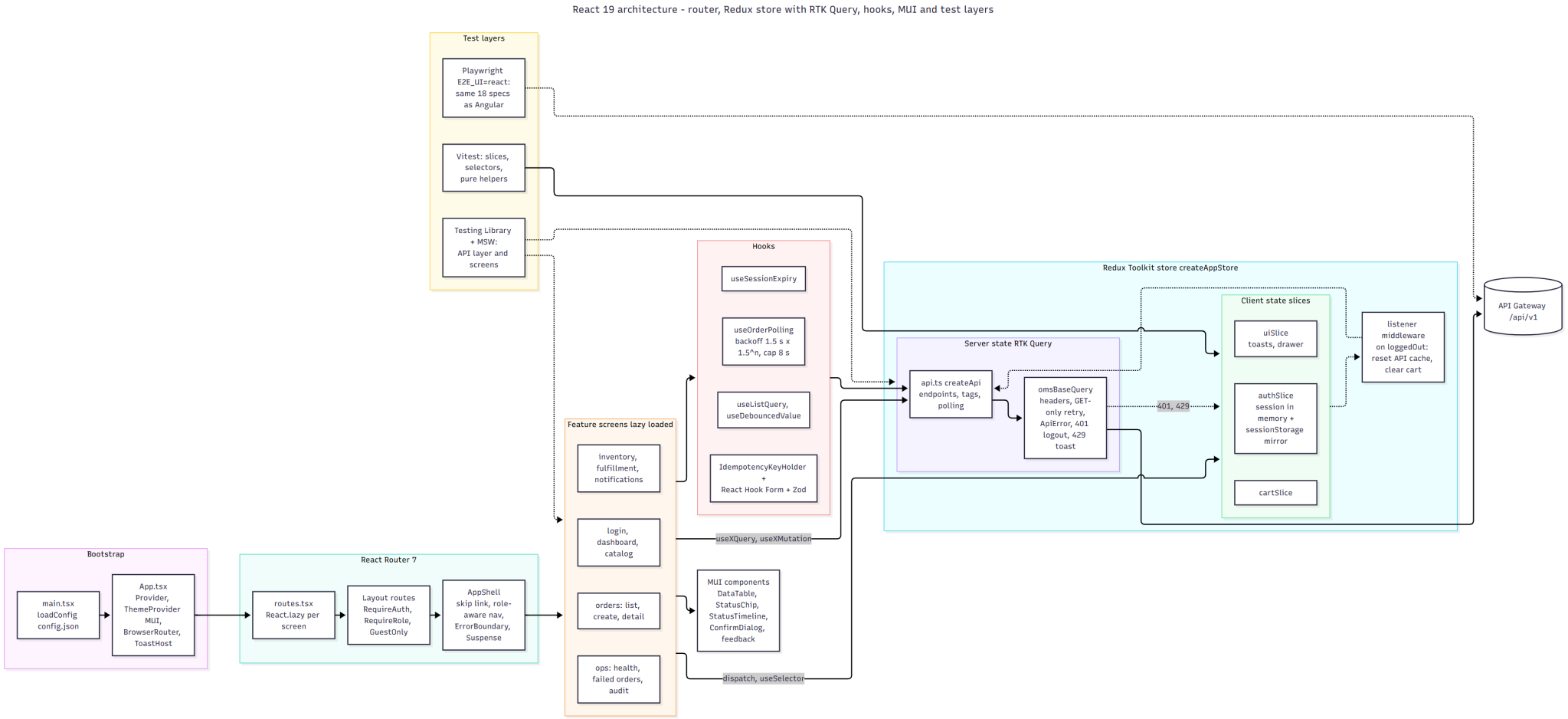


Figure 4. React architecture: router, Redux store with RTK Query, hooks, MUI and test layers

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Source: `frontend/oms-ui-react/src/routes.tsx` *(lines 31 to 45)*

```
      <Route element={<RequireAuth />}>
        <Route path="dashboard" element={<Dashboard />} />
        <Route path="catalog" element={<Catalog />} />
        <Route path="orders" element={<OrderList />} />
        <Route path="orders/new" element={<OrderCreate />} />
        <Route path="orders/:id" element={<OrderDetail />} />
        <Route path="notifications" element={<Notifications />} />
        <Route element={<RequireRole roles={STAFF} />}>
          <Route path="inventory" element={<Inventory />} />
          <Route path="fulfillment" element={<FulfillmentView />} />
          <Route path="failed-orders" element={<FailedOrders />} />
          <Route path="ops" element={<OpsDashboard />} />
          <Route path="audit" element={<Audit />} />
        </Route>
      </Route>
```

React state-management design

The React app splits state by who owns the truth, which is the central design decision of ADR-0013.

State	Owner	Mechanism	Rule
Server state: orders, products, stock, reservations, fulfillments, notifications, audit, ops, health	the backend	RTK Query cache (`createApi`), keyed by endpoint and arguments, tags for invalidation, polling for order detail	never copy server data into a slice; mutations invalidate tags; the cache is dropped on logout
Client state: session, cart, toasts, drawer	the browser	`createSlice` slices (`auth`, `cart`, `ui`), pure helper functions, `createSelector` selectors	serializable only; the session is mirrored to `sessionStorage` by middleware; reducers stay pure
Screen-local state: filters, page, sort, dialog open	the component	`useListQuery` (reducer), `useState`, React Hook Form with Zod	serialised into RTK Query arguments so a late response cannot overwrite a newer query
One logical submission	the order form	`IdempotencyKeyHolder`	same `Idempotency-Key` only after an unknown outcome with an unchanged body

Two side effects deserve a sentence each. `omsBaseQuery` is the single place that attaches the correlation id and bearer token (never on `/auth/token`), retries only GET and HEAD on network errors and 502, 503 and 504, converts failures to a serializable `ApiError`, logs out on a 401 and toasts on a 429. A listener middleware reacts to `loggedOut` by resetting the whole RTK Query cache, and the cart slice clears itself on the same action, so the next user of a tab sees nothing of the previous session.

Source: `frontend/oms-ui-react/src/store/index.ts` *(lines 27 to 33)*

```
const listener = createListenerMiddleware();
listener.startListening({
  actionCreator: loggedOut,
  effect: (_a, api) => {
    api.dispatch(omsApi.util.resetApiState());
  },
});
```

Source: `frontend/oms-ui-react/src/store/api.ts` *(lines 130 to 138)*

```
    createOrder: b.mutation<CreateOrderResult, { body: CreateOrderRequest; key: string }>({
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
      query: ({ body, key }) => ({ url: '/orders', method: 'POST', body, headers: { 'Idempotency-Key':
key } }),
      transformResponse: (body: Order, meta) => ({
        order: body,
        replay: meta?.response?.headers.get('Idempotent-Replay') === 'true',
        location: meta?.response?.headers.get('Location') ?? null,
      }),
      invalidatesTags: ['Orders'],
    }),
```

Angular and React side by side

Concern	React app (frontend/oms-ui-react)	Angular app (frontend/oms-ui)
Global client state	Redux Toolkit slices `authSlice`, `cartSlice`, `uiSlice`	signal stores `AuthStore`, `CartStore` (`providedIn: 'root'`)
Derived state	`createSelector`	`computed()`
Server state and caching	RTK Query: cache, tags, polling	hand-rolled `ListStore` and `OrderDetailStore` with RxJS `switchMap`; no cache
Polling with backoff	`pollingInterval` recomputed per completed fetch (`useOrderPolling`)	`pollUntil` (`expand` and `timer`) with `takeUntil`
HTTP cross-cutting concerns	`omsBaseQuery` (headers, GET retry, errors, 401, 429)	interceptor chain correlation, auth, retry, error
Routing and guards	React Router layout routes `RequireAuth`, `RequireRole`, `GuestOnly`; `React.lazy`	`canActivate` guards; `loadComponent`
Forms	React Hook Form with Zod schemas	typed Reactive Forms with validator functions
Error containment	`ErrorBoundary` per route	global `ErrorHandler`
UI kit	MUI	Angular Material
Unit and component tests	Vitest, Testing Library, MSW	Karma, Jasmine, `HttpTestingController`
Change propagation	re-render, memoized selectors	signals, zone-based change detection (no `OnPush`)
Idempotent order submit	`IdempotencyKeyHolder` in `orderForm.ts`	`IdempotencyKeyHolder` in `order-form.ts`

Dual-UI testing matrix

The same `tests/e2e/specs/order-lifecycle.spec.ts` (18 serial tests) runs against both UIs in two modes. `E2E_UI` (angular by default, or react) selects the UI served by the mock backend and the live URL.

UI	Mock backend (contract only)	Live stack (UI to gateway to real services on embedded Kafka and H2)
Angular	18 / 18	18 / 18
React	18 / 18	18 / 18

All four configurations were executed on 2026-10-19; the live stack used the gateway on port 18085 (`OMS_GATEWAY_PORT`) because a foreign process held 18080. The unit layers differ per framework and are not shared:

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Layer	Angular	React
Unit and component	149 of 149 (Karma and Jasmine)	207 of 207 (Vitest), coverage 96.3 % statements, 93.5 % branches, 94.5 % functions, 96.7 % lines
Static checks	`ng build`	ESLint 0 errors, `tsc -b` strict, `vite build`, `npm audit` 0 vulnerabilities
Not executed	accessibility audit, browsers other than Chrome	Docker image, compose profile `react`, Kubernetes manifest, CI and Jenkins runs, axe and Lighthouse, screen reader, narrow-screen visual check

Source: `tests/e2e/playwright.config.ts` *(lines 16 to 23)*

```
const ui = (process.env['E2E_UI'] ?? 'angular').toLowerCase();
if (ui !== 'angular' && ui !== 'react') throw new Error(`E2E_UI must be 'angular' or 'react' (got
'${ui}')`);
const DEFAULT_UI_DIR =
  ui === 'react'
    ? path.resolve(__dirname, '../../frontend/oms-ui-react/dist')
    : path.resolve(__dirname, '../../frontend/oms-ui/dist/oms-ui/browser');
const live = !!process.env['E2E_BASE_URL'];
const mockPort = process.env['E2E_MOCK_PORT'] ?? '4300';
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. REST API design

Conventions (all services)

Topic	Rule	Where implemented
Base path and versioning	`/api/v1`, additive-only changes inside v1 (ADR-0007)	Controllers, `docs/architecture/api-versioning.md`
Errors	RFC 7807 `application/problem+json` with `code`, `correlationId`, `errors[]`; no stack traces or SQL	`services/platform-common/src/main/java/com/enterprise/oms/common/error/GlobalExceptionHandler.java`
Correlation	`X-Correlation-Id` accepted or generated, echoed, placed in MDC	`common/web` filter in platform-common, gateway `EdgeFilters`
Idempotency	`Idempotency-Key` (8 to 64 chars) required on `POST /orders`	`services/order-service/src/main/java/com/enterprise/oms/order/service/OrderCommandService.java`
Pagination	`page` (0-based), `size` default 20 max 100, whitelisted `sort`; invalid values are 400, never silently clamped	platform-common paging helpers
Money and time	`BigDecimal` with scale 2, `USD`; ISO-8601 UTC timestamps	DTO records
Limits	JSON body max 64 KB (413), `POST /orders` rate limit per caller (429 + `Retry-After`)	gateway `RequestSizeLimitWebFilter`, order `ratelimit` package

Endpoint groups

Service	Examples	Roles
order	`POST /api/v1/orders`, `GET /api/v1/orders/{id}`, `POST /api/v1/orders/{id}/cancel`, `GET /api/v1/orders/{id}/history`	owner, OPERATOR, ADMIN
order admin	`GET /api/v1/admin/orders/failed`, `POST /api/v1/admin/orders/{id}/retry`, `GET /api/v1/admin/saga/{orderId}`, `POST /api/v1/admin/reconciliation/run`	OPERATOR, ADMIN
inventory	`GET /api/v1/products`, `PUT /api/v1/inventory/{productId}/stock`, `GET /api/v1/inventory/reservations`	any authenticated, OPERATOR, ADMIN
internal	`POST /api/v1/internal/products/validate`	SERVICE only, not routed by the gateway
fulfillment	`GET /api/v1/fulfillments`, `POST /api/v1/admin/fulfillments/{id}/retry`	OPERATOR, ADMIN
notification	`GET /api/v1/notifications`, `POST /api/v1/admin/notifications/{id}/retry`	customer (own), OPERATOR, ADMIN
operations	`GET /api/v1/ops/messaging`, `GET /api/v1/ops/audit`, gateway `GET /api/v1/ops/health`	OPERATOR, ADMIN

The complete, authoritative table is section 5 of `docs/architecture/00-shared-contract.md`; OpenAPI exports live in `specs/openapi/`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Design decisions worth defending

202 Accepted for order creation. The order is persisted and a saga row created in one transaction, then the API answers `202` with a `Location` header. Validation, reservation and fulfillment continue asynchronously. The client polls `GET /orders/{id}`. This keeps the request fast and honest: the order is accepted, not yet fulfilled. The Angular UI uses `pollUntil` with exponential backoff (`frontend/oms-ui/src/app/core/state/polling.ts`).

Idempotency key semantics. The scope is the `customerId` of the request body; the fingerprint is a SHA-256 over a canonical form (customer, lines sorted by product, trimmed address), so property order, whitespace and line order do not matter. Same key and same fingerprint returns the original `202` body with header `Idempotent-Replay: true`. Same key and a different fingerprint returns `422 IDEMPOTENCY_KEY_REUSED`. A request that fails validation or authorisation does not consume its key. Keys are retained for 24 hours (`oms.idempotency.retention`) and cleaned by `IdempotencyKeyCleanupJob`.

404 instead of 403 for other customers' orders. Returning 404 prevents order-id enumeration. Role-based URL rules run before request binding, so an unauthorised caller receives 403 before 400. This is the BOLA defence (see the threat model and the code-review exercise 02 in `docs/code-review/`).

Problem details. One error shape everywhere means the UI has one `errorInterceptor` (`frontend/oms-ui/src/app/core/http/interceptors.ts`) that turns it into an `ApiError`.

Optimistic concurrency on edits. `PUT /products/{id}` accepts an optional `version`; a mismatch returns `409 CONFLICT`. Orders carry a `version` column used for optimistic locking.

Example: create order and replay

```
POST /api/v1/orders
Authorization: Bearer <JWT>
Idempotency-Key: demo-key-0001
Content-Type: application/json

{"customerId":"c0000000-0000-0000-0000-000000000001",
 "items":[{"productId":"a0000000-0000-0000-0000-000000001001","quantity":2}],
 "shippingAddress":{"line1":"1 Main St","city":"Springfield","postalCode":"12345","country":"US"}}

HTTP/1.1 202 Accepted
Location: /api/v1/orders/<orderId>
```

Replaying the same request returns the same body plus `Idempotent-Replay: true`. Captured responses from a live stack are collected in `docs/api/examples.md` when the integration run has produced them.

Source: `services/order-service/src/main/java/com/enterprise/oms/order/api/OrderController.java` *(lines 68 to 97)*

```
    @PostMapping
    @PreAuthorize(ROLES)
    @Operation(summary = "Create an order (asynchronous, idempotent)",
            description = "Returns 202 with the order in status CREATED; the saga continues in the
background. "
                    + "The Idempotency-Key header (8-64 chars A-Za-z0-9_-) is required: replaying the
same key with "
                    + "the same body returns the original response with 'Idempotent-Replay: true'; the
same key with "
                    + "a different body is 422 IDEMPOTENCY_KEY_REUSED. A CUSTOMER can only order for
their own "
                    + "customerId (403 otherwise). Rate limited per caller (429 + Retry-After).")
    @ApiResponse(responseCode = "202", description = "Accepted; Location points to the order")
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    @ApiResponse(responseCode = "400", description = "VALIDATION_FAILED or IDEMPOTENCY_KEY_REQUIRED",
content = @Content)
    @ApiResponse(responseCode = "403", description = "customerId does not belong to the caller",
content = @Content)
    @ApiResponse(responseCode = "404", description = "Unknown customer", content = @Content)
    @ApiResponse(responseCode = "422", description = "IDEMPOTENCY_KEY_REUSED", content = @Content)
    @ApiResponse(responseCode = "429", description = "RATE_LIMITED", content = @Content)
    public ResponseEntity<OrderView> create(
            @Parameter(description = "8-64 chars of A-Za-z0-9_-", required = true)
            @RequestHeader(name = "Idempotency-Key", required = false) String idempotencyKey,
            @Valid @RequestBody CreateOrderRequest request) {
        CurrentUser user = CurrentUser.require();
        if (rateLimitProps.isEnabled()) {
            OrderRateLimiter.Decision d = rateLimiter.tryAcquire(user.id());
            if (!d.allowed()) {
                throw new OrderRateLimitedException(d.retryAfterSeconds());
            }
        }
        OrderCommandService.CreateResult result = commands.create(request, idempotencyKey, user);
        ResponseEntity.BodyBuilder response = ResponseEntity.accepted()
                .location(URI.create("/api/v1/orders/" + result.order().id()));
        if (result.replay()) {
            response.header("Idempotent-Replay", "true");
```

What is not done (honest list)

- No HATEOAS links: the API is resource-oriented REST at maturity level 2; hypermedia was judged low value for a UI that already knows the routes.
- No ETag or conditional GET; versions are explicit fields instead.
- Pagination is offset-based. Keyset pagination is recommended for very deep paging and is discussed in the database chapter of the interview guide.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 5. REST order creation: authorization, idempotency and async acceptance

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Captured API examples

Status: REAL. Every request/response below was produced by `node scripts/capture-api-examples.mjs` against the local no-Docker stack (`scripts/run-local.sh`: embedded Kafka, 4 services, gateway) on 2026-10-18, through the gateway at `http://localhost:18080`. Nothing is hand-written. Bearer tokens are redacted (`<redacted>`), the demo password is a placeholder, ids are the values generated in that run (orders are created fresh on every run, so ids differ each time; seed ids are fixed). Long arrays are shortened (marked). All data is synthetic.

Re-generate: start the stack, then `OMS_DEMO_PASSWORD=<printed value> OMS_JWT_SECRET=<only needed for the expired-token example> node scripts/capture-api-examples.mjs`.

Base path `/api/v1`. Common headers: `Authorization: Bearer <JWT>`, `Content-Type: application/json`, optional `X-Correlation-Id` (echoed back, generated when absent). Error shape (RFC 7807, `application/problem+json`): `type, title, status, detail, instance, code, correlationId, timestamp, errors[]`.

Item	Value
customer1 (Acme Industrial Supply)	`c0000000-0000-0000-0000-000000000001`
customer2 (Globex Logistics)	`c0000000-0000-0000-0000-000000000002`
SKU-1001 (stock 500)	`a0000000-0000-0000-0000-000000001001`
SKU-1019 (stock 3)	`a0000000-0000-0000-0000-000000001019`
SKU-1020 (stock 0)	`a0000000-0000-0000-0000-000000001020`
Demo users	`customer1`, `customer2` (CUSTOMER), `operator1` (OPERATOR), `admin1` (ADMIN)

1. Login (dev token endpoint, gateway profile dev only)

The password is the per-run random value printed by `scripts/run-local.sh`; it is replaced by a placeholder below.

Request

```
POST /api/v1/auth/token
Content-Type: application/json

{
  "username": "customer1",
  "password": "<OMS_DEMO_PASSWORD>"
}
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 1a3da4ef-8082-43f2-b939-df6ce430622c

{
  "accessToken": "<redacted>",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "username": "customer1",
  "roles": [
    "CUSTOMER"
  ],
  "customerId": "c0000000-0000-0000-0000-000000000001"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

2. Login with a wrong password: 401

Request

```
POST /api/v1/auth/token
Content-Type: application/json

{
  "username": "customer1",
  "password": "wrong-password"
}
```

Response: **HTTP 401**

```
content-type: application/problem+json
www-authenticate: Bearer
x-correlation-id: 1c2e7166-e71d-4080-b437-600992757e0d

{
  "type": "https://oms.example/problems/unauthenticated",
  "title": "Unauthorized",
  "status": 401,
  "detail": "Invalid credentials.",
  "instance": "/api/v1/auth/token",
  "code": "UNAUTHENTICATED",
  "correlationId": "1c2e7166-e71d-4080-b437-600992757e0d",
  "timestamp": "2026-10-18T15:37:05.907396700Z",
  "errors": []
}
```

3. Create order: success (202 Accepted)

Validation and the saga run asynchronously, so the order is returned in status `CREATED` with `totalAmount` 0.00 and unpriced items (contract section 13).

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-e3fd6717-6ca5-48c2-ac29-a178dd00e646

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "quantity": 2
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 202**

```
content-type: application/json
location: /api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde
x-correlation-id: 03f49000-8886-44e9-aa26-02d8700cc13c

{
  "id": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "CREATED",
  "statusReason": null,
  "totalAmount": 0,
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "sku": null,
      "name": null,
      "quantity": 2,
      "unitPrice": null,
      "lineTotal": null
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c",
  "createdAt": "2026-10-18T15:37:05.922899500Z",
  "updatedAt": "2026-10-18T15:37:05.922899500Z",
  "version": 0
}
```

4. Idempotent replay (same key, same body): 202 + Idempotent-Replay: true

Returns the ORIGINAL snapshot (status `CREATED`), not the order's current state.

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-e3fd6717-6ca5-48c2-ac29-a178dd00e646

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "quantity": 2
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 202**

```
content-type: application/json
location: /api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde
idempotent-replay: true
x-correlation-id: 5a379164-188d-47ef-a442-373cef23a98f

{
  "id": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "CREATED",
  "statusReason": null,
  "totalAmount": 0,
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "sku": null,
      "name": null,
      "quantity": 2,
      "unitPrice": null,
      "lineTotal": null
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c",
  "createdAt": "2026-10-18T15:37:05.922899500Z",
  "updatedAt": "2026-10-18T15:37:05.922899500Z",
  "version": 0
}
```

5. Idempotency key reused with a different body: 422

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-e3fd6717-6ca5-48c2-ac29-a178dd00e646

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "quantity": 3
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 422**

```
content-type: application/problem+json
x-correlation-id: 761b0d3c-3e5a-4a34-8138-b6fa8f9ae1c2

{
  "type": "https://oms.example/problems/idempotency-key-reused",
  "title": "Idempotency key reused with a different request",
  "status": 422,
  "detail": "Idempotency-Key was already used with a different request body",
  "instance": "/api/v1/orders",
  "code": "IDEMPOTENCY_KEY_REUSED",
  "correlationId": "761b0d3c-3e5a-4a34-8138-b6fa8f9ae1c2",
  "timestamp": "2026-10-18T15:37:05.996353600Z",
  "errors": []
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Missing Idempotency-Key: 400 IDEMPOTENCY_KEY_REQUIRED

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "quantity": 2
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 400**

```
content-type: application/problem+json
x-correlation-id: 50b33952-2a12-4eb8-9b8e-7f4f87b3220d

{
  "type": "https://oms.example/problems/idempotency-key-required",
  "title": "Idempotency-Key header required",
  "status": 400,
  "detail": "Header Idempotency-Key is required (8-64 chars of A-Z a-z 0-9 _ -)",
  "instance": "/api/v1/orders",
  "code": "IDEMPOTENCY_KEY_REQUIRED",
  "correlationId": "50b33952-2a12-4eb8-9b8e-7f4f87b3220d",
  "timestamp": "2026-10-18T15:37:06.012296300Z",
  "errors": []
}
```

7. Validation error: 400 VALIDATION_FAILED

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-16fab17c-3a88-4cae-87ef-c18a5c977a05

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "quantity": 0
    }
  ],
  "shippingAddress": {
    "line1": "",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 400**

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
content-type: application/problem+json
x-correlation-id: cbb52cc1-374a-414a-a96e-3559835d36bf

{
  "type": "https://oms.example/problems/validation-failed",
  "title": "Validation failed",
  "status": 400,
  "detail": "Validation failed",
  "instance": "/api/v1/orders",
  "code": "VALIDATION_FAILED",
  "correlationId": "cbb52cc1-374a-414a-a96e-3559835d36bf",
  "timestamp": "2026-10-18T15:37:06.170365100Z",
  "errors": [
    {
      "field": "items[0].quantity",
      "message": "must be greater than or equal to 1"
    },
    {
      "field": "shippingAddress.line1",
      "message": "must not be blank"
    }
  ]
}
```

8. Get order after the saga finished (FULFILLED)

Request

```
GET /api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: cfb77661-a862-4e11-9ddd-52bfd330271f

{
  "id": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "FULFILLED",
  "statusReason": null,
  "totalAmount": 49,
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "sku": "SKU-1001",
      "name": "Industrial Safety Gloves (12 pair)",
      "quantity": 2,
      "unitPrice": 24.5,
      "lineTotal": 49
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c",
  "createdAt": "2026-10-18T15:37:05.922900Z",
  "updatedAt": "2026-10-18T15:37:13.644259Z",
  "version": 3
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9. Order status history

Request

```
GET /api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde/history
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 1329d63b-e0dc-4638-a865-516719dc523e

[
  {
    "fromStatus": null,
    "toStatus": "CREATED",
    "reason": "Order created",
    "actor": "customer1",
    "occurredAt": "2026-10-18T15:37:05.927897Z"
  },
  {
    "fromStatus": "CREATED",
    "toStatus": "VALIDATED",
    "reason": null,
    "actor": "saga",
    "occurredAt": "2026-10-18T15:37:05.964709Z"
  },
  "... 3 more item(s) omitted"
]
```

10. Cross-customer access: 404 (not 403, to avoid enumeration)

Request

```
GET /api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde
Authorization: Bearer <redacted>
```

Response: **HTTP 404**

```
content-type: application/problem+json
x-correlation-id: bcb5db0a-7147-4c33-85fe-e33d29034051

{
  "type": "https://oms.example/problems/not-found",
  "title": "Resource not found",
  "status": 404,
  "detail": "Order 34b7449a-7ad4-44ac-952b-a8cd27391cde not found",
  "instance": "/api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde",
  "code": "NOT_FOUND",
  "correlationId": "bcb5db0a-7147-4c33-85fe-e33d29034051",
  "timestamp": "2026-10-18T15:37:14.126513300Z",
  "errors": []
}
```

11. Cancel a FULFILLED order: 409 INVALID_STATE_TRANSITION

Request

```
POST /api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde/cancel
Authorization: Bearer <redacted>
Content-Type: application/json

{
  "reason": "too late"
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Response: **HTTP 409**

```
content-type: application/problem+json
x-correlation-id: bacd92d7-b0a7-41b9-87c4-bb1cf06ccba4

{
  "type": "https://oms.example/problems/invalid-state-transition",
  "title": "Invalid state transition",
  "status": 409,
  "detail": "Order cannot transition from FULFILLED to CANCELLED",
  "instance": "/api/v1/orders/34b7449a-7ad4-44ac-952b-a8cd27391cde/cancel",
  "code": "INVALID_STATE_TRANSITION",
  "correlationId": "bacd92d7-b0a7-41b9-87c4-bb1cf06ccba4",
  "timestamp": "2026-10-18T15:37:14.175863100Z",
  "errors": []
}
```

12. Insufficient stock, step 1: accepted (202)

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: it-d75461d5-a165-4044-a787-2e668a0a8d8a

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001020",
      "quantity": 1
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 202**

```
content-type: application/json
location: /api/v1/orders/f88862ee-7774-4323-b61d-d74be7e5bcd6
x-correlation-id: c49989f6-6c76-4068-ad79-ce063df7b97e

{
  "id": "f88862ee-7774-4323-b61d-d74be7e5bcd6",
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "CREATED",
  "statusReason": null,
  "totalAmount": 0,
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001020",
      "sku": null,
      "name": null,
      "quantity": 1,
      "unitPrice": null,
      "lineTotal": null
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "c49989f6-6c76-4068-ad79-ce063df7b97e",
  "createdAt": "2026-10-18T15:37:14.216874500Z",
  "updatedAt": "2026-10-18T15:37:14.216874500Z",
  "version": 0
}
```

13. Insufficient stock, step 2: the order is REJECTED (SKU-1020 has stock 0)

Request

```
GET /api/v1/orders/f88862ee-7774-4323-b61d-d74be7e5bcd6
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 9ad44a6b-e2f6-43da-8181-0f220dcf4b38

{
  "id": "f88862ee-7774-4323-b61d-d74be7e5bcd6",
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "REJECTED",
  "statusReason": "Validation failed: SKU-1020: INSUFFICIENT_STOCK",
  "totalAmount": 0,
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001020",
      "sku": null,
      "name": null,
      "quantity": 1,
      "unitPrice": null,
      "lineTotal": null
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "c49989f6-6c76-4068-ad79-ce063df7b97e",
  "createdAt": "2026-10-18T15:37:14.216875Z",
  "updatedAt": "2026-10-18T15:37:14.321845Z",
  "version": 1
}
```

14. Customer creating an order for another customer: 403

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-ba854cec-f440-469c-a2e6-4aec9b01feb8

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001001",
      "quantity": 2
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
}
```

Response: **HTTP 403**

```
content-type: application/problem+json
x-correlation-id: 797f18cd-34b7-4cd1-9083-1a2a82f1ff0f

{
  "type": "https://oms.example/problems/forbidden",
  "title": "Forbidden",
  "status": 403,
  "detail": "customerId does not match the authenticated customer",
  "instance": "/api/v1/orders",
  "code": "FORBIDDEN",
  "correlationId": "797f18cd-34b7-4cd1-9083-1a2a82f1ff0f",
  "timestamp": "2026-10-18T15:37:14.972511600Z",
  "errors": []
}
```

15. Cancel an in-flight order: success (200)

Allowed from CREATED, VALIDATED, INVENTORY_RESERVED and CONFIRMED; a CONFIRMED order passes through COMPENSATION_PENDING while the reservation is released.

Request

```
POST /api/v1/orders/d89893a3-2546-4b24-970d-f382fac1bbef/cancel
Authorization: Bearer <redacted>
Content-Type: application/json

{
  "reason": "Ordered by mistake"
}
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 8bfa1366-7987-4202-8963-fa451b212ba5

{
  "id": "d89893a3-2546-4b24-970d-f382fac1bbef",
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "COMPENSATION_PENDING",
  "statusReason": "Ordered by mistake",
  "totalAmount": 129.99,
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001005",
      "sku": "SKU-1005",
      "name": "Cordless Drill 18V",
      "quantity": 1,
      "unitPrice": 129.99,
      "lineTotal": 129.99
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "467ac446-f0c8-484d-89ec-987a12634730",
  "createdAt": "2026-10-18T15:37:15.015620Z",
  "updatedAt": "2026-10-18T15:37:15.685281600Z",
  "version": 3
}
```


16. Failed fulfillment (shipping city FAILTOWN) ends FAILED, stock released

Final state reached FAILED; the history passes CONFIRMED > COMPENSATION_PENDING > FAILED.

Request

```
GET /api/v1/orders/200a13f4-be5a-449d-801f-741f7126c425
Authorization: Bearer <redacted>
```

Response: **HTTP 200**


```
content-type: application/json
x-correlation-id: 55e25dae-e1e5-47b5-9aef-f524a33a7739

{
  "id": "200a13f4-be5a-449d-801f-741f7126c425",
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "status": "FAILED",
  "statusReason": "Fulfillment failed: Shipping address not serviceable: city FAILTOWN (attempts 3)",
  "totalAmount": 89,
  "currency": "USD",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001004",
      "sku": "SKU-1004",
      "name": "Steel Toe Work Boots",
      "quantity": 1,
      "unitPrice": 89,
      "lineTotal": 89
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "FAILTOWN",
    "postalCode": "12345",
    "country": "US"
  },
  "correlationId": "4801a573-2c86-4469-b404-ebfd9eebee19",
  "createdAt": "2026-10-18T15:37:16.883116Z",
  "updatedAt": "2026-10-18T15:37:23.364526Z",
  "version": 4
}
```


17. Failed order status history

Request

```
GET /api/v1/orders/200a13f4-be5a-449d-801f-741f7126c425/history
Authorization: Bearer <redacted>
```

Response: **HTTP 200**


```
content-type: application/json
x-correlation-id: 2b7c9937-205c-4088-9a92-1fde6ac01a85

[
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  {
    "fromStatus": null,
    "toStatus": "CREATED",
    "reason": "Order created",
    "actor": "customer1",
    "occurredAt": "2026-10-18T15:37:16.889235Z"
  },
  {
    "fromStatus": "CREATED",
    "toStatus": "VALIDATED",
    "reason": null,
    "actor": "saga",
    "occurredAt": "2026-10-18T15:37:16.920830Z"
  },
  "... 4 more item(s) omitted"
]
```

18. Customer product list (paged, availability only)

Request

```
GET /api/v1/products?size=2&sort=sku,asc
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: cf0204c2-f866-42af-b7ac-a5b00b1c692f

{
  "content": [
    {
      "id": "a0000000-0000-0000-0000-000000001001",
      "sku": "SKU-1001",
      "name": "Industrial Safety Gloves (12 pair)",
      "description": "Cut-resistant work gloves, bulk pack",
      "category": "Safety",
      "price": 24.5,
      "currency": "USD",
      "active": true,
      "available": 496
    },
    {
      "id": "a0000000-0000-0000-0000-000000001002",
      "sku": "SKU-1002",
      "name": "Hard Hat, Class E",
      "description": "ANSI rated hard hat with ratchet suspension",
      "category": "Safety",
      "price": 18.75,
      "currency": "USD",
      "active": true,
      "available": 318
    }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 20,
  "totalPages": 10
}
```

19. Operator inventory item (stock fields visible)

Request

```
GET /api/v1/inventory/a0000000-0000-0000-0000-000000001019
Authorization: Bearer <redacted>
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 2a902f1c-3a34-4900-ab72-606a2b3f0522

{
  "id": "a0000000-0000-0000-0000-000000001019",
  "sku": "SKU-1019",
  "name": "Forklift Battery Charger",
  "description": "Industrial 24V charger (limited stock)",
  "category": "Material Handling",
  "price": 1450,
  "currency": "USD",
  "active": true,
  "onHand": 0,
  "reserved": 0,
  "available": 0,
  "version": 0
}
```

20. Customer cannot read inventory: 403

Request

```
GET /api/v1/inventory
Authorization: Bearer <redacted>
```

Response: **HTTP 403**

```
content-type: application/problem+json
x-correlation-id: 5a0bf29c-8f9e-4a9c-8443-2bb417505fb4

{
  "type": "https://oms.example/problems/forbidden",
  "title": "Forbidden",
  "status": 403,
  "detail": "You are not allowed to perform this operation.",
  "instance": "/api/v1/inventory",
  "code": "FORBIDDEN",
  "correlationId": "5a0bf29c-8f9e-4a9c-8443-2bb417505fb4",
  "timestamp": "2026-10-18T15:37:23.831568200Z",
  "errors": []
}
```

21. Stock adjustment below reserved/on-hand: 409 CONFLICT

Request

```
PUT /api/v1/inventory/a0000000-0000-0000-0000-000000001019/stock
Authorization: Bearer <redacted>
Content-Type: application/json

{
  "delta": -100000,
  "reason": "invalid adjustment demo"
}
```

Response: **HTTP 409**

```
content-type: application/problem+json
x-correlation-id: ee3c4c72-eb33-4ee3-a59a-682ae7db36a0

{
  "type": "https://oms.example/problems/conflict",
  "title": "Conflict",
  "status": 409,
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "detail": "Resulting on-hand -100000 would be below the 0 units already reserved for orders",
  "instance": "/api/v1/inventory/a0000000-0000-0000-0000-000000001019/stock",
  "code": "CONFLICT",
  "correlationId": "ee3c4c72-eb33-4ee3-a59a-682ae7db36a0",
  "timestamp": "2026-10-18T15:37:23.866748400Z",
  "errors": []
}
```


22. Saga of an order (operator)

Request

```
GET /api/v1/admin/saga/200a13f4-be5a-449d-801f-741f7126c425
Authorization: Bearer <redacted>
```

Response: **HTTP 200**


```
content-type: application/json
x-correlation-id: a419cdef-44cf-4126-8a04-0728dd6d5828

{
  "orderId": "200a13f4-be5a-449d-801f-741f7126c425",
  "sagaId": "6fa0a211-0e3d-4e35-a350-894c30ec9488",
  "state": "FAILED",
  "currentStep": "RELEASE_INVENTORY",
  "attempts": 0,
  "startedAt": "2026-10-18T15:37:16.884628Z",
  "updatedAt": "2026-10-18T15:37:23.368181Z",
  "lastError": null,
  "steps": [
    {
      "step": "VALIDATE",
      "status": "STARTED",
      "attempt": 0,
      "detail": "order accepted",
      "at": "2026-10-18T15:37:16.884628Z"
    },
    {
      "step": "VALIDATE",
      "status": "SUCCEEDED",
      "attempt": 1,
      "detail": "priced 1 line(s), total 89.00",
      "at": "2026-10-18T15:37:16.923865Z"
    },
    "... 8 more item(s) omitted"
  ]
}
```


23. Retry of a terminal order: 409

Request

```
POST /api/v1/admin/orders/200a13f4-be5a-449d-801f-741f7126c425/retry
Authorization: Bearer <redacted>
```

Response: **HTTP 409**


```
content-type: application/problem+json
x-correlation-id: 1664af7c-ee53-4606-9590-cf80e663e2e6

{
  "type": "https://oms.example/problems/invalid-state-transition",
  "title": "Invalid state transition",
  "status": 409,
  "detail": "Order cannot transition from FAILED to RETRY",
  "instance": "/api/v1/admin/orders/200a13f4-be5a-449d-801f-741f7126c425/retry",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "code": "INVALID_STATE_TRANSITION",
  "correlationId": "1664af7c-ee53-4606-9590-cf80e663e2e6",
  "timestamp": "2026-10-18T15:37:23.959299100Z",
  "errors": []
}
```


24. Failed-orders list (operator)

Request

```
GET /api/v1/admin/orders/failed?size=2
Authorization: Bearer <redacted>
```

Response: **HTTP 200**


```
content-type: application/json
x-correlation-id: c6a1e038-8709-4001-be9b-7cf151a24215

{
  "content": [
    {
      "id": "200a13f4-be5a-449d-801f-741f7126c425",
      "customerId": "c0000000-0000-0000-0000-000000000001",
      "status": "FAILED",
      "statusReason": "Fulfillment failed: Shipping address not serviceable: city FAILTOWN (attempts
3)",
      "totalAmount": 89,
      "currency": "USD",
      "items": [
        {
          "productId": "a0000000-0000-0000-0000-000000001004",
          "sku": "SKU-1004",
          "name": "Steel Toe Work Boots",
          "quantity": 1,
          "unitPrice": 89,
          "lineTotal": 89
        }
      ],
      "shippingAddress": {
        "line1": "1 Test Street",
        "city": "FAILTOWN",
        "postalCode": "12345",
        "country": "US"
      },
      "correlationId": "4801a573-2c86-4469-b404-ebfd9eebee19",
      "createdAt": "2026-10-18T15:37:16.883116Z",
      "updatedAt": "2026-10-18T15:37:23.364526Z",
      "version": 4
    },
    {
      "id": "f88862ee-7774-4323-b61d-d74be7e5bcd6",
      "customerId": "c0000000-0000-0000-0000-000000000001",
      "status": "REJECTED",
      "statusReason": "Validation failed: SKU-1020: INSUFFICIENT_STOCK",
      "totalAmount": 0,
      "currency": "USD",
      "items": [
        {
          "productId": "a0000000-0000-0000-0000-000000001020",
          "sku": null,
          "name": null,
          "quantity": 1,
          "unitPrice": null,
          "lineTotal": null
        }
      ],
      "shippingAddress": {
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
        "line1": "1 Test Street",
        "city": "Springfield",
        "postalCode": "12345",
        "country": "US"
      },
      "correlationId": "c49989f6-6c76-4068-ad79-ce063df7b97e",
      "createdAt": "2026-10-18T15:37:14.216875Z",
      "updatedAt": "2026-10-18T15:37:14.321845Z",
      "version": 1
    }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 35,
  "totalPages": 18
}
```


25. Fulfillment of the failed order (operator)

Request

```
GET /api/v1/fulfillments?orderId=200a13f4-be5a-449d-801f-741f7126c425
Authorization: Bearer <redacted>
```

Response: **HTTP 200**


```
content-type: application/json
x-correlation-id: e350884f-dde3-4166-b4a7-c599936d4293

{
  "content": [
    {
      "id": "794308d5-2afa-4675-9a7b-f92a4afb078a",
      "orderId": "200a13f4-be5a-449d-801f-741f7126c425",
      "status": "FAILED",
      "trackingNumber": null,
      "attempts": 3,
      "failureReason": "Shipping address not serviceable: city FAILTOWN",
      "events": [
        {
          "status": "PENDING",
          "at": "2026-10-18T15:37:17.934289Z"
        },
        {
          "status": "FAILED",
          "at": "2026-10-18T15:37:22.749023Z"
        }
      ],
      "createdAt": "2026-10-18T15:37:17.934289Z",
      "updatedAt": "2026-10-18T15:37:22.749023Z"
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1
}
```


26. Notifications of the order (customer, own only; recipient address never returned)

Request

```
GET /api/v1/notifications?orderId=34b7449a-7ad4-44ac-952b-a8cd27391cde&size=2
Authorization: Bearer <redacted>
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 0b662205-848f-4109-b8d4-dd8581179347

{
  "content": [
    {
      "id": "5bab6ef3-3752-47d3-9231-7bdbc8076306",
      "customerId": "c0000000-0000-0000-0000-000000000001",
      "orderId": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
      "channel": "EMAIL",
      "template": "ORDER_FULFILLED",
      "subject": "Order 34b7449a delivered - tracking OMS-50DFE420109C",
      "status": "SENT",
      "attempts": 1,
      "lastError": null,
      "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c",
      "sourceEventId": "d89dc13a-6701-4c6a-9580-b84a344bf50b",
      "createdAt": "2026-10-18T15:37:14.148832Z",
      "sentAt": "2026-10-18T15:37:14.260254Z"
    },
    {
      "id": "42b89708-b050-4f70-a24c-48cbbfcbfb54",
      "customerId": "c0000000-0000-0000-0000-000000000001",
      "orderId": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
      "channel": "EMAIL",
      "template": "FULFILLMENT_COMPLETED",
      "subject": "Order 34b7449a has shipped - tracking OMS-50DFE420109C",
      "status": "SENT",
      "attempts": 1,
      "lastError": null,
      "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c",
      "sourceEventId": "2c6d06c6-6bcb-4b4a-a3c5-2eea627073a9",
      "createdAt": "2026-10-18T15:37:13.634557Z",
      "sentAt": "2026-10-18T15:37:13.699281Z"
    }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 5,
  "totalPages": 3
}
```

27. Audit entries of an order (operator)

Request

```
GET /api/v1/ops/order/audit?entityType=Order&entityId=34b7449a-7ad4-44ac-952b-a8cd27391cde&size=2
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: f6f462f3-eb5a-439e-bfa8-89787e865500

{
  "content": [
    {
      "id": "1af33d5a-89ab-4f5c-973c-21765349e760",
      "occurredAt": "2026-10-18T15:37:13.646274Z",
      "actor": "saga",
      "entityType": "Order",
      "entityId": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
      "action": "STATUS_FULFILLED",
      "detail": "{\"from\":\"CONFIRMED\",\"to\":\"FULFILLED\",\"trigger\":\"FULFILLMENT_COMPLETED\"}",
      "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c"
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
    },
    {
      "id": "3ceebef5-c2a2-497f-a011-f782a289ed9b",
      "occurredAt": "2026-10-18T15:37:06.521947Z",
      "actor": "saga",
      "entityType": "Order",
      "entityId": "34b7449a-7ad4-44ac-952b-a8cd27391cde",
      "action": "STATUS_CONFIRMED",
      "detail": "{\"from\":\"INVENTORY_RESERVED\",\"to\":\"CONFIRMED\",\"trigger\":\"CONFIRM\"}",
      "correlationId": "03f49000-8886-44e9-aa26-02d8700cc13c"
    }
  ],
  "page": 0,
  "size": 2,
  "totalElements": 5,
  "totalPages": 3
}
```


28. Messaging status through the gateway rewrite (operator)

Request

```
GET /api/v1/ops/order/messaging
Authorization: Bearer <redacted>
```

Response: **HTTP 200**


```
content-type: application/json
x-correlation-id: ce138552-476c-4375-a2c8-040af25bc859

{
  "service": "order-service",
  "consumerGroup": "order-service",
  "topics": [
    {
      "topic": "oms.inventory.events.v1",
      "partitions": 3,
      "consumerLag": 0,
      "committedOffsets": {
        "0": 4,
        "1": 17,
        "2": 18
      }
    },
    {
      "topic": "oms.fulfillment.events.v1",
      "partitions": 3,
      "consumerLag": 0,
      "committedOffsets": {
        "0": 4,
        "1": 19,
        "2": 20
      }
    }
  ],
  "processedEvents": 82,
  "duplicatesIgnored": 0,
  "retries": 0,
  "deadLettered": 0,
  "outboxPending": 0,
  "lastProcessedAt": "2026-10-18T15:37:23.370196700Z",
  "dltTopics": [
    "oms.inventory.events.v1.DLT",
    "oms.fulfillment.events.v1.DLT"
  ],
  "lagAvailable": true
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

29. Aggregate health (operator)

Request

```
GET /api/v1/ops/health
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 85d4d5ad-0a52-4d6a-a7b8-8a64635736a4

{
  "status": "UP",
  "services": {
    "order": {
      "status": "UP",
      "detail": null
    },
    "inventory": {
      "status": "UP",
      "detail": null
    },
    "fulfillment": {
      "status": "UP",
      "detail": null
    },
    "notification": {
      "status": "UP",
      "detail": null
    }
  }
}
```

30. Reconciliation (admin)

Request

```
POST /api/v1/admin/reconciliation/run
Authorization: Bearer <redacted>
```

Response: **HTTP 200**

```
content-type: application/json
x-correlation-id: 07bc8c65-1bfa-4518-9f1b-7c34f939f764

{
  "examined": 0,
  "repaired": 0,
  "details": []
}
```

31. Reconciliation as OPERATOR: 403

Request

```
POST /api/v1/admin/reconciliation/run
Authorization: Bearer <redacted>
```

Response: **HTTP 403**

```
content-type: application/problem+json
x-correlation-id: e4982423-b7e9-4e3a-86fc-0db9d40e6133

{
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "type": "https://oms.example/problems/forbidden",
  "title": "Forbidden",
  "status": 403,
  "detail": "You are not allowed to perform this operation.",
  "instance": "/api/v1/admin/reconciliation/run",
  "code": "FORBIDDEN",
  "correlationId": "e4982423-b7e9-4e3a-86fc-0db9d40e6133",
  "timestamp": "2026-10-18T15:37:24.199022800Z",
  "errors": []
}
```

32. Internal endpoints are not routed through the gateway: 404

Request

```
POST /api/v1/internal/products/validate
Authorization: Bearer <redacted>
Content-Type: application/json

{
  "items": []
}
```

Response: **HTTP 404**

```
content-type: application/problem+json
x-correlation-id: e0cdc950-d8f6-42d1-92f9-b34b3de94e8c

{
  "type": "https://oms.example/problems/not-found",
  "title": "Not Found",
  "status": 404,
  "detail": "No such resource.",
  "instance": "/api/v1/internal/products/validate",
  "code": "NOT_FOUND",
  "correlationId": "e0cdc950-d8f6-42d1-92f9-b34b3de94e8c",
  "timestamp": "2026-10-18T15:37:24.203021500Z",
  "errors": []
}
```

33. No token: 401

Request

```
GET /api/v1/orders
```

Response: **HTTP 401**

```
content-type: application/problem+json
www-authenticate: Bearer
x-correlation-id: 807fc995-8d37-4040-9042-aadf2aeca882

{
  "type": "https://oms.example/problems/unauthenticated",
  "title": "Unauthorized",
  "status": 401,
  "detail": "Authentication is required.",
  "instance": "/api/v1/orders",
  "code": "UNAUTHENTICATED",
  "correlationId": "807fc995-8d37-4040-9042-aadf2aeca882",
  "timestamp": "2026-10-18T15:37:24.207019300Z",
  "errors": []
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

34. Forged token (signed with a random secret): 401

Request

```
GET /api/v1/orders
Authorization: Bearer <forged JWT>
```

Response: **HTTP 401**

```
content-type: application/problem+json
www-authenticate: Bearer
x-correlation-id: 99a0d725-9ca7-4645-9287-19c4c0ddc428

{
  "type": "https://oms.example/problems/unauthenticated",
  "title": "Unauthorized",
  "status": 401,
  "detail": "Authentication is required.",
  "instance": "/api/v1/orders",
  "code": "UNAUTHENTICATED",
  "correlationId": "99a0d725-9ca7-4645-9287-19c4c0ddc428",
  "timestamp": "2026-10-18T15:37:24.217543800Z",
  "errors": []
}
```

35. Expired token (correct signature, exp in the past): 401

Request

```
GET /api/v1/orders
Authorization: Bearer <expired JWT>
```

Response: **HTTP 401**

```
content-type: application/problem+json
www-authenticate: Bearer
x-correlation-id: 7187682c-f937-48d6-853d-e236ea061ca2

{
  "type": "https://oms.example/problems/unauthenticated",
  "title": "Unauthorized",
  "status": 401,
  "detail": "Authentication is required.",
  "instance": "/api/v1/orders",
  "code": "UNAUTHENTICATED",
  "correlationId": "7187682c-f937-48d6-853d-e236ea061ca2",
  "timestamp": "2026-10-18T15:37:24.222574500Z",
  "errors": []
}
```

36. Algorithm none token: 401

Request

```
GET /api/v1/orders
Authorization: Bearer <alg=none JWT>
```

Response: **HTTP 401**

```
content-type: application/problem+json
www-authenticate: Bearer
x-correlation-id: c309dd10-6076-44d4-b399-e5b2e4881480

{
  "type": "https://oms.example/problems/unauthenticated",
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
  "title": "Unauthorized",
  "status": 401,
  "detail": "Authentication is required.",
  "instance": "/api/v1/orders",
  "code": "UNAUTHENTICATED",
  "correlationId": "c309dd10-6076-44d4-b399-e5b2e4881480",
  "timestamp": "2026-10-18T15:37:24.231555900Z",
  "errors": []
}
```

37. Payload larger than 64 KB: 413 PAYLOAD_TOO_LARGE

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-3e34e9c3-5c49-4b1f-81ac-3d48b4a8175f

<70203 bytes of JSON, truncated here>
```

Response: **HTTP 413**

```
content-type: application/problem+json
x-correlation-id: 17af2604-dc18-4fe7-a7bf-43f025b32c0e

{
  "type": "https://oms.example/problems/payload-too-large",
  "title": "Payload Too Large",
  "status": 413,
  "detail": "Request body exceeds the 65536 byte limit.",
  "instance": "/api/v1/orders",
  "code": "PAYLOAD_TOO_LARGE",
  "correlationId": "17af2604-dc18-4fe7-a7bf-43f025b32c0e",
  "timestamp": "2026-10-18T15:37:24.240612900Z",
  "errors": []
}
```

38. Rate limited: 429 RATE_LIMITED with Retry-After (POST /orders is limited per caller)

Reached by sending small valid POST /orders requests (SKU-1016 x1, each creating a real order) back-to-back as one caller until its token bucket was empty (bucket capacity 20, refill 5/s, `oms.order.rate-limit.*`).

Request

```
POST /api/v1/orders
Authorization: Bearer <redacted>
Content-Type: application/json
Idempotency-Key: ex-<fresh key>

{
  "customerId": "c0000000-0000-0000-0000-000000000001",
  "items": [
    {
      "productId": "a0000000-0000-0000-0000-000000001016",
      "quantity": 1
    }
  ],
  "shippingAddress": {
    "line1": "1 Test Street",
    "city": "Springfield",
    "postalCode": "12345",
    "country": "US"
  }
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
}
```

Response: **HTTP 429**

```
content-type: application/problem+json
retry-after: 1
x-correlation-id: 2148022e-6e5c-4cbe-91e1-892a05a2dd67

{
  "type": "https://oms.example/problems/rate-limited",
  "title": "Rate limit exceeded",
  "status": 429,
  "detail": "Too many order requests; retry after 1s",
  "instance": "/api/v1/orders",
  "code": "RATE_LIMITED",
  "correlationId": "2148022e-6e5c-4cbe-91e1-892a05a2dd67",
  "timestamp": "2026-10-18T15:37:26.024165400Z",
  "errors": []
}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Database ER diagram

One database per service

Each service owns an isolated schema (`order_db`, `inventory_db`, `fulfillment_db`, `notification_db` on one PostgreSQL instance in Compose; separate in-memory H2 per service locally). The diagrams below are derived from the Flyway migrations, not from memory: shared tables from `services/platform-common/src/main/resources/db/common/V1__outbox_processed_audit.sql` appear in every service database, service tables come from each service's `V10` migration (order-service also `V11` for saga and idempotency tables and `V12` for indexes).

Service database	Migrations	Tables
order_db	V1, V10, V11, V12 (+ V1000 dev seed)	customers, orders, order_items, order_status_history, saga_instances, saga_step_log, idempotency_keys, outbox_events, processed_events, audit_events
inventory_db	V1, V10 (+ V1000 dev seed)	products, stock, inventory_reservations, inventory_reservation_items, stock_movements, outbox_events, processed_events, audit_events
fulfillment_db	V1, V10	fulfillments, fulfillment_events, outbox_events, processed_events, audit_events
notification_db	V1, V10	order_contacts, notifications, notification_attempts, processed_events, audit_events (outbox is present from V1 but unused: the service only consumes)

Design notes

- Foreign keys exist only inside a service. Across services there are only identifiers (`orders.customer_id` is local; `order_items.product_id` is a plain UUID owned by inventory, so it carries no foreign key).
- Optimistic locking uses a `version` column on orders, saga instances, products, stock, reservations and fulfillments.
- Uniqueness encodes business invariants: one saga per order, one reservation per order, one fulfillment per order, one notification per (source event, template), one idempotency record per (scope, key), one processed-event row per (consumer, event id).
- CHECK constraints on `stock` (non-negative, reserved at most on_hand) are the last line of defence for the oversell invariant.
- Money is `DECIMAL(19,2)` or `NUMERIC(12,2)`; timestamps are `TIMESTAMP WITH TIME ZONE`; the SQL stays portable so the same Flyway files run on H2 (PostgreSQL mode) and PostgreSQL 16.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

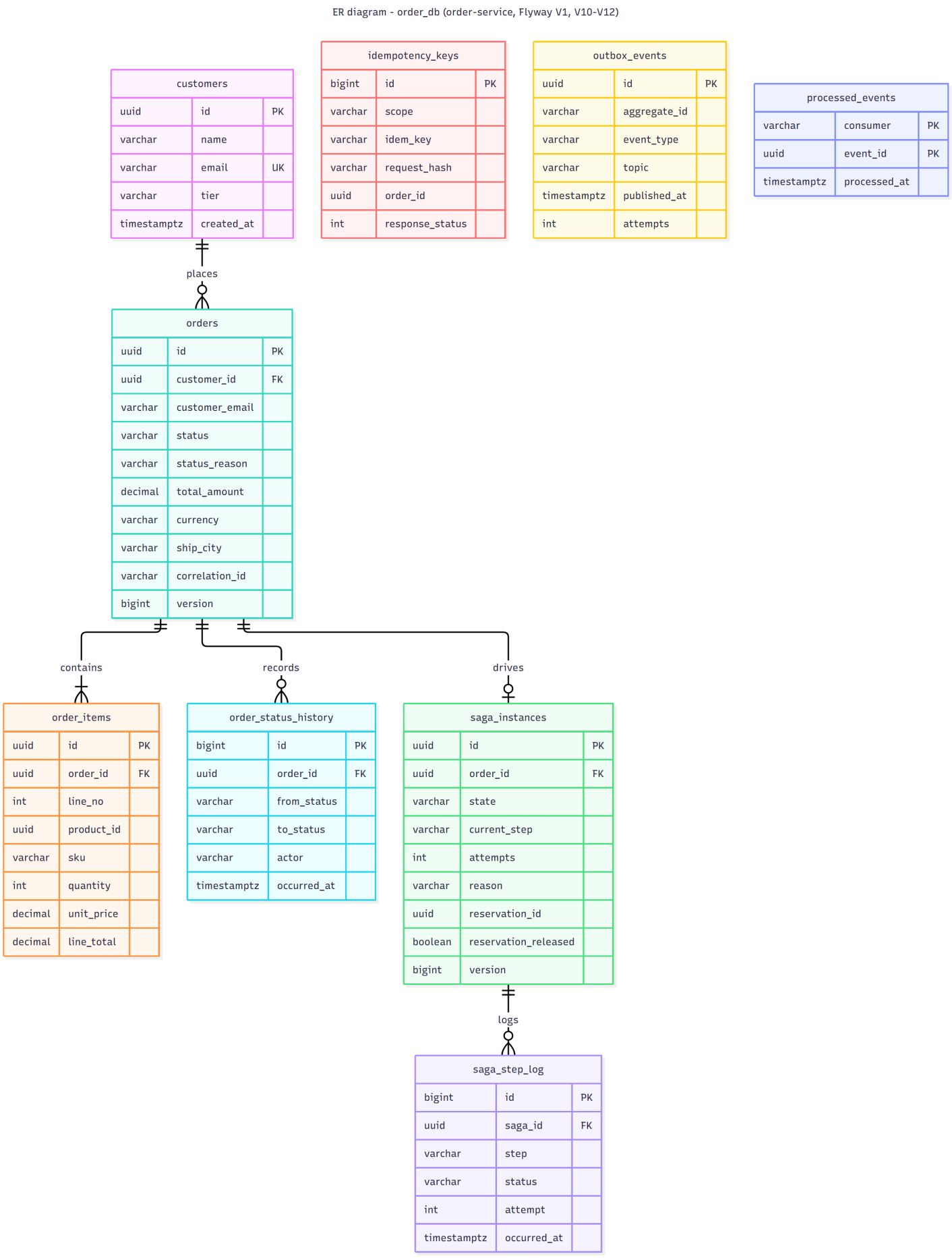


Figure 6. ER diagram: order_db

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

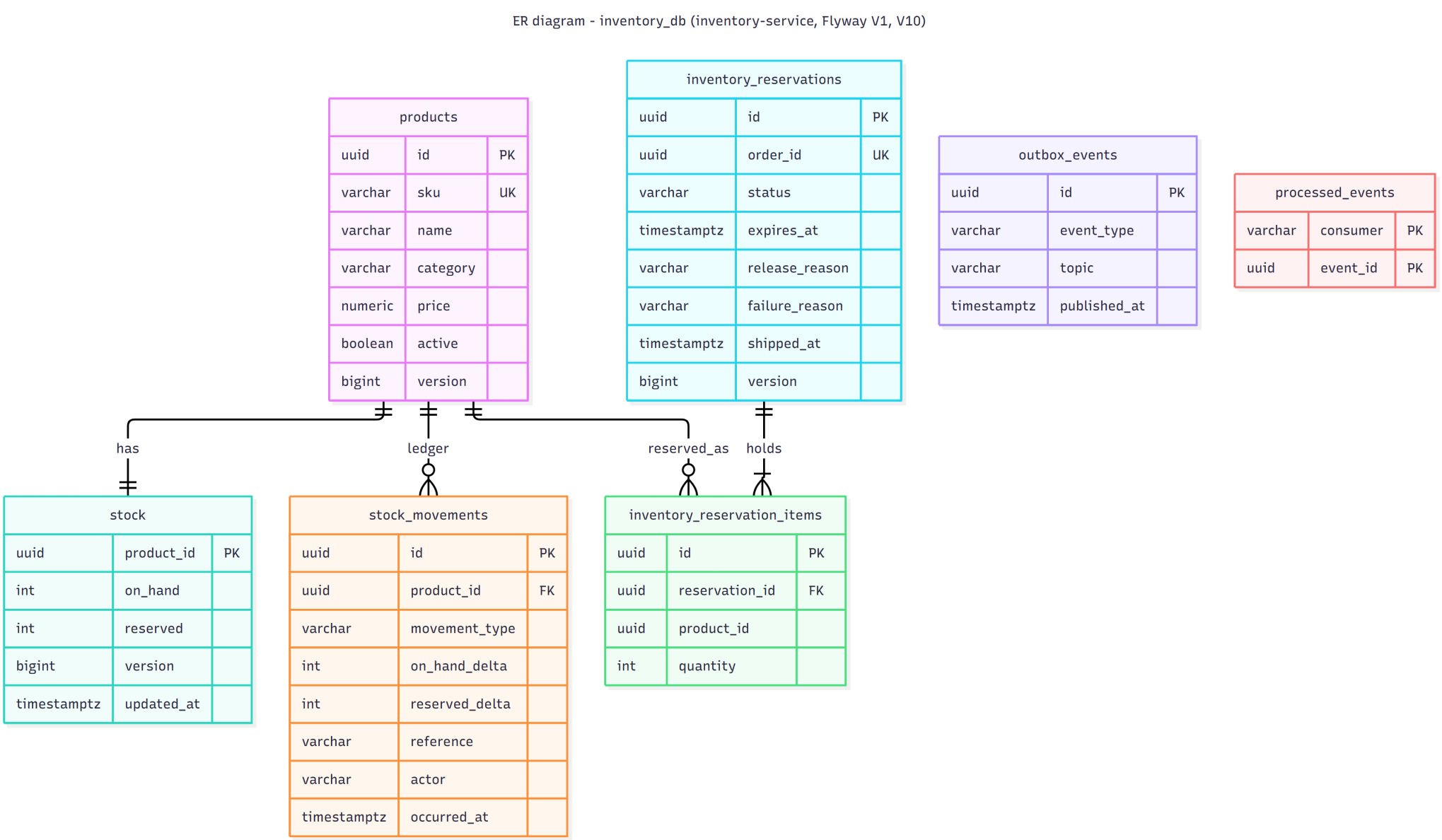


Figure 7. ER diagram: inventory_db

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

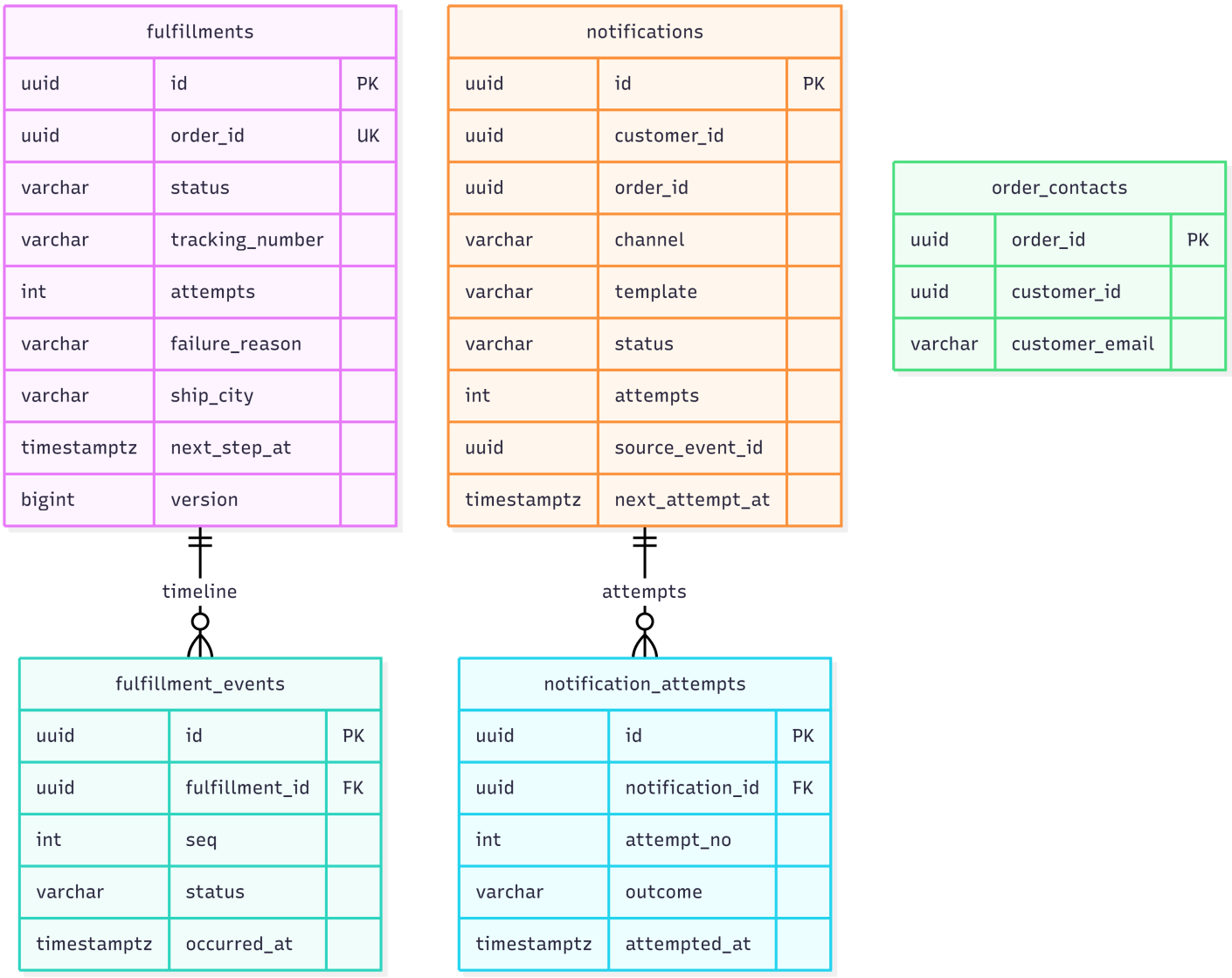


Figure 8. ER diagrams: fulfillment_db and notification_db

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Indexing highlights

Order list queries are served by composite indexes (`ix_orders_customer_status_created`, `ix_orders_status_created`), the saga recovery scan by `ix_saga_state_updated`, reservation expiry by `ix_reservations_status_expiry` (the EXPLAIN ANALYZE study in `docs/database/inventory-explain-analyze.md` reports the effect), the outbox relay by `ix_outbox_pending`, and the notification delivery job by `ix_notifications_due`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

7. Order lifecycle and state machine

Rules

The lifecycle is a pure, table-driven function `(status, trigger) -> decision` in `OrderStateMachine`: no Spring, no I/O, no clock. A decision is one of TRANSITION (valid), NO_OP (the trigger's effect is already in place: a duplicate or replayed event) or INVALID (a user command becomes 409 `INVALID_STATE_TRANSITION`; an asynchronous event is ignored and audited). Every transition writes `order_status_history`, `saga_step_log`, `audit_events` and, when relevant, an outbox event in ONE database transaction.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

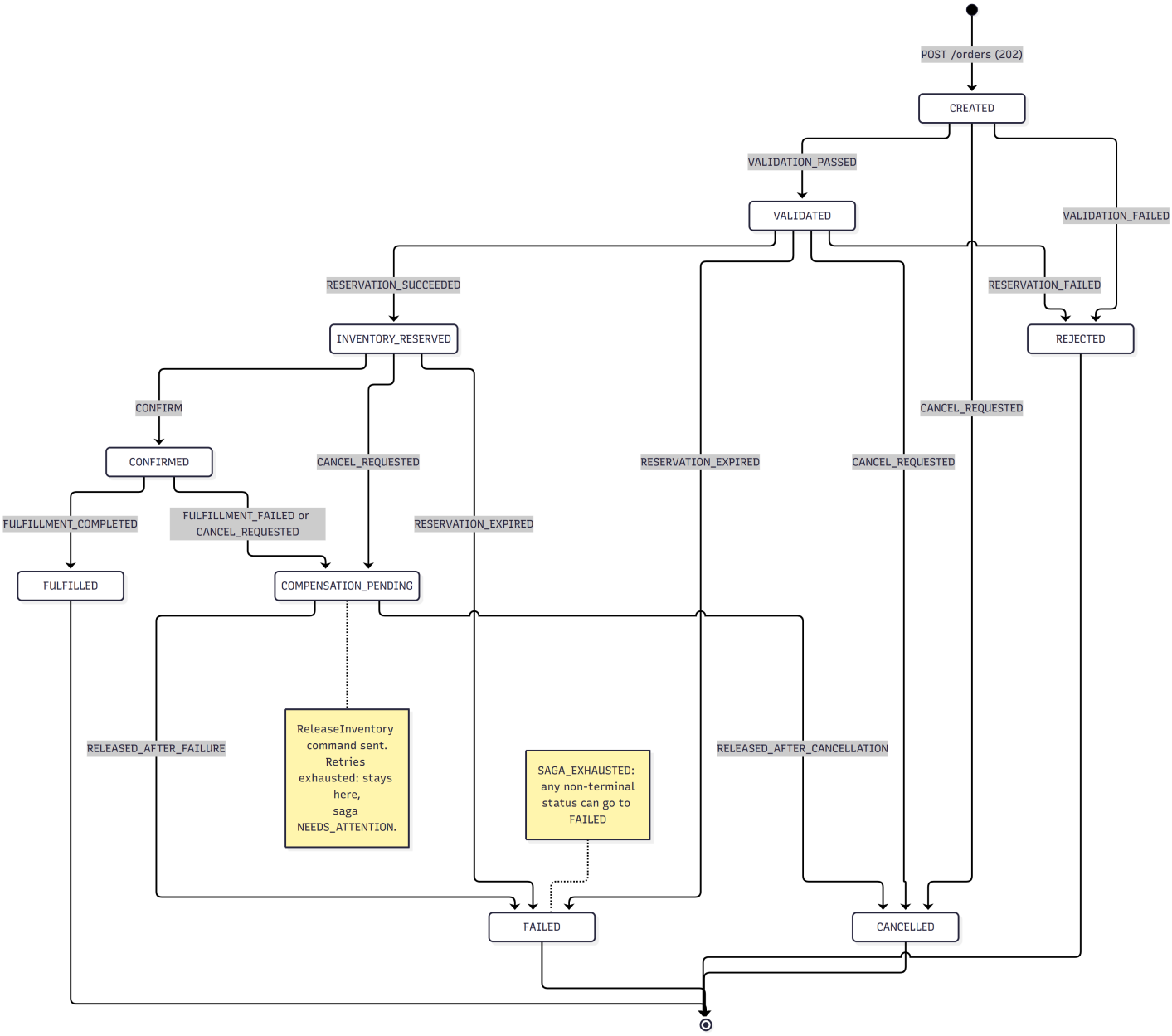


Figure 9. Order lifecycle state machine

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Terminal states are FULFILLED, REJECTED, CANCELLED and FAILED. COMPENSATION_PENDING is transitional: it means a `ReleaseInventory` command is outstanding.

Source: `services/order-service/src/main/java/com/enterprise/oms/order/domain/OrderStateMachine.java` *(lines 43 to 68)*

```
    static {
        on(OrderTrigger.VALIDATION_PASSED).move(CREATED, VALIDATED)
                .alreadyApplied(VALIDATED, INVENTORY_RESERVED, CONFIRMED, FULFILLED,
COMPENSATION_PENDING);
        on(OrderTrigger.VALIDATION_FAILED).move(CREATED, REJECTED)
                .alreadyApplied(REJECTED);
        on(OrderTrigger.RESERVATION_SUCCEEDED).move(VALIDATED, INVENTORY_RESERVED)
                .alreadyApplied(INVENTORY_RESERVED, CONFIRMED, FULFILLED, COMPENSATION_PENDING);
        on(OrderTrigger.RESERVATION_FAILED).move(VALIDATED, REJECTED)
                .alreadyApplied(REJECTED);
        on(OrderTrigger.CONFIRM).move(INVENTORY_RESERVED, CONFIRMED)
                .alreadyApplied(CONFIRMED, FULFILLED, COMPENSATION_PENDING);
        on(OrderTrigger.FULFILLMENT_COMPLETED).move(CONFIRMED, FULFILLED)
                .alreadyApplied(FULFILLED);
        on(OrderTrigger.FULFILLMENT_FAILED).move(CONFIRMED, COMPENSATION_PENDING)
                .alreadyApplied(COMPENSATION_PENDING, FAILED);
        // cancel: a command, never "already applied" (cancelling twice is a 409)
        on(OrderTrigger.CANCEL_REQUESTED)
                .move(CREATED, CANCELLED).move(VALIDATED, CANCELLED)
                .move(INVENTORY_RESERVED, COMPENSATION_PENDING).move(CONFIRMED, COMPENSATION_PENDING);
        on(OrderTrigger.RELEASED_AFTER_CANCELLATION).move(COMPENSATION_PENDING, CANCELLED)
                .alreadyApplied(CANCELLED);
        on(OrderTrigger.RELEASED_AFTER_FAILURE).move(COMPENSATION_PENDING, FAILED)
                .alreadyApplied(FAILED);
        on(OrderTrigger.RESERVATION_EXPIRED).move(VALIDATED, FAILED).move(INVENTORY_RESERVED, FAILED)
                .alreadyApplied(FAILED);
        // any non-terminal -> FAILED
```

Transition table (shared contract, section 6)

States: `CREATED, VALIDATED, INVENTORY_RESERVED, CONFIRMED, FULFILLED, REJECTED, CANCELLED, COMPENSATION_PENDING, FAILED`. Terminal: `FULFILLED, REJECTED, CANCELLED, FAILED`.

From	To	Trigger
CREATED	VALIDATED	validation OK (sync REST to inventory `/internal/products/validate`, circuit-breaker + retry)
CREATED	REJECTED	validation says invalid product/inactive/price mismatch
VALIDATED	INVENTORY_RESERVED	event `InventoryReserved`
VALIDATED	REJECTED	event `InventoryReservationFailed` (insufficient stock)
INVENTORY_RESERVED	CONFIRMED	orchestrator confirms, publishes `OrderConfirmed` + command `StartFulfillment`
CONFIRMED	FULFILLED	event `FulfillmentCompleted` → publish `OrderFulfilled`
CONFIRMED	COMPENSATION_PENDING	event `FulfillmentFailed` (after fulfillment's own retries) → command `ReleaseInventory`
CREATED, VALIDATED	CANCELLED	customer/operator cancel (if a reservation later arrives, orchestrator issues `ReleaseInventory`, order stays CANCELLED)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

From	To	Trigger
INVENTORY_RESERVED, CONFIRMED	COMPENSATION_PENDING	cancel → command `ReleaseInventory`
COMPENSATION_PENDING	CANCELLED	`InventoryReleased` and saga reason = CANCELLATION
COMPENSATION_PENDING	FAILED	`InventoryReleased` and saga reason = FULFILLMENT_FAILURE (or compensation retries exhausted → stays COMPENSATION_PENDING flagged for manual/reconciliation; admin retry available)
any non-terminal	FAILED	saga retries exhausted for non-compensable step (e.g. validation upstream down > max attempts)
Everything else → `InvalidStateTransitionException` (409). Transition logic is a pure domain class (`OrderStateMachine`) with unit tests covering every valid and invalid pair. Each transition writes `order_status_history`, `saga_step_log` , `audit_events` and (when applicable) an outbox event in ONE DB transaction. Transitions are idempotent: applying an already-applied event again is a no-op.		

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

8. Kafka event flows

Topology

Five topics with three partitions each (replication 1 locally, 3 in the production design), keyed by `orderId` so events for one order are ordered within a partition. Each failing consumer dead-letters to `<topic>.DLT`. Consumer group id equals the service name.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

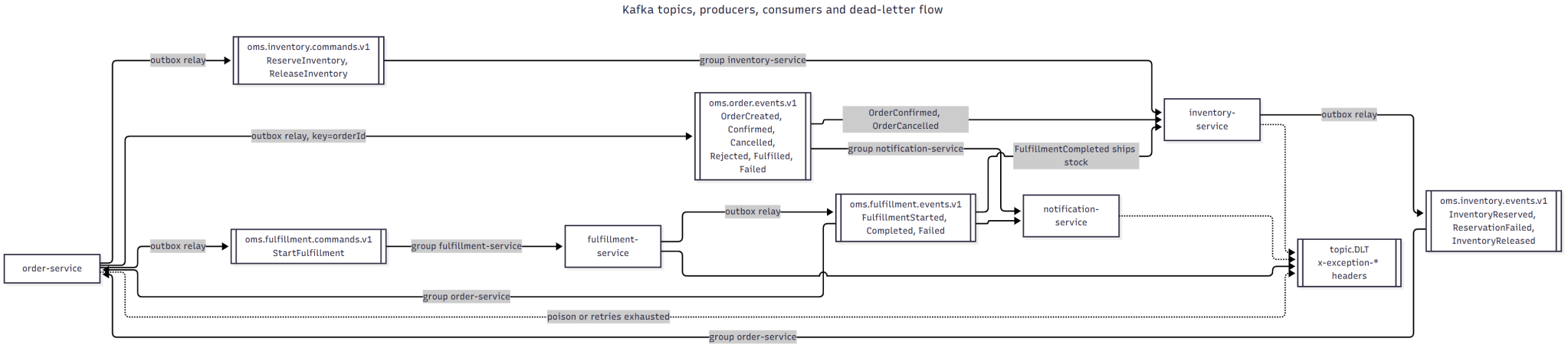


Figure 10. Kafka topics, producers, consumers and dead-letter flow

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Source: services/platform-common/src/main/java/com/enterprise/oms/common/events/Topics.java *(lines 6 to 23)*

```
public final class Topics {

    public static final String ORDER_EVENTS = "oms.order.events.v1";
    public static final String INVENTORY_COMMANDS = "oms.inventory.commands.v1";
    public static final String INVENTORY_EVENTS = "oms.inventory.events.v1";
    public static final String FULFILLMENT_COMMANDS = "oms.fulfillment.commands.v1";
    public static final String FULFILLMENT_EVENTS = "oms.fulfillment.events.v1";

    public static final String DLT_SUFFIX = ".DLT";
    public static final int PARTITIONS = 3;

    /** All business topics. */
    public static final List<String> ALL = List.of(
            ORDER_EVENTS, INVENTORY_COMMANDS, INVENTORY_EVENTS, FULFILLMENT_COMMANDS,
FULFILLMENT_EVENTS);

    /** All business topics followed by their dead-letter topics. */
    public static final List<String> ALL_WITH_DLT = ALL.stream()
            .flatMap(t -> java.util.stream.Stream.of(t, dlt(t)))
```

Contract: topics, envelope, payloads and delivery semantics

Topics (partitions 3, replication 1 locally / 3 prod; key = `orderId`):

Topic	Owner (producer)	Consumers (group)	Events
`oms.order.events.v1`	order-service	notification (`notification-service`), inventory (`inventory-service`, only `OrderConfirmed`,`OrderCancelled`)	`OrderCreated`, `OrderConfirmed`, `OrderCancelled`, `OrderRejected`, `OrderFulfilled`, `OrderFailed`
`oms.inventory.commands.v1`	order-service	inventory (`inventory-service`)	`ReserveInventory`, `ReleaseInventory`
`oms.inventory.events.v1`	inventory-service	order (`order-service`)	`InventoryReserved`, `InventoryReservationFailed`, `InventoryReleased`
`oms.fulfillment.commands.v1`	order-service	fulfillment (`fulfillment-service`)	`StartFulfillment`
`oms.fulfillment.events.v1`	fulfillment-service	order (`order-service`), inventory (`inventory-service`, `FulfillmentCompleted` → ship stock), notification	`FulfillmentStarted`, `FulfillmentCompleted`, `FulfillmentFailed`
`<topic>.DLT`	the failing consumer	operators	dead-lettered records with original headers + `x-exception-*`
Consumer group id = service name. Keys are orderId so per-order ordering holds within a partition.			

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Envelope (JSON, Jackson, class `EventEnvelope<T>` in platform-common): `{ "eventId":"uuid", "eventType":"OrderCreated", "schemaVersion":1, "occurredAt":"ISO", "correlationId":"uuid", "causationId":"uuid|null", "aggregateType":"Order", "aggregateId":"<orderId>", "producer":"order-service", "payload":{...} }` Kafka headers: `correlationId`, `eventType`, `schemaVersion`, `eventId`. Consumers use tolerant reader (`FAIL_ON_UNKNOWN_PROPERTIES=false`); unknown `eventType` → ignored + counted; malformed → DLT after non-retryable classification (no retries for deserialization/validation errors). JSON Schemas in `specs/events/*.schema.json`.

Payloads (field names are normative):

- `OrderCreated`: `{orderId, customerId, customerEmail, totalAmount, currency, items:[{productId, sku, quantity, unitPrice}], shippingCity}`
- `ReserveInventory` (command): `{orderId, items:[{productId, quantity}], reservationTtlSeconds}`; `ReleaseInventory`: `{orderId, reason}` (reason: `CANCELLATION|FULFILLMENT_FAILURE|EXPIRED`)
- `InventoryReserved`: `{orderId, reservationId, items:[{productId, quantity}], expiresAt}`; `InventoryReservationFailed`: `{orderId, reason, shortages:[{productId, requested, available}]}`; `InventoryReleased`: `{orderId, reservationId, reason}`
- `OrderConfirmed`: `{orderId, customerId, customerEmail, totalAmount}`; `StartFulfillment`: `{orderId, shippingAddress:{...}, items:[{productId, sku, quantity}]}`
- `FulfillmentStarted`: `{orderId, fulfillmentId}`; `FulfillmentCompleted`: `{orderId, fulfillmentId, trackingNumber}`; `FulfillmentFailed`: `{orderId, fulfillmentId, reason, attempts}`
- `OrderCancelled`: `{orderId, customerId, customerEmail, reason}`; `OrderRejected`: `{orderId, customerId, customerEmail, reason}`; `OrderFulfilled`: `{orderId, customerId, customerEmail, trackingNumber}`; `OrderFailed`: `{orderId, customerId, customerEmail, reason}`

Delivery semantics (document, do not overclaim): producers use the **transactional outbox** (row written in the same DB tx as state change; a relay publishes with `acks=all`, `enable.idempotence=true`, marks `published_at`) → at-least-once. Consumers are **idempotent** via `processed_events(consumer, event_id)` unique constraint inserted in the same tx as the business effect (+ state-machine idempotency) → effectively-once *business* effect. Manual-ack / `AckMode.RECORD`, `DefaultErrorHandler` with exponential backoff (4 attempts: 500ms ×2 up to 5s) then `DeadLetterPublishingRecoverer` → `<topic>.DLT`; non-retryable: deserialization, validation, `IllegalArgumentException`. Metrics (Micrometer, all services): `oms.events.processed{service, topic, type}`, `oms.events.duplicates`, `oms.events.retries`, `oms.events.dead_lettered`, `oms.outbox.pending` gauge, `oms.order.processing.duration` (timer; CREATED→terminal, order-service), `oms.saga.active` gauge, `oms.saga.compensations`.

Reservation lifecycle (inventory): `RESERVED` (counts in `reserved`, TTL `oms.inventory.reservation-ttl-seconds` default 900) → `COMMITTED` on `OrderConfirmed` (TTL cleared) → stock shipped (`on_hand -= q; reserved -= q`) on `FulfillmentCompleted`; `RELEASED` on `ReleaseInventory`/`OrderCancelled` (reserved -= q); `EXPIRED` by scheduler if still `RESERVED` after `expires_at` (reserved -= q, emits `InventoryReleased{reason:EXPIRED}`; order-service then moves an order still in VALIDATED/INVENTORY_RESERVED to FAILED). Duplicate `ReserveInventory` for the same orderId is a no-op returning the original outcome (unique `order_id` on reservations). Concurrency: atomic conditional SQL `UPDATE stock SET reserved = reserved + :q, version = version + 1 WHERE product_id = :id AND on_hand - reserved >= :q` (affected rows = 0 → insufficient) — demonstrated alongside `@Version` optimistic locking on admin stock edits (retry on `ObjectOptimisticLockingFailureException`) and a pessimistic `SELECT ... FOR UPDATE` variant documented/benchmarked in docs; items locked/updated in sorted productId order to avoid deadlocks. All-or-nothing per reservation (one tx).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Delivery semantics in depth

Decision records: ADR-0009 outbox, ADR-0005 idempotency, ADR-0002 events. Contract: section 7.

1. The three vocabularies

- At-most-once: send, never retry. Messages can be lost; no duplicates. Not acceptable for orders.
- At-least-once: retry until acknowledged. No loss (given durable storage), duplicates possible. This is what the platform provides on the wire.
- Exactly-once: each message's effect happens once. Kafka provides this only inside Kafka (idempotent producer within a session, transactions for read-process-write between topics). It cannot atomically include a PostgreSQL commit or an HTTP call to a third party. This platform does NOT claim exactly-once across database plus Kafka plus external systems.
- Effectively-once (what we claim): at-least-once delivery plus idempotent consumers, so the business effect is applied once even when the message is delivered more than once.

2. The chain

1. State change and event are written in one DB transaction: row in `outbox_events` (`OutboxService`, `services/platform-common/.../common/outbox`).
2. `OutboxRelay` publishes with `acks=all`, `enable.idempotence=true`, key = `orderId`, then marks `published_at`.
3. Consumers (`omsKafkaListenerContainerFactory`, `AckMode.RECORD`) process one record: in one DB transaction, insert `processed_events(consumer, event_id)` (unique) and apply the business effect via `IdempotentEventProcessor`, then the offset commits.
4. Failures: `DefaultErrorHandler` retries 4 times (500 ms doubling to 5 s) for transient errors; deserialization, validation and `IllegalArgumentException` are non-retryable; exhausted records are published to `<topic>.DLT` with `x-exception-*` and `x-original-*` headers by `DeadLetterPublishingRecoverer`.
5. The state machine and unique reservation per order make a repeated logical command harmless even when its `eventId` differs.

3. Failure windows

Window	What happens	Result	Signal
Crash before DB commit	Neither state nor outbox row exists	Nothing happened; client retries with the same Idempotency-Key	none needed
Crash after commit, before relay publishes	Outbox row waits	Published after restart; delayed, not lost	`oms.outbox.pending` rises
Broker down or `acks=all` not satisfied	Relay retries; row stays unpublished	Delay	outbox pending, relay errors
Broker acked, relay crashes before marking `published_at`	Row re-published	Duplicate on topic	consumers log duplicate, `oms.events.duplicates`
Producer retry inside a session	Idempotent producer dedupes by sequence	No duplicate from this cause	none
Consumer crashes after DB commit, before offset commit	Redelivery	Second delivery hits unique `processed_events` and is skipped	`oms.events.duplicates`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Window	What happens	Result	Signal
Consumer crashes before DB commit	Transaction rolled back	Redelivery applies it for the first time	`oms.events.retries`
Poison message	Non-retryable classification, straight to DLT	Others on the partition continue	`oms.events.dead_lettered`, DLT topic
Consumer processes but its own reply is lost	Saga step stuck	`SagaRecoveryJob` re-sends the idempotent command after `oms.saga.stuck-after-seconds`	`oms.saga.active`, step log attempts
Rebalance during processing	Possible duplicate processing by another instance	Unique constraint serializes: one wins, one skipped	duplicates
External call (simulated notification) succeeds but DB write fails	External effect may repeat on retry	Real providers need their own idempotency key; here simulated	`notification_attempts`
Partition order across orders	Not guaranteed	Not required; per-order ordering holds via key	

4. Honest limits

- Duplicates reach the topic by design; protection is entirely on the consumer side.
- `processed_events` and `idempotency_keys` must be retained at least as long as the maximum redelivery horizon (topic retention plus DLT replay window); purging earlier reopens duplicate risk.
- DLT replay re-injects old events; handlers must tolerate being applied late (the state machine rejects or ignores stale transitions).
- Ordering holds per order only while the key and partition count are unchanged.
- Business-level inconsistency from lost compensation is handled by recovery and reconciliation (saga-and-consistency.md), not by delivery guarantees.
- Verification status: unit and embedded-Kafka tests cover the logic locally. Tests that need Docker (Testcontainers PostgreSQL/Kafka) are skipped on the dev machine, and Compose, Kubernetes and monitoring manifests are statically validated only, so real-broker failure injection (killing a broker or relay mid-publish) has not been exercised there.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

9. Saga orchestration and compensation

Decision records: ADR-0003, ADR-0004, ADR-0009. Normative flow: contract sections 6 and 8.

1. What a saga is

A saga replaces one distributed transaction by a sequence of local transactions, each with a compensating action that semantically undoes it. There is no global rollback and no isolation across steps: intermediate states are visible, which is why orders have visible statuses and reservations have TTLs. Compensation is not "undo": a released reservation leaves a trace (`RELEASED`, `stock_movements`, audit rows).

2. Happy path (orchestrated by order-service)

Step	Who	Action	Order status after
(accept)	order-service	`POST /orders` with Idempotency-Key; write order, saga instance, outbox `OrderCreated` in one tx; reply 202	CREATED
VALIDATE	order-service to inventory (sync REST, circuit breaker + retry, SERVICE JWT)	validate products, active flag, prices	VALIDATED (or REJECTED)
RESERVE_INVENTORY	command `ReserveInventory` on `oms.inventory.commands.v1`; inventory reserves atomically (conditional UPDATE, sorted ids), emits `InventoryReserved` or `InventoryReservationFailed`		INVENTORY_RESERVED (or REJECTED)
CONFIRM_ORDER	order-service on `InventoryReserved`	publish `OrderConfirmed` and command `StartFulfillment`; inventory commits the reservation (TTL cleared)	CONFIRMED
FULFILL	fulfillment-service simulates picking, packing, shipping	emits `FulfillmentStarted`, then `FulfillmentCompleted` (stock shipped)	FULFILLED, event `OrderFulfilled`

Classes: `OrderSagaOrchestrator`, `OrderStateMachine`, `SagaRecoveryJob`, `ReconciliationService` in order-service (`services/order-service`, package `com.enterprise.oms.order`). Each transition writes `order_status_history`, `saga_step_log`, `audit_events` and any outbox event in one transaction, and applying the same event again is a no-op.

3. Compensation paths

Trigger	Sequence	End state
Insufficient stock	`InventoryReservationFailed`	REJECTED (nothing to undo; reservation transaction rolled back)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Trigger	Sequence	End state
Customer or operator cancel from CREATED/VALIDATED	cancel	CANCELLED; if a reservation arrives later the orchestrator issues `ReleaseInventory` and the order stays CANCELLED
Cancel from INVENTORY_RESERVED/ CONFIRMED	COMPENSATION_PENDING, command `ReleaseInventory{CANCELLATION}`, then `InventoryReleased`	CANCELLED
Fulfillment fails after its own retries (for example shipping city `FAILTOWN`)	`FulfillmentFailed`, COMPENSATION_PENDING, `ReleaseInventory{FULFILLMENT_FAILURE}`, `InventoryReleased`	FAILED
Reservation expires (TTL, default 900 s)	inventory scheduler emits `InventoryReleased{EXPIRED}`	order in VALIDATED/ INVENTORY_RESERVED moves to FAILED
Step never answers	`SagaRecoveryJob` re-sends the idempotent command after `oms.saga.stuck-after-seconds`; after `oms.saga.max-attempts`	FAILED if not compensable; COMPENSATION_PENDING flagged plus alert if compensable; operators use `POST /api/v1/admin/orders/{id}/retry`

4. Eventual consistency

- Between commits of different services the system is inconsistent by design: an order may be CONFIRMED while stock is already committed, or INVENTORY_RESERVED briefly after the customer clicked cancel. All such windows are bounded by retries, TTLs and recovery.
- Invariants that hold at all times inside a service (never oversell, one reservation per order, valid state transitions) are enforced by local transactions and constraints.
- Cross-service invariants hold eventually, and `ReconciliationService` checks them: order terminal but reservation not released; CONFIRMED order with no fulfillment progress; CANCELLED order with a RESERVED reservation. It repairs by issuing compensating commands, writes audit events, runs on a schedule and via `POST /api/v1/admin/reconciliation/run` (ADMIN) returning `{examined, repaired, details[]}`.

5. Trade-offs

Aspect	Gain	Cost
Local transactions only	Availability, short locks	No isolation across steps, visible intermediate states
Orchestration	Visible, recoverable flow	Orchestrator is a coordination point
Async commands over Kafka	Decoupling, backlog tolerance	Latency, duplicates, need for idempotency
Semantic compensation	Works across services	Not all actions are reversible (a sent email stays sent); notification is deliberately outside the compensable path
Reconciliation	Safety net for bugs and lost messages	Another job to run, monitor and keep correct

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. What users see

- `POST /orders` returns 202 and status `CREATED`, not "done". The Angular order detail screen polls with growing delay (`pollUntil`, `pollIntervalMs` 1500 up to `pollMaxIntervalMs` 8000) until a terminal status, shows a timeline from `GET /orders/{id}/history`, and the status reason on REJECTED or FAILED.
- Insufficient stock appears seconds later as REJECTED with a reason, not as a synchronous error.
- Cancel on a CONFIRMED order moves through COMPENSATION_PENDING before CANCELLED; the UI shows the intermediate status honestly.
- Operators see stuck or failed orders in the failed-orders screen (`GET /api/v1/admin/orders/failed`), the saga step log (`GET /api/v1/admin/saga/{orderId}`) and can retry.
- Customers get notification emails (simulated) driven by events; a failed notification never changes order state.
- Honest scope: the flow is exercised by unit and embedded-Kafka tests and by the local stack on H2; multi-instance, real-PostgreSQL and broker-failure behaviour has not been run on the dev machine because Docker is unavailable there.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Sequence diagrams

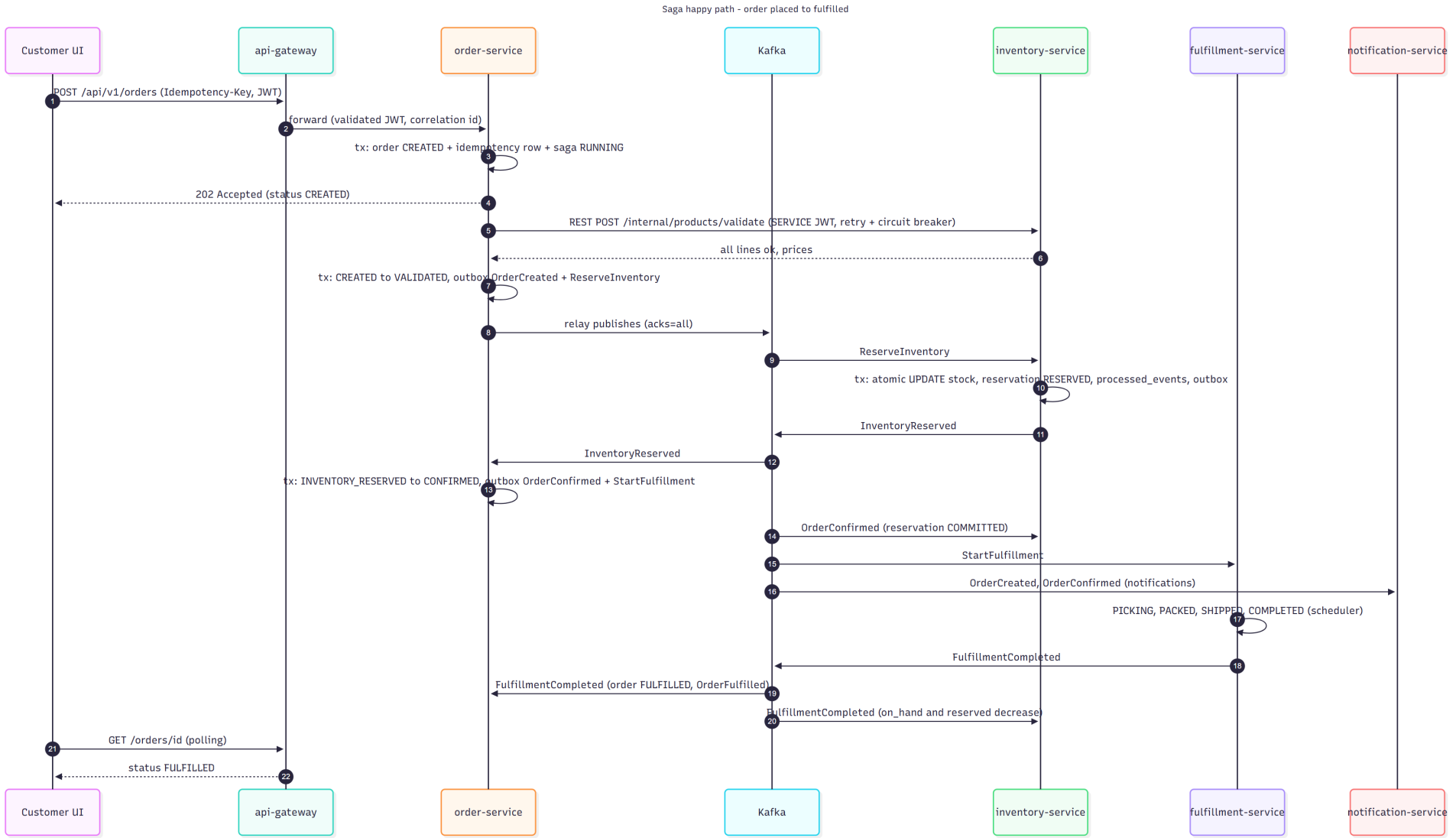


Figure 11. Saga happy path

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

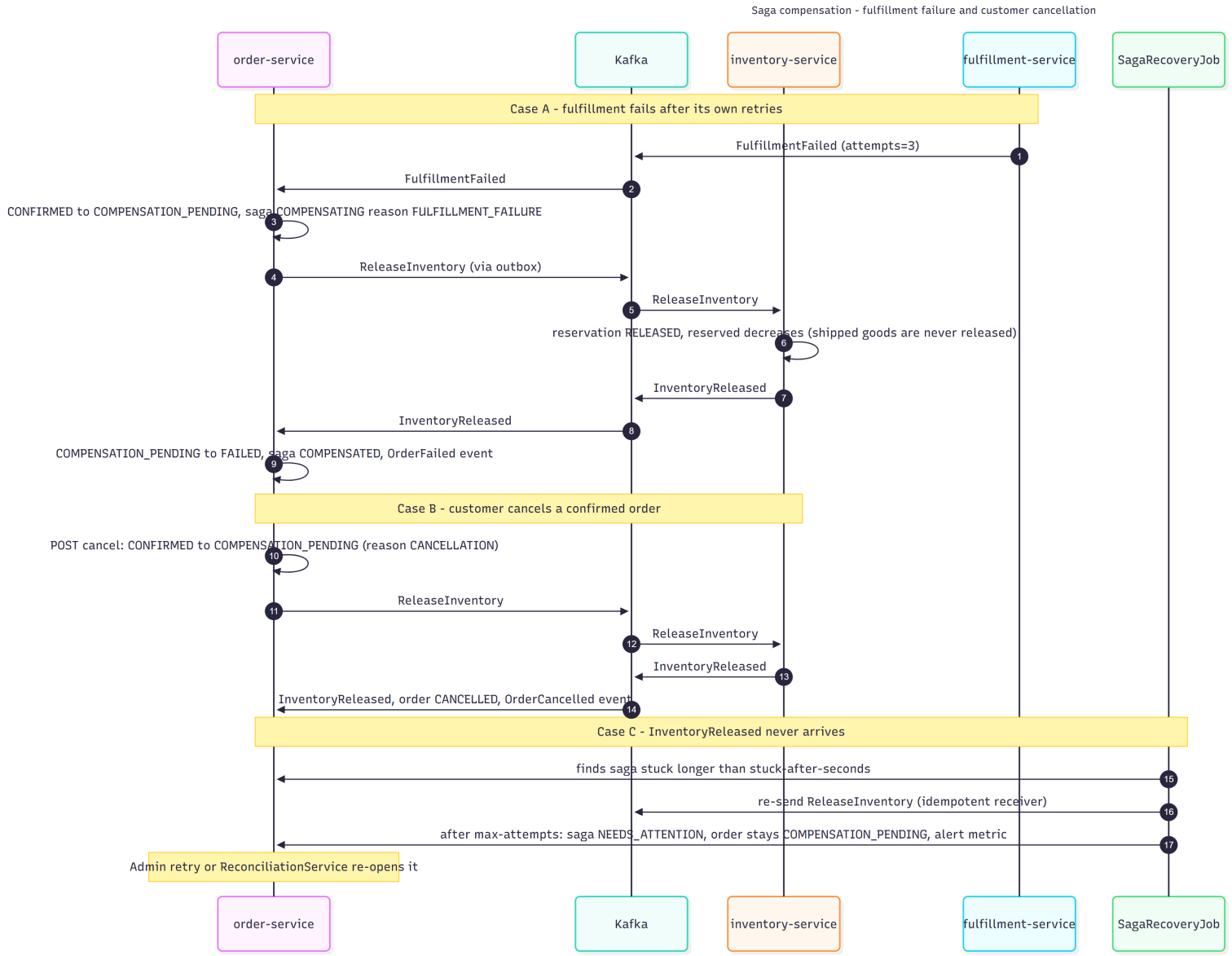


Figure 12. Saga compensation: fulfillment failure, cancellation and stuck release

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

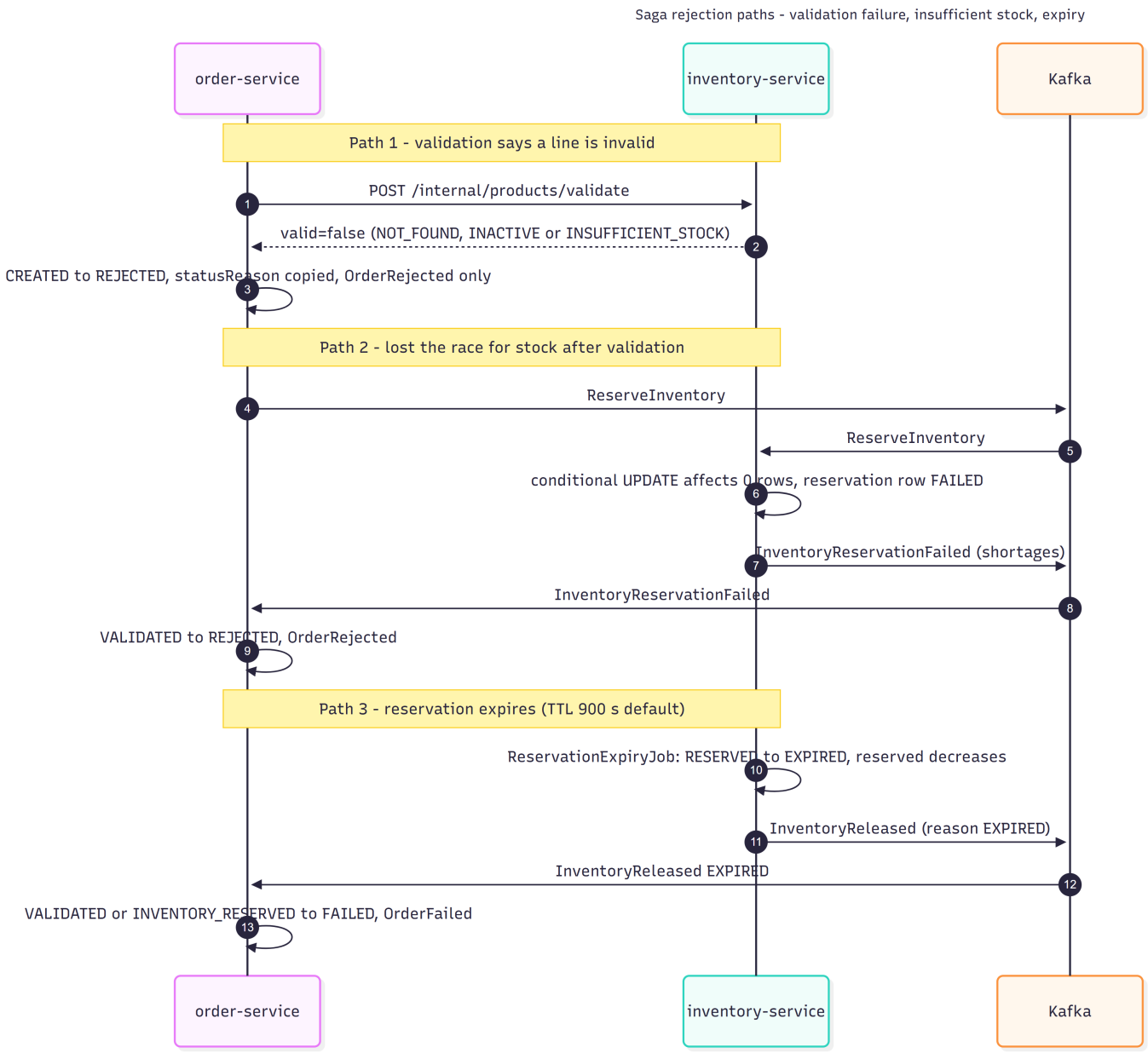


Figure 13. Saga rejection paths: validation, insufficient stock, expiry

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Contract: saga rules

- Orchestrated by `OrderSagaOrchestrator` in order-service; table `saga_instances(order_id unique, state, current_step, attempts, last_error, reason, version, timestamps)` and `saga_step_log`.
- Steps: `VALIDATE → RESERVE_INVENTORY → CONFIRM_ORDER → FULFILL`; compensation: `RELEASE_INVENTORY`.
- Retry: validation REST retry (Resilience4j, 3 attempts) + saga-level retry counter; scheduled `SagaRecoveryJob` re-drives steps stuck longer than `oms.saga.stuck-after-seconds` (default 60; tests small): re-sends the idempotent command; after `oms.saga.max-attempts` (default 5) → FAILED (non-compensable) or flagged COMPENSATION_PENDING + alert metric (compensable).
- Reconciliation (`ReconciliationService`): finds inconsistencies (order terminal but reservation not released; CONFIRMED order with no fulfillment progress; CANCELLED order with reservation RESERVED), repairs by issuing compensating commands; exposed via admin endpoint and scheduled job; results audited.
- Explicitly NOT a distributed ACID transaction: states are eventually consistent; compensations are semantic.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

10. Inventory concurrency handling

The problem

Two customers order the last three units at the same instant. A naive implementation reads the available quantity, compares it in Java, then writes the new reserved count. Both requests read the same value and both write, so more units are promised than exist (a lost update). The code-review exercise `docs/code-review/01-oversell-race.md` and the flawed-AI example `ai-engineering/flawed-ai-examples/01-check-then-act-oversell.md` show exactly this bug and its fix.

The model

Stock is split into `on_hand` (physical units) and `reserved` (promised to orders that are not shipped yet). Available units are `on_hand - reserved`. Every order owns exactly one row in `inventory_reservations` (unique `order_id`) with a lifecycle `RESERVED`, `COMMITTED`, `RELEASED`, `EXPIRED` or `FAILED`.

Event	Stock effect	Reservation status
`ReserveInventory` succeeds	`reserved += q`	`RESERVED` (TTL, default 900 s)
`ReserveInventory` short of stock	none (all-or-nothing)	`FAILED`
`OrderConfirmed`	none	`COMMITTED`, TTL cleared
`FulfillmentCompleted`	`on_hand -= q`, `reserved -= q`	stays `COMMITTED`, `shipped_at` set
`ReleaseInventory` or `OrderCancelled`	`reserved -= q`	`RELEASED`
TTL elapsed while `RESERVED`	`reserved -= q`	`EXPIRED`, `InventoryReleased` event

Strategy 1 (default): atomic conditional update

```
UPDATE stock SET reserved = reserved + :q, version = version + 1
WHERE product_id = :id AND on_hand - reserved >= :q
```

Zero affected rows means insufficient stock. The row lock taken by the UPDATE serialises concurrent writers; after waiting, the predicate is re-evaluated against the latest committed row, so overselling is impossible by construction. No read, no explicit lock, no retry loop.

Source: `services/inventory-service/src/main/java/com/enterprise/oms/inventory/reservation/AtomicUpdateReservationStrategy.java` *(lines 42 to 65)*

```
    public Result tryReserve(List<QuantityItem> lines) {
        Instant now = clock.instant();
        List<QuantityItem> taken = new ArrayList<>();
        boolean complete = true;
        for (QuantityItem line : lines) {
            if (stock.tryReserve(line.productId(), line.quantity(), now) == 1) {
                taken.add(line);
            } else {
                complete = false;
                break;
            }
        }
        if (complete) {
            return Result.ok();
        }
        for (int i = taken.size() - 1; i >= 0; i--) {
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
            QuantityItem t = taken.get(i);
            if (stock.release(t.productId(), t.quantity(), now) != 1) {
                throw new IllegalStateException("Compensation failed for product " + t.productId());
            }
        }
        return Result.short_(shortages.shortages(lines));
    }
}
```

Lines are processed in ascending product id order so two reservations with opposite item order cannot deadlock. If a later line fails, earlier lines are given back; this cannot fail or race because we still hold their row locks.

Strategy 2 and 3

Strategy	Mechanism	Strength	Weakness
`atomic` (default)	Conditional UPDATE	Simplest, fastest under contention, no retries	Business rule lives in SQL
`pessimistic`	`SELECT ... FOR UPDATE` (PESSIMISTIC_WRITE) in sorted order, then update	Easy to reason about, allows richer checks in Java	Holds locks longer, deadlock risk if order is not sorted
`optimistic`	`@Version` check, retry on conflict	No lock held while thinking	Retry storms on hot rows, needs bounded retries

`oms.inventory.reservation-strategy` selects one (`OMS_RESERVATION_STRATEGY`). All three have identical behaviour; the inventory README records a benchmark of their contention behaviour.

Defence in depth

- Database CHECK constraints in `services/inventory-service/src/main/resources/db/migration/V10__inventory_schema.sql`: both counters non-negative and `reserved <= on_hand`. If application code ever has a bug, the database refuses the write.
- Unique `order_id` makes a duplicate `ReserveInventory` return the original outcome and re-emit the same event, so redelivery is harmless.
- A release for an unknown order writes a `RELEASED` tombstone so a late or duplicate reserve can never take stock.
- `OrderConfirmed` arriving after expiry is not re-reserved; it is counted in `oms.inventory.reservation.late_commit` and left to reconciliation. Expiry versus confirm is decided by guarded UPDATEs: exactly one wins.
- Admin stock edits use optimistic `version` and reject a result below `reserved`.

Evidence

Concurrency tests live under `services/inventory-service/src/test`; their execution status for the current build is in the evidence section of this document. `performance/oversell.js` is a k6 script that hammers one product; whether it was run is stated in the validation report, not assumed.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Sequence: check-then-act versus atomic update

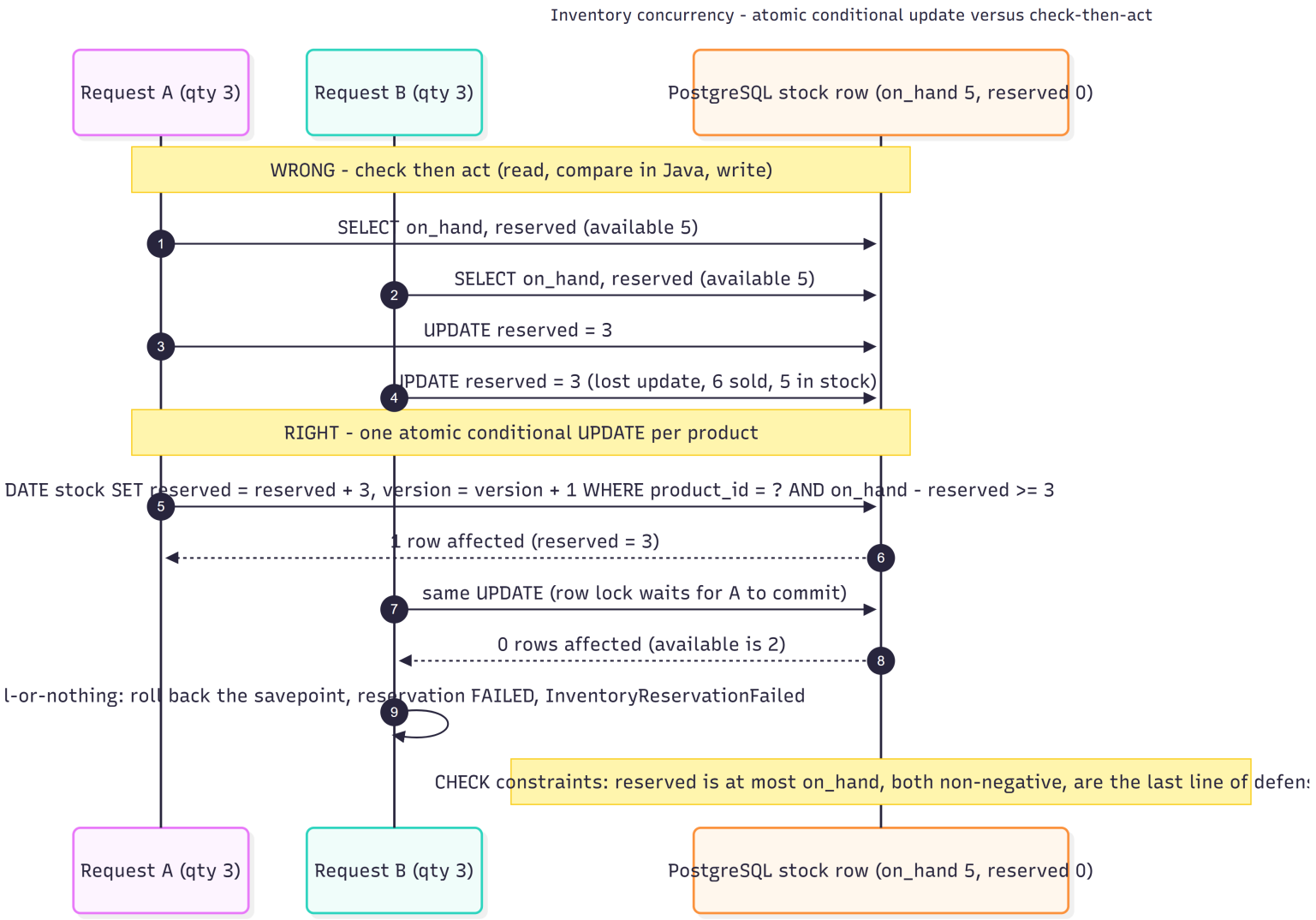


Figure 14. Inventory concurrency: lost update versus atomic conditional update

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

11. Security architecture

Authentication and authorization flow

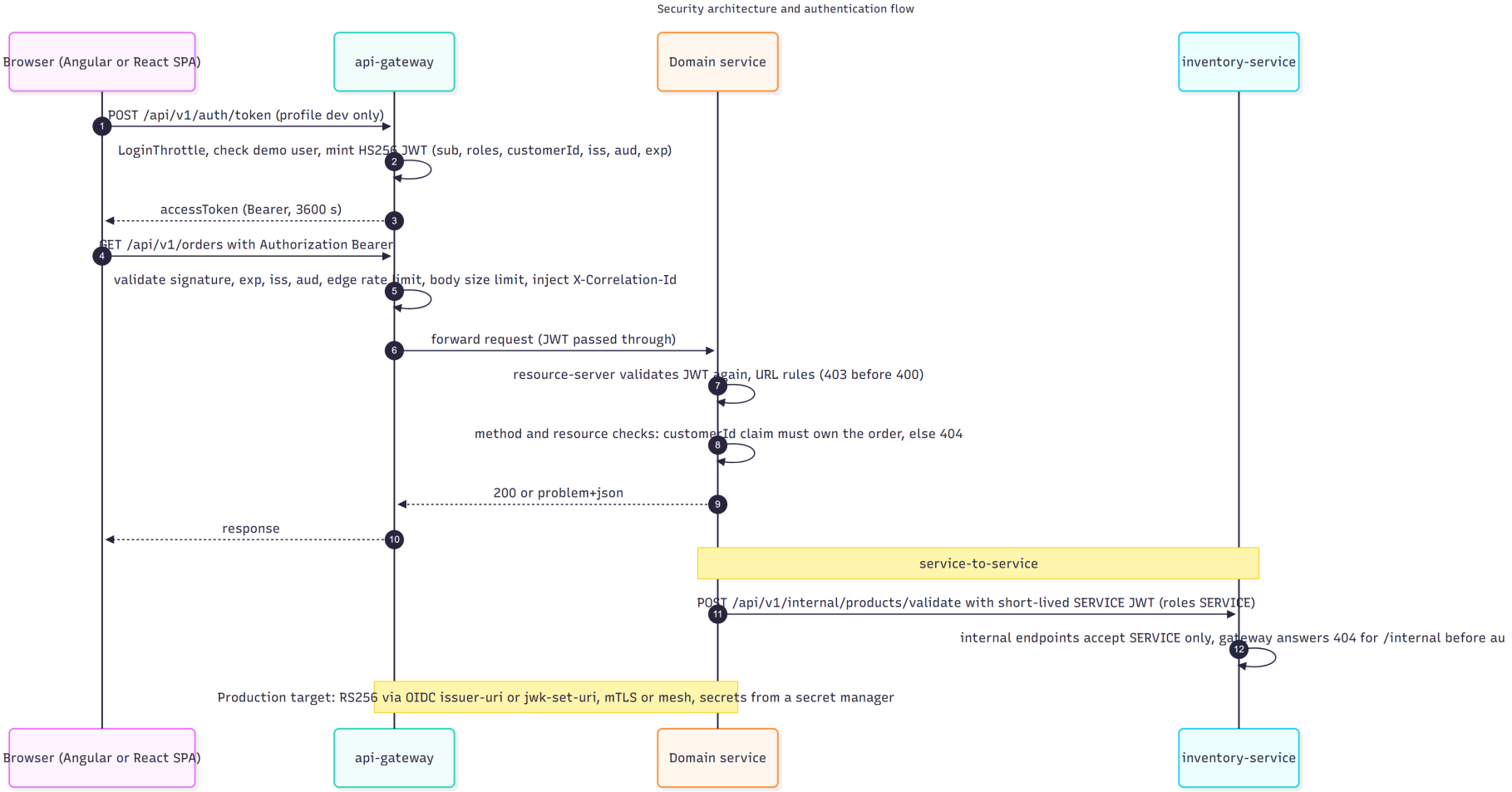


Figure 15. Security architecture and authentication flow

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Source: gateway/src/main/java/com/enterprise/oms/gateway/security/EdgeSecurityConfig.java *(lines 68 to 95)*

```
    @Bean
    SecurityWebFilterChain edgeSecurityFilterChain(ServerHttpSecurity http, ReactiveJwtDecoder decoder,
                                                   OmsSecurityProperties props, Problems problems, Environment env,
                                                   CorsConfigurationSource cors) {
        ServerAuthenticationEntryPoint entryPoint = (exchange, ex) -> problems.write(exchange, HttpStatus.UNAUTHORIZED,
                "UNAUTHENTICATED", "Unauthorized", "Authentication is required.",
                Map.of(HttpHeaders.WWW_AUTHENTICATE, "Bearer"));
        ServerAccessDeniedHandler denied = (exchange, ex) -> problems.write(exchange, HttpStatus.FORBIDDEN, "FORBIDDEN",
                "Forbidden", "You are not allowed to perform this operation.");
        boolean dev = env.acceptsProfiles(Profiles.of("dev"));

        http.csrf(ServerHttpSecurity.CsrfSpec::disable) // stateless bearer-token API: no cookies/sessions to ride on
                .httpBasic(ServerHttpSecurity.HttpBasicSpec::disable)
                .formLogin(ServerHttpSecurity.FormLoginSpec::disable)
                .logout(ServerHttpSecurity.LogoutSpec::disable)
                .requestCache(c -> c.requestCache(org.springframework.security.web.server.savedrequest.NoOpServerRequestC
                .securityContextRepository(NoOpServerSecurityContextRepository.getInstance())
                .cors(c -> c.configurationSource(cors))
                .headers(h -> h
                        .contentSecurityPolicy(csp -> csp.policyDirectives("default-src 'none'; frame-ancestors 'none'"))
                        .referrerPolicy(r -> r.policy(ReferrerPolicyServerHttpHeadersWriter.ReferrerPolicy.NO_REFERRER))
                        .frameOptions(f -> f.mode(org.springframework.security.web.server.header.XFrameOptionsServerHttpH
                        .hsts(Customizer.withDefaults())
                        .permissionsPolicy(p -> p.policy("geolocation=(), microphone=(), camera=()")))
                .exceptionHandling(e -> e.authenticationEntryPoint(entryPoint).accessDeniedHandler(denied))
                .oauth2ResourceServer(o -> o
                        .authenticationEntryPoint(entryPoint)
                        .accessDeniedHandler(denied)
```

Front-end security: token storage and Content Security Policy (An

Both UIs treat the browser as untrusted: route guards are UX only and the backend authorizes every call. They share CSP caveat.

Topic	Angular UI	React UI
Token storage	signal store in memory, mirrored to `sessionStorage`	Redux `auth` slice in memor
Exposure	readable by any script on the origin (XSS); per tab, cleared on close; never `localStorage`, never in URLs or logs	same
Production recommendation	authorization code with PKCE and a backend-for-frontend using HttpOnly, Secure, SameSite cookies	same
Where the token is attached	auth interceptor, API calls only, never `/auth/token`	`omsBaseQuery prepareHeade`
Logout	clears the session	clears the session, resets t
Redirect safety	n/a	`returnUrl` only accepts san
XSS surface	Angular escapes interpolation; no `innerHTML`	React escapes text nodes;
CSP (nginx)	`script-src 'self'`, `style-src 'self' 'unsafe-inline'`	same header; `'unsafe-inli` injects styles at runtime
Open item	CSP read from the file, image never built	nonce-based Emotion cach image never built

The React nginx policy as shipped in `frontend/oms-ui-react/nginx.conf`:

Source: frontend/oms-ui-react/nginx.conf *(lines 31 to 31)*

```
    add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src
'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'" always;
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Method: DFD with trust boundaries, STRIDE per element, then 10 named threats with controls separated into **Imple
only, and **Production-recommended**. Companion docs: `secure-coding-guide.md`, `docs/governance/risk-register.
Normative design: `docs/architecture/00-shared-contract.md`.

0. Verification legend and honest status

- **[code]** verified by reading classes under `services/platform-common/src/main/java/com/enterprise/oms/common/`
- **[tested]** a test class on disk exercises it (named in the threat).
- **[pending]** (historical marker, kept where a control is still not implemented or not run) first written when the serv
  As of 2026-10-18 all of them exist and have tests; the remaining open items are listed in `docs/known-limitations`
  tagged [pending] in this document that are verified in code now carry a status note.
- **[static]** only validated statically (kustomize/kubeconform/terraform validate); never applied to a cluster.
- Testcontainers tests are skipped on the dev machine (no Docker); embedded-Kafka and H2 tests do run.
- All data is synthetic. No real customer data or secrets are in the repository.

Status 2026-10-18 (re-checked against the code): all modules are implemented (gateway `EdgeSecurityConfig`,
`TokenController`; order-service `OrderSecurityConfig`, `RateLimitAdvice`; inventory, fulfillment and notification controll
0 failed, 29 skipped for Docker) and the live checks (`scripts/live-flows.mjs` 54/54: cross-customer 404/403, custon
at the edge, forged and `alg:none` tokens 401) ran against the local stack. Gateway coverage: `EdgeSecurityTest`,
`RoutingAndHeadersTest.internalPathsAreNeverRoutedEvenWithServiceOrAdminTokens`, `RateLimitTest`, `CorsTest` (wildca
`LoginThrottleTest`, `NonDevProfileTest`. Service coverage: `OrderSecurityTest` (role matrix of every endpoint group, c
tokens), `InventoryApiTest#adjustValidationAndAuthorization`, `ProductApiTest`, `NotificationApiTest`, `FulfillmentApi
`ReservationConcurrencyTest` (H2) plus the live oversell run; order-service idempotency is scoped by `(scope, idem_ke
open: PostgreSQL behaviour never run, OIDC/RS256 untested, consumer-side producer-ownership validation and dua
Kafka ACLs/TLS, NetworkPolicy enforcement unverified (no cluster), actuator health/prometheus unauthenticated.

1. Assets, actors, trust boundaries

Assets: orders and order history (A1), customer identity/email (A2), stock levels and reservations (A3), JWT signing s
and DLTs (A6), availability of the order path (A7). Actors: anonymous internet user, CUSTOMER, OPERATOR, ADMIN, S
cluster access, compromised dependency.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Data flow diagram

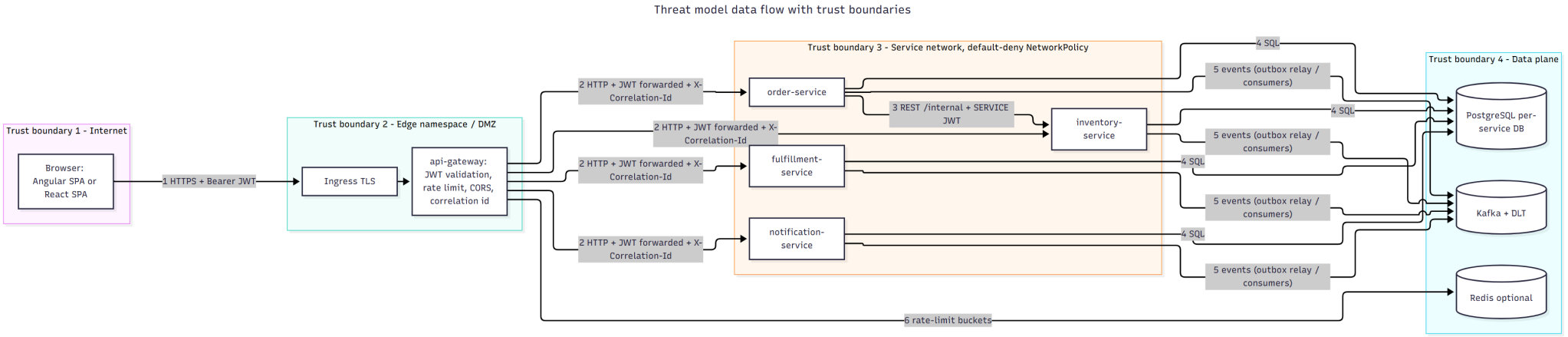


Figure 16. Threat model data flow with trust boundaries

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
Internet  | TB1 |  Edge  | TB2 |  Services  | TB3 |  Data
Browser --HTTPS--> Ingress --> Gateway --HTTP--> order/inventory/fulfillment/notification --> Postgres / Kafka / Redis
          (untrusted)      (authN, throttle)     (authZ again, backend-authoritative)        (per-service credentials)
```

Rule of this architecture: the gateway is a filter, not the only line of defence. Each service re-validates the JWT (`JwtS`
roles and ownership itself.

2. STRIDE by element

Element	S Spoofing	T Tampering	R Repudiation	I Info disclosure
Browser/SPA (Angular or React)	stolen token reuse	client-side state edits (cart, ids, Redux or signal store)	user denies action	token in storage/logs
Gateway	forged/expired JWT; alg confusion	header injection (`X-Correlation-Id`)	missing edge log	verbose errors
order/inventory/fulfillment/ notification APIs	service call without token	mass assignment, body tampering	actor not recorded	cross-customer reads, verbose errors/PII in log
Service-to-service REST	stolen SERVICE token	modified validate response	none	n/a
Kafka topics	producer impersonation	forged/replayed events	event provenance	topics readable by any client
PostgreSQL	shared credentials	SQL injection	audit row deletion	dumps/backups
Secrets (JWT secret, DB creds)	n/a	n/a	n/a	leak via env/log/repo
CI/CD and images	typosquat/ dependency	tampered image/action	unsigned builds	secrets in logs

3. The ten threats

Residual risk scale: Low / Medium / High after the listed controls, considering honest verification status.

T1 Unauthorized order access

- **Attack:** unauthenticated or wrongly-authenticated caller reads, creates or cancels orders: no token, expired, forg
  issuer, token from another system.
- **Impact:** confidentiality and integrity of orders; fraudulent orders/cancellations.
- **Controls implemented in code [code][tested]:** stateless resource server; `JwtTimestampValidator`, `JwtIssuerV`
  only (`MacAlgorithm.HS256`), so `alg: none` is rejected; `anyRequest().authenticated()`; 401/403 as problem+json wit
  `ProblemSecurityHandlers`); unknown roles ignored; method security enabled (`@EnableMethodSecurity`). Tests: `Secu`
  wrong audience, wrong issuer, other secret, tampered payload, `alg: none`, 403 via method security).
- **Demo-only:** HS256 with a shared secret; dev token endpoint with demo users (implemented in `gateway/auth/To`
  password from env, throttled by `LoginThrottle`, absent outside `dev`: `NonDevProfileTest`).
- **Production-recommended:** RS256 via IdP and JWKS (`oms.security.jwt.jwk-set-uri`), short token lifetime (dem
  token revocation/`jti` deny list for high-risk events, MFA at the IdP for OPERATOR/ADMIN.
- **Tests:** `SecurityApiTest`; per-endpoint 401/403 matrices: `OrderSecurityTest#roleMatrixOfEveryEndpointGroup`, ga
  `GatewayAuthTest`), `InventoryApiTest`, `NotificationApiTest`, `FulfillmentApiTest`.
- **Residual:** Low-Medium: endpoint matrices exist and pass on H2/embedded Kafka; OIDC mode and a real cluster

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

T2 Cross-customer exposure (IDOR / BOLA)

- **Attack:** customer1 requests `/api/v1/orders/{id}` of customer2, enumerates ids, passes another `customerId` in G body, reads another customer's notifications or customer record.
- **Impact:** disclosure of order contents and PII.
- **Controls in code [code]:** `CurrentUser.canAccessCustomer(UUID)` (staff, or customer whose token `customerId` equ as absent); `CurrentUser.find/require`. **Contract rules [code][tested in order-service]:** foreign order returns list filter is forced from the token, POST `customerId` must equal the token's, UUID ids (not sequential).
- **Demo-only:** none specific; seed customers are synthetic.
- **Production-recommended:** row-level ownership assertion repeated in repository queries (`WHERE customer_id =` check; security regression tests in CI; consider PostgreSQL row-level security as defence in depth.
- **Tests:** `SecurityApiTest.currentUserExposesIdRolesAndCustomerId` covers the identity extraction; e2e test "custor another customers order" runs in mock mode and live (18/18 on 2026-10-18). Also: `OrderSecurityTest#customerC` `OrderApiTest#cancellingAnotherCustomersOrderIs404AndChangesNothing`, `CustomerApiTest#customerReadsSelfOnly`
- **Residual:** Medium until the real services enforce and test it.

T3 Duplicate submission (replay, double click, retry)

- **Attack:** client or gateway retries `POST /orders`; attacker replays a captured request.
- **Impact:** duplicate orders and reservations; stock held; customer double-served.
- **Controls in code [code][tested]:** `IdempotencyKeys.require` (8-64 chars `[A-Za-z0-9_-]`, 400 on missing/malform Consumer side: `IdempotentEventProcessor` + `processed_events(consumer, event_id)` primary key; outbox relay m `PageRequestFactoryTest.idempotencyKeyValidation`, `IdempotentProcessorAndAuditTest`, `ConsumerKafkaTest`. **Pendi** scoped uniqueness (key must be unique per authenticated caller so one user cannot probe or hijack another's ke `Idempotent-Replay: true`, same key different body 422; unique `order_id` on reservations; unique (`source_event_i`
- **Demo-only:** none.
- **Production-recommended:** key TTL (for example 24 h) with cleanup; replay of a captured request after TTL is a should be rate-limited; request signing is out of scope.
- **Tests:** `IdempotencyApiTest` (replay, 422 reuse, 400 missing, concurrent same-key race: one order); e2e 5 and 5b requests with one key gave one order.
- **Residual:** Low-Medium.

T4 Inventory manipulation

- **Attack:** oversell via concurrent reservations; negative stock adjustment below reserved; a customer calls stock a `ReleaseInventory` commands drain or inflate stock; reservation hoarding (reserve and never confirm).
- **Impact:** financial/oversell loss, denial of stock to real buyers.
- **Controls [code][tested on H2]:** atomic conditional UPDATE `on_hand - reserved >= :q` (`AtomicUpdateReservati` strategies also exist); sorted-id order; all-or-nothing transaction; `PUT /inventory/{id}/stock` restricted to OPERAT writes `stock_movements` + audit (`AuditService`); reservation TTL (default 900 s) with `ReservationExpiryJob`; unique 1..1000, max 50 lines. DB CHECK constraints exist in `V10__inventory_schema.sql`. Not proven on PostgreSQL (row
- **Production-recommended:** alert on `reserved/on_hand` ratio, rate limit per customer on order creation, Kafka AC `oms.inventory.commands.v1`.
- **Tests:** `ReservationConcurrencyTest` (H2 threads, every strategy), `InventoryApiTest`, `ReservationLifecycleTest`; li parallel orders, stock 3: 3 FULFILLED, 27 REJECTED). PostgreSQL variants `InventoryPostgresIT`/`InventoryLocalPos` `performance/oversell.js` (k6) never run.
- **Residual:** Medium (no inventory code on disk yet).

T5 SQL injection

- **Attack:** malicious `q`, `sort`, `category`, `status` parameters; injection via JSON fields; second-order injection via store
- **Impact:** data theft/modification; full DB compromise of one service DB.
-

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Controls in code [code][tested]: `PageRequestFactory` rejects non-whitelisted sort fields and out-of-range pagin `PageRequestFactoryTest.rejectsOversizeNegativePageAndUnknownSortFields`); JPA with `spring.jpa.hibernate.ddl` and `GlobalExceptionHandler` keep SQL text out of responses; `OutboxEventRepository` uses named parameters (`@P` derived/`@Query` with bind parameters only; the conditional-UPDATE uses `:q`/`:id` binds.

- **Demo-only:** H2 in PostgreSQL mode (different parser than PostgreSQL).
- **Production-recommended:** least-privilege DB roles per service (no DDL at runtime role; Flyway uses a separat concatenation (already in CI: `p/java`), WAF rules at ingress.
- **Tests:** `PageRequestFactoryTest`; `ProductApiTest#sqlInjectionAttemptsInFiltersAreJustLiterals` (inventory). Prob `DROP TABLE--`, `sort=name;delete`) were not written (open).
- **Residual:** Low (parameterized by construction) once probes exist.

T6 Privilege escalation

- **Attack:** CUSTOMER calls ADMIN/OPERATOR endpoints; token claim tampering (`roles`); CUSTOMER token on `/api/` compromised service forges an ADMIN token.
- **Impact:** full data access, stock/product/customer changes, reconciliation triggers.
- **Controls in code [code][tested]:** signature verification (tampered payload and wrong-secret tokens rejected); (`internalEndpointsRequireServiceRole`); roles mapped from the `roles` claim only for the 4 known values; method Authorization matrix in contract section 4 [pending per endpoint]. UI guards (`frontend/oms-ui/src/app/core/auth`
- **Demo-only:** one shared HS256 secret across all services and the gateway: **any component holding it can mi ADMIN.** This is the largest structural weakness of the demo.
- **Production-recommended:** RS256 with the IdP holding the only private key; services verify with JWKS and can projected service-account tokens or mTLS (SPIFFE) rather than a shared secret; no ADMIN role without MFA; admi
- **Tests:** `SecurityApiTest` (above); `OrderSecurityTest#roleMatrixOfEveryEndpointGroup`, `#spoofedRoleHeadersParame` gateway `EdgeSecurityTest`.
- **Residual:** High in HS256 demo mode; Medium with RS256/JWKS.

T7 Forged events (Kafka)

- **Attack:** an actor with network access to the broker publishes `InventoryReserved`, `FulfillmentCompleted`, `OrderCon` fulfilled, release stock, or spam notifications; replays old events; sends malformed events to block a partition.
- **Impact:** state corruption bypassing all HTTP authorization.
- **Controls in code [code][tested]:** idempotent consumers (replay of the same `eventId` is a no-op); malformed JS partition; unknown `eventType` ignored and counted; tolerant reader; strict envelope parsing (`EventEnvelope`, `Malfo` rejects invalid transitions [pending]. Tests: `ConsumerKafkaTest`, `EventEnvelopeTest`. Network: default-deny Network Kafka [static].
- **Demo-only:** Compose and dev Kafka are PLAINTEXT with no authentication or ACLs; the envelope is unsigned; p
- **Production-recommended:** MSK with TLS plus SASL/IAM or mTLS; per-service principals with topic ACLs (only o `order.events`, only inventory writes `inventory.events`, and so on); consumer validates `producer` against topic own `events` schemas; optional message signing (JWS) for cross-trust-zone topics; alert on unexpected producer ids.
- **Tests:** `ConsumerKafkaTest` (poison, malformed, unknown type), `OrderResilienceKafkaIT`; schema validation of 218 `events.mjs` (offline, not a consumer-side check). Still not implemented: a consumer rejecting an event whose `pro` `known-limitations.md`).
- **Residual:** High in demo (any pod-level or compose-network actor); Medium with ACLs and TLS.

T8 Sensitive information in logs

- **Attack:** tokens, passwords, full emails, or PII written to application logs, audit `detail`, error responses, metrics la user input; tokens in URLs.
- **Impact:** credential theft via log access; privacy exposure; forged log lines.
- **Controls in code [code][tested]:** `LogMasking.maskEmail` (`a***@domain`), `maskToken`, `sanitize` (bearer, raw JWT, p `api-key=` pairs, emails), `stripNewlines` (log forging); `JwtSecret.toString()` does not print the secret; `server.erro` never in responses; `management.endpoint.health.show-details=never`; logging contract: JSON fields whitelist in `lo`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Gaps:** masking is opt-in (a developer must call it); audit `detail` is free text up to 8000 chars with no automatic sc `checklist.md`); `LogMasking` regexes do not cover phone numbers, addresses, or card-like numbers (none handled
- **Demo-only:** `run-local.sh` prints the generated demo password to the console by design.
- **Production-recommended:** central log scrubber in the shipper (Promtail/OTel processors), restricted log access customer identifiers as metric tags, trace attributes allow-list.
- **Tests:** `LogMaskingTest`, `OrderTransitionsLoggingTest`; live JSON logs (`docs/validation/correlation-trace-sample` (open): a log-scan integration test that fails on e-mail/JWT patterns in captured service logs.
- **Residual:** Medium (opt-in masking).

T9 Denial of service

- **Attack:** request floods; oversized bodies; deep pagination (`size=100000`); expensive searches; slow-loris; Kafka p inventory; reservation hoarding; log flooding.
- **Impact:** order path unavailable; cost.
- **Controls in code [code][tested]:** `RequestSizeLimitFilter` (64 KB, 413 problem+json, counts chunked bodies); `server.max-http-request-header-size=16KB`; form post limit 64KB; multipart disabled; outbound timeouts connect consumer retry bounded (4 attempts then DLT); graceful shutdown. Manifests [static]: resource requests/limits, H gateway rate limiting with Redis + in-memory fallback (`RATE_LIMITED`/429 + `Retry-After`), circuit breaker around
- **Demo-only:** rate limits with in-memory buckets are per-pod (not global).
- **Production-recommended:** WAF/CDN rate limiting at ingress, per-customer quotas, connection limits, bulkhead example), alerting on saturation, request timeouts at gateway (10 s per contract).
- **Tests:** `WebFiltersTest` (413 and chunked), `PageRequestFactoryTest`, `OutboxRelayResilienceTest`, gateway `RateLim` `performance/order-flow.js` never run (open).
- **Residual:** Medium.

T10 Misconfigured service-to-service access

- **Attack:** services reachable directly from any pod; `/api/v1/internal/**` exposed through the gateway or ingress; with long lifetime or reused; `management` endpoints (prometheus) reachable by anyone; shared DB credentials; ser headers; CORS `*`.
- **Impact:** bypass of gateway controls; lateral movement.
- **Controls in code [code][tested]:** `/api/v1/internal/**` SERVICE-only in every service (`internalEndpointsRequi` instead of trusting the gateway; `ServiceTokenProvider` mints short-lived SERVICE tokens (`sub: svc-order-service` `WebFiltersTest.correlationFilterGeneratesIdWhenMissingOrUnsafe`); `security headers` (nosniff, frame deny, no-r `networkpolicies.yaml` default-deny ingress+egress, DNS egress, ingress controller only to edge/ui, gateway and inventory on 18082), service-to-data allow rules; `automountServiceAccountToken: false`; non-root, read-only root namespace; per-service DB users in `oms-secrets` (keys `<svc>-db-username|password`). **Pending:** gateway must no
- **Demo-only:** `/actuator/prometheus` and health are `permitAll` on the service port; one shared HS256 secret; the m placeholder; NetworkPolicies need a CNI that enforces them and were never applied.
- **Production-recommended:** separate management port restricted by NetworkPolicy to the monitoring namespa identities; per-service credentials with least privilege; audience claim per service (`aud` currently `oms-api` for all); e CORS allow-list only (`OMS_GATEWAY_CORS_ALLOWED_ORIGINS`).
- **Tests:** `SecurityApiTest.internalEndpointsRequireServiceRole`; gateway `RoutingAndHeadersTest.internalPathsAreNeverRoutedEvenWithServiceOrAdminTokens` (the planned `InternalRouteB` at the edge. NetworkPolicy connectivity test on a real cluster remains open (no cluster).
- **Residual:** Medium-High until the policies are verified on a cluster.

4. Summary matrix

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Threat	Implemented and tested (platform-common, services, gateway)	Still open (2026-10-18)
T1 Unauthorized order access	JWT validation, 401/403 handlers, endpoint role matrices (`OrderSecurityTest`, `EdgeSecurityTest`)	OIDC/RS256 untested
T2 Cross-customer exposure	`CurrentUser.canAccessCustomer`, 404 rule and forced filters in order/ notification (`OrderSecurityTest`, live cross-customer checks)	none known
T3 Duplicate submission	`IdempotencyKeys`, `IdempotentEventProcessor`, outbox, `idempotency_keys` + unique constraints (`IdempotencyApiTest`)	PostgreSQL run of the constraints
T4 Inventory manipulation	atomic reservation, TTL, stock rules, audit (`ReservationConcurrencyTest`, `InventoryApiTest`, live oversell)	PostgreSQL row locking never run;
T5 SQL injection	sort whitelist, error hygiene, parameter binding; `ProductApiTest` injection probe	probes for the other list endpoints
T6 Privilege escalation	signature, internal SERVICE rule, per-endpoint roles (`OrderSecurityTest`)	RS256/JWKS untested; dual-key rot
T7 Forged events	idempotency, DLT, tolerant reader, 15 event schemas validated offline	producer-ownership check, Kafka A
T8 Sensitive logs	`LogMasking`, `LogMaskingTest`, masked live logs	log-scan test
T9 DoS	size limit, paging caps, timeouts, rate limiters (`RateLimitTest`, `RateLimitApiTest`), breakers (`InventoryClientResilienceTest`, gateway `ResilienceTest`, `BulkheadConcurrencyTest`)	k6 load test never run; Redis limite
T10 S2S misconfig	internal rule, short SERVICE tokens, gateway route block (tested), NetworkPolicies [static]	cluster verification (no cluster); OID token

5. Test matrix (existing vs still open)

Threat	Existing test (path under services/platform-common/src/test/java/com/enterprise/oms/common/ unless noted)	Still open
T1	`security/SecurityApiTest` (8 token-failure cases + 403); `OrderSecurityTest`; gateway `EdgeSecurityTest`	OIDC/JWKS test
T2	`security/SecurityApiTest.currentUserExposesIdRolesAndCustomerId`; `OrderSecurityTest#customerCannotReachOtherCustomersDataByAnyRoute`; `OrderApiTest`; `tests/e2e/specs/order-lifecycle.spec.ts` (live 18/18)	none
T3	`web/PageRequestFactoryTest.idempotencyKeyValidation`, `idempotency/IdempotentProcessorAndAuditTest`, `messaging/ConsumerKafkaTest`; order-service `IdempotencyApiTest`	PostgreSQL twin
T4	`ReservationConcurrencyTest`, `InventoryApiTest` (inventory-service); `tests/e2e/oversell-live.mjs`	`InventoryPostgresIT` run, k6 o
T5	`web/PageRequestFactoryTest`; inventory `ProductApiTest#sqlInjectionAttemptsInFiltersAreJustLiterals`	probes on other list endpoints
T6	`security/SecurityApiTest` (tampered, other secret, `alg:none`, unknown roles, internal); `OrderSecurityTest` role matrix	RS256 test
T7	`messaging/ConsumerKafkaTest`, `events/EventEnvelopeTest`	producer-ownership validation
T8	`observability/LogMaskingTest`, order-service `OrderTransitionsLoggingTest`	log-scan test
T9	`web/WebFiltersTest`, `web/PageRequestFactoryTest`, `outbox/OutboxRelayResilienceTest`, gateway `RateLimitTest`, order `RateLimitApiTest`	k6 load test
T10	`security/SecurityApiTest.internalEndpointsRequireServiceRole`; gateway `RoutingAndHeadersTest`	cluster NetworkPolicy test

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. OWASP Top 10 (2021) mapping

ID	Category	OMS exposure and control	Status
A01	Broken Access Control	role checks, `CurrentUser.canAccessCustomer`, 404 for foreign resources, SERVICE-only internal APIs	platform parts an `EdgeSecurityTest`
A02	Cryptographic Failures	TLS at ingress [static]; HS256 secret >= 32 bytes enforced (`JwtSecret`); secrets never in code; HSTS header; plaintext Kafka/DB inside the demo network	partial; demo plai
A03	Injection	parameter binding, sort whitelist, JSON binding; no dynamic SQL by rule	`PageRequestFacto` `ProductApiTest`);
A04	Insecure Design	threat model, idempotency, saga compensation, state machine, 404-not-403	this document
A05	Security Misconfiguration	secure defaults (`oms-defaults.properties`: no stack traces, health details never, multipart off), non-root read-only containers, PSA restricted, NetworkPolicies; demo permitAll prometheus; base ConfigMap defaults to `dev` profile	[code]/[static]; se
A06	Vulnerable and Outdated Components	Trivy fs/image, `npm audit`, Dependabot recommended, version pinning; Actions and images not SHA/digest-pinned yet	CI defined, never
A07	Identification and Authentication Failures	JWT validation of exp/iss/aud/signature; fail-fast secret; generic login error in UI; no default demo password	[tested]; MFA/lock
A08	Software and Data Integrity Failures	idempotent consumers, unsigned events (gap), CI supply chain (tags not SHAs), npm `--ignore-scripts` in CI	partial
A09	Security Logging and Monitoring Failures	audit_events + `/ops/audit`, correlation id, JSON logs, `LogMasking`, alert rules in `observability/prometheus/rules/oms-alerts.yml`	[code]; alerts [sta
A10	Server-Side Request Forgery	no user-supplied URLs are fetched; service URLs are static config; NetworkPolicy egress limits	Low exposure

7. OWASP API Security Top 10 (2023) mapping

ID	Category	OMS control	Status
API1	Broken Object Level Authorization	`canAccessCustomer`, 404 for foreign orders, forced list filters, UUID ids	helper and enc live)
API2	Broken Authentication	JWT validators, no `alg:none`, no default secret/password	[tested]
API3	Broken Object Property Level Authorization	role-shaped Product JSON (customers get `available` only), no mass assignment (DTO records, not entities)	[code]; custom
API4	Unrestricted Resource Consumption	64 KB body limit, page size <= 100, timeouts, rate limit, saga max attempts	size/paging [te order `RateLimi` real Redis)
API5	Broken Function Level Authorization	admin/ops/internal routes separated by role; gateway does not route internal	internal rule [te
API6	Unrestricted Access to Sensitive Business Flows	idempotency on order creation, quantity/line limits, reservation TTL, rate limiting	[tested] (`Idemp` limit tests)
API7	Server Side Request Forgery	not applicable (no user-controlled outbound URLs)	n/a
API8	Security Misconfiguration	CORS allow-list (same-origin UI), security headers, problem+json without internals, actuator exposure limited to health/info/prometheus	[code]; CORS [

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ID	Category	OMS control	Status
API9	Improper Inventory Management	OpenAPI exports in `specs/openapi` (exist: 4 service exports + generated gateway file), `/v3/api-docs` is public in demo (restrict in prod), additive-only v1, one version live	partial
API10	Unsafe Consumption of APIs	validation of upstream responses, timeouts, circuit breaker on inventory call, tolerant-reader plus schema validation for events	breaker [teste schemas exist consumer-side

8. Demo-only vs production controls

Area	Demo / dev (this repo)	Production require
Token signing	HS256, shared `OMS_JWT_SECRET`	RS256 + JWKS from I
Token issuance	gateway dev endpoint (`dev` profile) with demo users and `OMS_DEMO_PASSWORD`	IdP login (OIDC code
Seed data	Flyway `V1000+` in `dev`	none in prod
Kafka	PLAINTEXT, no ACLs	TLS + SASL/mTLS + A
Database	in-cluster, plaintext, single replica; H2 locally	RDS Multi-AZ, TLS, pe
Redis	optional, in-memory fallback	authenticated TLS Ela
Actuator	health/prometheus `permitAll` on service port; `/v3/api-docs` public	management port +
Service identity	SERVICE JWT from shared secret	SPIFFE/mTLS or proje
NetworkPolicy	authored, not applied; `10.0.0.0/8` placeholder	enforced CNI, tight CI
Images/CI	tags, major-tag Actions	digests, SHA-pinned A
Observability	sampling 1.0 in dev, console password print	sampling 0.05, no se
Audit table	append-only by convention	DB role without UPDA
Ingress/TLS	example hostnames	managed certificates

Inherited defaults flagged earlier and **resolved in the integration pass** (section 10): `base/configmap.yaml` no long overlay sets its own); `/v3/api-docs` and Swagger UI are switched off by the `prod` profile of every service (test: `ProdPr` `JwtSecurityAutoConfiguration` permits `/swagger-ui/**` and `/v3/api-docs/**` at the security layer, which is harmless endpoints themselves are disabled there.

9. Secrets management

Topic	Design
The signing secret	env `OMS_JWT_SECRET`, at least 32 bytes. **No default** in the prod profile: `JwtSecret.reso` `JwtSecretTest`: failsFastWhenAbsent, failsFastWhenShorterThan32Bytes, `applicationS` generates a random in-memory secret and is for tests/single-process only; it must ne recommended).
Local	`scripts/run-local.sh` generates a random secret per run; `OMS_DEMO_PASSWORD` is genera secrets at runtime (`JwtTestTokens`, `JwtSecret.random()`).
Kubernetes	Pods read `oms-secrets` through `secretKeyRef` only (keys `jwt-secret`, `-db-username
Production source of truth	External Secrets Operator reading AWS Secrets Manager (`examples/externalsecret.ex` `aws/terraform/secrets_observability.tf` generates the RDS password in Secrets Man Secrets acceptable for GitOps (sealed output safe to commit, controller key is not). En encryption) and restrict `get secrets` RBAC.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Topic	Design
Rotation (HS256, interim)	Support two keys during rotation: add a second accepted verification key (planned en not implemented), roll the new signing key to the issuer last, wait one token lifetime (support, rotation = rolling restart of ALL services plus gateway at once and all outstar every 90 days and immediately on suspected exposure.
Rotation (DB credentials)	ESO refresh interval plus rolling restart (Spring needs a restart to rebuild the pool) or
JWKS migration (target)	1) Introduce IdP (OIDC). 2) Set `oms.security.jwt.issuer-uri` and/or `jwk-set-uri` on se the HS256 beans (`JwtTokenFactory`, `ServiceTokenProvider`) are not created. 3) Gatewa endpoint removed from prod builds. 4) Replace SERVICE JWT minting by client-creden `OMS_JWT_SECRET` from `oms-secrets`. Key rollover is then the IdP's JWKS rotation (publish
Detection	gitleaks in CI (`secret-scan` job); pre-commit hook recommended; never log secrets (`Lo`
Incident	Suspected leak: rotate signing key, invalidate tokens, review `audit_events` for ADMIN `governance/post-incident-review-template.md`).

10. Integration-pass security findings and caveats (live stack, 202

10.1 Environment profile defaults (fixed)

`infrastructure/kubernetes/base/configmap.yaml` shipped `SPRING_PROFILES_ACTIVE: "dev,json-logs"`; any overlay th production pods with the dev token endpoint, seed data and Swagger. The base is now `json-logs` only; the dev and (with a comment), prod sets `prod,json-logs`. Static check only: `kubectl kustomize`/apply was never run (no cluster).

10.2 OpenAPI and Swagger UI are off in prod (fixed, tested)

`springdoc.api-docs.enabled=false` and `springdoc.swagger-ui.enabled=false` in the `prod` profile of order-, inventory-, (previously only the UI was off in three of them, and api-docs stayed on in all four). Regression test per service: `Prod` `application*.yml` through Spring Boot's config-data loader with `spring.profiles.active=prod` and asserts both flags disable them. Limits of the evidence: it proves the configuration, not an HTTP 404 from a running prod-profile servic available). The gateway has no springdoc dependency at all and has no route for `/v3/api-docs` or `/swagger-ui`; verifi `swagger-ui.html` answer 401 without a token and 404 with one; `/swagger-ui/index.html` 404), and `scripts/export-o` directly.

10.3 HS256 shared secret: caveat and production path

Demo mode signs and verifies every token with ONE symmetric secret (`OMS_JWT_SECRET`) that the gateway and all fou holder of the secret (any compromised service, anyone who can read the env/Secret object) can mint a token for an is no per-service identity, no key id and no clean rotation (rotation = restart everything at once, all tokens invalid; du secret is random per run, never in the repo, and startup fails when it is missing or shorter than 32 bytes; that makes for production.

Production move: RS256 with JWKS from an OIDC provider (Keycloak/Cognito/Entra). Wiring that exists: platform-con servlet services) and the gateway (`EdgeSecurityConfig`, reactive) switch to `NimbusJwtDecoder.withJwkSetUri` / `JwtDec` `oms.security.jwt.issuer-uri` and/or `oms.security.jwt.jwk-set-uri` is set, keep the audience and timestamp validat HS256 beans. **Not tested:** there is no test, and no IdP was available, for either OIDC path (verified by reading the c `jwk-set-uri`/`issuer-uri`). Known gaps to close before relying on it: (1) in OIDC mode `ServiceTokenProvider` is not cre inventory validation call) sends no SERVICE token and inventory's `/api/v1/internal/**` (role SERVICE) would answer mTLS) must be added; (2) the `roles` and `customerId` claim names must be mapped from the IdP's claims; (3) the dev builds (already only active in profile `dev`).

10.4 /actuator/prometheus exposure

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

`/actuator/health/**`, `/actuator/info` and `/actuator/prometheus` are `permitAll` on every service's main port (`JwtSecu` carry business labels (topic names, outcome counts, route/URI tags, JVM and pool details). Verified live: the gateway `**` (`GET :18080/api/v1/ops/order/../../../actuator/prometheus` is 404), but the gateway's OWN `/actuator/health` a unauthenticated on its public port 18080 (the Prometheus scrape in `scripts/check-metric-names.mjs` reads it withou service port directly can scrape that service. In Kubernetes the Ingress only forwards `/api` (to the gateway) and `/` (to reachable from outside by manifest (static check only, never applied). Production requirements: (a) put actuator on `management.server.port`) that is not in the Service/Ingress and is reachable only from the Prometheus namespace th authentication on `/actuator/prometheus` (basic auth or bearer for the scraper) and keep only `health/liveness|readi` tags with raw ids (templates only, as Spring MVC does) and keep PII out of metric labels. The Kubernetes manifests `servicemonitor.yaml`) were validated statically only; neither option (a) nor (b) is implemented in code, so this remain a closed one.

11. React front end notes (frontend/oms-ui-react, added 2026-10-

The browser trust boundary (TB1) now contains either of two SPAs (Angular or React); both use the same gateway c applies unchanged. React-specific points, verified in the code unless marked otherwise:

- **XSS.** React escapes text nodes; the app renders all server strings (`statusReason`, names, notification bodies) as t `dangerouslySetInnerHTML` and no `eval`. Residual risk: a future developer adding `dangerouslySetInnerHTML` or a vuln vulnerabilities on 2026-10-19; Trivy and CodeQL jobs in CI never ran). The lint rules do not forbid `dangerouslySet`
- **Token storage.** The access token lives in the Redux `auth` slice (memory) and is mirrored to `sessionStorage` (per log the user out. Script running on the origin can read it, so any XSS can steal it (the same trade-off as the Angul URLs, never logged; the password is never stored). Production: authorization code with PKCE and a BFF issuing H JavaScript never sees a token.
- **Token leakage paths.** The bearer token is attached only by `omsBaseQuery` to same-origin `/api/v1` calls and neve accepts only same-app relative paths (no `//`, no scheme, no backslash), which blocks open-redirect abuse. On log resets the RTK Query cache and clears the cart, so a later user of the same tab cannot see the previous session's
- **CSP caveat.** nginx sends `default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; connect` `object-src 'none'`. `style-src 'unsafe-inline'` is required because MUI/Emotion injects `<style>` tags at runtime (exception). Consequence: an injected style can restyle or exfiltrate through CSS side channels, which is weaker th remain blocked by `script-src 'self'` (the build emits no inline scripts). Fix: an Emotion cache with a per-respons done. The header was read from `nginx.conf`; the image was never built, so the live header was NOT observed.
- **Guards are UX.** `RequireAuth` and `RequireRole` only hide screens; the backend still returns 401/403/404 (T1, T2, T DevTools gains no data.
- **Supply chain.** All dependencies are pinned to exact versions with a lockfile; `npm ci --ignore-scripts` is the inte (MUI, Emotion, RTK, React Router, RHF, Zod, MSW, Vitest), so the attack surface of the dev toolchain is larger tha
- **Contract drift.** `src/models.ts` is a copy of the types; a drift would show up as wrong rendering, not as a security example the null prices of REJECTED orders, defect D-03) can still blank a screen: covered by tests.
- **Not done:** automated accessibility and security scans of the running UI (ZAP, axe), a nonce-based CSP, SRI (no t other than desktop Chrome.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

12. Docker and Kubernetes deployment

Honest status first

Docker was **not available** on the development machine. Dockerfiles, Compose files, Kubernetes manifests and Terraform were written and validated statically (kustomize build, kubeconform, terraform validate, compose config, checked by `scripts/validate-infra.sh` and CI job `infra-validate`) where those tools could be fetched. Whether any image was actually built or any container actually ran is stated in the Evidence section; do not assume it. Anything marked "reference" below was never applied to a live cluster.

Container images

One Dockerfile per component in `infrastructure/docker/`. Key properties of `order-service.Dockerfile`:

- Two stages: the Spring Boot fat jar is split into layers with `-Djarmode=tools extract --layers`, so dependency layers are cached and only the small application layer changes per build.
- JRE-only Alpine base pinned to an exact version tag (digest pinning is documented as a production step).
- Non-root numeric user (uid 10001) so Kubernetes `runAsNonRoot` can verify it.
- `-XX:MaxRAMPercentage=75.0 -XX:+UseG1GC -XX:+ExitOnOutOfMemoryError` and graceful shutdown (`SIGTERM` reaches the JVM because of `exec`).
- `HEALTHCHECK` on `/actuator/health/readiness` for Docker and Compose.

Source: `infrastructure/docker/order-service.Dockerfile` *(lines 30 to 39)*

```
USER 10001:10001
ENV SERVER_PORT=${APP_PORT} \
    SPRING_PROFILES_ACTIVE=json-logs \
    SERVER_SHUTDOWN=graceful \
    SPRING_LIFECYCLE_TIMEOUT_PER_SHUTDOWN_PHASE=25s \
    TZ=UTC \
    JAVA_OPTS="-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0 -XX:InitialRAMPercentage=50.0 -XX:
+ExitOnOutOfMemoryError -XX:+UseG1GC -Djava.io.tmpdir=/tmp -Djava.security.egd=file:/dev/urandom -
Dfile.encoding=UTF-8"
EXPOSE 18081
# Container-level healthcheck (Compose/Docker). Kubernetes ignores HEALTHCHECK and uses its own
probes.
HEALTHCHECK --interval=15s --timeout=3s --start-period=60s --retries=5 \
```

Docker Compose

`infrastructure/compose/docker-compose.yml` runs PostgreSQL 16 (four databases created by `postgres/init/01-create-databases.sh`), Kafka in KRaft mode with topics created by `kafka/create-topics.sh`, Redis, the four services, the gateway and the frontend; `docker-compose.observability.yml` adds Prometheus, Grafana, Loki, Promtail, Tempo and the OpenTelemetry collector. The no-Docker local path (`scripts/run-local.sh`) uses the `local-stack` module with embedded Kafka and H2 and is what was actually executed.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Kubernetes

Layout (`infrastructure/kubernetes/`): `base/` (namespace, one Deployment and Service per component, ConfigMap, Ingress, NetworkPolicies, ServiceAccount), `overlays/dev|test|prod` (Kustomize patches for replicas, resources, images and settings), `dev-dependencies/` (PostgreSQL, Kafka, Redis for dev clusters only), `keda/` (Kafka-lag autoscaling), `monitoring/` (ServiceMonitor and PrometheusRule), `examples/` (secret templates; no real secrets).

Concern	What the manifests do
Zero-downtime rollout	`maxUnavailable: 0`, readiness gate before traffic
Slow JVM start	`startupProbe` so liveness does not kill a booting pod (Flyway on first boot)
Probes	Spring Boot liveness and readiness groups under `/actuator/health/*`
Pod security	`runAsNonRoot`, container `securityContext`, dedicated ServiceAccount
Resources	requests and limits per container; JVM sized with percentage flags
Network	NetworkPolicies, default deny with explicit allows
Config and secrets	ConfigMap for non-secret settings; secrets from a Secret or External Secrets (templates only)
Autoscaling	HorizontalPodAutoscalers scale 2 to 6 replicas on CPU; the KEDA ScaledObject for Kafka lag is an example file that is not part of any overlay
Rollback	`kubectl rollout undo` and the decision tree in `docs/governance/deployment-and-rollback.md`

Source: `infrastructure/kubernetes/base/order-service.yaml` *(lines 100 to 125)*

```
          startupProbe:                     # allows slow JVM + Flyway on first boot without liveness
kills
            httpGet:
              path: /actuator/health/liveness
              port: http
            periodSeconds: 5
            failureThreshold: 36            # 3 minutes
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: http
            periodSeconds: 10
            timeoutSeconds: 3
            failureThreshold: 3
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: http
            periodSeconds: 20
            timeoutSeconds: 3
            failureThreshold: 3
          lifecycle:
            preStop:
              exec:
                command: ["sh", "-c", "sleep 10"]   # let endpoints/ingress drop this pod before
SIGTERM drain
          securityContext:
            allowPrivilegeEscalation: false
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Rollout and database changes

Migrations follow expand/contract so the previous version keeps working during a rolling update and rollback is safe (see deployment-and-rollback). Kafka consumers rebalance on pod restart; offsets are committed after processing, so a restarted consumer resumes without loss and idempotent consumers absorb redelivery.

AWS reference

`infrastructure/cloud/aws/terraform/` defines VPC across three availability zones with public, private and data subnets, EKS with managed node groups, RDS PostgreSQL Multi-AZ, MSK, ElastiCache, ECR, Secrets Manager and IRSA. It is a reference: `terraform validate` is the strongest check performed; no `apply` was run and there is no cost estimate in this repository. Known gap recorded by the documentation review: the production overlay expects a SASL/SCRAM listener while the Terraform enables IAM authentication for MSK, and no MSK IAM client library is in the service poms. Resolve this before any real deployment.

Trade-offs

- Kustomize over Helm: fewer moving parts, plain YAML diffs, less templating power.
- Lag-based scaling (KEDA) cannot help beyond the partition count: 3 partitions means at most 3 useful consumers per group, while the HPA ceiling is 6 replicas.
- Managed Kafka and PostgreSQL cost more than self-hosting but remove the hardest operational work.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Deployment diagram

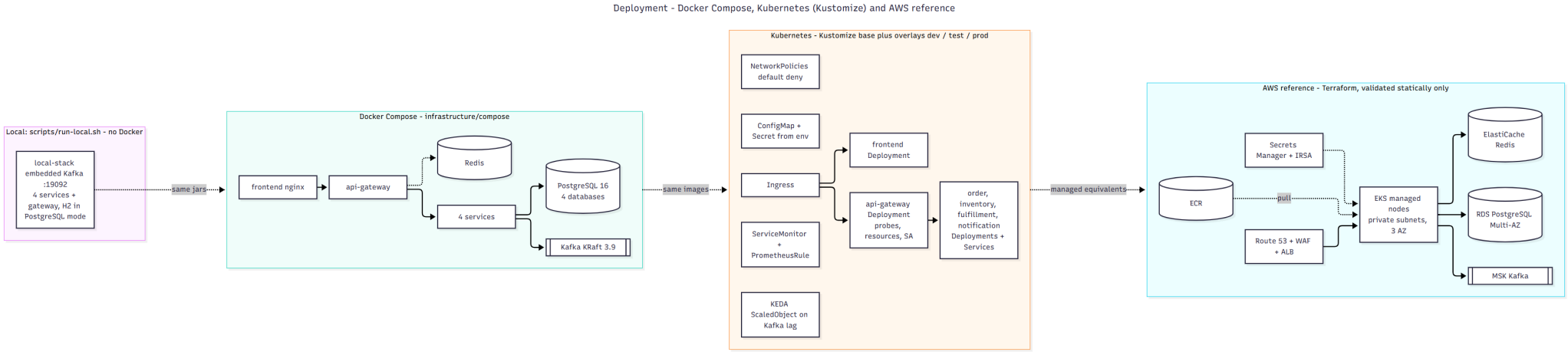


Figure 17. Deployment: local, Docker Compose, Kubernetes and AWS reference

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Deployment and rollback procedure

Scope: how a release moves through dev, test and prod and how to back it out. Companion: `infrastructure/kubernetes/README.md`, `.github/workflows/ci.yml`, `docs/security/threat-model.md`.

0. Honest status

- The Kustomize base and overlays (`infrastructure/kubernetes/{base, overlays/dev, overlays/test, overlays/prod}`) render and pass `kubeconform -strict` (CRDs skipped). **Nothing has been applied to a cluster.** Probes, NetworkPolicies, PDB, HPA, KEDA and the dev-dependency StatefulSets are unverified at runtime.
- `ci.yml` has never run on a GitHub runner; its `deploy-test` job is a gated placeholder (skips when `KUBE_CONFIG_TEST_B64` is unset).
- The services being deployed (order, inventory, fulfillment, notification, gateway) are all implemented now and their jars exist (`services/*/target/<module>-1.0.0.jar`, `gateway/target/api-gateway-1.0.0.jar`, built by the 2026-10-18 reactor run) and run as JVMs in the local stack. **No Docker image was ever built** (no Docker) and no jar ran inside a container, so image names, Dockerfile layering and container flags remain unverified (`infrastructure/README.md`; `docs/known-limitations.md`).
- The prod overlay contains PLACEHOLDER endpoints (`oms-pg.cluster-PLACEHOLDER...`, `b-1.oms-msk.PLACEHOLDER...`). It is a reference, not a deployable environment.
- Procedures below that say "rehearse" (Sprint 4 story S11.2) were **not rehearsed**: no cluster was available, so they are untested as of 2026-10-18.

1. Environments and overlays

Env	Overlay	Namespace	Replicas	Data stores	Profile / notable patches
local	none (scripts/run-local.sh)	n/a	1 each	H2 + embedded Kafka (`local-stack`)	`local`; generates random `OMS_JWT_SECRET`, prints demo password
compose	`infrastructure/compose/docker-compose.yml`	n/a	1 each	Postgres 16, Kafka KRaft, Redis	`${VAR:?}` makes start fail on unset secrets; ports bound to 127.0.0.1
dev	`overlays/dev`	`oms-dev`	1 (PDBs deleted so drains are not blocked; HPA 1-2)	`dev-dependencies/` in-cluster Postgres/Kafka/Redis (single replica, not HA, plaintext)	tracing sampling 1.0
test	`overlays/test`	`oms-test`	base (2)	in-cluster `dev-dependencies/`	HPA max 3; sampling 0.5
prod	`overlays/prod`	`oms-prod`	order/inventory/gateway 3, fulfillment/notification 2, frontend 2; HPA 3-12	managed RDS / MSK / ElastiCache (placeholders); no in-cluster data	`SPRING_PROFILES_ACTIVE=prod, json-logs`; `DoNotSchedule` hostname spread plus zone spread; PDB `minAvailable: 2` for order/inventory/gateway; sampling 0.05

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Finding to fix before first prod use: `base/configmap.yaml` sets `SPRING_PROFILES_ACTIVE: "dev,json-logs"`; dev and test inherit it (intended) but any new overlay that forgets to patch it would enable the dev token endpoint and seed data. Mitigation: prod patch exists; add a `validate-infra.sh` assertion that rendered prod has no `dev` profile (risk R-08).

2. Pipeline (design)

```
PR: java unit tests -> frontend lint/test/build -> CodeQL + Semgrep + Trivy fs + npm audit + gitleaks -
> infra-validate
main: + java-integration (Testcontainers on Linux runner) -> docker build + Trivy image scan (fail on
fixable HIGH/CRITICAL)
      -> push to GHCR -> smoke-local (embedded Kafka + services on runner) -> deploy-test (gated,
environment: test)
tag v*: semver = tag; image tag <semver> (non-tag builds: <VERSION>-<sha8>)
prod: promote the SAME image digest that passed test; manual approval; set image by digest in overlays/
prod
```

Versioning: single semver base in `/VERSION`; image tag `<semver>-<gitsha8>`.

3. Rolling update mechanics (from base/*.yaml)

Mechanism	Setting	Why
Strategy	RollingUpdate `maxSurge: 1`, `maxUnavailable: 0`	capacity never drops during rollout
Startup probe	`/actuator/health/liveness`, 5 s period, 36 failures (3 min)	slow JVM + Flyway on first boot must not be killed by liveness
Readiness probe	`/actuator/health/readiness`, 10 s, 3 failures	traffic only to ready pods
Liveness probe	`/actuator/health/liveness`, 20 s, 3 failures	restart wedged JVM
preStop	`sleep 10`	endpoints/ingress drop the pod before SIGTERM
Graceful shutdown	`SERVER_SHUTDOWN=graceful`, `SPRING_LIFECYCLE_TIMEOUT_PER_SHUTDOWN_PHASE=25s`, `terminationGracePeriodSeconds: 45`	finish in-flight requests, stop Kafka listeners (10 + 25 < 45)
History	`revisionHistoryLimit: 5`	allows `rollout undo`
PDB	`minAvailable: 1` base, 2 prod (order/inventory/gateway)	voluntary disruption safety

Readiness gates (what must be true before a pod receives traffic or the rollout advances)

1. Flyway migrations applied (app only reports ready after context start).
2. Readiness group UP. Decision: the readiness group does not include Kafka (verified in `services/order-service/src/main/resources/application.yml`: `readiness.include: readinessState, db`; the other services were not checked individually). Rationale: do NOT include Kafka in readiness (a broker blip would drain every pod and amplify the outage); expose it through `/api/v1/ops/messaging` and alerts instead.
3. Rollout verification beyond probes (manual or scripted): `GET /api/v1/ops/health` (gateway aggregate, OPERATOR token) shows all services UP; DLT depth unchanged; `oms.outbox.pending` not rising; error rate and p95 on Grafana "OMS - API RED"; consumer lag stable on "OMS - Kafka Messaging".
4. Soak: hold 10 minutes at 100 percent before declaring done; no canary tooling is configured (gap; Argo Rollouts/Flagger would be the extension).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Release procedure (per environment)

```
# 1. set images (CI does this; prod by digest)
cd infrastructure/kubernetes/overlays/<env>
kustomize edit set image ghcr.io/example-org/oms/order-service=<registry>/order-service:<semver>-
<sha8>
# 2. render and validate offline
kubectl kustomize . | kubeconform -strict -ignore-missing-schemas -
# 3. server-side dry run, then apply
kubectl apply -k . --dry-run=server
kubectl apply -k .
# 4. watch
kubectl -n oms-<env> rollout status deploy/order-service --timeout=300s
```

Order of rollout for a release that changes messages or schema: (1) consumers first (inventory, fulfillment, notification, then order's listeners) because event schemas are additive and consumers are tolerant readers; (2) producers; (3) gateway; (4) frontend. Apply one Deployment at a time in prod and verify between steps.

Secrets (`oms-secrets`: `jwt-secret`, `<svc>-db-username|password`, `postgres-password`, `redis-password`, optional `demo-password` for dev) must exist before apply. dev/test: `infrastructure/kubernetes/examples/create-dev-secret.sh`. prod: External Secrets from AWS Secrets Manager (`examples/externalsecret.example.yaml`). Never commit Secret objects (`validate-infra.sh` checks no overlay renders one).

5. Database migrations: expand / contract

Flyway runs at service start (`classpath:db/common` + `classpath:db/migration`). During a rolling update old and new pods run against the same schema, so every migration must be backward compatible with the previous release.

Phase	Release	Allowed	Forbidden in the same release
Expand	N	add table, add nullable/defaulted column, add index (PG: build concurrently in a separate manual step for large tables), add new enum value in code-tolerant way	dropping/renaming anything the N-1 code uses; adding NOT NULL without default
Migrate	N	dual-write code; backfill in batches (separate job, not inside the startup migration for large tables)	long locks
Contract	N+1 or later	after N-1 is fully gone and a rollback to N-1 is no longer planned: drop old column, add NOT NULL, remove dual-write	doing this together with expand

Rules specific to this repo: V1..V8 only platform-common; service migrations V10+; seeds V1000+ and dev profile only; never edit an applied file (Flyway checksum failure); SQL must stay portable to H2 (PostgreSQL mode). Flyway has no automatic down-migrations here: rollback of schema means a forward "fix" migration, which is why expand/contract matters. Take a managed-DB snapshot before any contract-phase migration.

Multiple replicas starting at once: Flyway takes a lock, so one applies and the others wait; the startup probe window (3 min) must exceed the longest migration.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

6. Kafka consumer concerns during deploy and rollback

Concern	Behaviour / design	Action
Rebalance on every pod restart	consumer group = service name; 3 partitions; rolling one pod at a time keeps pauses short	keep `maxUnavailable: 0`; avoid restarting all pods at once
Max useful replicas	partitions = 3: more than 3 consumers per group sit idle	cap HPA/KEDA at 3 for consumers (note: base HPA allows 6, prod overlay 12; for consumer-heavy services prefer the KEDA example capped at 3 or accept idle pods)
In-flight message when pod is killed	uncommitted offset => redelivery; `processed_events` makes the business effect once	none, but verify `oms.events.duplicates` rises only briefly
New event type or field	consumers are tolerant readers; unknown `eventType` ignored and counted	deploy consumers before producers
Removing/renaming a field	breaks old consumers	forbidden within `schemaVersion` 1; use a new `schemaVersion` with a dual-publish period
Rollback after producing new events	old pods read new events (additive)	confirm `oms.events.processed` for unknown types is acceptable; if the new event carries data the old code needs, rollback is not safe (use fix-forward)
Poison message after deploy	4 attempts, then `<topic>.DLT`	inspect via `GET /api/v1/ops/messaging` (`dltTopics`); replay after fix (**no replay tool exists**: replay is a manual procedure, `docs/known-limitations.md` item 3); DLT growth blocks the release gate
Outbox backlog	`OutboxRelay` retries in order; `oms.outbox.pending` gauge	if pending climbs after rollout, check broker reachability before rolling back app
Offsets reset	never reset a group in prod without incident approval (would replay; safe only because of idempotency, but expensive)	incident commander approval
Consumer group id change	creates a new group and re-reads from `auto.offset.reset`	never rename the service/group casually

7. Rollback decision tree

```
Release is live and a signal fires
|
+-- Is there customer-visible harm (5xx, wrong data, stuck orders) or data corruption risk?
|     no  -> continue soak, open ticket, fix forward in next release
|     yes -> go on
|
+-- Is it data corruption / wrong business effect being written (oversell, wrong status, duplicate
charges-equivalent)?
|     yes -> STOP INGRESS FIRST: scale gateway to 0 or set maintenance route; pause consumers (scale
to 0);
|            declare incident (post-incident-review-template.md); then choose A or C below
|     no  -> go on
|
+-- Did this release include a migration?
|     no  -> A: kubectl rollout undo (fast, safe)
|     yes -> was it expand-only (additive)?
|              yes -> A: rollout undo; old code ignores new columns
|              no  -> contract/destructive: DO NOT undo blindly.
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
|                    restore snapshot only if data loss is acceptable; otherwise
|                    C: fix forward with a new migration + hotfix image
|
+-- Did new code publish new event versions that old code cannot process?
|     yes -> C: fix forward (old pods would dead-letter them)
|     no  -> A
|
+-- Is the problem configuration/secret only?
      yes -> B: revert the ConfigMap/Secret change (GitOps revert), restart pods, no image change

A: kubectl -n oms-<env> rollout undo deploy/<name>; rollout status; verify gateway ops health; check
DLT, lag, outbox
B: revert commit that changed config; kubectl apply -k; kubectl rollout restart deploy/<name>
C: branch from the release tag, minimal fix + test, expedited pipeline, deploy by normal procedure
```

Post-rollback verification: `rollout history` shows previous revision; readiness green; `GET /api/v1/admin/orders/failed` (OPERATOR) reviewed for orders damaged during the bad window; run `POST /api/v1/admin/reconciliation/run` (ADMIN; implemented in `order/api/AdminController`, backed by `ReconciliationService`, tested in `ReconciliationTest`) to repair semantic inconsistencies; a rollback does not undo business effects already applied by the bad version.

8. Dev-to-prod differences and demo-only controls to remove

Item	Demo / dev	Prod requirement
Profile	`dev` (token endpoint, seed data)	`prod,json-logs`, no dev token endpoint, no seed
JWT	HS256 shared secret	RS256/JWKS from an IdP (`oms.security.jwt.jwk-set-uri`)
Secrets	generated per run / `create-dev-secret.sh`	External Secrets or Sealed Secrets, rotation
Data stores	in-cluster, single replica, plaintext, no backups	RDS (Multi-AZ, backups, TLS), MSK (3 brokers, TLS/SASL), ElastiCache (auth token, TLS)
Network CIDR	`ipBlock 10.0.0.0/8` placeholder	tighten to data subnets
Images	version tag	digest-pinned, signed, scanned
Actions	major tags	pinned to commit SHA
Ingress/TLS	`*.example.test` hosts	real certificates (cert-manager), WAF
Prometheus endpoint	permitAll on service port in demo	management port restricted by NetworkPolicy

9. Backout checklist (printable)

- Freeze further deploys; announce in the incident channel.
- Capture evidence: `kubectl get pods,events`, Grafana snapshot, DLT depth, outbox pending, lag.
- Decide A/B/C using the tree; record the decision and who approved.
- Execute; verify readiness and ops health.
- Reconcile data; list affected order ids.
- Open post-incident review within 2 working days.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

13. CI/CD pipeline

Pipeline

The workflow in `.github/workflows/ci.yml` runs build and test, frontend build, four security scanners, infrastructure validation, image build with an image scan, a local smoke test on the embedded-Kafka stack, and a test-environment deploy that is a documented placeholder. Honest status: **the workflow has not run on a GitHub runner**; its YAML and the validation scripts it calls were checked locally where possible.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

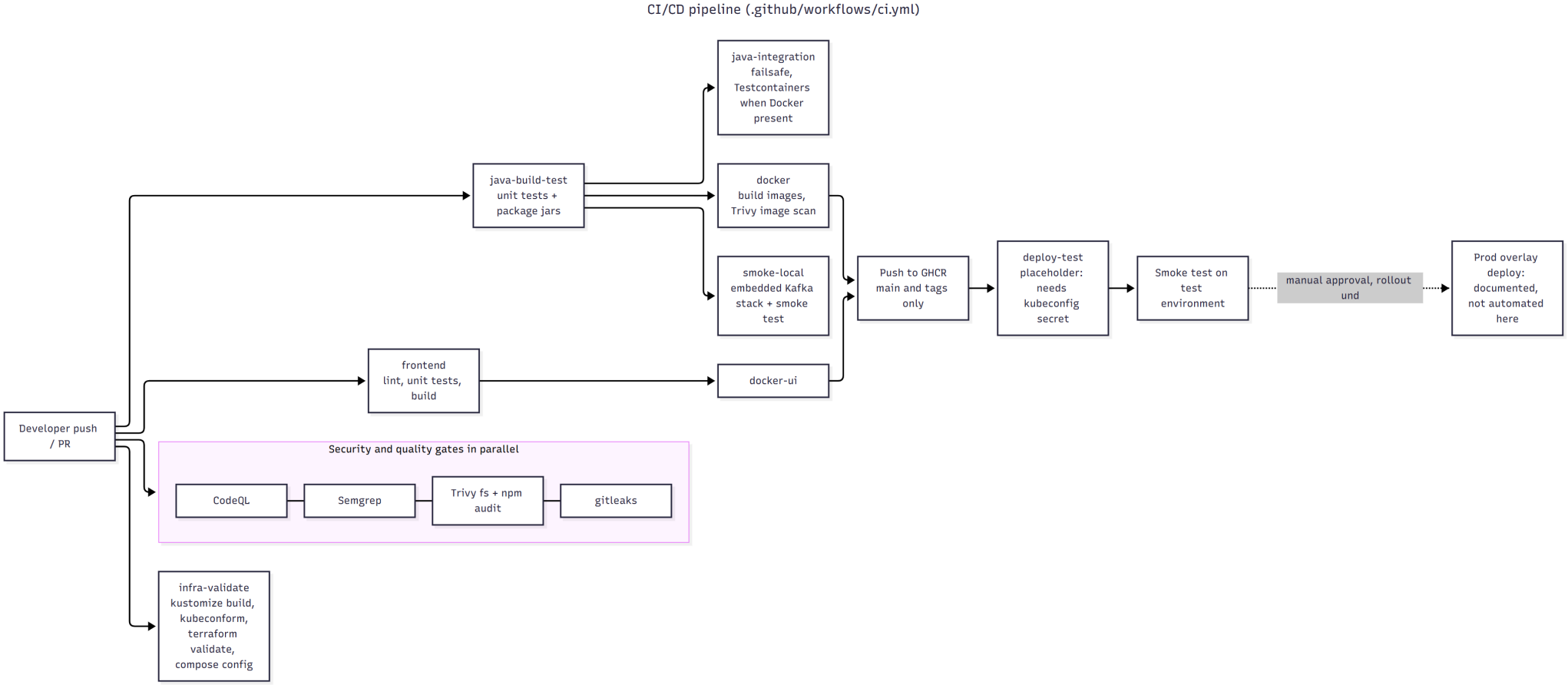


Figure 18. CI/CD pipeline

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Stage purposes

Stage	Purpose	Failure policy
java-build-test	Compile with Java 17, run unit and slice tests, package jars once	Fails the pipeline
java-integration	Failsafe tests, Testcontainers where Docker exists	Fails the pipeline
frontend	Install, lint, unit test, production build of the Angular app	Fails the pipeline
codeql, semgrep	Static analysis for code vulnerabilities	Report and gate per configuration
dependency-scan	Trivy on Maven and npm lockfiles and IaC, `npm audit`	HIGH or CRITICAL fixable findings fail
secret-scan	gitleaks	Any finding fails
infra-validate	kustomize build, kubeconform, terraform validate, compose config, observability checks	Fails the pipeline
docker, docker-ui	Build images from the already-built jars, scan them, push to GHCR on main and tags only	Image scan fails on fixable HIGH or CRITICAL
smoke-local	Start the embedded-Kafka stack and run `scripts/smoke-test.mjs`	Fails the pipeline
deploy-test	Placeholder apply to a non-production cluster plus smoke test	Needs a kubeconfig secret that is not provided

Release and rollback

Environments are Kustomize overlays (dev, test, prod). Production promotes an image by digest, applies after a server-side dry run, and watches the rollout; the rollback decision tree is in section 12. Database migrations follow expand and contract so that rolling back the application never needs a database rollback. Blue-green and canary are described conceptually in the interview guide; the repository implements rolling updates only.

Source: `.github/workflows/ci.yml` (lines 47 to 68)

```
java-build-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { distribution: temurin, java-version: "17", cache: maven }
    - name: Unit tests
      run: mvn -B -ntp -f services/pom.xml test
    - name: Package jars (tests already ran)
      run: mvn -B -ntp -f services/pom.xml package -DskipTests
    - uses: actions/upload-artifact@v4
      with:
        name: jars
        retention-days: 7
        if-no-files-found: error
        path: |
          services/*/target/*-1.0.0.jar
          gateway/target/*-1.0.0.jar
    - uses: actions/upload-artifact@v4
      if: always()
      with:
        name: unit-test-reports
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

14. Observability architecture

Three signals, one correlation id

Metrics (Micrometer to Prometheus), logs (Logstash JSON encoder to Promtail to Loki) and traces (OpenTelemetry bridge to the collector to Tempo) share one correlation id and the W3C trace context. Grafana provides the single pane: five provisioned dashboards (API RED, JVM, Hikari pool, messaging, saga funnel). Alert rules live in `observability/prometheus/rules/oms-alerts.yml` and each alert maps to a runbook.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

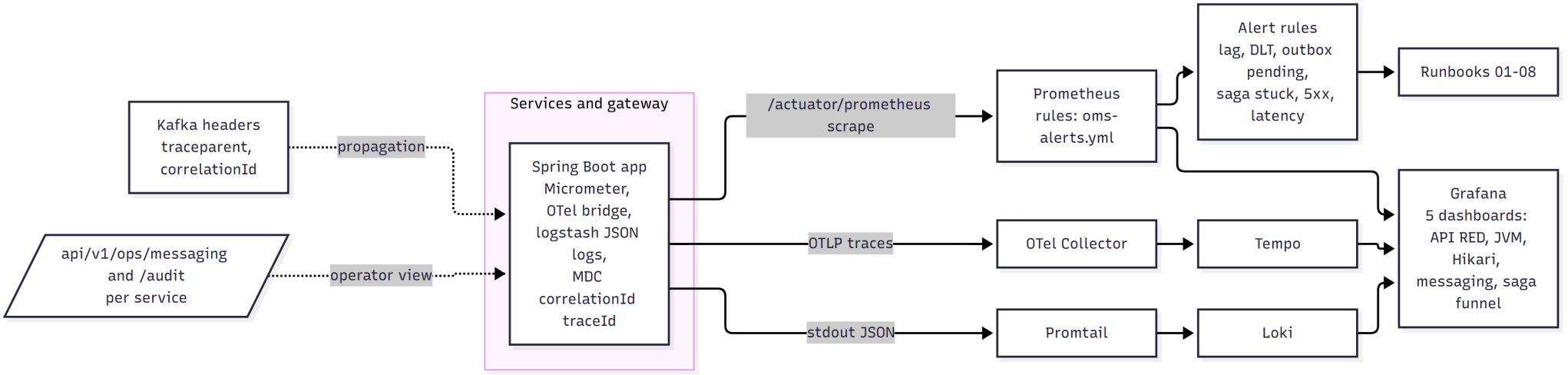


Figure 19. Observability architecture: logs, metrics, traces

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

What to say about maturity

- Implemented and unit tested: correlation id filter, log masking, Micrometer metrics for events, outbox, saga, inventory reservations, operational endpoints.
- Verified against the live stack: all 59 dashboard and alert expressions resolve to metric names present in a real scrape of the five services (`scripts/check-metric-names.mjs`; the first run found 5 missing names, fixed as D-05 to D-07), plus a correlation trace across five services (`docs/validation/correlation-trace-sample.md`).
- Statically checked: dashboards and compose wiring (`scripts/check-observability.py`).
- Not executed: the Docker observability stack (no Prometheus, Grafana, Tempo or Loki ran, so dashboards were not rendered and alert rules were not evaluated). The `traceparent` is not continuous across Kafka hops; the correlation id is the join key.

Source: `observability/prometheus/rules/oms-alerts.yml` *(lines 6 to 35)*

```
      - alert: OmsServiceDown
        expr: up{job=~"oms-.*"} == 0
        for: 2m
        labels: { severity: critical }
        annotations:
          summary: "{{ $labels.job }} instance {{ $labels.instance }} is down"
          description: "Prometheus cannot scrape the actuator endpoint for 2 minutes."

  - name: oms-api
    rules:
      - alert: OmsHighErrorRate
        expr: |
          sum by (job) (rate(http_server_requests_seconds_count{job=~"oms-.*",status=~"5.."}[5m]))
            /
          sum by (job) (rate(http_server_requests_seconds_count{job=~"oms-.*"}[5m])) > 0.05
        for: 5m
        labels: { severity: critical }
        annotations:
          summary: "5xx error ratio above 5% on {{ $labels.job }}"
          description: "Error ratio over the last 5m is {{ $value | humanizePercentage }}."
      - alert: OmsHighP95Latency
        # needs management.metrics.distribution.percentiles-histogram.http.server.requests=true
        expr: |
          histogram_quantile(0.95,
            sum by (job, le) (rate(http_server_requests_seconds_bucket{job=~"oms-.*",uri!~"/
actuator.*"}[5m]))) > 1.0
        for: 10m
        labels: { severity: warning }
        annotations:
          summary: "p95 latency above 1s on {{ $labels.job }}"
          description: "p95 is {{ $value | humanizeDuration }}."
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

15. Failure and recovery flows

Failure catalogue

Distributed systems fail in partial, boring ways. The platform assumes every hop can fail and gives each failure window a named, tested mechanism.

#	Failure	Detection	Automatic recovery	Manual fallback
1	Process crashes after the DB commit, before publishing	Unpublished rows in `outbox_events`, metric `oms.outbox.pending`	Outbox relay publishes on restart (at-least-once)	Inspect `GET /api/v1/ops/messaging` (`outboxPending`)
2	Broker unavailable	Relay errors, `attempts` and `last_error` on outbox rows	Relay keeps retrying; service stays up, orders are accepted	Runbook 02
3	Duplicate delivery of an event	`oms.events.duplicates`	`processed_events` primary key plus state-machine NO_OP	Runbook 05
4	Poison message (bad JSON, validation error)	`oms.events.dead_lettered`	Classified non-retryable, sent straight to `<topic>.DLT`	Inspect DLT headers (`x-exception-*`, original topic, partition, offset), fix, replay
5	Handler throws transiently (DB blip)	`oms.events.retries`	`DefaultErrorHandler`: 4 attempts, 500 ms doubling to 5 s, then DLT	Replay from DLT
6	Inventory REST call fails or is slow	Resilience4j metrics, saga step log `FAILED` or `RETRIED`	Retry (3 attempts) and circuit breaker; saga attempt counter	Admin retry endpoint
7	Saga step never completes (lost reply, consumer down)	Saga older than `oms.saga.stuck-after-seconds`	`SagaRecoveryJob` re-sends the idempotent command; after `oms.saga.max-attempts` order `FAILED` (non-compensable) or saga `NEEDS_ATTENTION` (compensable)	`POST /admin/orders/{id}/retry`, runbook 01
8	Order and reservation disagree	`ReconciliationService` queries	Repairs by issuing compensating commands, audits results	`POST /admin/reconciliation/run`
9	Reservation never confirmed	`expires_at` passed	`ReservationExpiryJob` expires it, releases stock, emits `InventoryReleased{EXPIRED}`; order goes to `FAILED`	None needed
10	Downstream service down at the edge	Gateway circuit breaker, 503 `UPSTREAM_UNAVAILABLE`	Circuit opens, `Retry-After: 5`, other services unaffected	Runbook 06
11	Fulfillment fails	`FulfillmentFailed` after its own 3 attempts	Saga compensates: release inventory, order `FAILED`	Admin fulfillment retry
12	Notification provider down	`notifications.status` `RETRYING` or `FAILED`	Backoff 1 s doubling to 30 s, 5 attempts	Admin notification retry

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Why these mechanisms are sufficient and where they stop

- **Outbox** closes the dual-write hole (state committed but event lost). It does not give exactly-once delivery: duplicates are possible and are handled downstream.
- **Idempotent consumers** make duplicates harmless for the business effect. The `processed_events` insert shares the transaction with the effect, so a crash between the two cannot record "processed" without applying the effect.
- **Recovery and reconciliation** handle failures that no retry can see, such as a reply that was never sent. They are periodic jobs and therefore bounded by their interval.
- **Compensation is semantic**: releasing a reservation does not undo history; the audit trail shows both the reservation and the release.
- Kafka replication factor is 1 locally and 3 in the production design; the local topology is not highly available.

Key code

Source: `services/platform-common/src/main/java/com/enterprise/oms/common/outbox/OutboxRelay.java` *(lines 24 to 63)*

```
public class OutboxRelay {

    private static final Logger log = LoggerFactory.getLogger(OutboxRelay.class);

    private final OutboxEventRepository repository;
    private final OutboxService outboxService;
    private final KafkaTemplate<String, String> kafka;
    private final TransactionTemplate tx;
    private final OutboxProperties props;

    public OutboxRelay(OutboxEventRepository repository, OutboxService outboxService,
                       KafkaTemplate<String, String> kafka, TransactionTemplate tx, OutboxProperties
props) {
        this.repository = repository;
        this.outboxService = outboxService;
        this.kafka = kafka;
        this.tx = tx;
        this.props = props;
    }

    @Scheduled(fixedDelayString = "${oms.outbox.poll-interval-ms:500}", initialDelayString =
"${oms.outbox.poll-interval-ms:500}")
    public void poll() {
        try {
            publishPending();
        } catch (RuntimeException e) {
            log.warn("Outbox relay cycle failed (will retry): {}", e.toString());
        }
    }

    @Scheduled(fixedDelayString = "${oms.outbox.cleanup-interval-ms:60000}", initialDelayString =
"${oms.outbox.cleanup-interval-ms:60000}")
    public void cleanup() {
        if (!props.isCleanupEnabled()) {
            return;
        }
        try {
            Instant before = Instant.now().minus(props.getRetention());
            Integer n = tx.execute(s -> repository.deletePublishedBefore(before));
            if (n != null && n > 0) {
                log.debug("Outbox cleanup removed {} published rows", n);
            }
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
        } catch (RuntimeException e) {
```

Order of operations for an on-call engineer

1. Is the service ready? (`/actuator/health/readiness`, gateway `GET /api/v1/ops/health`.)
2. Is the broker reachable and is lag growing? (`/api/v1/ops/messaging`, Prometheus.)
3. Is anything in a DLT? Read the exception headers before replaying.
4. Which orders are stuck? (`/admin/orders/failed`, then `/admin/saga/{orderId}` for the step log.)
5. Follow the matching runbook and finish with a post-incident review (`docs/governance/post-incident-review-template.md`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Figure 20. Failure and recovery flows

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

16. Testing strategy

Strategy

The test pyramid is deliberately bottom-heavy. Pure domain logic (the order state machine, fulfillment state machine, shortage calculation, idempotency fingerprint, backoff policy) is tested exhaustively and in milliseconds. Spring slices and full-context tests with H2 in PostgreSQL mode and embedded Kafka verify wiring, persistence and messaging. A small number of browser tests and load scripts sit at the top.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Testing pyramid for the platform

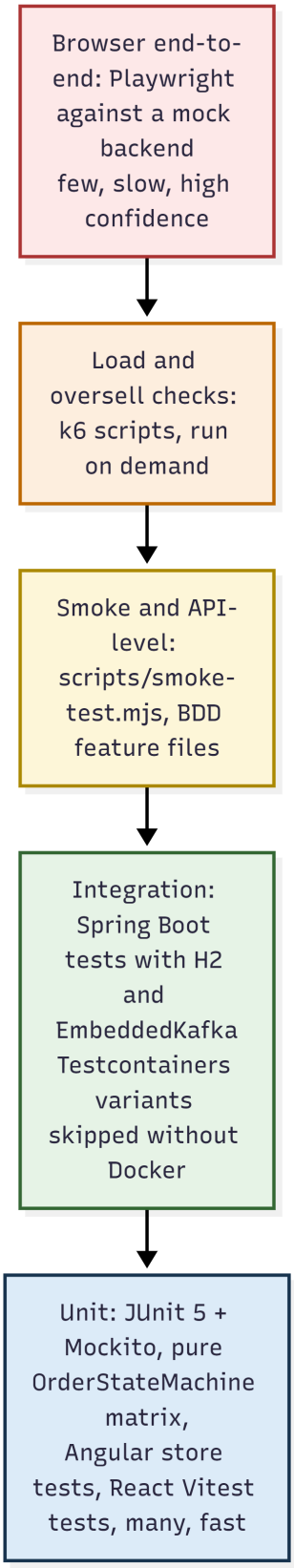


Figure 21. Testing pyramid for the platform

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Techniques used

Technique	Where	What it proves
Table-driven state machine tests	`services/order-service/src/test` (state machine tests), TDD walk-through in `docs/tdd/order-state-machine-tdd.md`	Every valid and invalid status and trigger pair behaves as documented, duplicates are no-ops
Concurrency test with `ExecutorService` and `CountDownLatch`	`services/inventory-service/src/test/java/com/enterprise/oms/inventory/ReservationConcurrencyTest.java`	No oversell across the three reservation strategies
Mockito unit tests	`services/inventory-service/src/test/java/com/enterprise/oms/inventory/service/ReservationServiceUnitTest.java`	Orchestration logic in isolation (mocks, argument captors, interaction verification)
MockMvc and WebTestClient API tests	`services/*/src/test` (`*ApiTest`)	Status codes, problem+json bodies, authorization matrix
Embedded Kafka integration tests	`services/order-service/src/test` (`*KafkaIT`)	Saga across topics, retry, DLT, resilience
Testcontainers PostgreSQL and Kafka	`*PostgresIT` classes, annotated `disabledWithoutDocker`	PostgreSQL-specific behaviour (skipped when Docker is absent)
BDD scenarios	`tests/bdd/*.feature`, executed through `InventoryBddScenariosTest` and the fulfillment equivalent	Business-readable acceptance criteria
Angular unit tests	`frontend/oms-ui/src/app/**/*.spec.ts` (Karma and Jasmine)	Stores, pure reducers, guards, interceptors
React unit and component tests	`frontend/oms-ui-react/src/**/*.test.ts(x)` (Vitest, Testing Library, MSW)	Slices, selectors, RTK Query layer, guards, forms, idempotency, polling, every screen; 207 of 207 pass, coverage 96.3 % statements
Browser end-to-end	`tests/e2e` (Playwright, mock backend and live mode)	18/18 against the mock backend and 18/18 live (UI to gateway to real services); the live run found a real blank-page defect
Load and oversell	`tests/e2e/oversell-live.mjs` (run), `performance/order-flow.js`, `performance/oversell.js` (k6, never run)	Live burst of 30 parallel orders for stock 3: 3 FULFILLED, 27 REJECTED; k6 scripts only syntax-checked
Smoke test	`scripts/smoke-test.mjs`	Local stack answers end to end

TDD is shown as a worked practice: `docs/tdd/` records the red, green, refactor steps for the state machine and the inventory reservation, and the flawed-AI regression tests were written failing first.

Quality gates

JaCoCo is configured in the reactor for coverage reports. CodeQL, Semgrep, Trivy, `npm audit` and gitleaks are defined in CI but, as stated in the limitations section, CI has not run on a runner.

Evidence for this build

The tables below are generated when the PDF is built from the actual `surefire-reports` and `failsafe-reports` XML files under each module's `target/` directory, so a rebuild after another test run reports the new numbers. Skipped counts are shown as they are: skipped is not passed.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Front-end testing: React numbers and the dual-UI end-to-end matrix

The React UI was verified on 2026-10-19: **207 of 207 Vitest tests pass** (9 files: slices and selectors, error normalization, the RTK Query layer against a Mock Service Worker, guards, forms, idempotency, polling, every screen), coverage **96.3 % statements, 93.5 % branches, 94.5 % functions, 96.7 % lines**, ESLint 0 errors, production build OK and `npm audit` 0 vulnerabilities. The Angular UI has 149 of 149 Karma tests.

Playwright, same 18 specs	Mock backend	Live stack
Angular (`E2E_UI=angular`)	18 / 18	18 / 18
React (`E2E_UI=react`)	18 / 18	18 / 18

The live stack used the gateway on port 18085 because a foreign process held 18080. Not executed for the React UI: the Docker image, the compose profile, the Kubernetes manifest, CI runs, accessibility audits and a narrow-screen visual check. Unit tests are per framework; only the end-to-end layer is shared, which is what proves behavioural parity.

Backend test results found on disk (surefire = unit and slice tests, failsafe = integration tests)

Module	Surefire	Failsafe (integration)	Newest report
platform-common	68 run, 0 failed, 0 skipped	no report	2026-10-18 17:11 UTC
order-service	436 run, 0 failed, 0 skipped	27 run, 0 failed, 7 skipped	2026-10-18 17:17 UTC
inventory-service	151 run, 0 failed, 1 skipped	32 run, 0 failed, 18 skipped	2026-10-18 17:21 UTC
fulfillment-service	44 run, 0 failed, 0 skipped	1 run, 0 failed, 1 skipped	2026-10-18 17:22 UTC
notification-service	60 run, 0 failed, 0 skipped	1 run, 0 failed, 1 skipped	2026-10-18 17:24 UTC
api-gateway	142 run, 0 failed, 0 skipped	no report	2026-10-18 17:26 UTC
platform-test-support	5 run, 0 failed, 1 skipped	no report	2026-10-18 17:12 UTC
local-stack	3 run, 0 failed, 0 skipped	no report	2026-10-18 17:12 UTC
Total	**909 run, 0 failed, 2 skipped**	**61 run, 0 failed, 27 skipped**	

Counts come from the JUnit XML in `target/surefire-reports` and `target/failsafe-reports`. A report shows what ran at the time of the last Maven run on that module, which may differ from the current source. Skipped integration tests are normally the Testcontainers PostgreSQL and Kafka classes, skipped because Docker is not installed on the development machine; they prove nothing about PostgreSQL behaviour.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Validation report (docs/validation/validation-report.md)

Date of all runs: **2026-10-18** (local time +05:30), except section 7 (React front end, **2026-10-19**). Machine: Windows 11 Enterprise, Git Bash, Node v24.13.1, GraalVM JDK 17.0.12 (by full path, `JAVA_HOME` set inline per command only), Maven 3.9.12, Chrome 155 (Playwright `channel: chrome`, headless), Playwright 1.64.1, Angular 20. **No Docker.** Everything below ran against the local no-Docker stack: embedded KRaft Kafka (`local-stack`, port 19092), order/inventory/fulfillment/notification services (H2 in PostgreSQL mode, same Flyway SQL), gateway on 18080, built Angular UI served by `scripts/serve-ui.mjs` on 4200. All data is synthetic. Secrets were random per run (`OMS_JWT_SECRET`, `OMS_DEMO_PASSWORD`), never written to a deliverable (searched the repo for the run's actual values: no hit).

1. Result at a glance

Area	Result
Backend reactor `mvn verify` (final)	BUILD SUCCESS, **968 test cases, 939 executed, 939 passed, 0 failed, 29 skipped (Docker/Postgres/benchmark)**; table in `backend-test-summary.md`
Frontend unit tests (Angular)	149 / 149 pass; `ng build` OK
React frontend (2026-10-19, section 7)	Vitest 207 / 207, coverage 96.3 % statements; ESLint 0 errors; build OK; `npm audit` 0; Playwright 18/18 on mock and live for each UI
Smoke (`scripts/smoke-test.mjs`)	10 / 10 checks, repeated on 3 stacks
API flows (`scripts/live-flows.mjs`)	54 / 54 checks (first run 53/54: the one failure was my wrong expectation, see 3.6)
Oversell (`tests/e2e/oversell-live.mjs`)	30 parallel orders for SKU-1019 (stock 3): 3 FULFILLED, 27 REJECTED, never above 3 committed (201 samples); passed twice on the fixed build
Resilience (`tests/e2e/resilience-live.mjs`)	passed (order parked while fulfillment down, completes after restart, exactly one fulfillment row)
Playwright **live** (UI -> gateway -> real services)	**18 / 18** (final stack); first live run found 2 real defects, see 3.9
Playwright **mock** (mock backend)	**18 / 18** (re-run at the end)
Postman collection executed by Node runner	41 / 41
Event schemas vs live Kafka	218 records (final capture; 364 in an earlier one), all valid, 15 / 15 event types observed, 0 in DLTs
Metric names in `observability/` vs live scrape	33 names referenced: 32 found, 1 external (`up`), **0 missing** (first run: 5 missing, fixed)
Defects found and fixed	11 (`integration-defects.md`), plus a list of known issues not fixed
Static infra validation (`scripts/validate-infra.sh`)	47 PASS, 0 FAIL, 2 SKIP (docker compose config, gitleaks)

2. Commands to reproduce (from the repo root)

```
# 1. build + test everything (about 19 min; log under .run/, which is git-ignored)
JAVA_HOME=/c/Users/sahukari.rao/.jdks/graalvm-jdk-17.0.12 ~/.m2/wrapper/dists/apache-maven-3.9.12-bin/5nmfsn99br87k5d4ajlekdq10k/apache-maven-3.9.12/bin/mvn -B -f services/pom.xml verify
node scripts/test-summary.mjs --md docs/validation/backend-test-summary.md
(cd frontend/oms-ui && npm test && npx ng build)
# 2. start the stack (random secrets; password printed once; takes ~2.5 min on this machine)
bash scripts/run-local.sh --no-build --json-logs --ui          # --ui serves the built UI on :4200
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
set -a; . .run/secrets.env; set +a                                  # exports OMS_DEMO_PASSWORD (+ JWT
secret) into THIS shell only
# 3. live checks
node scripts/smoke-test.mjs
node scripts/live-flows.mjs
node tests/e2e/oversell-live.mjs 30
node tests/e2e/resilience-live.mjs
(cd tests/e2e && E2E_BASE_URL=http://localhost:4200 E2E_DEMO_PASSWORD="$OMS_DEMO_PASSWORD" npx
playwright test)
(cd tests/e2e && npx playwright test)                               # mock mode (no E2E_BASE_URL), starts
mock-backend on :4300
node scripts/run-api-collection.mjs
bash scripts/capture-events.sh && node scripts/validate-events.mjs
node scripts/capture-api-examples.mjs                               # rewrites docs/api/examples.md
(OMS_JWT_SECRET also exported -> expired-token example)
node scripts/export-openapi.mjs --check && node scripts/gen-gateway-openapi.mjs
node scripts/check-metric-names.mjs
node scripts/correlation-trace.mjs <correlationId> <orderId>   # needs --json-logs
# 4. generators and static checks
node scripts/gen-event-schemas.mjs && node scripts/gen-api-collection.mjs
bash scripts/validate-infra.sh && python scripts/check-observability.py
# 5. stop everything (releases the jars)
bash scripts/stop-local.sh
```

3. Checks that were run

3.1 Build and unit/integration tests

Check	Command	Actual result
Baseline reactor	`mvn -B -f services/pom.xml verify` (19:06-19:45, 38 min 41 s while I was editing in parallel)	BUILD SUCCESS; 896 unit + 61 IT cases, 0 failures, 29 skipped
Final reactor	same command (21:13-21:32, 18 min 39 s, idle machine)	BUILD SUCCESS; 907 unit + 61 IT = 968 cases, 939 executed, 0 failures, 29 skipped (`backend-test-summary.md`)
New regression tests, targeted	`mvn -o -pl ... test -Dtest=...` before the final run	`ProdProfileDocsDisabledTest` (4 services), `MessagingMetricsTest`, `PlatformDefaultsTest`, `BulkheadConcurrencyTest` (also shown to FAIL with the old 25-call limit), `OrderTransitionsLoggingTest`, `FulfillmentServiceTest` all pass
Frontend	`npm test` (Chrome Headless 155) / `npx ng build`	149 of 149 SUCCESS; build OK

3.2 Stack start

`scripts/run-local.sh` failed the first time (D-01, a Node crash in the readiness probe). After the fix it started cleanly three times from the shell and a fourth time at the end, when port 18080 turned out to be occupied by **a different project's process** (`cisco-fde-ai-ops-platform/.../ops-platform-1.0.0.jar`, PID 39348, started by someone else after my previous stop): my gateway could not bind and `run-local.sh` still reported "ready" because the foreign app answered the health URL on that port. I did NOT stop that process. For the last check (the Postman collection run) I started my gateway on port 18090 with the same jar and environment and ran the collection against it; everything else in this report ran on port 18080 before the conflict appeared. Fix made afterwards: `run-local.sh` now refuses to start when any of the ports 19092/18080-18084 is already taken (tested with a dummy listener on 18080: exit 1, nothing started; the foreign process had disappeared again a few minutes after I saw it).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

3.3 Smoke and API flows (through the gateway)

`node scripts/smoke-test.mjs` PASS 10/10 (401 without token, login, products, create 202, idempotent replay + header, missing key 400, FULFILLED, history). `node scripts/live-flows.mjs` PASS 54/54: happy path to FULFILLED (history path `CREATED>VALIDATED>INVENTORY_RESERVED>CONFIRMED>FULFILLED`, saga COMPLETED with 8 steps, reservation COMMITTED+shipped, stock 498 -> 496, tracking number, 5 notifications SENT, 5 audit entries, customer 403 on saga/fulfillment/audit); insufficient stock (SKU-1020 stock 0, SKU-1019 over-ask 4 > 3 with stock untouched, all-or-nothing mixed order, listed in `/admin/orders/failed`); idempotency (replay with `Idempotent-Replay: true`, 422 reuse, 400 missing, 8 parallel requests with one key -> one order); cancel (immediately: `CREATED>VALIDATED>CANCELLED`; at CONFIRMED: `...CONFIRMED>COMPENSATION_PENDING>CANCELLED` with reservation RELEASED; terminal: 409 `INVALID_STATE_TRANSITION`; operator cancel); FAILTOWN (`CONFIRMED>COMPENSATION_PENDING>FAILED`, reservation RELEASED/FULFILLMENT_FAILURE, available restored, fulfillment FAILED attempts=3, retry of terminal order 409); cross-customer (404 on get/history/cancel/profile, forced own-orders filter, 403 creating for another customer, customers see `available` only, `/internal/**` 404 at the edge); notification retry path (customer3 `fail.example.test`: every notification SENT only after attempt 3, transient RETRYING with `lastError` observed, cleared on success); ops (4x messaging: processed > 0, dead-lettered 0, outboxPending 0, lagAvailable true; aggregate health UP; reconciliation admin 200 / operator 403).

3.4 Concurrency

Run 1 (before fix): `{"202":29, "503":1}`: the 503 was the gateway's hidden 25-call bulkhead (D-02); inventory invariants still held (3 FULFILLED, 26 REJECTED, max committed 3). Runs 2 and 3 (fixed gateway, fresh stock): burst of 30 accepted in ~4 s, **FULFILLED 3, REJECTED 27**, `max(shipped+reserved) = 3`, `min(available) = 0`, 30 reservation rows of which exactly 3 RESERVED/COMMITTED, ledger `onHand 3 -> 0, reserved 0 -> 0`. Orders were spread over four identities (customer1, customer2, operator1, admin1) because `POST /orders` is rate limited per subject (capacity 20): this is a measured limit of the design, not a test artefact. Numbers are from `.run/oversell*.out` of the actual runs; the script recomputes every number (no constants).

3.5 Resilience

`node tests/e2e/resilience-live.mjs`: scenario A (stop fulfillment-service, place order, 8 s later still `CONFIRMED`, restart): order FULFILLED 39 s after the start command (includes JVM boot), exactly one fulfillment row. Scenario B (stop triggered when the order is first seen CONFIRMED; it was still CONFIRMED when the stop completed): FULFILLED after restart, no duplicate row. This shows that the `StartFulfillment` command waits durably in Kafka and the consumer resumes at its committed offset.

3.6 Playwright

Live: first run 1 failed (blank order page, D-03) and the next surfaced two spec-wording differences (D-04); then 18/18 on two stacks, the last one after the final rebuild. One spec expectation changed for the real backend: retrying a TERMINAL failed order is 409 (accepted alongside the mock's success). Mock: 18/18 (2.9 min). `live-flows.mjs` first run failed one check (`lastError` after success): wrong expectation, the service clears `lastError` on success; the script now observes the RETRYING state instead.

3.7 Contract artefacts

- **OpenAPI:** `specs/openapi/{order, inventory, fulfillment, notification}-service.json` exported from `/v3/api-docs` of the running services (12/10/5/4 paths); `--check` reports no drift on the final stack. `specs/openapi/api-gateway.json` is **generated, not exported** (the reactive gateway has no springdoc) and describes only the gateway's own endpoints (token, ops/health) plus the proxied route table.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- **Events:** `specs/events/*.schema.json` (15 event schemas + envelope, written to match contract section 7); `scripts/capture-events.sh` consumed every `oms.*` topic from the live broker with a throw-away consumer group (Java single-file program, `tools/capture`), `scripts/validate-events.mjs` validated 218 records: 218 valid, 0 invalid, all 15 types observed, 0 DLT records. The committed sample is `tests/e2e/samples/kafka-events.sample.jsonl` (30 records). The validator is a small purpose-built subset implementation (`scripts/lib/mini-jsonschema.mjs`, no `$ref`, no `additionalProperties` restriction on purpose: tolerant reader); a negative test (missing `items`, non-UUID `orderId`, wrong `producer`) is reported invalid. No general-purpose JSON Schema engine was used.
- **API examples:** `docs/api/examples.md` regenerated by `scripts/capture-api-examples.mjs` from live responses: 38 examples (success, 400 validation, 400 missing key, 422 reuse, replay, 404, 403, 409 invalid cancel, 409 stock conflict, 401 none/forged/expired/`alg:none`, 413, 429, saga, audit, messaging, health). Tokens redacted.
- **Collection:** `tests/api-collection/oms.postman_collection.json` (41 requests), `oms.postman_environment.template.json` (placeholders only), `oms.http` (REST Client), generated from one catalog by `scripts/gen-api-collection.mjs`. The collection was executed by `scripts/run-api-collection.mjs`, my own small interpreter: 41/41 pass. **It was not opened in Postman or newman, and `oms.http` was not run in the VS Code REST Client**, so the Postman test scripts and the `.http` variable syntax are unverified in their native tools.

3.8 Observability

- `node scripts/check-metric-names.mjs` scraped all five `/actuator/prometheus` endpoints (313 series names) and checked the 59 expressions of 5 dashboards and 12 alert rules. First run: MISSING `kafka_consumer_fetch_manager_records_lag_max`, `oms_events_dead_lettered_total`, `oms_events_duplicates_total`, `oms_events_retries_total`, `spring_kafka_listener_seconds_bucket`, and `http_server_requests_seconds_bucket` existed on 2 of 5 apps only. After the fixes (D-05..D-07): 0 missing. The EXTERNAL list is just Prometheus' own `up`; the dashboards use no kafka-exporter/postgres-exporter series, so no infra metric needed to be excluded.
- Correlation trace: `docs/validation/correlation-trace-sample.md` (27 real JSON log lines, 5 services, e-mail masked). traceId is NOT stable across Kafka hops here and no OTel collector/Tempo/Loki/Grafana/Prometheus server ran, so dashboards were validated by metric name against the live scrape, not rendered, and alert rules were not evaluated by Prometheus.
- `python scripts/check-observability.py`: OK (5 dashboards / 40 panels, 12 alert rules).

3.9 Security

- Kubernetes base ConfigMap: profile default `json-logs`; `kustomize build` renders `dev,json-logs` (dev, test), `prod,json-logs` (prod), base `json-logs`; `scripts/validate-infra.sh` 47 PASS / 0 FAIL / 2 SKIP, including kubeconform on the rendered overlays.
- Prod docs off: `ProdProfileDocsDisabledTest` x4 (config-level). Live gateway probe (dev profile): `/v3/api-docs` 401 without token, 404 with token; `/swagger-ui.html` 401; `/swagger-ui/index.html` 404; `/api/v1/internal/products/validate` 404; path traversal into `/actuator` via `/ops/order/..` 404; `/actuator/env` 401; gateway's own `/actuator/health` 200 and `/actuator/prometheus` open (documented).
- `docs/security/threat-model.md` section 10: HS256 caveat, RS256/JWKS path (code reviewed, **untested**, gaps listed), prometheus exposure.

4. NOT EXECUTED (stated plainly)

Not done	Why / consequence
Docker, Docker Compose, image builds (all Dockerfiles), nginx image of the UI	Docker is not installed. `docker compose config` was SKIPPED by `validate-infra.sh`; compose files were only structure-checked.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Not done	Why / consequence
`kubectl apply` / any cluster, NetworkPolicy enforcement, HPA/KEDA, Ingress, ServiceMonitor, PrometheusRule on a cluster	no cluster. Manifests were rendered with kustomize and checked with kubeconform (static) only.
`terraform plan/apply`	only `fmt -check` and `validate` ran (no credentials).
k6 (`performance/*.js`)	k6 binary not available; only `node --check` syntax of the scripts. No load/latency numbers exist beyond the 30-order burst above.
Real GitHub Actions run, Jenkins	workflow/Jenkinsfile only parsed.
Real Kafka cluster (multi-broker, replication 3, TLS/SASL, ACLs)	only the embedded single-node KRaft broker; delivery semantics were observed on it (no duplicates, no DLT records), not proven for a cluster.
Redis (rate limiter, product cache), `oms.redis.enabled=true`	in-memory fallback only; `RedisOrderRateLimiterTest` etc. are in the suite but not against a real Redis.
OIDC / JWKS / RS256 (gateway and services)	no IdP; code path exists, no test, no run.
PostgreSQL (any), Testcontainers PostgreSQL/Kafka tests	28 tests skipped (`disabledWithoutDocker`); all runtime evidence is on H2 in PostgreSQL mode. Row locking (`pessimistic` strategy), Postgres-specific plans and `docs/database/*` EXPLAIN numbers are not re-verified.
OpenTelemetry collector, Tempo, Loki/promtail, Grafana, Prometheus server	not started; see 3.8. End-to-end distributed trace not shown.
Postman / newman / VS Code REST Client themselves	only my Node interpreter of the collection.
`ng serve` (dev proxy)	the built UI served by `scripts/serve-ui.mjs` was used instead.
Multiple gateway/service replicas, rolling restarts, graceful-shutdown drain under load	single instance of everything.
Browsers other than Chrome 155, accessibility audit, mobile viewports	not run.
Long soak, memory leak and GC behaviour	not run; JVMs capped at 384 MB each for the demo.
`git commit` / `git push`	nothing committed or pushed (the git root is the home directory with everything untracked); deliverables are files in the working tree.

5. Other honest observations

- Stack start takes about 2.5 minutes on this machine (six JVMs at once); `run-local.sh` waits up to 300 s per service.
- `POST /orders` is rate limited per caller (20 burst, 5/s): fine for users, and it shaped the oversell test design.
- Contract deviations and additions from this pass are in `docs/architecture/00-shared-contract.md` section 15.

6. Finalization pass (after the integration run)

Changes made after the runs above, and what was re-verified:

-

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Infrastructure alignment: JVM tmpdir is now `/tmp` everywhere (Dockerfiles, Compose `tmpfs`, Kubernetes `emptyDir` at `/tmp`); the prod overlay, NetworkPolicy and Terraform security group agree on MSK IAM authentication (port 9098, no SCRAM); Terraform gained `aws_secretsmanager_secret.service_db` (containers only) for the per-service DB credentials the ExternalSecret example references. `bash scripts/validate-infra.sh` was re-run: **47 PASS, 0 FAIL, 2 SKIP** (docker compose config, gitleaks), including `terraform fmt -check` and `terraform validate`.

- **Histograms (D-07):** re-checked in code: platform-common defaults cover the four servlet services and the gateway sets its own; nothing further needed.
- **New code:** `RetentionPurgeJob` (platform-common, `oms.retention.*`) with `RetentionPurgeJobTest` (2 cases, H2). `platform-common` suite re-run green (68 cases); notification and fulfillment module Kafka/API/context tests re-run green with the new auto-configuration active. The full reactor WAS re-run afterwards (`mvn -B -f services/pom.xml verify`, 2026-10-18, BUILD SUCCESS): **970 cases (909 unit + 61 IT), 941 executed, 941 passed, 0 failed, 29 skipped**; `backend-test-summary.md` was regenerated from that run. Older 968/939 figures elsewhere in these docs are the earlier run.
- **Documentation:** governance docs refreshed against the code, `docs/known-limitations.md` added, PDFs rebuilt (architecture PDF gained Appendix B with the 11 defects; interview PDF gained Q145 to Q150 on them).

7. React front end verification (frontend/oms-ui-react, 2026-10-19)

Context: the original brief mandated Angular. The user later explicitly asked for a React + Redux Toolkit front end as well; it was added as a second, feature-equivalent UI on the unchanged backend and gateway contract (ADR-0013). Nothing under `services/` or `gateway/` was changed for it. Same machine as above (Windows 11, Git Bash, Node v24), no Docker.

Check	Result (executed 2026-10-19)
Vitest unit and component tests (`npm test`, jsdom + MSW)	**207 / 207 pass** in 9 files
Coverage (`npm run test:coverage`, v8)	**96.3 % statements, 93.5 % branches, 94.5 % functions, 96.7 % lines**
ESLint (typescript-eslint, react-hooks, jsx-a11y)	0 errors
Production build (`tsc -b` strict + `vite build`)	OK
`npm audit` (including dev dependencies)	0 vulnerabilities
Playwright, the same 18 specs, React UI, mock backend (`E2E_UI=react`)	**18 / 18**
Playwright, React UI, live stack (gateway -> services -> embedded Kafka -> H2)	**18 / 18**
Playwright, Angular UI, mock backend (re-run for the matrix)	**18 / 18**
Playwright, Angular UI, live stack	**18 / 18**
`bash scripts/validate-infra.sh` (re-run)	**47 PASS, 0 FAIL, 2 SKIP** (docker compose config, gitleaks)

Live-run note: the live stack used the gateway on port **18085** (`OMS_GATEWAY_PORT=18085`) because a foreign process already held 18080. The UIs were served on 4200 (Angular) and 4201 (React) with a same-origin `/api` proxy.

NOT executed for the React UI (stated plainly):

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- The React Docker image build, the compose profile `react`, and the Kubernetes `frontend-react.yaml` apply (no Docker, no cluster). Only `kustomize build` and `kubeconform` (part of the 47 passing infra checks) touched the manifest.
- The CI and Jenkins changes for the React UI (text only; no GitHub Actions or Jenkins run).
- Accessibility audits: no axe, no Lighthouse, no screen-reader pass. Only lint rules and role-based Playwright locators.
- A narrow-screen (mobile) visual check; only desktop Chrome was used.
- Browsers other than Chrome; contrast measurement; load or soak tests of the UI.

Known limitations of the React UI are in `docs/known-limitations.md` (types copied not generated, `style-src 'unsafe-inline'` for MUI/Emotion, token in `sessionStorage`).

See also `docs/known-limitations.md` and the root `README.md`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

17. AI-assisted development workflow

Principle

AI assistants are fast juniors with excellent recall and no accountability. They draft; humans own the result. The workspace in `ai-engineering/` makes that operational: give the assistant the right context cheaply, make it work from a written spec, and review its output against a catalogue of the mistakes assistants actually make in this domain.

The workflow

1. **Spec first.** Write or load a feature spec (`ai-engineering/templates/feature-spec-template.md`, worked examples `specs/features/order-cancellation.md` and `specs/features/inventory-reservation.md`). Acceptance criteria are testable sentences mapped to test classes.
2. **Layered context.** Always load `ai-engineering/context/00-repo-context.md` (small); add only the module files the task needs (architecture, Java, Angular, API, security, migrations, tests, performance, Kafka, saga rules). This keeps tokens low and answers on-topic (`guides/token-optimization.md`).
3. **Role prompts.** Templates for implementation, testing, security and documentation agents live in `ai-engineering/agents/`; reusable prompts in `ai-engineering/prompts/prompt-library.md`.
4. **Draft, then read every line.** The author compares the draft with the flawed-example catalogue.
5. **Automated gates.** Tests, scanners and CI must pass; the assistant never merges.
6. **Human review** with `ai-engineering/checklists/pr-checklist.md` and the governance code-review checklist.
7. **Definition of done** (`ai-engineering/checklists/definition-of-done.md`): docs updated, evidence recorded.
8. **Feed back lessons** into context files or the flawed catalogue.

The flawed-AI catalogue

Twelve realistic mistakes with corrected versions. Ten are runnable Java (`ai-engineering/flawed-ai-examples/java`, with regression tests); two are Angular snippets.

#	Mistake an assistant makes	Why it is dangerous	Fix
01	Check-then-act stock reservation	Oversells under concurrency	Atomic conditional UPDATE
02	`@Transactional` on a method called through `this`	Proxy bypassed, no transaction	Call through another bean or restructure
03	Publish to Kafka inside the DB transaction	Dual write: event without commit or commit without event	Transactional outbox
04	Non-idempotent consumer	Duplicate effects on redelivery	`processed_events` in same transaction
05	`customerId` taken from the request body	BOLA: read or create for others	Derive identity from the JWT, 404 for others
06	Logging tokens and emails	PII and credential leakage	Masking, never log secrets
07	Unbounded pagination and N+1 queries	Memory and latency blow-ups	Paging limits, fetch joins or batch
08	SQL built by string concatenation	SQL injection	Bound parameters

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

#	Mistake an assistant makes	Why it is dangerous	Fix
09	Catch Exception and acknowledge	Silent message loss	Let the container retry then DLT
10	Mutable singleton state	Cross-request data races	Stateless beans, immutable values
11	Angular subscriptions never unsubscribed	Memory leaks, duplicate handlers	`takeUntilDestroyed`, async pipe, signals
12	Route guard treated as security	Client checks are bypassable	Backend-authoritative authorization

Reviewing AI-generated code: a five-minute method

- Does it compile against the real APIs? Assistants invent method names and dependency versions; run the build.
- Does every state change have an owner, a transaction boundary and an idempotency story?
- Where does identity come from? Is every resource access checked?
- What happens on retry, on duplicate, on partial failure, on slow dependency?
- Are inputs bounded and parameters bound? Are secrets and personal data kept out of logs and fixtures?
- Is there a test that would have caught the bug the assistant just fixed?

Risks and controls

Risk	Control
Hallucinated APIs or versions	Compile, run tests, read library docs
Secrets pasted into prompts	Never include secrets, gitleaks in CI, `.env` excluded from the ZIP
Prompt injection from external text (issues, web pages, logs)	Treat external content as data, never as instructions; human approves actions
Licence or provenance concerns	Review generated snippets; prefer standard library and documented patterns
Over-trust	Human accountability recorded in the PR; AI used as reviewer second opinion, not approver

Honesty statement

The agent definitions are templates and were not executed as an automated pipeline. The Java flawed examples were executed with Maven and JUnit; the Angular snippets were not. This repository itself was produced with AI assistance and coordinated work-streams, which is exactly why the review catalogue and evidence labels exist.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

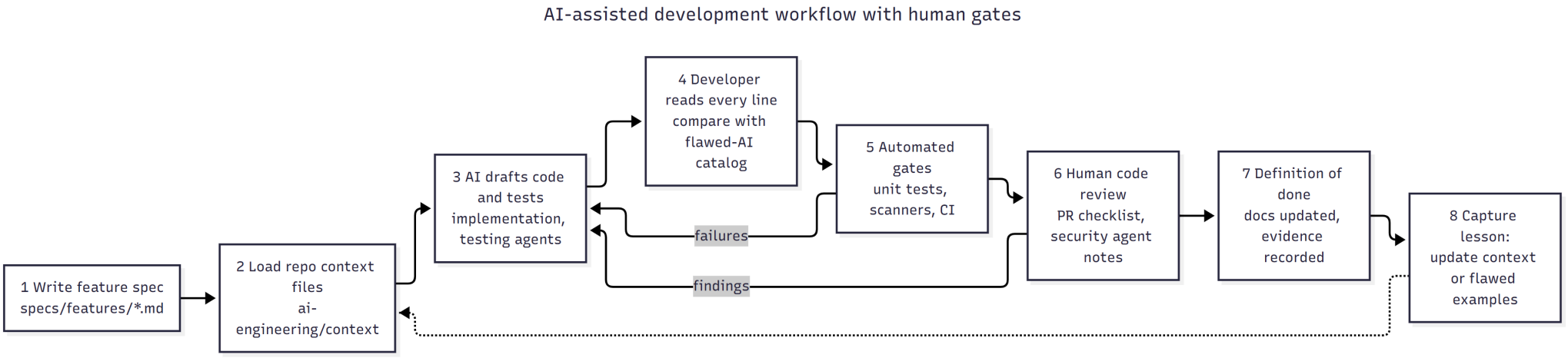


Figure 22. AI-assisted development workflow with human gates

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

18. Architecture Decision Records

MADR-style records for the OMS platform. New ADRs start from 0000-template.md. Never edit an accepted ADR's decision; supersede it with a new one and update the status line.

ADR	Title	Status	Date
0001	Four services plus a gateway instead of a modular monolith	Accepted	2026-10
0002	Kafka event design (envelope, topics, keys, versioning)	Accepted	2026-10
0003	Orchestrated saga for order processing	Accepted	2026-10
0004	Database per service	Accepted	2026-10
0005	Idempotency at API, consumer and state machine	Accepted	2026-10
0006	Concurrency control for stock reservation	Accepted	2026-10
0007	URL-path API versioning, additive-only evolution	Accepted	2026-10
0008	CI/CD pipeline and deployment model	Accepted	2026-10
0009	Transactional outbox for event publication	Accepted	2026-10
0010	JWT, OIDC-compatible resource servers, backend-authoritative authz	Accepted	2026-10
0011	Observability stack (Prometheus, Grafana, Loki, Tempo, OTel)	Accepted	2026-10
0012	Angular state management with signals, not NgRx	Accepted	2026-10
0013	Two front ends (Angular and React + Redux) on one backend contract	Accepted	2026-10

Companion documents: shared contract, API versioning, event schema evolution, delivery semantics, saga and consistency, Angular architecture, React architecture.

Reading order for interviews or onboarding: 0001, 0004, 0002, 0009, 0005, 0003, 0006, then the rest.

ADR-0001: Four services plus a gateway instead of a modular monolith

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

The OMS must demonstrate order intake, inventory reservation, simulated fulfillment and customer notification. Those four capabilities have different change rates, different scaling profiles (inventory reads and reservations are hot; notification is bursty and tolerates delay) and different failure behaviour (a notification outage must never block order acceptance). The project is also a reference for distributed-systems patterns (saga, outbox, idempotent consumers), which only exist as real problems once there is a network and more than one database.

Honest framing: a team of this size, with this traffic, would normally ship a modular monolith first. This repository deliberately chooses services because the learning and demonstration goals need the distributed failure modes to be real.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Decision drivers

- Independent deployability and failure isolation for notification and fulfillment.
- Data ownership boundaries that are enforced by the runtime, not by convention (see ADR-0004).
- Need to demonstrate eventual consistency, saga and delivery semantics honestly.
- Operational cost must stay manageable on a developer laptop without Docker (embedded Kafka and H2, see `services/local-stack`).

Considered options

1. Modular monolith (one deployable, module-per-capability, one schema with per-module table prefixes).
2. Four services plus an edge gateway (chosen).
3. Many fine-grained services (separate customer service, product service, payment service, and so on).

Decision outcome

Option 2: `order-service`, `inventory-service`, `fulfillment-service`, `notification-service`, and an `api-gateway` (Spring Cloud Gateway, physically in `gateway/`). Service boundaries follow data ownership in contract section 3. Shared technical code (envelope, outbox, idempotency, security, error model) lives in the library `services/platform-common`, never shared business code and never shared entities.

Rules that keep this from becoming a distributed monolith:

- No synchronous call chains deeper than one hop. The only sync call is order to inventory product validation (`/api/v1/internal/products/validate`), protected by a circuit breaker, retry and timeouts (connect 2s, read 3s).
- All state changes that other services care about travel as events (ADR-0002) via the outbox (ADR-0009).
- No cross-service database access (ADR-0004).
- The library `platform-common` carries only cross-cutting infrastructure; changing it is a coordinated, versioned release (`1.0.0` across the reactor).

Consequences

- Good: failure isolation (notification down does not stop orders), independent scaling (KEDA Kafka-lag scaling is configured in `infrastructure/kubernetes/keda`), clear ownership, realistic demonstration of saga/outbox/idempotency.
- Bad: eventual consistency everywhere (see `docs/architecture/saga-and-consistency.md`), higher operational surface (4 databases, Kafka, gateway), harder local debugging, duplicated boilerplate (mitigated by platform-common), end-to-end tests are slower and need orchestration. Distributed tracing becomes mandatory (ADR-0011).

When to merge back into a modular monolith

Merge when most of these hold: a single team owns all four services; no service has ever scaled independently; more than half of releases touch two or more services together (lock-step deploys); the saga adds more incidents than it prevents; infrastructure cost or on-call burden dominates. Mechanically the merge is cheap if boundaries were respected: each service already owns its tables, so modules keep the same schemas, replace Kafka commands with in-process calls behind the same interfaces, and keep the outbox only for genuinely external consumers. The state machine and idempotency logic stay unchanged.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alternatives rejected

- Modular monolith: the better default for real teams, rejected here only because the reference goals (independent failure, real eventual consistency, per-service scaling) cannot be demonstrated in one process.
- Many fine-grained services: multiplies network hops and sagas without a business reason; customer data sits with orders, product data sits with stock because they change together.

Links

- `docs/architecture/00-shared-contract.md` sections 3, 7, 8
- `services/platform-common`, `services/local-stack`, `gateway/`
- ADR-0003, ADR-0004, ADR-0009

ADR-0002: Kafka event design (envelope, topics, keys, versioning)

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

Services communicate state changes and saga commands asynchronously. We need one wire format, a topic layout that makes ordering and ownership clear, and a versioning strategy that lets producers and consumers deploy independently.

Decision drivers

- Per-order ordering (an order must never see `InventoryReleased` before `InventoryReserved`).
- Single owner per topic (one producer service) so schemas have one authority.
- Independent deployment: old consumers must survive new producers.
- Debuggability: every record carries correlation and causation identifiers.

Considered options

1. One topic per event type.
2. One topic per aggregate or interaction (chosen), several event types per topic, discriminated by `eventType`.
3. A single global topic.
4. Avro/Protobuf with a schema registry versus JSON with committed JSON Schema.

Decision outcome

Topology (contract section 7): `oms.order.events.v1`, `oms.inventory.commands.v1`, `oms.inventory.events.v1`, `oms.fulfillment.commands.v1`, `oms.fulfillment.events.v1`, plus `<topic>.DLT`. Constants live in `Topics` (`services/platform-common/src/main/java/com/enterprise/oms/common/events`). Commands and events are separate topics so authorization, retention and consumers differ. Naming: `oms.<domain>.<events|commands>.v<major>`.

- Key: `orderId`, 3 partitions (replication 1 locally, 3 in prod). All events for one order land in one partition, so per-order ordering holds. There is no ordering guarantee across orders and none is needed.
-

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Envelope: `EventEnvelope<T>` with `eventId, eventType, schemaVersion, occurredAt, correlationId, causationId, aggregateType, aggregateId, producer, payload`. Kafka headers duplicate `correlationId, eventType, schemaVersion, eventId` (plus `traceparent`) so routing and tracing do not require parsing the body.

- Serialization: JSON via Jackson (`EventJson`), JSON Schema files committed under `specs/events`. Human-readable in DLTs and logs; no registry to operate on a laptop.
- Versioning: `schemaVersion` inside the envelope for additive change; the topic suffix `.v1` bumps only for breaking change, with dual publishing during migration. Details and a worked example are in `docs/architecture/event-schema-evolution.md`.
- Consumers use a tolerant reader (`FAIL_ON_UNKNOWN_PROPERTIES=false`). Unknown `eventType` is ignored and counted; malformed records are classified non-retryable (`NonRetryableEventException`, `MalformedEventException`) and go straight to the DLT. Routing is by `EventRouter` per consumer.
- Consumer group id equals the service name. Retries: `DefaultErrorHandler`, 4 attempts, 500 ms doubling to 5 s, then `DeadLetterPublishingRecoverer`.

Consequences

- Good: strict per-order ordering, simple ownership, tolerant evolution, readable DLTs, no registry dependency.
- Bad: JSON is larger and unchecked at publish time unless contract tests run (they must); multiple event types per topic means consumers must filter (cheap, but each consumer discards events it does not handle); a hot order key cannot be spread across partitions; changing partition count later reshuffles keys.
- Not claimed: Kafka itself does not provide exactly-once across the database and Kafka. Delivery is at-least-once with idempotent consumers; see ADR-0005 and `docs/architecture/delivery-semantics.md`.

Alternatives rejected

- One topic per event type: 15+ topics for four services, loses cross-event ordering for one order (reserved and released could arrive out of order on different topics).
- Single global topic: all consumers read everything, no per-topic ACLs, noisy lag metrics.
- Avro/Protobuf plus registry: stronger compatibility enforcement, but extra infrastructure that cannot run in the no-Docker local stack; JSON Schema plus contract tests gives most of the safety. Revisit if the number of producers grows.

Links

- Contract section 7; `services/platform-common` packages `events`, `messaging`
- `docs/architecture/event-schema-evolution.md`, `docs/architecture/delivery-semantics.md`
- ADR-0005, ADR-0009

ADR-0003: Orchestrated saga for order processing

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

An order spans validation, inventory reservation, confirmation, fulfillment and, on failure, compensation (release inventory). There is no distributed ACID transaction (ADR-0004). We need a coordination style whose state is visible, recoverable and testable.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Decision drivers

- Operators must answer "where is order X stuck and why" with one query.
- Compensation paths (cancel, fulfillment failure, reservation expiry) must be explicit and exhaustively tested.
- Recovery of lost messages must be possible without guesswork.
- Keep services decoupled: inventory and fulfillment should not know about each other's workflows.

Considered options

1. Choreography: each service reacts to events and emits new ones, no central owner.
2. Orchestration: order-service owns a saga instance and sends commands (chosen).
3. Workflow engine (Temporal, Camunda, Step Functions).

Decision outcome

Orchestration inside order-service. `OrderSagaOrchestrator` drives `VALIDATE -> RESERVE_INVENTORY -> CONFIRM_ORDER -> FULFILL` with compensation `RELEASE_INVENTORY`. State is persisted in `saga_instances` and `saga_step_log`; the order lifecycle is the pure `OrderStateMachine` (contract section 6) so every transition is unit-testable and idempotent. Commands go out on `oms.inventory.commands.v1` and `oms.fulfillment.commands.v1`; participants reply with events on their own topics. A scheduled `SagaRecoveryJob` re-sends the idempotent command for steps stuck beyond `oms.saga.stuck-after-seconds`; after `oms.saga.max-attempts` the saga is FAILED (non-compensable) or flagged COMPENSATION_PENDING (compensable), and `ReconciliationService` repairs residual inconsistencies.

Choreography is still used where there is no coordination need: notification-service just consumes `oms.order.events.v1` and `oms.fulfillment.events.v1`; inventory consumes `OrderConfirmed`/`OrderCancelled` as a safety net.

Consequences

- Good: one table shows the whole flow; compensation is a first-class, tested path; participants stay simple and idempotent; recovery has a single owner.
- Bad: order-service is a coordination hot spot and a central point of knowledge about the flow; it must be highly available; command/reply topology adds topics; the orchestrator itself must be idempotent and crash-safe (state and outbox written in one transaction, ADR-0009).

Alternatives rejected

- Choreography: with four services and three compensation triggers, the flow becomes implicit across consumers; cyclic event dependencies and "who releases the stock?" ambiguity appear. Fine for 2-3 steps, poor for observability here.
- Workflow engine: the right answer for long, complex flows in production, but adds a heavy runtime that cannot run in the Docker-less local stack and hides the pattern this project exists to teach.

Links

- Contract sections 6 and 8; `docs/architecture/saga-and-consistency.md`
- order-service package `com.enterprise.oms.order` (service module under `services/order-service`)
- ADR-0005, ADR-0009

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

ADR-0004: Database per service

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

Each service owns a distinct set of aggregates (contract section 3). Sharing tables would let teams bypass APIs, couple releases through schema changes and make "who owns this row" unanswerable.

Decision drivers

- Enforced encapsulation: schema changes are local.
- Independent migration cadence (Flyway per service).
- Failure isolation: a long inventory transaction must not hold locks that order queries wait on.
- One PostgreSQL instance in Compose/K8s is acceptable for cost; logical separation is mandatory.

Considered options

1. Shared database, shared schema.
2. Shared PostgreSQL instance, separate database and credentials per service (chosen for Compose/K8s).
3. Separate PostgreSQL clusters per service.

Decision outcome

`order_db`, `inventory_db`, `fulfillment_db`, `notification_db` created by `infrastructure/compose/postgres/init/01-create-databases.sh`, each with its own user. Locally each service uses its own in-memory H2 in `MODE=PostgreSQL` running the same Flyway SQL. Flyway layout: `V1..V8` reserved for `platform-common` (`outbox_events`, `processed_events`, `audit_events` in `classpath:db/common`), service migrations from `V10`, seed data from `V1000`. Migrations are written portably (no PostgreSQL-only types) so H2 and PostgreSQL agree.

Data needed from another service is obtained by (a) event payloads that carry what the consumer needs (for example `customerEmail` in `OrderCreated`, so notification never calls order), (b) the single synchronous internal validate call, or (c) local denormalized copies (order_items snapshot `sku, name, unitPrice` at order time). Never by a join across databases.

Consequences

- Good: clear ownership, local transactions stay short, independent schema evolution and backup/restore.
- Bad: no cross-service joins or constraints (referential integrity across services is eventual and verified by reconciliation); reporting needs events or read models; duplicated reference data; one shared PostgreSQL instance is still a shared failure and resource domain (noisy neighbour); H2 PostgreSQL mode is an approximation, so PostgreSQL-specific behaviour is only verified by Testcontainers tests, which are skipped on the dev machine because Docker is not installed there.

Alternatives rejected

- Shared schema: fastest to build, but turns services into a distributed monolith and makes the saga unnecessary and therefore undemonstrable.
- Separate clusters per service: correct for strict isolation, but triples operational cost for a demo; the logical separation preserves the design and can be split later by changing JDBC URLs.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Links

- Contract sections 3, 9, 12; `services/platform-common/src/main/resources/db/common`
- `infrastructure/compose/postgres/init`
- ADR-0001, ADR-0009

ADR-0005: Idempotency at the API, the consumer and the state machine

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

Duplicates arrive from three places: clients retrying `POST /orders` after a timeout, Kafka redelivering after a rebalance or failed offset commit (at-least-once), and the saga recovery job deliberately re-sending commands. Each must be harmless.

Decision drivers

- A retried order submission must never create a second order.
- A redelivered event must never apply its business effect twice.
- Idempotency state must commit atomically with the effect it protects.
- Must work without Redis (Redis is optional and off by default).

Considered options

1. Rely on Kafka exactly-once semantics (transactions) only.
2. Three layers: API `Idempotency-Key`, consumer `processed_events`, state-machine idempotency (chosen).
3. Natural-key uniqueness only (for example unique order number).

Decision outcome

Three complementary layers.

1. API: `Idempotency-Key` header is mandatory on `POST /api/v1/orders` (8-64 chars `[A-Za-z0-9_-]`). Table `idempotency_keys` (order-service) stores key, request body hash, response status/body, and the order id. Same key and same body returns 202 with the original body and `Idempotent-Replay: true`; same key and different body returns 422 `IDEMPOTENCY_KEY_REUSED`; missing key returns 400 `IDEMPOTENCY_KEY_REQUIRED`. The unique constraint on the key makes concurrent first-requests race safely: the loser fails the insert and re-reads the winner's result. The key insert and the order insert share one transaction. Exceptions: `IdempotencyKeyRequiredException`, `IdempotencyKeyReusedException` in `services/platform-common/.../common/error`.
2. Consumer: `processed_events(consumer, event_id)` with a unique constraint, inserted in the same transaction as the business change by `IdempotentEventProcessor` (`services/platform-common/.../common/idempotency`). A duplicate violates the constraint, the transaction is skipped, `oms.events.duplicates` increments. Notification adds a second guard: unique (`source_event_id`, `template`).
3. State machine: `OrderStateMachine` treats an already-applied transition as a no-op, and reservations are unique per `order_id`, so a duplicate `ReserveInventory` returns the original outcome. This covers cases where the same logical command is re-issued with a new `eventId` (saga recovery re-sends).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Together these give an effectively-once business effect on top of at-least-once delivery. This is not Kafka exactly-once; see `docs/architecture/delivery-semantics.md` for the honest limits.

Consequences

- Good: safe client retries, safe redelivery, safe recovery job; no external store required; failure windows are covered by database atomicity.
- Bad: `idempotency_keys` and `processed_events` grow and need retention jobs (delete older than the maximum redelivery horizon); a key reused after expiry would be treated as new; one extra insert per consumed event; clients must generate good keys (UUIDs recommended) and the UI does so per submission attempt.

Alternatives rejected

- Kafka transactions only: cover Kafka-to-Kafka read-process-write, not the database effect or HTTP clients; does not protect `POST /orders`.
- Natural keys only: there is no business-provided unique order number at submission time, and it would not detect same-key-different-body misuse.

Links

- Contract sections 5, 6, 7; frontend `frontend/oms-ui/src/app/features/orders/order-create.ts`
- ADR-0006, ADR-0009; `docs/architecture/delivery-semantics.md`

ADR-0006: Concurrency control for stock reservation

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

Many orders compete for the same `stock` rows. Overselling is a correctness bug; long locks are a throughput bug. Two kinds of writers exist: the reservation hot path (many, short) and admin stock edits (rare, human speed).

Decision drivers

- Never reserve more than `on_hand - reserved`.
- No deadlocks on multi-line orders (up to 50 lines).
- Throughput under contention on popular SKUs.
- Reservation must be all-or-nothing per order.

Considered options

1. Atomic conditional UPDATE (compare in the WHERE clause).
2. Pessimistic locking: `SELECT ... FOR UPDATE`, check in Java, then update.
3. Optimistic locking with `@Version`, retry on conflict.

Decision outcome

Hot path uses option 1:

```
UPDATE stock
   SET reserved = reserved + :q, version = version + 1
 WHERE product_id = :id AND on_hand - reserved >= :q
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Affected rows = 0 means insufficient stock; the whole reservation transaction rolls back and `InventoryReservationFailed` (with shortages) is emitted. The database serializes concurrent updates on the same row and re-evaluates the predicate against the committed value, so no application-level lock or retry loop is needed. Lines are processed in ascending `productId` order so two orders touching overlapping products acquire row locks in the same order and cannot deadlock. One transaction covers the reservation row, items, stock updates, `stock_movements` and the outbox event.

Admin stock edits use option 3: `stock.version` as JPA `@Version`; on `ObjectOptimisticLockingFailureException` the service retries a bounded number of times, then returns 409 `CONFLICT`. A result below `reserved` is rejected. Optimistic locking suits rare writes where a lost update must be detected rather than prevented.

Option 2 is kept as a selectable variant (`PessimisticLockReservationStrategy`, exercised by `ReservationConcurrencyTest` on H2 for every strategy), not the default. A strategy benchmark exists (`StrategyBenchmarkTest`/`BenchmarkRunner` in inventory-service, opt-in and skipped by default) and `performance/oversell.js` (k6) targets the oversell scenario; neither produced numbers (k6 never run, PostgreSQL never run), so no measured comparison exists.

Consequences

- Good: single round trip, no read-then-write race, no lock held across application code, deadlock-free by ordering, the invariant lives in one SQL predicate.
- Bad: logic in SQL is less portable to non-relational stores and less visible than Java; the `version` column is bumped by the hot path too, so an admin edit racing with reservations can see optimistic conflicts (acceptable, retried); same-row contention still serializes on very hot SKUs; H2 `MODE=PostgreSQL` row-lock behaviour differs slightly from PostgreSQL, so contention tests that matter need Testcontainers (skipped where Docker is absent).

Alternatives rejected

- Pessimistic as default: holds row locks while Java checks and while the transaction continues; amplifies contention and deadlock risk; needs the same sorted-order discipline anyway.
- Optimistic on the hot path: under contention most attempts fail and retry, wasting work and producing retry storms on popular SKUs.

Links

- Contract section 7 (reservation lifecycle); inventory-service under `services/inventory-service`
- ADR-0005, ADR-0004

ADR-0007: URL-path API versioning with additive-only evolution

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

External clients (the Angular UI, scripts, possibly partners) depend on the REST surface behind the gateway. We need a visible version scheme and a cheap, enforceable compatibility discipline.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Decision drivers

- Visibility in logs, routes, gateway rules and curl examples.
- Ability to run two major versions side by side behind the gateway.
- Most changes should not require a new major version.

Considered options

1. URL path (`/api/v1/...`).
2. Header or media-type versioning (`Accept: application/vnd.oms.v1+json`).
3. Query parameter (`?version=1`).
4. No versioning, rely on discipline.

Decision outcome

URL-path major version, base path `/api/v1`. Within v1 only additive, backward-compatible changes are allowed (new optional request fields, new response fields, new endpoints, new enum values documented as extensible only where clients are tolerant). Removing or renaming fields, changing types or semantics, or tightening validation requires `/api/v2`. Deprecation uses `Deprecation` and `Sunset` response headers plus a minimum notice period. Enforcement: committed OpenAPI exports in `specs/openapi/<service>.json`, an OpenAPI diff step in CI that fails on breaking changes, and consumer-driven contract tests. The gateway routes by path prefix, so v2 can be added as a new route to a new controller set without touching v1. Full rules and tables are in `docs/architecture/api-versioning.md`.

Consequences

- Good: obvious to humans and tooling, trivial gateway routing and rate-limit rules, cacheable, easy to test per version.
- Bad: URL is not "pure REST" (resource identity includes version); two live versions double the test and support surface; clients must tolerate unknown fields (the UI models are written that way); enum growth is a breaking change for strict clients, so enums are documented as open.

Alternatives rejected

- Media-type versioning: elegant but hard to see in logs, awkward in browsers and Postman, and gateway routing by header is more fragile.
- Query parameter: easy to omit, pollutes caching keys, no clear default.
- No versioning: no escape hatch when a breaking change becomes unavoidable.

Links

- Contract section 5; `docs/architecture/api-versioning.md`; `specs/openapi` (exports are generated from each service's `/v3/api-docs`)
- ADR-0002 (events use topic and `schemaVersion` versioning instead)

ADR-0008: CI/CD pipeline and deployment model

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

Four JVM services, a gateway and an Angular UI must be built, tested, scanned, containerized and deployed repeatably. The dev machine has no Docker, so much of the infrastructure can only be validated statically.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Decision drivers

- Fast feedback: unit tests on every push, slower checks in parallel jobs.
- Supply-chain hygiene: dependency, secret, SAST and image scanning before publishing.
- Declarative, reviewable deployment with environment overlays.
- Honesty about what is verified and what is not.

Considered options

1. GitHub Actions plus Kustomize overlays, GHCR images (chosen), with a Jenkinsfile mirror.
2. Helm charts.
3. GitOps controller (Argo CD, Flux) as the only deployer.
4. Single "build everything and deploy" script.

Decision outcome

`.github/workflows/ci.yml` jobs: `version`, `java-build-test` (unit tests, jars), `java-integration` (failsafe), `frontend`, `codeql`, `semgrep`, `dependency-scan` (Trivy filesystem plus `npm audit`), `secret-scan` (gitleaks), `infra-validate` (kustomize build, kubeconform, terraform validate, compose config, observability checks), `docker` and `docker-ui` (build, Trivy image scan failing on fixable HIGH/CRITICAL, push to GHCR on main and tags only), `smoke-local` (embedded Kafka plus 4 services plus gateway on H2), and `deploy-test` (guarded by a GitHub `environment: test` with required reviewers; secrets are environment-scoped; a placeholder step documents what is needed). `infrastructure/ci/Jenkinsfile` mirrors the stages for teams on Jenkins.

Deployment: Kustomize base in `infrastructure/kubernetes/base` with `overlays/dev`, `overlays/test`, `overlays/prod`; network policies, KEDA Kafka-lag scaling (`infrastructure/kubernetes/keda`), monitoring rules (`infrastructure/kubernetes/monitoring`). Images are built per service from `infrastructure/docker/*.Dockerfile`. AWS reference infrastructure is Terraform under `infrastructure/cloud/aws/terraform` (EKS, ECR, network, secrets). Database migrations run via Flyway at service start (ADR-0004); expand-then-contract is required so old and new versions coexist during rolling updates. Rollback is `kubectl rollout undo` or re-deploying the previous image tag; schema rollbacks are forward-fixes, never down-migrations.

Honest status: the pipeline YAML, Kubernetes manifests, Compose files and Terraform are statically validated (parsing, kustomize build, schema checks). They have not been executed against a real cluster or cloud account from the dev machine. Testcontainers-based tests are annotated `disabledWithoutDocker = true` and are skipped locally; they run only where Docker is available (CI runners).

Consequences

- Good: parallel gates, immutable image tags, reviewable overlays, no secrets in the repo, a deliberate approval gate before deploy.
- Bad: Kustomize has no templating power or release history (Helm would); two CI definitions can drift; no automated canary or progressive delivery yet; the deploy job is a placeholder until a real non-production cluster exists; first real deployment will likely surface issues static checks cannot see.

Alternatives rejected

- Helm: richer, but values-file sprawl for a 5-component system; Kustomize is enough and renders to plain YAML for review.
- GitOps-only: the right production end state, deferred; the current job structure can feed it by committing image tags.
- Single script: no parallelism, no per-gate reporting.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Links

- `.github/workflows/ci.yml`, `infrastructure/ci/Jenkinsfile`, `infrastructure/kubernetes`, `infrastructure/compose`, `infrastructure/docker`, `infrastructure/cloud`
- ADR-0010 (secrets), ADR-0011 (monitoring manifests)

ADR-0009: Transactional outbox for event publication

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture

Context

A state change and its event must not diverge. Writing to the database and then to Kafka is a dual write: a crash between them either loses the event (state changed, nothing published) or publishes a phantom (event sent, transaction rolled back).

Decision drivers

- No lost events after a committed state change.
- No events for rolled-back changes.
- Works with plain PostgreSQL/H2 and Spring Kafka.
- Honest delivery guarantee.

Considered options

1. Dual write (save, then send).
2. Transactional outbox with a polling relay (chosen).
3. Outbox with CDC (Debezium).
4. Kafka transactions plus a database transaction (best-effort 2PC).

Decision outcome

Table `outbox_events` (Flyway `V1__outbox_processed_audit.sql` in `services/platform-common/src/main/resources/db/common`) exists in every producing service. `OutboxService` inserts the row in the same transaction as the state change, including saga step log and audit rows. `OutboxRelay` (package `com.enterprise.oms.common.outbox`, config `OutboxProperties`) polls unpublished rows in order, publishes through `omsKafkaTemplate` (`acks=all`, `enable.idempotence=true`, key = aggregate id), waits for the broker acknowledgement, then sets `published_at`. Gauge `oms.outbox.pending` and the `outboxPending` field of `/api/v1/ops/messaging` expose backlog.

Guarantee: at-least-once publication. If the relay crashes after Kafka acknowledged but before `published_at` is committed, the row is published again. Consumers therefore must be idempotent (ADR-0005). The producer-side idempotence setting removes duplicates caused by producer retries within a session; it does not remove the relay re-publish case. Kafka does not give exactly-once across the database and Kafka here, and this design does not claim it.

Consequences

- Good: no lost or phantom events, simple to reason about and test, survives broker outages (rows accumulate, backlog gauge alerts), keeps ordering per aggregate when the relay sends in insertion order for a key.
- Bad: added latency (poll interval), extra writes and table growth (needs a purge of old published rows), the relay is another thing to scale and monitor, polling load on the database, ordering across multiple relay instances needs care (single active relay per service or row-level claim with `FOR UPDATE SKIP LOCKED` on PostgreSQL), duplicates are possible by design.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Alternatives rejected

- Dual write: loses or invents events on every crash window.
- CDC (Debezium): lower latency and no polling, but needs Kafka Connect and PostgreSQL logical replication that cannot run in the Docker-less local stack; a valid production upgrade path because the outbox table shape stays the same.
- Kafka plus DB transaction coordination: no standard cross-resource atomic commit; complexity without a guarantee we could defend.

Links

- Contract sections 7 and 12; `services/platform-common` package `outbox`
- ADR-0002, ADR-0005; `docs/architecture/delivery-semantics.md`

ADR-0010: JWT bearer tokens, OIDC-compatible resource servers, backend-authoritative authorization

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture, security

Context

The platform needs authentication at the edge, authorization inside each service, service-to-service identity, and a story for production identity providers, without shipping secrets or default passwords.

Decision drivers

- Defence in depth: gateway validates, every service validates again.
- Authorization must not depend on the UI.
- Swap HS256 demo tokens for a real OIDC provider by configuration only.
- No hard-coded secrets; fail fast when misconfigured.

Considered options

1. Opaque session cookies with a central session store.
2. Self-contained JWTs validated by every service (chosen).
3. Gateway-only authentication with trusted internal headers.
4. mTLS only for service identity.

Decision outcome

- Tokens: JWT with claims `sub`, `roles` (`CUSTOMER|OPERATOR|ADMIN|SERVICE`), `customerId`, `iss=oms-dev-issuer`, `aud=oms-api`, `exp`, `iat`, `jti`. Demo mode is HS256 with the secret from `OMS_JWT_SECRET` (at least 32 bytes). There is no default in the prod profile: startup fails if it is missing or short (`JwtSecret`, `OmsSecurityProperties`), unless `oms.security.jwt.allow-ephemeral-secret=true`, which is for tests and single-process runs only. `scripts/run-local.sh` generates a random secret per run.
- OIDC: setting `oms.security.jwt.issuer-uri` or `jwk-set-uri` switches resource servers to RS256 via JWKS; the HS256 beans (`JwtTokenFactory`, `ServiceTokenProvider`) are then not created. Code in `services/platform-common/.../common/security`.
- Layers: the gateway validates the JWT, applies CORS, rate limiting and correlation ids; each service re-validates (never trusts a header from the gateway) and authorizes with method security plus resource-level checks. `/api/v1/internal/**` requires `SERVICE` and is not routed by the gateway.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Service identity: short-lived SERVICE JWT minted by `ServiceTokenProvider` (`sub=svc-order-service`).
- Authorization matrix lives in contract section 4. Cross-tenant access returns 404, not 403, to prevent enumeration. 401 and 403 bodies are RFC 7807 via `ProblemSecurityHandlers`.
- Dev token endpoint exists only in the gateway `dev` profile; demo passwords come from `OMS_DEMO_PASSWORD`, never from code.
- UI: guards in Angular are UX only (ADR-0012); the token lives in a signal mirrored to `sessionStorage`. Known trade-off: readable by XSS; production should use authorization-code with PKCE through a BFF with HttpOnly cookies, plus strict CSP.

Consequences

- Good: stateless horizontal scaling, no session store, clear production path (OIDC), tamper-resistant authorization, fail-fast config.
- Bad: JWTs cannot be revoked before `exp` (mitigate with short lifetimes and, in production, introspection or a deny list keyed by `jti`); HS256 shares one secret across all services (any holder can mint tokens; acceptable only for demo, not for production); sessionStorage token exposure to XSS; each service pays JWT verification cost; key rotation for HS256 requires coordinated restart.

Alternatives rejected

- Server-side sessions: fits browsers but not service-to-service calls and needs a shared store.
- Trust gateway headers: any bypass of the gateway (misrouted network policy) becomes full privilege escalation.
- mTLS only: strong service identity but carries no end-user roles; complementary, not a replacement (the K8s network policies already restrict east-west traffic).

Links

- Contract section 4, 12; `services/platform-common/src/main/java/com/enterprise/oms/common/security`
- `infrastructure/kubernetes/base/networkpolicies.yaml`; `frontend/oms-ui/src/app/core/auth/auth.store.ts`
- ADR-0012

ADR-0011: Observability stack (Prometheus, Grafana, Loki, Tempo, OpenTelemetry)

- Status: Accepted
- Date: 2026-10
- Deciders: platform architecture, operations

Context

An asynchronous, multi-service flow fails in ways no single log shows: a stuck saga, a growing consumer lag, a duplicate storm. Operators need metrics for symptoms, logs for detail and traces for causality, linked by one correlation id.

Decision drivers

- Vendor-neutral instrumentation (OpenTelemetry, Micrometer).
- Open-source backends that run in Compose and Kubernetes.
- Cost control: logs and traces are sampled or retained briefly.
- Privacy: no tokens, passwords or full emails in telemetry.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Considered options

1. Prometheus + Grafana + Loki + Tempo behind an OpenTelemetry Collector (chosen).
2. ELK stack (Elasticsearch, Logstash, Kibana, APM).
3. A commercial SaaS APM.

Decision outcome

- Metrics: Micrometer with the Prometheus registry on `/actuator/prometheus`. Custom metrics from the contract: `oms.events.processed`, `oms.events.duplicates`, `oms.events.retries`, `oms.events.dead_lettered`, `oms.outbox.pending`, `oms.order.processing.duration`, `oms.saga.active`, `oms.saga.compensations` (`MessagingMetrics`, `Metrics` in `services/platform-common`). Dashboards (provisioned from `observability/grafana/dashboards`): API RED, JVM, HikariCP, messaging, saga funnel. Alert rules in `observability/prometheus/rules/oms-alerts.yml` and `infrastructure/kubernetes/monitoring/prometheusrule.yaml`.
- Logs: JSON via logstash-logback-encoder with `service, correlationId, traceId, spanId`; email masking by `LogMasking`; shipped by Promtail to Loki (`observability/loki`).
- Traces: `micrometer-tracing-bridge-otel` with OTLP export to the collector (`observability/otel/otel-collector.yml`) and Tempo (`observability/tempo/tempo.yml`). Context crosses Kafka through the `traceparent` and `correlationId` headers; the gateway injects `X-Correlation-Id`, which is also echoed to clients and shown in the UI error toasts.
- Operational API: `/api/v1/ops/messaging` (lag, processed, duplicates, retries, DLT counts, outbox pending) and `/api/v1/ops/audit` in every service; `MessagingStatusService` degrades to `lagAvailable:false` rather than failing.
- Scaling signal: KEDA scales consumers on Kafka lag (`infrastructure/kubernetes/keda`).
- Tracing is off locally by default and sampling is configurable.

Status honesty: configuration files, dashboards and rules are statically validated (YAML/JSON parse and CI checks). They have not been observed running against live Prometheus, Loki or Tempo on the dev machine, since Docker is not installed there.

Consequences

- Good: portable instrumentation, three signals correlated by ids, no licence cost, alerts as code.
- Bad: four or more stateful components to run and size; Loki/Tempo retention must be configured or storage grows; label cardinality discipline needed (never label metrics by orderId); sampling can drop the very trace you need; dashboards drift unless regenerated (`observability/grafana/build-dashboards.mjs` is the source).

Alternatives rejected

- ELK: heavier to operate, stronger full-text search; Loki is cheaper because it indexes labels only.
- Commercial APM: fastest to adopt but cost and data-residency concerns and cannot be bundled in the reference repo.

Links

- Contract section 10 and 5.5; `observability/`, `infrastructure/compose/docker-compose.observability.yml`
- `docs/architecture/delivery-semantics.md` (metrics per failure window)

ADR-0012: Angular state management with signals and small stores, not NgRx

- Status: Accepted
- Date: 2026-10

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Deciders: frontend lead

Context

`frontend/oms-ui` (Angular 20, Angular Material) has about a dozen screens. Server data dominates; client-only state is small (auth, cart, list query/paging, polling status). The team has React/Redux background and wants a clear mapping. (Update 2026-10-19: a real React + Redux Toolkit UI now exists as a second front end, see ADR-0013; this ADR still governs the Angular app only.)

Decision drivers

- Minimal boilerplate for the amount of shared state we have.
- Testability: state transitions as pure functions.
- Fine-grained change detection without a heavy runtime.
- Debuggability.

Considered options

1. NgRx (store, effects, selectors, devtools).
2. NgRx SignalStore.
3. Plain Angular signals in injectable stores plus RxJS for IO (chosen).
4. Component-local state only.

Decision outcome

Signals in `providedIn: 'root'` stores for cross-screen state (`AuthStore` in `core/auth/auth.store.ts`, `CartStore` in `core/state/cart.store.ts`) and per-screen stores for server-backed screens (`list-store.ts` for paginated lists, `order-detail.store.ts`). Reducer-style pure update functions (`listLoading`, `listLoaded`, `listFailed`, `withFilters`, `withPage`, `withSort`) are exported and unit-tested without TestBed. IO and polling use RxJS (`pollUntil` in `core/state/polling.ts`, backoff 1.5x capped by `pollMaxIntervalMs`). Derived values use `computed`. No global store: server state is refetched, not mirrored.

Revisit triggers: more than a few stores writing the same entity, need for time-travel/devtools, cross-tab undo, or an offline write queue. Then adopt NgRx SignalStore first (same signal model), full NgRx only for event-sourced client logic.

Consequences

- Good: very little code, no action/reducer ceremony, signals integrate with templates and zoneless-ready change detection, tests are plain TypeScript.
- Bad: no built-in devtools or action log; conventions rather than enforced structure (discipline needed to keep stores small and updates pure); risk of ad hoc stores multiplying; no standard pattern for optimistic updates or caching of server state (hand-rolled).

Alternatives rejected

- NgRx: excellent for large teams and complex client state, but here it would add actions, effects and selectors for data that is refetched anyway.
- Component-only state: breaks down for auth, cart and list state that survive navigation.

React/Redux mapping (summary)

Redux / React	This project
store slice	a store class with `signal()` fields
reducer	pure functions such as `listLoaded`, `withFilters`
selector / reselect	`computed()`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Redux / React	This project
thunk / saga	RxJS pipelines, `pollUntil`, `switchMap`
`useSelector`	reading the signal in a template or `computed`
middleware	HTTP interceptors

Full table: `docs/architecture/angular-architecture.md`.

Links

- `frontend/oms-ui/src/app/core/state`, `frontend/oms-ui/src/app/core/auth`
- ADR-0010

ADR-0013: Two front ends (Angular and React + Redux) on one backend contract

- Status: Accepted
- Date: 2026-10
- Deciders: frontend lead, platform lead

Context

`frontend/oms-ui` (Angular 20, ADR-0012) was the only UI and the earlier rule was "Angular only". The target job description asks for React with Redux, and the same people maintain the platform. A second UI is cheap to justify only if it does not fork the backend, the contract, or the test suite. The gateway API, the problem+json error model, role rules, idempotency and the order state machine are already verified; they must not change for a UI.

Decision drivers

- One API contract, no UI-specific endpoints, no backend changes.
- Evidence that both UIs behave the same: the SAME end-to-end specs must pass against both.
- Idiomatic code in each framework (not an Angular port written in JSX).
- Same security posture (CSP, same-origin `/api`, backend-authoritative authorization).
- Low cost to keep one UI and drop the other.

Considered options

1. Replace Angular with React.
2. Keep Angular only and document React equivalents (the mapping table in the Angular README).
3. Add `frontend/oms-ui-react` as a second, feature-equivalent UI that shares nothing but the contract (chosen).
4. A shared UI core (TypeScript package) consumed by both.

Decision outcome

Option 3. `frontend/oms-ui-react` is React 19 + TypeScript (strict) + Vite, Redux Toolkit, React Router, React Hook Form + Zod and MUI. It talks to the gateway through the same `/api/v1` paths, with the same behaviours: `Idempotency-Key` per submit attempt (reused only after an unknown outcome and an unchanged body), problem+json normalization, 401 logout, 429 toast with `Retry-After`, GET-only retry, role-aware navigation, order polling until a terminal status, and null-price tolerance for REJECTED orders (defect D-03).

State split: **RTK Query for server state** (cache, de-duplication, invalidation, polling), **slices for client state** (`auth`, `cart`, `ui`). No server data is copied into slices. See react-architecture.md for the mapping to the Angular stores.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Parity is enforced by tests, not by prose: `tests/e2e` is parameterized with `E2E_UI=angular|react`, and the 18 lifecycle specs run unchanged against both, in mock-backend mode and against the live stack. The React markup was written to satisfy the existing role/label/text locators plus the few `data-testid`s Angular already exposes, so no adapter or forked spec was needed.

Deployment: separate image (`oms-ui-react`, same nginx-unprivileged contract), a compose profile `react` (host port 4201), an optional Kubernetes Deployment/Service that the Ingress does not route (the Angular UI owns `/`), and separate CI jobs. Removing the React UI means deleting one directory, one compose service, one kustomize resource and its CI jobs.

Consequences

- Good: the contract is exercised by two independent clients, which finds contract ambiguity (for example the null prices of REJECTED orders). Interviews can compare the two stacks on identical behaviour.
- Good: no backend change; the React UI sits behind the same gateway, CORS-free (same origin through nginx or the Vite proxy).
- Bad: every UI feature is written twice and must be kept in step (mitigated by the shared e2e suite, not eliminated). Two dependency trees to patch (`npm audit` runs for both in CI).
- Bad: UI-level unit tests are not shared (Karma vs Vitest); only the e2e layer is.
- Bad: MUI's emotion runtime injects styles, so the CSP keeps `style-src 'unsafe-inline'` (as Angular Material already required); scripts stay `script-src 'self'`.

Alternatives rejected

- Replacing Angular: loses a verified UI and the Angular-specific evidence the repository already has.
- Mapping table only: does not demonstrate working React/Redux code.
- Shared UI core package: the two frameworks share only types and a handful of pure functions (timeline, idempotency holder); the packaging overhead exceeds the reuse. The types are copied verbatim from the contract instead (`src/models.ts`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

19. Architecture mind map

One page to recall the whole platform. Use it as the opening sketch in an interview, then drill into whichever branch the interviewer picks.

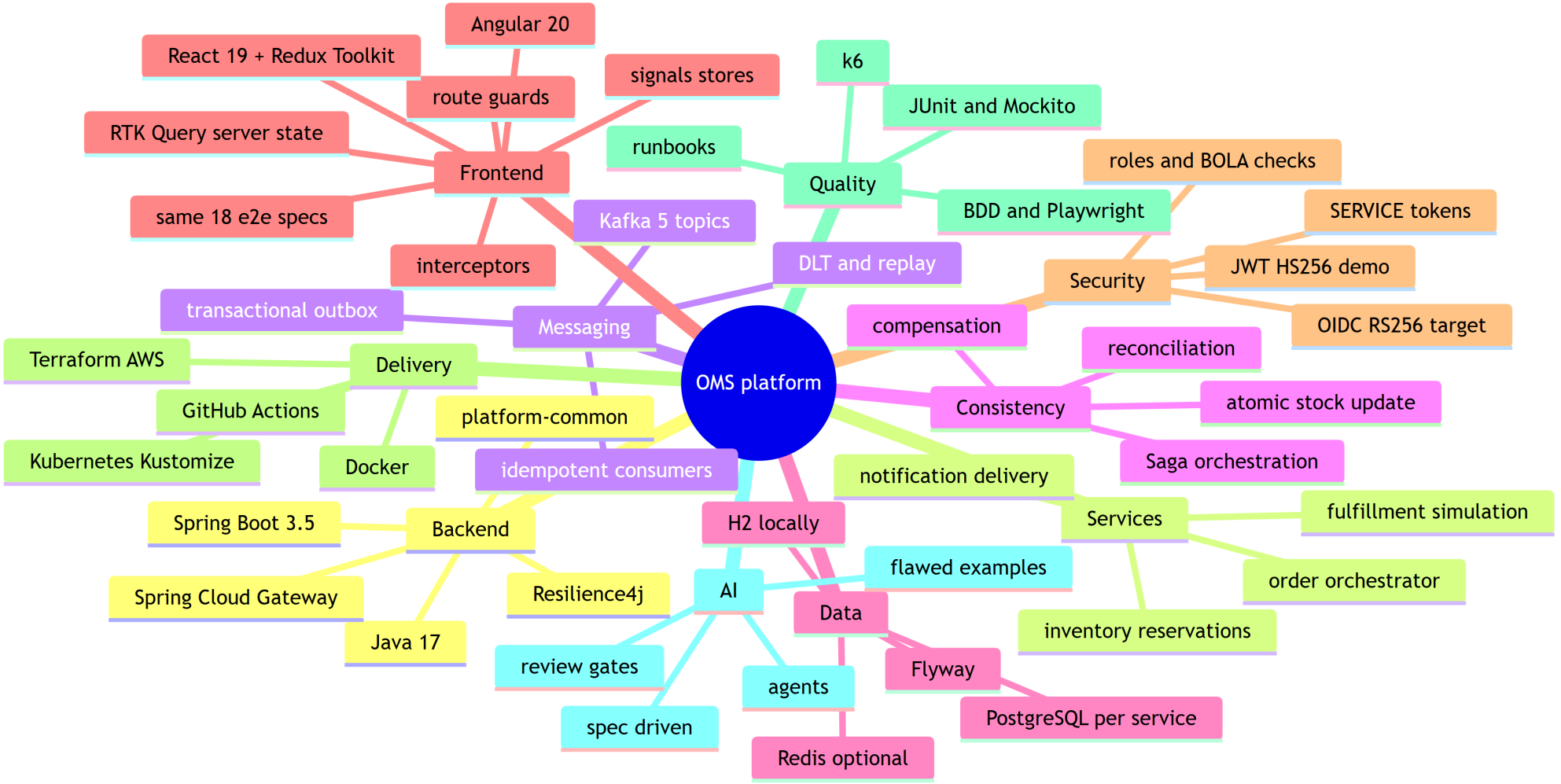


Figure 23. Architecture mind map

Generated by AI. AI can make mistakes. Validate before relying upon this document.

© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

20. Troubleshooting scenarios

How to use this section

Each scenario is a realistic production symptom. Start from the symptom, form a hypothesis from metrics, confirm with one cheap query, then act. The full runbooks follow in this section; every command in them is illustrative and was not run against a live cluster (stated in the runbook index).

Symptom to first action

Symptom	First hypothesis	First check	Runbook
Orders stay in `VALIDATED` for minutes	Reservation reply lost or consumer down	`GET /api/v1/admin/saga/{orderId}`, consumer lag, `SagaRecoveryJob` logs	01
Consumer lag keeps growing	Slow handler, poison pill, rebalance loop, DB pool exhausted	`GET /api/v1/ops/messaging`, `kafka-consumer-groups --describe`, Hikari pending	02
`InventoryReservationFailed` spike or negative availability	Hot product, strategy change, or a bug that bypasses the atomic update	Compare `reserved` and `on_hand`, reservation metrics	03
p99 latency up on every endpoint	DB lock contention, slow query, pool exhaustion	`hikaricp.connections.pending`, `pg_stat_activity`	04
Customers get two emails	Redelivery without effective dedupe, or unique key missing	`oms.events.duplicates`, `notifications` unique key	05
503 from the gateway	Circuit open or readiness failing	`GET /api/v1/ops/health`, pod events	06
Rollout stuck or crash loop	Config, secret, migration or probe failure	`kubectl rollout status`, previous logs	07
Pod OOMKilled or long GC pauses	Heap sizing, leak, unbounded query	heap flags, `jcmd`, metrics	08

A repeatable investigation method

1. State the impact in one sentence and the start time.
2. Check what changed (deploy, config, traffic, dependency).
3. Use the RED view (rate, errors, duration) per service, then USE (utilisation, saturation, errors) for the database pool, CPU and consumer lag.
4. Follow one request: take the `X-Correlation-Id` from the response, search logs for it, open the trace.
5. Stabilise first (scale, roll back, pause a consumer), then fix, then write the review.

Debugging exercises included in the repository

Seven code-review exercises (`docs/code-review/`) and two database performance write-ups (`docs/database/`) double as troubleshooting practice: oversell race, BOLA via body `customerId`, transactional self-invocation, Kafka publish inside a transaction, acknowledging offsets on exception, unbounded pagination with N+1 queries, PII in logs, plus the inventory EXPLAIN ANALYZE study and the slow-query, lock-contention and pool-exhaustion investigation.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Runbook index

Operational guides for the OMS (order, inventory, fulfillment, notification services and the gateway). Each runbook doubles as a **practice exercise**: it says how to reproduce the failure in the local stack and which test guards against regression.

Honesty notes (read first)

- Every command in these documents is **illustrative: not executed against a live cluster.** The development machine has no Docker and no Kubernetes; no cluster exists for this project. Namespace `oms`, deployment names (`order-service`, ...), and the Prometheus job labels are conventions assumed from the contract; adjust to the real manifests in `infrastructure/kubernetes/`.
- All modules now exist on disk (checked 2026-10-18). Implemented and verified in code: `/api/v1/ops/messaging` and `/api/v1/ops/audit` (platform-common `OpsMessagingController`, `OpsAuditController`), the gateway aggregate `/api/v1/ops/health` (`OpsHealthController`), order-service `order/api/AdminController` (`GET /api/v1/admin/orders/failed`, `POST /api/v1/admin/orders/{id}/retry`, `GET /api/v1/admin/saga/{orderId}`, `POST /api/v1/admin/reconciliation/run`), and the metrics `oms.events.*`, `oms.outbox.pending`, `oms.saga.active`, `oms.saga.compensations`, `oms.saga.recovered`, `oms.saga.alerts`, `oms.order.processing.duration` (found in the main sources; dashboard/alert metric names matched a live scrape with 0 missing).
- `scripts/run-local.sh` and `scripts/stop-local.sh` exist and were run on 2026-10-18 (stack start takes about 2.5 minutes; it refuses to start if ports 19092/18080-18084 are taken). The stack start and the live flows are verified (`docs/validation/validation-report.md`); the failure-injection steps in each practice exercise are reproduction recipes, and only the fulfillment-down scenario (`tests/e2e/resilience-live.mjs`) and the FAILTOWN/insufficient-stock flows were actually run. Kubernetes commands remain unexecuted (no cluster).

Index

#	Runbook	Typical alert	Default severity
01	Orders stuck in processing	`oms.saga.active` high, stuck saga	SEV2
02	Kafka consumer lag	lag growing, `oms.outbox.pending` growing	SEV2
03	Reservation failures under concurrency	spike of `InventoryReservationFailed`, oversell suspicion	SEV1 if oversell, else SEV3
04	Database latency spikes	`hikaricp.connections.pending` > 0, p99 up	SEV2
05	Duplicate order events	`oms.events.duplicates` spike, duplicate emails	SEV3
06	Service unavailable (503/ readiness DOWN)	readiness failing, 5xx	SEV1/SEV2
07	Failed deployment	rollout stuck, crash loop	SEV2
08	JVM memory / OOM	heap > 90%, OOMKilled, GC pauses	SEV2

Severity guide

Level	Definition	Examples here	Response	Update cadence
SEV1	Customer-facing outage or data-integrity loss	all orders failing; stock oversold; gateway down	page immediately, incident commander, 24x7	every 15 min

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Level	Definition	Examples here	Response	Update cadence
SEV2	Major degradation, workaround exists or limited scope	one service down with saga retrying; lag > 5 min; DB latency	page in hours of coverage, start within 15 min	every 30 min
SEV3	Minor impact, no data loss	duplicate emails, single DLT record	ticket, next business day	daily
SEV4	Cosmetic / no impact	dashboard glitch	backlog	none

Escalate one level if: customer data exposure is suspected (also notify security), the issue is not understood after 30 minutes, or two services are affected.

On-call basics

1. **Acknowledge** the page; state who has it.
2. **Assess impact** before diagnosing: which services, since when, how many orders (`oms.saga.active`, `http.server.requests` 5xx ratio).
3. **Stabilise before fixing:** roll back the last deploy, scale, or pause the traffic source; do not hot-fix in production.
4. **Collect evidence before restarting:** thread dump, heap dump if memory, `ops/messaging` output, one correlationId trace. A restart destroys evidence.
5. **Communicate:** one channel, timestamps, hypotheses and actions. Never paste tokens, passwords, or customer emails into the channel; use ids.
6. **Do not edit data by hand** if an admin endpoint exists (retry, reconciliation); those write audit rows (`/api/v1/ops/audit`).
7. **Close with a postmortem** (blameless): timeline, root cause, what detected it, what should have, action items with owners and dates.

Access model: OPERATOR and ADMIN roles only. DLT inspection and reconciliation are ADMIN. Obtain tokens from the identity provider or, in dev profile only, `POST /api/v1/auth/token`; the examples below write `<JWT>`.

Command cheat sheet (all illustrative: not executed against a live cluster)

```
# Kubernetes (namespace oms assumed)
kubectl -n oms get pods -o wide
kubectl -n oms describe pod <pod>
kubectl -n oms logs deploy/order-service --since=15m | grep <correlationId>
kubectl -n oms logs <pod> --previous                        # after a crash
kubectl -n oms top pod
kubectl -n oms rollout status deploy/order-service
kubectl -n oms rollout history deploy/order-service
kubectl -n oms rollout undo deploy/order-service
kubectl -n oms port-forward svc/order-service 18081:18081

# Health and metrics (ports from the contract; locally on localhost)
curl -s localhost:18081/actuator/health/readiness
curl -s localhost:18081/actuator/health/liveness
curl -s localhost:18081/actuator/prometheus | grep '^oms_'

# Operational endpoints (OPERATOR/ADMIN token)
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/order/messaging | jq
curl -s -H "Authorization: Bearer <JWT>" "localhost:18080/api/v1/ops/order/audit?
entityType=Order&entityId=<orderId>" | jq
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/admin/saga/<orderId> | jq
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/health | jq
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
# Kafka
kafka-consumer-groups.sh --bootstrap-server localhost:19092 --describe --group order-service
kafka-topics.sh --bootstrap-server localhost:19092 --list
kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic oms.inventory.events.v1.DLT \
  --from-beginning --max-messages 5 --property print.key=true --property print.headers=true

# PostgreSQL
psql -h <host> -U <ops_user> -d order_db -c "select pid,state,wait_event_type,now()-query_start as
age,left(query,80) from pg_stat_activity where state<>'idle' order by age desc;"

# JVM
jcmd <pid> VM.flags ; jcmd <pid> Thread.print ; jstat -gcutil <pid> 1000 5
jcmd <pid> GC.heap_dump /tmp/heap.hprof      # contains data: handle as confidential, delete after use
```

Ports locally: gateway 18080, order 18081, inventory 18082, fulfillment 18083, notification 18084, Kafka 19092.

Metrics referenced (contract section 7 plus Micrometer defaults)

`oms.events.processed`, `oms.events.duplicates`, `oms.events.retries`, `oms.events.dead_lettered`, `oms.events.ignored` (all verified in `MessagingMetrics`), `oms.outbox.pending`, `oms.order.processing.duration`, `oms.saga.active`, `oms.saga.compensations`, `jvm.memory.used`, `jvm.gc.pause`, `hikaricp.connections.active|pending|max`, `http.server.requests`, Kafka consumer metrics / lag. In Prometheus, dots become underscores and counters get `_total` (for example `oms_events_dead_lettered_total`, `oms_outbox_pending`).

Related

- Review scenarios that produce these incidents: ../code-review/README.md
- Debugging method: ../mentoring/debugging.md
- Shared design: ../architecture/00-shared-contract.md

Runbook 01 - Orders stuck in processing

All commands: illustrative, not executed against a live cluster. The order-service saga/admin endpoints are implemented (`order/api/AdminController`: `/api/v1/admin/saga/{orderId}`, `/orders/{id}/retry`, `/orders/failed`, `/reconciliation/run`) and covered by `AdminApiTest`; recovery is `order/saga/SagaRecoveryJob` (`oms.saga.stuck-after-seconds` default 60, `oms.saga.max-attempts` default 5) and `ReconciliationService`.

Symptoms

- Customers report orders that never leave CREATED / VALIDATED / INVENTORY_RESERVED / CONFIRMED.
- `oms.saga.active` rises and does not drain; `oms.order.processing.duration` has no new samples (it records CREATED to terminal).
- `GET /api/v1/admin/orders/failed` lists COMPENSATION_PENDING orders.
- No error rate in `http.server.requests`: the HTTP side is healthy, the asynchronous side is not.

Metrics and PromQL

```
oms_saga_active                                          # gauge: rising without bound = stuck
rate(oms_order_processing_duration_seconds_count[5m])    # completions per second; 0 while orders
arrive = stalled
increase(oms_saga_compensations_total[15m])
increase(oms_events_dead_lettered_total[15m])
oms_outbox_pending{service="order-service"}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
sum by (topic) (rate(oms_events_processed_total[5m]))
```

Diagnostics

```
# 1. Which step is stuck? (OPERATOR/ADMIN)
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/admin/saga/<orderId> | jq
'{state,currentStep,attempts,lastError,steps}'

# 2. Order, history and audit
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/orders/<orderId>/history | jq
curl -s -H "Authorization: Bearer <JWT>" "localhost:18080/api/v1/ops/order/audit?
entityType=Order&entityId=<orderId>" | jq

# 3. Messaging health of each hop
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/order/messaging     | jq
'{consumerGroup,outboxPending,retries,deadLettered,topics}'
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/inventory/messaging | jq
'{consumerGroup,outboxPending,retries,deadLettered,topics}'

# 4. Readiness of everything
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/health | jq

# 5. Pods and logs
kubectl -n oms get pods
kubectl -n oms logs deploy/order-service --since=30m | grep <correlationId>
kubectl -n oms logs deploy/inventory-service --since=30m | grep <correlationId>
```

Database (order_db), stuck sagas older than the threshold (default 60 s):

```
select order_id, state, current_step, attempts, last_error, updated_at
  from saga_instances
 where state not in ('COMPLETED','FAILED','CANCELLED','REJECTED')   -- adapt to the real saga state
names
   and updated_at < now() - interval '2 minutes'
 order by updated_at;
select count(*) from outbox_events where published_at is null;
```

Root-cause tree

```
Order stuck
|- saga currentStep = VALIDATE
|    |- inventory REST unreachable / breaker open -> attempts rising, lastError UPSTREAM_UNAVAILABLE -
> runbook 06 (inventory down)
|    '- order-service thread/connection starvation -> runbook 04 / 08
|- currentStep = RESERVE_INVENTORY
|    |- order outbox pending high -> relay stuck or broker down -> runbook 02
|    |- inventory consumer lag high -> runbook 02
|    |- record in oms.inventory.commands.v1.DLT -> read x-exception-*; poison vs transient
|    '- reservation exists but InventoryReserved not consumed by order -> check
oms.inventory.events.v1 lag and order DLT
|- currentStep = CONFIRM_ORDER / FULFILL
|    |- fulfillment down or FAILTOWN shipping city (deterministic failure) -> expected
FulfillmentFailed -> COMPENSATION_PENDING
|    '- FulfillmentCompleted not consumed -> lag on oms.fulfillment.events.v1
'- state COMPENSATION_PENDING for long
     |- ReleaseInventory lost/failed -> SagaRecoveryJob should re-send; check attempts vs oms.saga.max-
attempts
     '- retries exhausted -> flagged for manual action
```

Mitigation

1. Fix the underlying dependency first (broker, inventory, DB). The `SagaRecoveryJob` re-drives stuck steps automatically using idempotent commands, once the dependency is back.
2. For individual orders, use the admin retry (idempotent, retries the current saga step, audited):

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
curl -s -X POST -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/admin/orders/<orderId>/retry
```

3. For inconsistencies (terminal order with reservation still RESERVED, CONFIRMED with no fulfillment progress), run reconciliation (ADMIN):

```
curl -s -X POST -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/admin/reconciliation/run | jq '{examined,repaired,details}'
```

4. Do not update `orders.status` by SQL: it skips history, audit, outbox. If unavoidable, SEV1 decision with two people and a recorded audit note.
5. Communicate customer impact using order ids only.

Prevention

- Alert: `oms_saga_active` increasing for 10 min; stuck-saga count > 0 for 5 min; `oms_events_dead_lettered_total` increase > 0.
- Keep `oms.saga.stuck-after-seconds` and `max-attempts` tuned (defaults 60 and 5) and tested.
- Every step command and event handler idempotent (see `docs/mentoring/design-patterns.md`).
- Scheduled reconciliation with results audited.

Practice exercise

Goal: make an order stick at RESERVE_INVENTORY, observe, then recover.

1. Start the local stack: `scripts/run-local.sh` (prints the generated demo password once; verified on 2026-10-18). Obtain a customer token from `POST /api/v1/auth/token` (dev profile).
2. Stop inventory-service only (kill its PID from `.run/`; `scripts/stop-local.sh` stops everything).
3. Create an order for SKU-1001 (see `docs/api/examples.md`). Expect 202 and status CREATED/VALIDATED depending on whether validation (REST) can succeed. With inventory down, validation retries then the saga records `lastError`.
4. Inspect `/api/v1/admin/saga/<orderId>` and `ops/order/messaging`; note `attempts` and `outboxPending`.
5. Restart inventory-service. Watch the order reach CONFIRMED then FULFILLED without manual action (recovery job).
6. Repeat, but set `oms.saga.max-attempts=1` and see FAILED / COMPENSATION_PENDING flagged; recover with admin retry.

Questions: Which log line proves the command was re-sent? Why does a duplicate command not double-reserve?

Regression test pointers

- `OrderStateMachineTest` (13 tests, exists, order-service).
- `SagaRecoveryTest` (12 tests, exists): `aStuckCompensationGetsReleaseInventoryResentWithItsOriginalReason`, `theJobRedrivesOnlySagasThatMadeNoProgressForTheConfiguredTime`, `validatedSagaThatNeverProgressesFailsAndDefensivelyReleases`; `ReconciliationTest` (5); live: `tests/e2e/resilience-live.mjs` (order parked while fulfillment is down completes after restart).
- Consumer duplicate test pattern: `services/platform-common/src/test/java/com/enterprise/oms/common/messaging/ConsumerKafkaTest.java` (exists).

Runbook 02 - Kafka consumer lag

All commands: illustrative, not executed against a live cluster. `/api/v1/ops/messaging` is implemented in `services/platform-common` (`OpsMessagingController`, `MessagingStatusService`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Symptoms

- `consumerLag` in `GET /api/v1/ops/messaging` growing for a topic; `lagAvailable:false` means the AdminClient could not reach the broker (that itself is a finding, not an excuse).
- Downstream effects delayed: notifications late, reservations late, fulfillment late (see runbook 01).
- `oms.events.processed` rate falls to zero while producers still publish; `oms.outbox.pending` rises if the broker, not the consumer, is the problem.
- `oms.events.retries` increasing: consumers are failing and backing off (500 ms x2 up to 5 s, 4 attempts) before DLT.

Metrics and PromQL

```
sum by (topic) (rate(oms_events_processed_total[5m]))
increase(oms_events_retries_total[10m])
increase(oms_events_dead_lettered_total[10m])
oms_outbox_pending
# Kafka client metric (names vary by exporter; example with Micrometer binder)
max by (topic, partition) (kafka_consumer_fetch_manager_records_lag{client_id=~".*order-service.*"})
histogram_quantile(0.99, sum by (le) (rate(http_server_requests_seconds_bucket{uri!~"/actuator.*"}
[5m])))
```

Diagnostics

```
# Lag per partition for a group (group id = service name)
kafka-consumer-groups.sh --bootstrap-server localhost:19092 --describe --group inventory-service
kafka-consumer-groups.sh --bootstrap-server localhost:19092 --list
# Look at: CURRENT-OFFSET vs LOG-END-OFFSET, LAG, CONSUMER-ID (empty = no active member), HOST

# Application view
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/inventory/messaging | jq
#   -> topics[].consumerLag, committedOffsets, retries, deadLettered, outboxPending, lastProcessedAt,
dltTopics

# Dead letters
kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic oms.inventory.events.v1.DLT \
  --from-beginning --max-messages 5 --property print.key=true --property print.headers=true
kafka-run-class.sh kafka.tools.GetOffsetShell --bootstrap-server localhost:19092 --topic
oms.inventory.events.v1.DLT

# Pod state and rebalances
kubectl -n oms get pods -l app=inventory-service
kubectl -n oms logs deploy/inventory-service --since=15m | grep -i -E 'rebalance|partitions (assigned|
revoked)|CommitFailed|max.poll'
```

Root-cause tree

```
Lag growing
|- LAG on all partitions, CONSUMER-ID empty -> no consumers
|    |- pods crash-looping / not ready -> runbook 06/07
|    '- listener container stopped after fatal error -> restart; fix cause
|- LAG on one partition only -> hot key (one orderId storm) or poison record at head
|    '- check retries metric and logs for same offset repeating
|- Consumers alive, processed rate low
|    |- handler slow: DB latency, row locks (runbook 03/04), remote call inside handler
|    |- repeated rebalances: processing exceeds max.poll.interval.ms -> consumer kicked
|    '- retry backoff on a failing record blocks that partition (up to ~8 s per record x attempts)
|- Producer side: oms.outbox.pending growing, consumers idle
|    |- relay not running (scheduling disabled / exception loop) -> app logs for OutboxRelay
|    '- broker unreachable / acks=all cannot be satisfied (min.insync.replicas) -> broker health
'- Poison pills landing in DLT -> lag drains but deadLettered rises; classify by x-exception-class
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Mitigation

1. Restore capacity: scale consumers up to the partition count (3). More than 3 per group is idle.

```
kubectl -n oms scale deploy/inventory-service --replicas=3
```

2. Fix the slow dependency (DB, lock contention) rather than raising poll intervals blindly.
3. Poison record: confirm it lands in the DLT (the error handler does this automatically after retries; non-retryable types go straight). Fix the producer or handler; replay valid records only after the fix. Replays are safe because consumers dedupe by `(consumer, event_id)` in `processed_events`.
4. Broker problems: check broker health and disk; the outbox buffers events safely (rows remain unpublished) and the relay resumes. Do not delete outbox rows.
5. Never reset offsets (`kafka-consumer-groups.sh --reset-offsets`) on a production group without ADMIN approval and a written reason: skipping events skips saga steps. If required, stop the consumers first, and use `--dry-run` before `--execute`.

Prevention

- Alert on lag growth rate, not only absolute lag; alert on `oms_events_dead_lettered_total` increase and `oms_outbox_pending` > threshold for 5 min.
- Keep handlers short; no remote calls; bounded DB work.
- Non-retryable classification kept current (`KafkaConsumerConfigSupport.NON_RETRYABLE`).
- Load test with `performance/` (k6) and record consumer throughput per partition.

Practice exercise

1. Local stack (`scripts/run-local.sh`, embedded Kafka on 19092; the stack start was verified on 2026-10-18, this lag exercise itself was not run).
2. Pause the inventory consumer by stopping inventory-service; post 20 orders (distinct Idempotency-Keys).
3. Run `kafka-consumer-groups.sh --describe --group inventory-service` and `ops/order/messaging`: observe lag on `oms.inventory.commands.v1`.
4. Restart inventory-service and watch lag drain; compute drain rate.
5. Poison test: publish `{not json` to `oms.inventory.commands.v1` with `kafka-console-producer.sh`; confirm the record appears in `oms.inventory.commands.v1.DLT` with `x-exception-class` and no retries.

Expected: lag grows then drains; the malformed record is dead-lettered once, `oms.events.dead_lettered` +1, `oms.events.retries` unchanged.

Regression test pointers (exist)

- `services/platform-common/src/test/java/com/enterprise/oms/common/messaging/ConsumerKafkaTest.java` (retry, DLT, non-retryable behaviour on EmbeddedKafka).
- `services/platform-common/src/test/java/com/enterprise/oms/common/outbox/OutboxRelayKafkaTest.java` (relay publishes headers and key).

Runbook 03 - Inventory reservation failures under concurrency

All commands: illustrative, not executed against a live cluster. inventory-service is implemented (`inventory/service/ReservationService`, three strategies in `inventory/reservation/`, `inventory/web/InventoryController`); the concurrency evidence is H2 only, nothing was run on PostgreSQL.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Symptoms

Two distinct problems share this runbook; tell them apart first.

A. **Legitimate rejections spike:** many orders REJECTED with `statusReason` "insufficient stock" for a hot SKU. Expected when demand exceeds stock; not an incident unless stock data is wrong.

B. **Integrity violation (SEV1):** `reserved > on_hand`, negative `available`, more RESERVED reservation quantity than stock, or fulfillment unable to ship confirmed orders.

Others: deadlock errors in logs (`deadlock detected`), `ObjectOptimisticLockingFailureException` storms on stock edits, reservation timeouts, `hikaricp.connections.pending` during bursts.

Metrics and PromQL

```
sum(increase(oms_events_processed_total{type="InventoryReservationFailed"}[10m]))
sum(increase(oms_events_processed_total{type="InventoryReserved"}[10m]))
rate(oms_events_retries_total{topic="oms.inventory.commands.v1"}[5m])
max(hikaricp_connections_pending{service="inventory-service"})
histogram_quantile(0.99, sum by (le) (rate(http_server_requests_seconds_bucket{service="inventory-
service"}[5m])))
```

Diagnostics

```
# Stock and reservations (OPERATOR/ADMIN)
curl -s -H "Authorization: Bearer <JWT>" "localhost:18080/api/v1/inventory?sku=SKU-1019" | jq
curl -s -H "Authorization: Bearer <JWT>" "localhost:18080/api/v1/inventory/reservations?
status=RESERVED&size=50" | jq
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/inventory/messaging | jq
```

Invariant checks in `inventory_db` (read-only):

```
-- Must return zero rows
select product_id, on_hand, reserved from stock where reserved > on_hand or reserved < 0 or on_hand <
0;

-- Reserved quantity per stock row vs sum of live reservation items (adapt table/column names to the
real schema)
select s.product_id, s.reserved as stock_reserved, coalesce(sum(i.quantity),0) as live_items
  from stock s
  left join inventory_reservation_items i on i.product_id = s.product_id
  left join inventory_reservations r on r.id = i.reservation_id and r.status in
('RESERVED','COMMITTED')
 group by s.product_id, s.reserved
having s.reserved <> coalesce(sum(i.quantity),0);
```

Locks and waits (PostgreSQL):

```
select a.pid, a.state, a.wait_event_type, a.wait_event, now()-a.query_start as age, left(a.query,100)
as query
  from pg_stat_activity a where a.datname = 'inventory_db' and a.state <> 'idle' order by age desc;

select l.pid, l.locktype, l.mode, l.granted, a.query
  from pg_locks l join pg_stat_activity a on a.pid = l.pid
 where not l.granted;

select pg_blocking_pids(pid) as blockers, pid, left(query,80) from pg_stat_activity where
cardinality(pg_blocking_pids(pid)) > 0;
```

Deadlock evidence: PostgreSQL server log lines `deadlock detected` with the two statements and process ids; application log lines for the retried/redelivered command with the same correlationId.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Root-cause tree

```
Reservation problem
|- Rejections spike, invariants hold (A)
|    |- genuine demand > stock -> business: restock via PUT /inventory/{id}/stock
|    |- reservations never released: expired/cancelled reservations not cleaned -> check EXPIRED
scheduler, ReleaseInventory consumption, runbook 01
|    '- stock edit lowered on_hand (adjust rejected below reserved by design)
|- Invariants broken (B)
|    |- check-then-act code path (read, compare in Java, write) -> code-review scenario 01
|    |- direct SQL/admin script bypassing the conditional UPDATE
|    '- partial multi-line commit after failure (missing rollback) -> scenario 01
|- Deadlocks
|    '- lines locked/updated in non-sorted productId order, or FOR UPDATE variant mixed with the
atomic one
|- Optimistic lock storms (stock admin edit)
|    '- hot row edited concurrently with reservations that bump version; retry logic missing or
unbounded
'- Latency during bursts -> hot-row contention + long transactions (remote call inside tx) -> runbook
04
```

Mitigation

- Case A: nothing to fix technically; confirm stock numbers with the business owner. Adjust stock with the audited endpoint, e.g. restock:

```
curl -s -X PUT -H "Authorization: Bearer <JWT>" -H "Content-Type: application/json" \
  -d '{"delta": 50, "reason": "restock PO-synthetic-1"}' localhost:18080/api/v1/inventory/
a0000000-0000-0000-0000-000000001019/stock
```

- Case B (SEV1): stop the bleed by pausing order intake for the affected SKU (deactivate product, ADMIN `DELETE /api/v1/products/{id}`) or scaling `inventory-service` consumers to 0 while keeping the outbox intact; collect the invariant query output; open an incident. Repair via reconciliation and audited stock adjustments, not ad hoc SQL. Affected orders need business decisions (cancel the latest ones: cancel releases reservations through the saga).
- Deadlocks: PostgreSQL aborts one transaction; the Kafka retry handles it. Fix ordering in code.

Prevention

- The single conditional UPDATE `... WHERE on_hand - reserved >= :q`, sorted productId order, all-or-nothing per reservation in one transaction (contract section 7).
- Unique `order_id` on reservations makes duplicate commands no-ops.
- CHECK constraint `reserved <= on_hand` where the portable SQL allows it, as a last line of defence.
- Nightly invariant query job with an alert on non-empty result.
- Concurrency test in CI (below).

Practice exercise

1. Local stack up (`scripts/run-local.sh`; verified on 2026-10-18). Seed has SKU-1019 stock 3 and SKU-1020 stock 0.
2. Fire 20 parallel order creations for SKU-1019 quantity 1 (distinct Idempotency-Keys), for example with a loop of backgrounded `curl` calls or `performance/` k6.
3. Expect exactly 3 orders to reach CONFIRMED path and 17 REJECTED; check `reserved = 3`, `available = 0`.
4. Run the invariant SQL; expect zero rows.
5. Variation: swap in the check-then-act code from code-review scenario 01 on a scratch branch and repeat; observe `reserved` anomalies. Never do this on shared environments.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Regression test pointers

- Contention test: `docs/code-review/01-oversell-race.md` (20 threads, stock 3, exactly 3 succeed); PostgreSQL twin extends `AbstractPostgresIT` (exists in `platform-test-support`, skipped without Docker).
- Opposing multi-line orders deadlock test (exists): `ReservationConcurrencyTest#oppositeItemOrdersDoNotDeadlock`, run for every strategy on H2; the live 30-order oversell run (`tests/e2e/oversell-live.mjs`: 3 FULFILLED, 27 REJECTED, never above 3 committed) passed twice.
- `PUT /inventory/{id}/stock` below reserved returns 409 `CONFLICT` (exists): `InventoryApiTest#adjustingBelowTheReservedQuantityIsRejectedWith409AndChangesNothing`.

Runbook 04 - Database latency spikes

All commands: illustrative, not executed against a live cluster. Local/test databases are H2 (`MODE=PostgreSQL`); the diagnostics below apply to PostgreSQL 16 in Compose/Kubernetes. Each service owns its own database (`order_db`, `inventory_db`, `fulfillment_db`, `notification_db`).

Symptoms

- `http.server.requests` p99 up across endpoints of one service; CPU of the app low.
- `hikaricp.connections.pending` > 0 and `hikaricp.connections.active` equal to max; log lines `HikariPool-1 - Connection is not available, request timed out after 30000ms`.
- Kafka lag growing on that service's consumers (handlers wait on the DB), `oms.outbox.pending` rising (relay's DB queries slow).
- Readiness may flap if the DB health indicator times out.

Metrics and PromQL

```
max by (service) (hikaricp_connections_active) / max by (service) (hikaricp_connections_max)
max by (service) (hikaricp_connections_pending)
histogram_quantile(0.99, sum by (le, service, uri) (rate(http_server_requests_seconds_bucket[5m])))
histogram_quantile(0.99, sum by (le, service) (rate(hikaricp_connections_acquire_seconds_bucket[5m])))
rate(oms_events_processed_total[5m])
oms_outbox_pending
```

Diagnostics

```
curl -s localhost:18081/actuator/health | jq '.components.db'
curl -s localhost:18081/actuator/prometheus | grep -E 'hikaricp_connections|jvm_threads_states'
jcmd <pid> Thread.print | grep -c 'HikariPool.*getConnection'     # threads waiting for a connection
```

PostgreSQL (use a read-only ops role; `pg_stat_statements` must be enabled in `shared_preload_libraries` and `CREATE EXTENSION pg_stat_statements` done in the database):

```
-- 1. Who is running what, longest first
select pid, state, wait_event_type, wait_event, now()-query_start as age, left(query,120) as query
  from pg_stat_activity
 where datname = 'order_db' and state <> 'idle'
 order by age desc;

-- 2. Idle in transaction (leaked or long transactions hold locks and connections)
select pid, now()-xact_start as xact_age, left(query,120)
  from pg_stat_activity where state = 'idle in transaction' order by xact_age desc;

-- 3. Blocking chains
select pid, pg_blocking_pids(pid) as blocked_by, left(query,80) from pg_stat_activity where
cardinality(pg_blocking_pids(pid)) > 0;
select l.locktype, l.mode, l.granted, a.pid, left(a.query,80)
  from pg_locks l join pg_stat_activity a on a.pid = l.pid where not l.granted;
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
-- 4. Slowest statements by total and mean time (PG 13+ column names)
select calls, round(total_exec_time::numeric,0) as total_ms, round(mean_exec_time::numeric,2) as
mean_ms, rows, left(query,100)
  from pg_stat_statements order by total_exec_time desc limit 10;

-- 5. Missing index smell: large sequential scans
select relname, seq_scan, seq_tup_read, idx_scan from pg_stat_user_tables order by seq_tup_read desc
limit 10;

-- 6. Connection count vs limit
select count(*) as connections, current_setting('max_connections') as max from pg_stat_activity;
```

Plan check for a suspect query: `EXPLAIN (ANALYZE, BUFFERS) <query>` on a replica or off-peak; never run `ANALYZE` of write statements in production without a transaction you roll back.

Root-cause tree

```
DB latency
|- Pool exhausted but DB itself fast (pg_stat_activity mostly idle / idle in transaction)
|    |- connections held during remote calls (REST to inventory, 3 s read timeout x retries) -> move
call outside the tx
|    |- leaked connection / transaction never closed
|    '- pool too small for the real concurrency (Little's law) or too many replicas x pool >
max_connections
|- DB busy
|    |- slow statement (pg_stat_statements): missing index, N+1 burst, unbounded list -> code-review
scenario 06
|    |- lock waits: hot rows (stock), long transactions, migrations -> pg_locks / blocking pids
|    |- autovacuum behind: bloated outbox / processed_events / saga_step_log (never purged)
|    '- resource: CPU, IO, memory, storage latency on the node
|- Recent change: new migration (table rewrite, index build without CONCURRENTLY), new query, new
release -> runbook 07
'- Traffic: retry storm after an upstream blip multiplies load
```

Mitigation

1. Cancel/terminate the offender only after capturing it:

```
select pg_cancel_backend(<pid>);       -- polite: cancels the query
select pg_terminate_backend(<pid>);    -- last resort: kills the session
```

2. Idle-in-transaction leaks: restart the leaking pod after taking a thread dump; set `idle_in_transaction_session_timeout` as a guardrail.
3. If caused by a release, roll back (runbook 07).
4. Shed load: scale consumers down temporarily (the outbox and Kafka buffer), reduce rate limits at the gateway; do not just raise the Hikari pool size (it moves the queue into the database).
5. Add the missing index with `CREATE INDEX CONCURRENTLY` (outside Flyway's transaction; plan a migration) after confirming with `EXPLAIN`.
6. Purge old data in batches (published outbox rows, old `processed_events`, per policy), never one giant DELETE.

Prevention

- Remote calls never inside DB transactions; connection timeout and leak-detection threshold set in Hikari.
- Statement-count tests for list endpoints; indexes reviewed with `EXPLAIN` on PostgreSQL (H2 plans differ).
- Retention jobs for outbox, processed_events, saga_step_log.
- Alert on pool pending > 0 for 2 minutes, DB connections > 80% of max, p99 latency SLO burn.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Migrations reviewed for lock level and table rewrites; `lock_timeout` set for DDL.

Practice exercise

1. Local stack uses H2, so reproduce the application-side symptom, not the PostgreSQL views: set `spring.datasource.hikari.maximum-pool-size=2` for order-service and inject latency in the inventory validate call (a test stub with a 3 s sleep) while issuing 10 parallel order creations. Expect `hikaricp_connections_pending > 0` and timeouts in the order-service log. (Not executed: the services exist and run locally, but this latency-injection exercise was not run.)
2. For the PostgreSQL views, use a Testcontainers PostgreSQL test (extends `AbstractPostgresIT`; skipped without Docker, so run it where Docker exists): open a transaction updating one `stock` row, do not commit, start a second update, and query `pg_locks` / `pg_blocking_pids` from a third connection.
3. Write down which of the root-cause branches each experiment demonstrates.

Regression test pointers

- Hikari saturation scenario (open): no test asserts that the inventory validate call is made before the transaction opens. Related, exists: `InventoryClientResilienceTest#aSlowInventoryHitsTheReadTimeoutAndIsRetried` (read timeout and retry) and `#theCircuitOpensAfterRepeatedFailuresAndRejectsCallsWithoutHittingTheServer`.
- Statement-count assertion for list endpoints (exists): `OrderRepositoryTest#searchingAPageOfOrdersWithItemsTakesAConstantNumberOfStatements`; background in `docs/code-review/06-unbounded-pagination-n-plus-one.md`.
- `AbstractPostgresIT` base class exists in `services/platform-test-support`.

Runbook 05 - Duplicate order events

All commands: illustrative, not executed against a live cluster.

Context: delivery is at-least-once (outbox relay, consumer retries, rebalances). Duplicates on the wire are **normal**; duplicate *business effects* are the bug. Consumers insert `(consumer, event_id)` into `processed_events` in the same transaction as the effect; the second delivery increments `oms.events.duplicates` and does nothing.

Symptoms

- `oms.events.duplicates` spikes (usually benign, see below).
- Customer receives two emails for one event (notification effect duplicated), or two reservations/ fulfillments for one order (serious).
- Two orders created for one customer action (HTTP-level duplicate: missing or reused Idempotency-Key handling).
- Order history shows the same transition twice.

Metrics and PromQL

```
sum by (topic, type) (increase(oms_events_duplicates_total[15m]))
sum by (topic) (increase(oms_events_processed_total[15m]))
# duplicate ratio: benign if low and tied to a restart/rebalance window
sum(increase(oms_events_duplicates_total[15m])) / sum(increase(oms_events_processed_total[15m]))
increase(oms_events_retries_total[15m])
changes(kube_pod_container_status_restarts_total{namespace="oms"}[30m])
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Diagnostics

```
# What did the service see?
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/notification/messaging | jq
'{processedEvents,duplicatesIgnored,retries,deadLettered}'

# Is it the same eventId? Compare Kafka records for the order (key = orderId)
kafka-console-consumer.sh --bootstrap-server localhost:19092 --topic oms.order.events.v1 \
  --from-beginning --property print.key=true --property print.headers=true --timeout-ms 10000 | grep
<orderId>
# Same eventId header twice => redelivery/relay re-publish (benign if effect once).
# Different eventIds for the same logical change => producer emitted two events (a bug upstream).

# Audit trail for the order
curl -s -H "Authorization: Bearer <JWT>" "localhost:18080/api/v1/ops/order/audit?
entityType=Order&entityId=<orderId>" | jq
```

```
-- Was the event recorded as processed? (notification_db; columns per platform-common V1 migration)
select consumer, event_id, event_type, topic, processed_at from processed_events where event_id =
'<eventId>';

-- Notification uniqueness guard: (source_event_id, template) must be unique
select source_event_id, template, count(*) from notifications group by 1,2 having count(*) > 1;

-- Outbox: was the same row published repeatedly?
select id, event_type, attempts, published_at, last_error from outbox_events where aggregate_id =
'<orderId>' order by created_at;
```

HTTP-level duplicates: look for the same `Idempotency-Key` in the gateway/order logs; replays return 202 with `Idempotent-Replay: true`.

Root-cause tree

```
Duplicate effect observed
|- eventId identical, effect applied twice
|    |- processed_events insert not in the same tx as the effect (self-invocation, missing
@Transactional) -> code-review scenario 03
|    |- effect performed outside the processOnce callback (e.g. email "sent" before/after the tx)
|    |- consumer id (first argument of processOnce) differs between code paths -> dedupe key mismatch
|    '- non-idempotent external side effect (real email provider) -> needs provider idempotency key
|- eventId different, same logical change
|    |- producer built two events (retry in business code that re-appends to outbox)
|    '- eventId generated at relay time instead of at append time
|- Same eventId, effect once, duplicates counter up -> benign redelivery
|    |- consumer restart/rebalance replays uncommitted offsets
|    '- relay crashed after send, before marking published_at
'- Two HTTP-created orders
     |- client generated a new Idempotency-Key per retry
     '- key scope or request-hash comparison bug
```

Mitigation

- Benign duplicates: no action; note the cause (deploy/rebalance) in the ticket.
- Duplicate notifications: they are simulated in this platform (no real provider is contacted); for a real provider, apologise per business policy and fix the dedupe.
- Duplicate fulfillment/reservation: reservations are guarded by a unique `order_id`; fulfillments should be too. Cancel the surplus via the audited paths (order cancel releases reservations), not by SQL.
- Producer bug: fix and deploy; no replay needed for the outbox (rows are not republished once `published_at` is set).
- Never delete rows from `processed_events` to "retry" an event: that re-enables duplicates for that id. To reprocess intentionally, use admin retry endpoints that create new, audited work.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Prevention

- Idempotency test for every consumer (same event twice, effect once).
- `processed_events` insert and effect in one transaction, verified by rollback test.
- Unique constraints as the final safety net (reservations.order_id, notifications `(source_event_id, template)`).
- Retention on `processed_events` longer than the maximum redelivery window (topic retention).
- Clients must reuse the same Idempotency-Key when retrying the same request (documented in `docs/api/examples.md`).

Practice exercise

1. Local stack (`scripts/run-local.sh`; stack start verified on 2026-10-18; the replay steps below were not run by hand, but the same behaviour is asserted by `NotificationKafkaTest`/`NotificationServiceTest`).
2. Create an order; copy the `OrderCreated` record from `oms.order.events.v1` (console consumer with headers).
3. Re-publish the same JSON (same `eventId`) twice to `oms.order.events.v1` with `kafka-console-producer.sh`. Expect `oms.events.duplicates` +2 and no new notification rows.
4. Change `eventId` in the payload (new UUID) and publish: a second notification is created or blocked by `(source_event_id, template)` uniqueness depending on design; record what you observe and explain why.
5. Replay the HTTP request with the same Idempotency-Key and body: 202 + `Idempotent-Replay: true`. Change the body: 422 `IDEMPOTENCY_KEY_REUSED`.

Regression test pointers

- `services/platform-common/src/test/java/com/enterprise/oms/common/idempotency/IdempotentProcessorAndAuditTest.java` (exists: processOnce semantics).
- `.../messaging/ConsumerKafkaTest.java` (exists).
- Order creation idempotency matrix (exists): `IdempotencyApiTest` (11 tests, including replay, key reuse 422, missing key 400 and concurrent same-key submissions); live: 8 parallel requests with one key produced one order (`scripts/live-flows.mjs`). Teaching background: `docs/mentoring/testing.md` section 4.

Runbook 06 - Service unavailable (503 / readiness DOWN)

All commands: illustrative, not executed against a live cluster.

Symptoms

- Clients get 503 `UPSTREAM_UNAVAILABLE` (gateway cannot reach a service, or order-service cannot reach inventory), 504 after the gateway's 10 s response timeout, or 502 from the ingress.
- `GET /api/v1/ops/health` shows a service `DOWN` (the endpoint always returns HTTP 200 and reports status in the body).
- Kubernetes: pods `NotReady`, `CrashLoopBackOff`, `OOMKilled`, or no endpoints behind the Service.
- Circuit breaker open on the order -> inventory call; sagas record `lastError` and retry.

Metrics and PromQL

```
up{job=~"order-service|inventory-service|fulfillment-service|notification-service|api-gateway"} == 0
sum by (service) (rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
  / sum by (service) (rate(http_server_requests_seconds_count[5m]))
kube_deployment_status_replicas_available{namespace="oms"} <
kube_deployment_spec_replicas{namespace="oms"}
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

```
resilience4j_circuitbreaker_state{state="open"} == 1
changes(kube_pod_container_status_restarts_total{namespace="oms"}[15m])
```

Diagnostics

```
# 1. Aggregate view
curl -s -H "Authorization: Bearer <JWT>" localhost:18080/api/v1/ops/health | jq

# 2. Direct probes (bypass the gateway)
curl -s -o /dev/null -w '%{http_code}\n' localhost:18081/actuator/health/liveness
curl -s localhost:18081/actuator/health/readiness | jq
curl -s localhost:18081/actuator/health | jq '.components'      # db, kafka, diskSpace etc. if details
are exposed on the management port

# 3. Kubernetes
kubectl -n oms get pods -o wide
kubectl -n oms describe pod <pod> | sed -n '/Events/,$p'
kubectl -n oms get endpoints order-service
kubectl -n oms logs <pod> --previous | tail -100
kubectl -n oms get events --sort-by=.lastTimestamp | tail -20

# 4. Dependencies from inside the cluster
kubectl -n oms exec deploy/order-service -- wget -qO- http://inventory-service:18082/actuator/health/
readiness
```

Interpretation: liveness fails -> the container is restarted (deadlock, OOM, fatal startup error). Readiness fails while liveness passes -> a dependency is down (DB, Kafka) or the app is still starting; it is removed from the Service endpoints, which is what you want. A service restarting in a loop with `OMS_JWT_SECRET` missing or shorter than 32 bytes is **intended** fail-fast behaviour (contract); fix the secret, never relax the check.

Root-cause tree

```
503 / DOWN
|- Gateway itself
|    |- pods down -> everything unavailable (SEV1)
|    '- route target wrong/timeout 10 s -> backend slow, not down
|- Single service pods NotReady
|    |- CrashLoopBackOff
|    |    |- startup failure: missing/short OMS_JWT_SECRET, bad datasource URL, Flyway migration
failure (checksum/ordering), port clash
|    |    |- OOMKilled -> runbook 08
|    |    '- recent deploy -> runbook 07
|    |- Readiness fails, liveness OK
|    |    |- DB unreachable / credentials rotated -> check secret mount, DB health
|    |    '- Kafka admin check failing (should not fail readiness; if it does, config bug)
|    '- No pods scheduled: resource quota, node pressure, image pull error
|- Dependency-induced
|    |- inventory down -> order VALIDATE step retries (Resilience4j 3 attempts) then circuit opens;
orders accumulate (runbook 01)
|    '- Postgres down -> all services DOWN; Kafka consumers pause; outbox buffers nothing new (no
writes)
'- Network: NetworkPolicy blocks management port / service DNS issue
```

Mitigation

1. Identify the smallest failing component and its last change (`kubectl rollout history`).
2. Last deploy suspected: `kubectl -n oms rollout undo deploy/<name>` (runbook 07).
3. Dependency down: restore it; services recover on their own (readiness returns, breakers half-open, `SagaRecoveryJob` re-drives).
4. Capacity: `kubectl -n oms scale deploy/<name> --replicas=<n>` when pods are healthy but overloaded.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

5. Gateway timeouts: check backend p99 and DB (runbook 04) before raising timeouts.
6. Customer-facing messaging while degraded: orders are accepted asynchronously (202) and processed when dependencies return; do not tell customers orders are lost.

Prevention

- Readiness probe depends on what the service truly needs (DB), liveness only on the process; startup probe for slow Flyway migrations.
- PodDisruptionBudget and at least 2 replicas for gateway and order-service.
- Alert on `up == 0`, 5xx ratio, available replicas < desired, breaker open > 5 min.
- Fail-fast on bad config stays enabled; configuration validated in CI (`scripts/validate-infra.sh` exists; static validation only).
- Game days: periodically kill inventory-service and verify sagas recover.

Practice exercise

1. Local stack (`scripts/run-local.sh`, verified on 2026-10-18; the kill-inventory step itself was not run, the fulfillment-down variant was: `tests/e2e/resilience-live.mjs`).
2. `curl localhost:18080/api/v1/ops/health` -> all UP.
3. Kill inventory-service (PID in `.run/`). Re-run: inventory `DOWN`, HTTP still 200. Place an order: it is accepted (202); check the saga/lastError and the order-service log for breaker and retry lines.
4. Restart inventory-service; confirm the order proceeds without intervention.
5. Start order-service with `OMS_JWT_SECRET` set to a 10 character value (synthetic, throwaway); expect startup failure with a clear message. Restore a generated 32+ byte secret.

Regression test pointers

- `JwtSecret` fail-fast behaviour (platform-common security tests; verify in `services/platform-common/src/test`).
- Order-to-inventory adapter (exists): `InventoryClientResilienceTest` (11 tests: retry, read timeout, circuit opens and closes again, 4xx not retried).
- Gateway `ops/health` aggregation returns 200 with a DOWN body when a backend is down (exists): `OpsHealthTest#oneServiceDownIsDegradedAndStillHttp200`, `#allDownIsDown`, `OpsHealthUnreachableTest`; also `gateway/ResilienceTest`.

Runbook 07 - Failed deployment

All commands: illustrative, not executed against a live cluster. Manifests are expected under `infrastructure/kubernetes/` (Kustomize base + overlays), CI in `.github/workflows/ci.yml`. At the time of writing these have only been **statically validated** (`scripts/validate-infra.sh`), never applied to a cluster.

Symptoms

- `kubectl rollout status` hangs or reports `deployment exceeded its progress deadline`.
- New pods `CrashLoopBackOff`, `ImagePullBackOff`, `CreateContainerConfigError`, or `Running` but never Ready.
- Error rate or latency rises right after a release; consumer lag grows after the rollout.
- Flyway failure in startup logs (`Migration checksum mismatch`, `Detected resolved migration not applied`, `Validate failed`).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Metrics and PromQL

```
kube_deployment_status_replicas_unavailable{namespace="oms"} > 0
kube_deployment_status_observed_generation != kube_deployment_metadata_generation
sum by (service) (rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
changes(kube_pod_container_status_restarts_total{namespace="oms"}[10m])
sum by (topic) (rate(oms_events_processed_total[5m]))
increase(oms_events_dead_lettered_total[10m])
```

Compare the minutes after the rollout to the baseline before it.

Diagnostics

```
kubectl -n oms rollout status deploy/order-service --timeout=120s
kubectl -n oms rollout history deploy/order-service
kubectl -n oms get rs -l app=order-service
kubectl -n oms describe pod <new-pod> | sed -n '/Events/,$p'
kubectl -n oms logs <new-pod> | head -80                  # startup errors: config, secret, Flyway
kubectl -n oms logs <new-pod> --previous | tail -80
kubectl -n oms get deploy order-service -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
kubectl -n oms get secret oms-jwt -o jsonpath='{.metadata.name}{"\n"}'     # existence only; never
print secret values
kubectl -n oms exec deploy/order-service -- env | grep -c OMS_JWT_SECRET    # count only, do not
display
curl -s localhost:18081/actuator/health/readiness | jq
```

Database migration state (order_db):

```
select installed_rank, version, description, success, installed_on from flyway_schema_history order by
installed_rank desc limit 5;
```

Root-cause tree

```
Deployment failed
|- Image: tag missing, registry credentials, architecture mismatch -> ImagePullBackOff
|- Config: missing env/secret (OMS_JWT_SECRET absent or < 32 bytes -> intentional fail-fast), wrong
datasource URL, bad Kafka bootstrap
|- Flyway
|    |- failed migration (success = false): needs manual repair decision with DBA
|    |- checksum mismatch: someone edited an applied migration (never edit; add a new version)
|    |- version ordering: platform-common V1..V8, service V10+, seed V1000+ (dev only); a new
migration below the max applied version fails validation
|    '- long migration holds locks -> readiness probe timeout -> runbook 04
|- Probes: initialDelay too short for migrations -> restarts; use a startup probe
|- Resources: requests exceed quota/node capacity (Pending), limits too low (OOMKilled -> runbook 08)
|- Behavioural regression passing probes: higher 5xx/latency, DLT growth, schema/event incompatibility
|    '- incompatible event/API change: v1 changes must be additive, consumers tolerant (contract
section 5 and 7)
'- Mixed versions during rollout (old + new pods): contract-breaking change not backward compatible
```

Mitigation

1. **Roll back first, investigate second:**

```
kubectl -n oms rollout undo deploy/order-service
kubectl -n oms rollout status deploy/order-service
```

2. If a migration already ran and the old version cannot work with the new schema, rollback of the pod is not enough: that is why migrations must be backward compatible (expand, then contract in a later release). Escalate to SEV1 and involve the DBA; do not manually edit `flyway_schema_history` without sign-off.
3. A failed Flyway migration: `flyway repair` only after fixing the cause and with approval; prefer a corrective new migration.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

4. Behavioural regression: roll back; replay is automatic because the outbox/Kafka buffered events and consumers are idempotent. Check DLTs for records produced by the bad version.
5. Record the failed version, timeline and decision in the incident ticket.

Prevention

- CI gates: unit + embedded Kafka tests, Flyway migrate against H2 and (where Docker exists) PostgreSQL, OpenAPI diff, event schema check.
- Rolling update with `maxUnavailable: 0`, readiness gating, `progressDeadlineSeconds`, canary where possible.
- Expand/contract migrations; never edit applied migrations; migration review checklist (lock level, table rewrite, index `CONCURRENTLY`).
- Config validation at startup (fail-fast) and in CI manifests lint.
- Post-deploy smoke test: `scripts/smoke-test.sh` / `smoke-test.mjs` exist and are meant for the local stack; adapt the base URL for an environment.

Practice exercise

1. Local only (no cluster): in a scratch copy, add a migration `V10__x.sql` to a service with a deliberate SQL error, start the service via the local scripts, observe startup failure and the Flyway message. Then edit an already-applied migration file and restart against the same H2 file DB to see a checksum mismatch (in-memory H2 resets on restart; use a file-based URL for this exercise).
2. Start a service with no `OMS_JWT_SECRET` and observe fail-fast.
3. If a cluster is available later: apply an overlay with a non-existent image tag, observe `ImagePullBackOff`, and practise `rollout undo`.

Regression test pointers

- Flyway migration test on H2 (`MODE=PostgreSQL`) and PostgreSQL Testcontainers twin (skipped without Docker); migration version-range convention test (open: not written). Real migration tests on H2: `SeedMigrationTest` (order-service), `SeedDataTest` (inventory); the PostgreSQL twins (`*PostgresIT`) were never executed.
- `scripts/validate-infra.sh` for static manifest validation (exists).
- Backward-compat test: consumer ignores unknown fields and unknown event types (`EventEnvelopeTest` and router tests in platform-common, exist).

Runbook 08 - JVM memory problems

All commands: illustrative, not executed against a live cluster. Runtime is Java 17. Heap dumps contain customer data (synthetic in this project, real in any real deployment): treat them as confidential, store encrypted, delete after analysis.

Symptoms

- `OOMKilled` pod restarts (container limit hit, exit code 137) or `java.lang.OutOfMemoryError: Java heap space` in logs.
- `jvm.memory.used{area="heap"}` close to `jvm.memory.max` and not falling after GC.
- Long or frequent GC pauses (`jvm.gc.pause`), latency spikes that correlate, throughput drop.
- Kafka consumers kicked from the group because a pause exceeded `max.poll.interval.ms` (lag and rebalances; runbook 02).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Metrics and PromQL

```
jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes{area="heap"}
sum by (service) (jvm_memory_used_bytes{area="nonheap"})
rate(jvm_gc_pause_seconds_sum[5m]) / rate(jvm_gc_pause_seconds_count[5m])      # mean pause
max_over_time(jvm_gc_pause_seconds_max[5m])
rate(jvm_gc_pause_seconds_sum[5m])                                              # fraction of time
paused; > 0.1 is a problem
jvm_gc_live_data_size_bytes                                                     # live set after major
GC; rising = leak
container_memory_working_set_bytes{namespace="oms"} / on(pod)
kube_pod_container_resource_limits{resource="memory"}
jvm_threads_live_threads
```

Reading it: a sawtooth heap that returns to a stable floor is healthy. A floor that creeps upward across GCs is a leak. Heap fine but container working set near limit means native/off-heap/metaspace/thread stacks or `-Xmx` set too close to the limit.

Diagnostics

```
# What are the JVM flags and sizes?
kubectl -n oms exec deploy/order-service -- jcmd 1 VM.flags
kubectl -n oms exec deploy/order-service -- jcmd 1 VM.info | head -40
kubectl -n oms top pod

# GC activity (5 samples, 1 s)
kubectl -n oms exec deploy/order-service -- jstat -gcutil 1 1000 5
#   O (old gen %) pinned near 100 and FGC increasing => memory pressure / leak

# Class histogram: cheap, no full dump, shows top retained-by-count classes
kubectl -n oms exec deploy/order-service -- jcmd 1 GC.class_histogram | head -30

# Heap dump (pauses the JVM for seconds; do it on one pod you have removed from traffic, if possible)
kubectl -n oms exec deploy/order-service -- jcmd 1 GC.heap_dump /tmp/heap.hprof
kubectl -n oms cp order-service-<pod>:/tmp/heap.hprof ./heap.hprof     # then delete from the pod

# Threads (stack memory, runaway pools)
kubectl -n oms exec deploy/order-service -- jcmd 1 Thread.print | grep -c '^"'

# Native memory tracking only if the JVM was started with -XX:NativeMemoryTracking=summary
kubectl -n oms exec deploy/order-service -- jcmd 1 VM.native_memory summary
```

Preventive JVM flags worth having in every environment: `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps` (with a mounted, restricted volume), `-XX:MaxRAMPercentage=70` (leave room for metaspace, threads, direct buffers), `-Xlog:gc*:file=/tmp/gc.log`. Analyse dumps with Eclipse MAT: look at the dominator tree and "leak suspects".

Root-cause tree

```
Memory pressure
|- Heap leak (live set grows)
|    |- unbounded collections / caches without eviction (in-memory rate limiter buckets keyed by
customer/IP, product cache without max size)
|    |- unbounded list endpoint loading entities (code-review scenario 06)
|    |- large payloads: whole aggregates in events, big JSON bodies (limit is 64 KB by filter)
|    |- listener holding references (e.g. accumulating failed records in a list)
|    '- ThreadLocal/MDC not cleaned in pooled threads
|- Heap too small for workload (live set stable but near max) -> resize, not leak
|- Off-heap / native
|    |- Metaspace (class loader leak in dev tools), direct buffers (Netty in the gateway), thread
stacks (thread-per-task pools)
|    '- container limit vs MaxRAMPercentage mismatch -> OOMKilled with a healthy heap
|- GC tuning symptoms: allocation rate high (N+1 loading, big result sets), humongous allocations (G1)
'- Load: traffic spike or retry storm
```


Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Mitigation

1. Capture evidence (histogram, GC log, heap dump) from one pod **before** restarting it; the restart clears the problem for hours and destroys the proof.
2. Restart or roll pods one at a time to restore service; scale out if the live set is just too large for one pod.
3. Temporary relief: raise the memory limit and `MaxRAMPercentage` together; record it as a stopgap with an owner and date.
4. Identify the leak with the dump; fix (bound the cache, page the query, clean the ThreadLocal); add a test.
5. Gateway: check in-memory rate-limit bucket growth when `oms.redis.enabled=false` (the local default); in production Redis stores buckets.

Prevention

- Every cache and in-memory map has a maximum size and an eviction policy; reviews ask "what bounds this?".
- Paged queries only; entities never accumulated in memory.
- Alerts: heap used > 85% after GC for 10 min, `jvm_gc_live_data_size_bytes` trend, GC pause fraction, OOMKilled restarts.
- Memory limits defined from measurement (k6 load profile in `performance/`), documented with the observed live set.
- `HeapDumpOnOutOfMemoryError` configured, with access control on the dump volume.

Practice exercise

1. Local JVMs (no Docker). Start one service with a small heap by adding `-Xmx128m` to its launch (inline environment `JAVA_TOOL_OPTIONS=-Xmx128m` for that process only; do not export globally; the local scripts launch the JVMs for you; the small-heap exercise was not run, the demo JVMs are capped at 384 MB each).
2. Introduce the flawed unbounded listing from `docs/code-review/06-unbounded-pagination-n-plus-one.md` on a scratch branch, seed 100k orders using a throwaway script, and call the endpoint repeatedly.
3. Observe with `jstat -gcutil <pid> 1000` and `jcmd <pid> GC.class_histogram | head`. Capture a dump with `jcmd <pid> GC.heap_dump <path-outside-repo>` and open it in MAT (dominator tree).
4. Fix with paging and re-run: old gen stabilises.

Regression test pointers

- Statement and result-size bound tests for list endpoints (exists): `OrderRepositoryTest#searchingAPageOfOrdersWithItemsTakesAConstantNumberOfStatements`, `PageRequestFactoryTest` (rejects oversize pages); see code-review scenario 06.
- Bounded-cache unit test (exists): `ProductCacheTest sizeIsBoundedAndATtlOfZeroDisablesCaching` (inventory-service `InMemoryProductCache`).
- k6 soak profile in `performance/` with an assertion on heap floor trend (documented procedure; not executed).

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

21. Limitations and future enhancements

Known limitations (by design or by environment)

Area	Limitation	Consequence	What production would need
Identity	HS256 shared-secret JWT and a dev-only token endpoint	Any holder of the secret can mint any identity; services share the secret	OIDC provider, RS256 via JWKS, short-lived tokens, per-service client credentials
Service-to-service	SERVICE JWT from the same secret, network trust	No cryptographic workload identity	mTLS or a service mesh plus NetworkPolicies, or OAuth client credentials
Discovery and config	Static route URIs, no Eureka and no Config Server	Fine on Kubernetes and Compose; not dynamic on VMs	Kubernetes DNS (already), or Eureka and Config where there is no orchestrator
Kafka	Embedded single broker locally, replication factor 1; consumer lag partition-bound (3 partitions)	No HA locally; scaling consumers beyond 3 per group gives nothing	RF 3, `min.insync.replicas=2`, more partitions planned up front, rack awareness
Delivery	At-least-once plus idempotent consumers, not exactly-once	A duplicate is possible and absorbed, not prevented	Same, or Kafka transactions where read-process-write is in Kafka only
Trace propagation	`correlationId` is propagated end to end and was shown across all five services in a real trace sample; the W3C `traceparent` is NOT continuous across Kafka hops (a new trace id per hop in the local run)	Distributed traces break at the outbox hop; use `correlationId` as the join key	Persist trace context in outbox headers and restore it in the relay
Database	PostgreSQL-only behaviour is not proven by H2 tests; PostgreSQL integration tests need Docker	Local green is not PostgreSQL green	Run Testcontainers in CI, run `CREATE INDEX CONCURRENTLY` style migrations with PostgreSQL-specific scripts
Multi-replica jobs	The outbox relay has no row claiming and the recovery, reconciliation and expiry jobs have no scheduler lock	With several replicas duplicates are published or work is repeated; tolerated by idempotency but wasteful	Row claiming (SELECT ... FOR UPDATE SKIP LOCKED) or a scheduler lock
DLT replay	No replay tool; dead letters are inspected manually	Recovery is a manual procedure	A replay command or admin endpoint with audit
Data growth	A `RetentionPurgeJob` now purges `processed_events` (14 days) and `audit_events` (365 days) via `oms.retention.*`; tested on H2 only, retention values are assumptions	Purge never ran against PostgreSQL at volume	Tune retention to redelivery and compliance windows, consider partitioning

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Area	Limitation	Consequence	What production would need
React frontend	Types copied (not generated) from the contract; `style-src 'unsafe-inline'` for MUI/ Emotion; token readable by script (`sessionStorage`); no axe, Lighthouse or screen-reader audit; Docker image, compose profile `react`, Kubernetes manifest and CI jobs never executed; only desktop Chrome	Two UIs can drift; a CSP exception; XSS exposure of the token; unknown WCAG status; deployment assets unproven	Generate types from the OpenAPI export, nonce-based Emotion cache, PKCE plus BFF with HttpOnly cookies, run axe and Lighthouse, build and scan the image
Frontend	zone.js change detection, components not using `OnPush`, GET retries without jitter	Higher change-detection cost, retry synchronisation under load	`OnPush` or zoneless, jittered backoff
OpenAPI	All four services are exported from live `/v3/api-docs`; the gateway spec is generated, not exported	Contract drift risk remains if the export is not diffed in CI	Run `scripts/export-openapi.mjs --check` in CI
Infrastructure	Compose, Kubernetes and Terraform were validated statically; Docker was unavailable	Untested in a live cluster	Rehearse on a real cluster, apply Terraform in a sandbox account
MSK auth	Terraform and the prod overlay now agree on IAM (port 9098), but the services ship no `aws-msk-iam-auth` library and set no SASL properties	Prod Kafka connection is not usable until the client side is added	Add the library and `SASL_SSL` properties, then test against MSK
Deployment safety	`progressDeadlineSeconds` is not set although the runbook recommends it	A stuck rollout takes longer to be flagged	Set it and alert on it
Load and capacity	k6 scripts exist; no recorded production-scale run	Capacity numbers are design targets	Run against a production-like environment, record results
CI	`.github/workflows/ci.yml` has never run on a GitHub runner; `deploy-test` is a placeholder	Pipeline correctness is unproven	Run it, pin Actions to commit SHAs, add environment protection
Business scope	No payment, tax, pricing rules, carriers or real notifications	Not a complete commerce system	Integrate providers behind the existing ports

Documentation freshness

The governance documents (HLD, LLD, test strategy, risk register, runbook index) were refreshed against the code after integration. The authoritative sources, in order, are the code, the shared contract (`docs/architecture/00-shared-contract.md`), the validation report and then the narrative documents. When they disagree, trust the code. The full list of limitations is in `docs/known-limitations.md`; its additional items are:

- Outbox relay and scheduled jobs do not claim rows (duplicate work with replicas, safe because of idempotency).
- No DLT replay tool.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

- Angular `OnPush` is not used.
- React UI: types are copied not generated, `style-src 'unsafe-inline'` remains for MUI/Emotion, no accessibility audit, and the React image, compose profile, Kubernetes manifest and CI jobs were never executed.
- OIDC mode is untested and lacks the SERVICE token for order to inventory calls.
- `/actuator/health` and `/actuator/prometheus` are unauthenticated.
- The KEDA example is not in any overlay, and the HPA range 2 to 6 exceeds the 3 partitions per topic.
- Docker, Kubernetes, Terraform apply, k6 and GitHub CI were never executed.
- A cancel arriving during fulfillment is resolved by the state machine and compensation, not by a cross-service lock.

Future enhancements (prioritised)

1. **Security hardening**: OIDC with RS256, per-service identities, mTLS or mesh, secret rotation, dependency pinning by digest.
2. **Reliability**: persist trace context through the outbox, row-claim the scheduled jobs, build a DLT replay tool, add `progressDeadlineSeconds`, run chaos tests (broker loss, DB failover).
3. **Scale**: keyset pagination, read replicas for reporting, partition planning, Redis caching with explicit invalidation, load-test baselines.
4. **Product**: payment and shipping adapters, partial fulfillment, backorders, multi-warehouse stock.
5. **Delivery**: GitOps (Argo CD or Flux), progressive delivery (canary), SBOM and image signing, preview environments.
6. **Frontend**: generate both UIs' types from `specs/openapi/*.json`, nonce-based Emotion CSP, `OnPush` or zoneless, Signal Forms when stable, accessibility audit, run the live-mode end-to-end suite in CI (it passed 18/18 locally against the real stack).
7. **Observability**: SLO burn-rate alerts, exemplars linking metrics to traces, structured business metrics per tenant.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix A. Job-description traceability matrix

Maps typical Senior Full Stack Java and Angular job requirements to what this repository actually implements. Evidence rules: **Executed** means a JUnit report, Playwright result or similar artefact exists on disk from a real run (the class and the test count are named); **Documented / not executed** means files exist (and may have been statically validated) but nothing was run; **Conceptual** means design discussion only. Every path in the source-code column is checked by `tools/pdf/check-refs.mjs`. Executed counts are from `target/surefire-reports` and `target/failsafe-reports` and from the live integration run (`docs/validation/validation-report.md`): reactor 968 cases, 939 executed, 0 failed, 29 skipped (all Docker/Postgres/benchmark), plus 2 retention tests added afterwards; Angular frontend 149/149; React frontend 207/207 (2026-10-19); live end-to-end 18/18 for each of the two UIs; smoke 10/10; API flows 54/54; oversell burst 3 FULFILLED and 27 REJECTED. The PDF evidence table regenerates the per-module counts at build time.

37 requirements are mapped. Each card lists: implemented feature; source-code location; test or validation evidence; interview talking point; remaining limitations.

Java 17

Field	Detail
Implemented feature	Java 17 release target, records for DTOs and events, table-driven domain classes, switch expressions
Source-code location	`services/pom.xml`, `services/platform-common/src/main/java/com/enterprise/oms/common/events/EventEnvelope.java`, `services/order-service/src/main/java/com/enterprise/oms/order/domain/OrderStateMachine.java`
Test or validation evidence	Executed: whole backend compiles with `maven.compiler.release` 17; `OrderStateMachineTest` 207 run, 0 failed
Interview talking point	Records give immutable value types; pure domain classes are exhaustively testable
Remaining limitations	No sealed hierarchies or pattern matching for switch (a Java 21 feature) in production code

Spring Boot

Field	Detail
Implemented feature	Boot 3.5.16 services, auto-configuration library, profiles, Actuator, graceful shutdown
Source-code location	`services/platform-common/src/main/java/com/enterprise/oms/common/autoconfigure/OmsCoreAutoConfiguration.java`, `services/order-service/src/main/resources/application.yml`
Test or validation evidence	Executed: `OperationalEndpointsTest`, `ActuatorAndDocsTest` and the other API tests under `services/*/src/test`
Interview talking point	Custom auto-configuration keeps four services consistent without copy-paste
Remaining limitations	Library coupling across services (one version)

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Spring Cloud (Gateway, Resilience4j; no Eureka or Config Server)

Field	Detail
Implemented feature	Spring Cloud Gateway with static route URIs, JWT at the edge, rate limit, circuit breaker; Resilience4j retry and circuit breaker on the inventory REST call
Source-code location	`gateway/src/main/java/com/enterprise/oms/gateway/config/RoutesConfig.java`, `gateway/src/main/resources/application.yml`, `services/order-service/src/main/java/com/enterprise/oms/order/saga/RestInventoryClient.java`
Test or validation evidence	Executed: gateway `ResilienceTest` 3 run, `EdgeSecurityTest` 59 run, order `InventoryClientResilienceTest` 10 run, all 0 failed
Interview talking point	Eureka and Config Server are deliberately absent: Kubernetes DNS and ConfigMaps/Secrets already provide discovery and configuration
Remaining limitations	No dynamic discovery or central refreshable config; would be added on VM estates

Microservices

Field	Detail
Implemented feature	Four services plus gateway, database per service, event-driven integration (ADR-0001, ADR-0004)
Source-code location	`docs/adr/0001-microservices-vs-modular-monolith.md`, `docs/adr/0004-database-per-service.md`, `services/pom.xml`
Test or validation evidence	Executed: per-module test suites; live end-to-end flows: smoke 10/10, API flows 54/54, resilience run passed (see Appendix B for the 11 defects it found)
Interview talking point	Boundaries follow data ownership; the cost is eventual consistency
Remaining limitations	Single repo, shared library, no independent release pipelines

REST APIs

Field	Detail
Implemented feature	Versioned REST, problem+json, pagination, idempotency key, 202 async creation, 404 for foreign resources
Source-code location	`services/order-service/src/main/java/com/enterprise/oms/order/api/OrderController.java`, `services/platform-common/src/main/java/com/enterprise/oms/common/error/GlobalExceptionHandler.java`
Test or validation evidence	Executed: `IdempotencyApiTest` 11 run, `ExceptionHandlerApiTest`, `OrderApiTest`, 0 failed
Interview talking point	Idempotent POST and honest async semantics
Remaining limitations	No HATEOAS; offset pagination only

Angular

Field	Detail
Implemented feature	Angular 20 standalone app, lazy routes, signal stores, functional guards and interceptors, Material UI
Source-code location	`frontend/oms-ui/src/app/app.routes.ts`, `frontend/oms-ui/src/app/core/auth/auth.store.ts`, `frontend/oms-ui/src/app/core/http/interceptors.ts`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Field	Detail
Test or validation evidence	Executed: Karma unit-test coverage output exists under `frontend/oms-ui/coverage`; Karma 149 of 149 pass, `ng build` OK; Playwright e2e 18/18 against the mock backend AND 18/18 live (UI -> gateway -> real services), where the live run found a real blank-page defect (D-03)
Interview talking point	Signals for state, interceptors for cross-cutting HTTP concerns, guards are UX not security
Remaining limitations	No OnPush or zoneless; one browser (Chrome 155), no accessibility audit

Kafka

Field	Detail
Implemented feature	Five topics, keyed by orderId, outbox relay, idempotent consumers, retry and DLT
Source-code location	`services/platform-common/src/main/java/com/enterprise/oms/common/outbox/OutboxRelay.java`, `services/platform-common/src/main/java/com/enterprise/oms/common/messaging/KafkaConsumerConfigSupport.java`, `infrastructure/compose/kafka/create-topics.sh`
Test or validation evidence	Executed (embedded Kafka): `OutboxRelayKafkaTest` 3, `ConsumerKafkaTest` 5, `NotificationKafkaTest` 9 run, 0 failed; live: 218 captured records all schema-valid, 15/15 event types, 0 in DLTs; fulfillment-down resilience run passed
Interview talking point	At-least-once plus idempotent consumers gives an effectively-once business effect
Remaining limitations	Embedded single broker; RF 1 locally; not exactly-once; no DLT replay tool; relay has no row claiming

PostgreSQL

Field	Detail
Implemented feature	Schema per service, constraints, composite indexes, atomic updates, EXPLAIN ANALYZE study
Source-code location	`services/inventory-service/src/main/resources/db/migration/V10__inventory_schema.sql`, `services/order-service/src/main/resources/db/migration/V12__indexes.sql`, `docs/database/inventory-explain-analyze.md`
Test or validation evidence	Executed on H2 (PostgreSQL mode); the EXPLAIN study documents a PostgreSQL run by its author; PostgreSQL Testcontainers tests (`InventoryPostgresIT`) skipped without Docker per failsafe report
Interview talking point	Index choice from plans, not guesses
Remaining limitations	H2 does not prove PostgreSQL isolation behaviour

Cloud deployment

Field	Detail
Implemented feature	AWS reference in Terraform (VPC, EKS, RDS, MSK, ElastiCache, ECR, Secrets Manager)
Source-code location	`infrastructure/cloud/aws/terraform/eks.tf`, `infrastructure/cloud/README.md`, `infrastructure/cloud/diagrams/aws-reference.mmd`
Test or validation evidence	Documented / not executed: `terraform fmt -check` and `validate` PASS in `scripts/validate-infra.sh`; no plan or apply
Interview talking point	Multi-AZ, private subnets, IRSA, managed Kafka and database

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Field	Detail
Remaining limitations	Terraform and prod overlay now agree on MSK IAM (9098) but the client library is missing; no cost model

CI/CD

Field	Detail
Implemented feature	GitHub Actions: build, test, scans, images, infra validation, smoke, deploy placeholder; Jenkinsfile mirror
Source-code location	`.github/workflows/ci.yml`, `infrastructure/ci/Jenkinsfile`, `docs/adr/0008-ci-cd-and-deployment.md`
Test or validation evidence	Documented / not executed (workflow never ran on a runner); the validation script it calls passes locally (47 PASS, 0 FAIL, 2 SKIP)
Interview talking point	Shift-left scanning, immutable images, promotion by overlay
Remaining limitations	deploy-test is a placeholder; Actions not pinned to SHA

Automated testing

Field	Detail
Implemented feature	Pyramid across unit, slice, embedded Kafka, BDD, e2e, load scripts
Source-code location	`docs/governance/test-strategy.md`, `services/order-service/src/test/java/com/enterprise/oms/order/saga/SagaOrchestrationTest.java`, `tests/bdd/order-scenarios.feature`
Test or validation evidence	Executed: `SagaOrchestrationTest` 23 run, 0 failed; full counts in the PDF evidence table
Interview talking point	Test the state machine exhaustively, integrate narrowly
Remaining limitations	PostgreSQL ITs skipped; k6 not run

Mockito

Field	Detail
Implemented feature	Mockito-based unit tests for services and strategies
Source-code location	`services/inventory-service/src/test/java/com/enterprise/oms/inventory/service/ReservationServiceUnitTest.java`, `services/inventory-service/src/test/java/com/enterprise/oms/inventory/reservation/ReservationStrategyUnitTest.java`
Test or validation evidence	Executed: `ReservationServiceUnitTest` 6 run, 0 failed
Interview talking point	Mock collaborators at boundaries, assert behaviour not implementation
Remaining limitations	Some tests use real Spring contexts rather than mocks by design

Docker

Field	Detail
Implemented feature	Multi-stage layered Dockerfiles, non-root, healthcheck, Compose stack
Source-code location	`infrastructure/docker/order-service.Dockerfile`, `infrastructure/compose/docker-compose.yml`
Test or validation evidence	Documented / not executed (Docker not installed); `/tmp` tmpdir alignment fixed statically; no image was built

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Field	Detail
Interview talking point	Layer caching, JVM container flags, non-root
Remaining limitations	Images not built here; base image not digest-pinned

Kubernetes

Field	Detail
Implemented feature	Kustomize base and overlays, probes, NetworkPolicies, KEDA, ServiceMonitor
Source-code location	`infrastructure/kubernetes/base/order-service.yaml`, `infrastructure/kubernetes/overlays/prod/kustomization.yaml`, `infrastructure/kubernetes/keda/scaledobject-kafka-lag.yaml`
Test or validation evidence	Documented / not executed: kustomize renders of dev/test/prod/monitoring pass `kubeconform -strict` and a policy check; never applied
Interview talking point	Startup probe, zero-unavailable rollout, default-deny network
Remaining limitations	Never applied to a live cluster; HPA 2-6 exceeds 3 partitions; KEDA example not in an overlay

Observability

Field	Detail
Implemented feature	Micrometer metrics, JSON logs, correlation id, OTel, Prometheus rules, Grafana dashboards
Source-code location	`observability/prometheus/rules/oms-alerts.yml`, `observability/grafana/dashboards/oms-api-red.json`, `services/platform-common/src/main/java/com/enterprise/oms/common/observability/Metrics.java`
Test or validation evidence	Executed: `LogMaskingTest` 3 run; all 59 dashboard/alert expressions checked against a live scrape (0 missing after fixes D-05 to D-07); correlation trace across 5 services; no Prometheus/Grafana/Tempo server ran
Interview talking point	RED and USE views, runbooks tied to alerts
Remaining limitations	traceparent not continuous across Kafka hops (correlationId is the join key)

Agile delivery

Field	Detail
Implemented feature	Sprint plan, Definition of Ready and Done, risk register, ADRs
Source-code location	`docs/governance/sprint-plan.md`, `docs/governance/definition-of-ready-and-done.md`, `docs/governance/risk-register.md`
Test or validation evidence	Documented / not executed (planning artefacts, no real sprint ran)
Interview talking point	Explain the process you would run and why
Remaining limitations	Not evidence of a delivered team cadence

Security

Field	Detail
Implemented feature	JWT edge and service validation, role and ownership checks, rate limits, body limits, masked logs

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Field	Detail
Source-code location	`gateway/src/main/java/com/enterprise/oms/gateway/security/EdgeSecurityConfig.java`, `services/platform-common/src/main/java/com/enterprise/oms/common/security/JwtSecret.java`, `docs/security/threat-model.md`
Test or validation evidence	Executed: `EdgeSecurityTest` 59, `OrderSecurityTest` 10, `SecurityApiTest`, `JwtSecretTest`, 0 failed
Interview talking point	Backend-authoritative authorisation; 404 against enumeration
Remaining limitations	HS256 demo tokens; no OIDC provider, no mTLS

Design patterns

Field	Detail
Implemented feature	Outbox, saga, state machine, strategy (3 reservation strategies), idempotent receiver, tolerant reader, circuit breaker
Source-code location	`services/inventory-service/src/main/java/com/enterprise/oms/inventory/reservation/ReservationStrategy.java`, `services/platform-common/src/main/java/com/enterprise/oms/common/idempotency/IdempotentEventProcessor.java`, `docs/mentoring/design-patterns.md`
Test or validation evidence	Executed: `ReservationConcurrencyTest` 15 run, 0 failed; `IdempotentProcessorAndAuditTest` 4 run
Interview talking point	Name the pattern, the problem and the trade-off
Remaining limitations	Patterns documented more than benchmarked

Production troubleshooting

Field	Detail
Implemented feature	Eight runbooks, ops endpoints, saga inspector, reconciliation
Source-code location	`docs/runbooks/02-kafka-consumer-lag.md`, `services/platform-common/src/main/java/com/enterprise/oms/common/messaging/OpsMessagingController.java`, `services/order-service/src/main/java/com/enterprise/oms/order/saga/SagaRecoveryJob.java`
Test or validation evidence	Executed: `SagaRecoveryTest` 12 run, `ReconciliationTest` 5 run, 0 failed; runbook commands not executed on a cluster
Interview talking point	Symptom, hypothesis, cheap check, action
Remaining limitations	Runbook commands are illustrative

Technical ownership

Field	Detail
Implemented feature	ADRs, HLD and LLD, post-incident template, deployment and rollback procedure
Source-code location	`docs/adr/README.md`, `docs/governance/deployment-and-rollback.md`, `docs/governance/post-incident-review-template.md`
Test or validation evidence	Documented / not executed
Interview talking point	Own the decision, the evidence and the follow-up
Remaining limitations	Templates, not real incidents

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

React.js and Redux

Field	Detail
Implemented feature	React 19 + TypeScript (strict) + Vite second front end, feature-equivalent to the Angular UI: screens as lazy routes, Redux store with `auth`, `cart` and `ui` slices, hooks for polling, list queries and session expiry, MUI components. Added after the user explicitly asked for React (the original brief mandated Angular), see ADR-0013
Source-code location	`frontend/oms-ui-react/src/App.tsx`, `frontend/oms-ui-react/src/routes.tsx`, `frontend/oms-ui-react/src/store/authSlice.ts`, `frontend/oms-ui-react/src/store/cartSlice.ts`, `frontend/oms-ui-react/src/hooks/polling.ts`, `frontend/oms-ui-react/src/features/orders/orderForm.ts`, `docs/architecture/react-architecture.md`, `docs/adr/0013-dual-frontend-angular-and-react.md`
Test or validation evidence	Executed 2026-10-19: Vitest 207 of 207 pass (coverage 96.3 % statements, 93.5 % branches); ESLint 0 errors; production build OK; Playwright 18/18 against the mock backend and 18/18 live with `E2E_UI=react`
Interview talking point	Same behaviour in both frameworks, proven by the same 18 e2e specs; hooks and reducers versus signal stores
Remaining limitations	Contract types copied not generated; no axe/Lighthouse audit; token in `sessionStorage`; Docker image and Kubernetes manifest never run

Redux Toolkit and RTK Query

Field	Detail
Implemented feature	Server state in RTK Query (`createApi`, tags, polling, one base query for headers, GET-only retry, problem+json errors, 401 and 429 handling); client state in `createSlice` slices with `createSelector`; listener middleware resets the cache on logout
Source-code location	`frontend/oms-ui-react/src/store/api.ts`, `frontend/oms-ui-react/src/store/baseQuery.ts`, `frontend/oms-ui-react/src/store/index.ts`, `frontend/oms-ui-react/src/store/uiSlice.ts`, `frontend/oms-ui-react/src/store/slices.test.ts`, `frontend/oms-ui-react/src/store/api.test.ts`
Test or validation evidence	Executed 2026-10-19: slice, selector and API-layer tests (MSW: headers, GET-only retry, 401 and 429 side effects, invalidation) are part of the 207 passing Vitest tests
Interview talking point	Server cache versus client state; do not copy server data into slices; tag invalidation instead of manual refetch
Remaining limitations	No normalized entity cache; no persisted cache beyond the session mirror

React Router

Field	Detail
Implemented feature	React Router 7 layout routes with `React.lazy` screens and role guards (`RequireAuth`, `RequireRole`, `GuestOnly`); guards are UX only
Source-code location	`frontend/oms-ui-react/src/routes.tsx`, `frontend/oms-ui-react/src/components/guards.tsx`, `frontend/oms-ui-react/src/layout/AppShell.tsx`
Test or validation evidence	Executed 2026-10-19: guard and role-aware navigation tests in the 207 Vitest tests; Playwright navigation and guard specs 18/18 mock and live
Interview talking point	Route guards are not security; the backend authorizes every call
Remaining limitations	No route-level data loaders; client-side routing only

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

React Testing Library, Vitest and MSW

Field	Detail
Implemented feature	Tests render the whole app at a URL with a seeded store and a mock service worker for the gateway; pure logic tested without React
Source-code location	`frontend/oms-ui-react/src/app.test.tsx`, `frontend/oms-ui-react/src/features/orders/orders.test.tsx`, `frontend/oms-ui-react/src/test/server.ts`, `frontend/oms-ui-react/src/test/render.tsx`, `frontend/oms-ui-react/vite.config.ts`
Test or validation evidence	Executed 2026-10-19: 207 of 207 in 9 files; v8 coverage 96.3 % statements, 93.5 % branches, 94.5 % functions, 96.7 % lines
Interview talking point	Test behaviour through the user's eyes; MSW instead of mocking modules
Remaining limitations	Three harmless act(...) notices remain; jsdom is not a real browser

Dual-UI end-to-end parity

Field	Detail
Implemented feature	The same 18 Playwright specs run against either UI, selected by `E2E_UI` (angular or react), in mock and live mode
Source-code location	`tests/e2e/playwright.config.ts`, `tests/e2e/specs/order-lifecycle.spec.ts`, `tests/e2e/mock-backend/server.js`, `scripts/run-local.sh`
Test or validation evidence	Executed 2026-10-19: React mock 18/18, React live 18/18, Angular mock 18/18, Angular live 18/18 (live stack gateway on port 18085 because 18080 was taken by a foreign process)
Interview talking point	One contract, two clients: specs prove parity, not prose
Remaining limitations	Only desktop Chrome; the specs cover the 18 flows, not every screen; contract-drift risk between the two `models` copies

AI-assisted development

Field	Detail
Implemented feature	Spec-driven workflow, layered context, agent templates, 12 flawed-AI examples with tests
Source-code location	`ai-engineering/README.md`, `ai-engineering/flawed-ai-examples/README.md`, `ai-engineering/guides/spec-driven-workflow.md`
Test or validation evidence	Executed: the Java flawed-example project was run with JUnit (per its README); agent templates not executed
Interview talking point	AI drafts, humans verify with catalogue and tests
Remaining limitations	Angular examples not executed; no measured productivity data

Redis

Field	Detail
Implemented feature	Optional rate-limit store and product cache with in-memory fallback
Source-code location	`services/order-service/src/main/java/com/enterprise/oms/order/ratelimit/RedisOrderRateLimiter.java`, `services/inventory-service/src/main/java/com/enterprise/oms/inventory/cache/RedisProductCache.java`, `infrastructure/kubernetes/dev-dependencies/redis.yaml`

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Field	Detail
Test or validation evidence	Executed with a mocked client: `RedisOrderRateLimiterTest`, `ProductCacheTest`; no live Redis run
Interview talking point	Degrade to in-memory instead of failing
Remaining limitations	Never used for reservation decisions

Flyway

Field	Detail
Implemented feature	Versioned migrations per service, shared V1 library migration, dev-only seed
Source-code location	`services/platform-common/src/main/resources/db/common/V1__outbox_processed_audit.sql`, `services/order-service/src/main/resources/db/migration/V10__customers_orders.sql`, `services/order-service/src/main/resources/db/seed/V1000__seed_customers.sql`
Test or validation evidence	Executed: migrations applied on H2 in every Spring test; `SeedMigrationTest`
Interview talking point	Expand and contract for zero-downtime releases
Remaining limitations	Not applied to PostgreSQL in this environment

OpenAPI

Field	Detail
Implemented feature	springdoc per service, exported specs, disabled in prod profile
Source-code location	`services/order-service/src/main/java/com/enterprise/oms/order/api/OpenApiConfig.java`, `scripts/export-openapi.mjs`, `specs/openapi/inventory-service.json`
Test or validation evidence	Executed: `ActuatorAndDocsTest`, `ProdProfileDocsDisabledTest`; all four service exports plus a generated gateway spec are committed; `export-openapi.mjs --check` shows no drift
Interview talking point	Contract as artefact, exported not hand-written
Remaining limitations	Gateway spec is generated, not exported

TDD and BDD

Field	Detail
Implemented feature	Red-green-refactor write-ups, Gherkin features executed as tests
Source-code location	`docs/tdd/order-state-machine-tdd.md`, `tests/bdd/inventory-scenarios.feature`, `services/inventory-service/src/test/java/com/enterprise/oms/inventory/InventoryBddScenariosTest.java`
Test or validation evidence	Executed: BDD test classes ran (the XML report lists scenario tests even where the text report shows 0)
Interview talking point	Tests as executable specification
Remaining limitations	Feature files are simulated in JUnit rather than a Cucumber runner

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Load testing

Field	Detail
Implemented feature	k6 scripts for order flow and an oversell hammer
Source-code location	`performance/order-flow.js`, `performance/oversell.js`, `performance/README.md`
Test or validation evidence	Documented / not executed: k6 not installed, scripts syntax-checked only; the only load evidence is the live 30-order oversell burst (3 FULFILLED, 27 REJECTED, never above 3 committed)
Interview talking point	Prove no oversell under load with an invariant check
Remaining limitations	No capacity numbers measured

Concurrency control

Field	Detail
Implemented feature	Atomic conditional update, pessimistic and optimistic alternatives, sorted lock order
Source-code location	`services/inventory-service/src/main/java/com/enterprise/oms/inventory/reservation/AtomicUpdateReservationStrategy.java`, `docs/adr/0006-concurrency-control.md`
Test or validation evidence	Executed: `ReservationConcurrencyTest` 15 run (5 scenarios x 3 strategies), 0 failed on H2; live oversell: 30 parallel orders, stock 3 -> 3 FULFILLED, 27 REJECTED, passed twice
Interview talking point	Let the database arbitrate
Remaining limitations	PostgreSQL behaviour under load not proven here

Idempotency and exactly-once effects

Field	Detail
Implemented feature	API idempotency keys, processed_events, state-machine no-ops
Source-code location	`services/order-service/src/main/java/com/enterprise/oms/order/service/OrderCommandService.java`, `docs/adr/0005-idempotency-strategy.md`, `docs/architecture/delivery-semantics.md`
Test or validation evidence	Executed: `IdempotencyApiTest` 11 run, `IdempotentProcessorAndAuditTest` 4 run
Interview talking point	Three layers of idempotency
Remaining limitations	`RetentionPurgeJob` purges processed_events (H2 test only)

Saga and eventual consistency

Field	Detail
Implemented feature	Orchestrated saga, compensation, recovery job, reconciliation
Source-code location	`services/order-service/src/main/java/com/enterprise/oms/order/saga/OrderSagaOrchestrator.java`, `docs/architecture/saga-and-consistency.md`, `docs/adr/0003-saga-orchestration-vs-choreography.md`
Test or validation evidence	Executed: `SagaOrchestrationTest` 23, `SagaRecoveryTest` 12, `ReconciliationTest` 5, 0 failed
Interview talking point	Orchestration for visibility, choreography discussed in ADR
Remaining limitations	Orchestrator is a single-service coordinator

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

API gateway and edge security

Field	Detail
Implemented feature	Rate limit, size limit, CORS allow-list, circuit breaker, correlation id
Source-code location	`gateway/src/main/java/com/enterprise/oms/gateway/ratelimit/RateLimitWebFilter.java`, `gateway/src/main/java/com/enterprise/oms/gateway/filter/RequestSizeLimitWebFilter.java`, `gateway/README.md`
Test or validation evidence	Executed: `RateLimitTest`, `CorsTest`, `EdgeSecurityTest` 59 run, 0 failed
Interview talking point	Defence in depth: edge plus service checks
Remaining limitations	In-memory limiter unless Redis enabled

Code review and quality

Field	Detail
Implemented feature	Review checklist, seven flawed-PR exercises, AI review catalogue
Source-code location	`docs/governance/code-review-checklist.md`, `docs/code-review/README.md`, `ai-engineering/checklists/pr-checklist.md`
Test or validation evidence	Documented / not executed (teaching material; the Java flawed examples have tests, see AI row)
Interview talking point	Review for failure modes, not style
Remaining limitations	Exercise diffs are reconstructed, not real PRs

Mentoring and leadership

Field	Detail
Implemented feature	Mentoring guides on SOLID, design patterns, debugging, security, testing, performance
Source-code location	`docs/mentoring/README.md`, `docs/mentoring/solid.md`, `docs/mentoring/debugging.md`
Test or validation evidence	Documented / not executed
Interview talking point	Teach with the failing test first
Remaining limitations	No evidence of real mentees

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

Appendix B. Integration defects found and fixed

Eleven defects were found by running the real stack end to end after every module had a green test suite. They are the most useful interview material in this project, because each one is a concrete answer to 'tell me about a bug your tests did not catch'. The text below is the integration run's own record; the closing list names issues that were NOT fixed.

Every module was green in isolation (896 unit + 61 integration test cases at the start). The defects below were found only by running the whole stack (embedded Kafka + 4 services + gateway + built Angular UI, no Docker) and driving it through the gateway, the browser and the metric/log endpoints. Each fix lives in the owning module, has a regression test where one is practical (stated per item), and was verified again on the live stack afterwards. "Severity" is my judgement for a production system.

#	Area	Severity	Found by
D-01	scripts/run-local.sh readiness probe	High (stack never starts on this box)	first `run-local.sh` run
D-02	gateway concurrency limit	High	30 parallel orders (`oversell-live.mjs`)
D-03	Angular order page	High (blank page)	Playwright live run, spec 4
D-04	e2e mock ids / spec wording	Low	Playwright live run, specs 7 and 16
D-05	failure-path counters absent	Medium	`check-metric-names.mjs`
D-06	Kafka client metrics not bound	Medium	`check-metric-names.mjs`
D-07	latency histograms missing on 3 of 5 apps	Medium	`check-metric-names.mjs`
D-08	no lifecycle logs in order/fulfillment; no correlation id in scheduler logs	Medium	correlation trace
D-09	K8s base config defaulted to profile `dev`	High (security)	security flag 6a
D-10	`/v3/api-docs` served in `prod`	Medium (security)	security flag 6b
D-11	run-local readiness timeout too short	Low	first run

D-01 wait-for.mjs crashed Node on Windows, run-local.sh aborted a healthy stack

- **Symptom:** `run-local.sh` printed `Assertion failed: !(handle->flags & UV_HANDLE_CLOSING), file src\win\async.c, line 76` and `fulfillment-service did not become ready on port 18083`, then stopped everything, although the service was up.
- **Root cause:** `scripts/lib/wait-for.mjs` called `process.exit(0)` immediately after a `fetch()` that used `AbortSignal.timeout()`; on Windows with Node 24 this trips a libuv assertion (exit code 134), which the caller read as "not ready".
- **Fix:** the probe uses `node:http` with a socket timeout and sets `process.exitCode` instead of calling `process.exit()`.
- **Test:** no automated regression test (it is a Windows/Node runtime interaction). Evidence: `run-local.sh` started the stack three times afterwards and `node scripts/lib/wait-for.mjs http <url> 3` exits 0/1 cleanly.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

D-02 Gateway shed load at 26 concurrent requests per downstream service (503 UPSTREAM_UNAVAILABLE)

- **Symptom:** in the first 30-parallel-order burst, 29 requests got 202 and one got `503 UPSTREAM_UNAVAILABLE`; the gateway log said `Upstream unavailable: BulkheadFullException`. The 30 requests spanned only ~1.8 s, nothing was down.
- **Root cause:** Spring Cloud CircuitBreaker's reactive Resilience4j integration always wraps the circuit-breaker filter in a *semaphore bulkhead* with the library default of 25 concurrent calls and no waiting (`ReactiveResilience4JAutoConfiguration` creates the provider even though nothing configures it). The limit was an invisible hard cap per route.
- **Fix:** `GatewayProperties.Bulkhead` (`oms.gateway.bulkhead.max-concurrent-calls`, env `OMS_GATEWAY_BULKHEAD_MAX_CALLS`, default 200; `max-wait` default 500 ms) applied through a `Customizer<ReactiveResilience4jBulkheadProvider>` in `RoutesConfig`; explicit `resilience4j-bulkhead` dependency in `gateway/pom.xml`.
- **Test:** `BulkheadConcurrencyTest` (60 parallel requests to a stub that answers after 600 ms, all must be 200). I confirmed it fails with the old limits (`-Doms.gateway.bulkhead.max-concurrent-calls=25 -Doms.gateway.bulkhead.max-wait=0ms` produced 503s) and passes with the fix. Live re-run: 30/30 accepted.

D-03 Order detail page crashed (blank page, endless spinner) for orders rejected at validation

- **Symptom:** Playwright spec "insufficient stock (SKU-1020)" timed out: the status chip stayed empty. Browser console: `TypeError: Cannot read properties of null (reading 'toFixed')`.
- **Root cause:** contract section 13 says `sku/name/unitPrice/lineTotal` are `null` until validation has priced the order and stay null if it is rejected there. The UI model typed them as non-null and `order-detail.ts` called `i.unitPrice.toFixed(2)`. The mock backend priced everything, so no earlier test hit it.
- **Fix:** `OrderItem` fields are `| null` in `core/models.ts`; `OrderDetail.money()` renders `-` for null prices.
- **Test:** new `order-detail.spec.ts` (renders a REJECTED order with null prices, expects status chip, reason and `-` cells). Frontend unit tests: 149 pass (148 before). Live re-run of the spec: pass.

D-04 e2e assets were written against the mock only

- The mock backend used the invalid product id pattern `p0000000-...` (contract section 9 already corrected it to `a0000000-...0010NN`); fixed in `tests/e2e/mock-backend/server.js` and in `docs/runbooks/03-...md`.
- Spec wording that differed on the real backend and is now accepted in both modes: retrying a TERMINAL failed order is `409 INVALID_STATE_TRANSITION` on the real order-service (contract section 13) and the UI shows "That action is not possible in the current state of the order"; notification subjects are `Order <id8> failed` rather than `Order failed`. Both modes pass (mock 18/18, live 18/18).

D-05 oms_events_retries_total, oms_events_dead_lettered_total, oms_events_duplicates_total did not exist on a healthy service

- **Root cause:** Micrometer registers a counter on its first increment. Dashboards showed "No data" and `increase(...)>0` alert expressions had no series.
- **Fix:** `MessagingMetrics.preRegister(topics)` creates the series at 0 for every topic in `oms.messaging.topics`, called from `OmsMessagingAutoConfiguration`.
- **Test:** `MessagingMetricsTest` (3 cases). Live: the three names now appear on all four services.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

D-06 kafka_consumer_fetch_manager_records_lag_max (consumer-lag alert and dashboard) was never exposed

- **Root cause:** Spring Boot binds Kafka client metrics only to the consumer factory it auto-configures; platform-common builds its own `DefaultKafkaConsumerFactory`, so no `kafka_consumer_*` meters existed.
- **Fix:** `KafkaConsumerConfigSupport.setMeterRegistry` adds a `MicrometerConsumerListener`.
- **Test:** no dedicated unit test; verified live (`scripts/check-metric-names.mjs`: present on ports 18081-18084 with `topic`/`partition` labels). Note: Micrometer rewrites dots in that label (`oms_inventory_events_v1`) while `oms_events_*` keep dots (`oms.inventory.events.v1`); not changed.

D-07 Latency histogram buckets existed only on order- and inventory-service

- **Root cause:** `percentiles-histogram.http.server.requests` was set in two services' own yml, so the p95/p99 panels and `OmsHighP95Latency` had no data for fulfillment, notification and the gateway; `spring_kafka_listener_seconds_bucket` (messaging dashboard) did not exist anywhere.
- **Fix:** platform-common `oms-defaults.properties` enables `http.server.requests` and `spring.kafka.listener` histograms for all servlet services; the gateway sets `http.server.requests` in its own `application.yml`.
- **Test:** `PlatformDefaultsTest`; live: buckets on all five ports.

D-08 order-service and fulfillment-service logged nothing about an order's lifecycle

- **Symptom:** the correlation trace of one order contained only gateway, inventory and notification lines.
- **Root cause:** no INFO logging on the success path in the two services that own the saga and the fulfillment; additionally the fulfillment scheduler thread has no MDC, so any line it wrote would have had no correlation id.
- **Fix:** `OrderTransitions` logs every transition (and creation) with the MDC correlation id; `FulfillmentService` logs acceptance and each step with the stored correlation id, set into the MDC around the line (and restored).
- **Test:** `OrderTransitionsLoggingTest`, `FulfillmentServiceTest.schedulerStepsLogTheStoredCorrelationIdEvenWithoutMdc`. Live: `docs/validation/correlation-trace-sample.md` now shows all five services.

D-09 / D-10 Security flags (see docs/security/threat-model.md section 10)

- D-09 `infrastructure/kubernetes/base/configmap.yaml` defaulted `SPRING_PROFILES_ACTIVE` to `dev,json-logs`. Base is now `json-logs`; dev/test overlays set `dev,json-logs` explicitly; prod overlay unchanged (`prod,json-logs`). Static validation only (no cluster).
- D-10 `springdoc.api-docs.enabled` was never disabled in prod (and `swagger-ui` was not disabled in order-service's prod block either). Now both are `false` in the `prod` profile of all four services; test `ProdProfileDocsDisabledTest` x4 (config-level proof, not an HTTP 404 from a prod-profile process: prod needs PostgreSQL).

D-11 run-local.sh waited 180 s per service and never checked for busy ports

- Starting six JVMs at once on this machine takes 108-132 s per service (log: `Started ... in 108.652 seconds`), so a slow start looked like a failure. Timeout raised to 300 s. Second half, found at the very end: when a FOREIGN process (another project's gateway) held port 18080, the health URL answered and the script reported "ready" for a gateway that had failed to bind; the script now refuses to start if any of 19092/18080-18084 is taken (checked with a dummy

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.

listener).

Known issues NOT fixed (honest list)

- Gateway logs one `WARN` from `PrometheusMeterRegistry` (`http.client.requests.active` tag keys conflict with Spring Cloud Gateway's `http_client_requests_active_seconds`); the conflicting meter is simply not registered; no functional impact found.
- A separate `traceId` is created at every Kafka hop in this local run (no collector, tracing export off); end-to-end distributed traces were not demonstrated.
- `/actuator/prometheus` and `/actuator/health` are unauthenticated on every service port and on the gateway's public port (threat model 10.4).
- OIDC/JWKS mode is wired in code (gateway + platform-common) but untested; in that mode order -> inventory calls would lack a SERVICE token (threat model 10.3).
- Startup of the whole stack needs ~2.5 minutes on this machine; memory is capped at 384 MB per JVM in `run-local.sh`.

Generated by AI. AI can make mistakes. Validate before relying upon this document.
© 2026 Cotiviti, Inc. All rights reserved. All proprietary information shall remain the sole and exclusive property of Cotiviti, Inc.